ПОЛНЫЙ КУРС
ДЛЯ НАЧИНАЮЩИХ

C++

ОСНОВЫ ПРОГРАММИРОВАНИЯ

ПОЛНЫЙ КУРС
ДЛЯ НАЧИНАЮЩИХ ПРОГРАММИСТОВ

C++

ОСНОВЫ ПРОГРАММИРОВАНИЯ

МОСКВА

УДК 004.43
ББК 32.973.26-018.1
Н22

Книга «Основи на програмирането със C++» на болгарском языке распространяется в соответствии с открытой лицензией **CC-BY-SA** *(https://creativecommons.org/licenses/by-sa/4.0/).*
Книга доступна в электронном виде на официальном сайте: https://cpp-book.softuni.bg.
Исходный код доступен по ссылке: https://github.com/SoftUni/Programming-Basics-Book-CPP-BG.

Авторский коллектив:
Андрей Царев, Божидар Колев, Борис Горанов, Венцислав Петров, Георгий Лацев, Георгий Шавов, Денис Миланов, Димитар Кацарский, Димо Димов, Димо Чанев, Зорница Йоханова, Игнес Симеонова, Йордан Добрев, Любомир Господинов, Мирела Дамьянова, Николай Костов, Пламен Динев, Светлин Наков, Цветан Иванов.

Наков, Светлин.
С++. Основы программирования / С. Наков [и др.]. — Москва , 2024. — 480 с. : ил. — (Полный курс для начина-ющих программистов).

Книга «С++. Основы программирования» знакомит читателей с написанием программного кода в среде разработки (IDE) на начальном уровне, обучает использованию переменных и данных, операторов и выражений, работе с консолью (считывание входных данных и вывод выходных данных), применению условных операторов (if, if-else, switch-case), циклов (for, while, do-while) и функций (объявление и вызов функций, передача параметров и возврат значений). Основной упор в издании сделан на упражнениях: здесь приведено множество подробно прокомментированных примеров кода, дополненных тщательно отобранными практическими задачами и судейской системой для мгновенной автоматической оценки написанного вами кода. Издание призвано помочь новичкам сделать первые шаги в освоении С++ — мультипарадигменного высокопроизводительного языка программирования общего назначения с широчайшим набором возможностей, дальнейшее изучение которого открывает путь в такие области, как разработка высоконагруженных приложений, нейронных сетей, банковских, поисковых и ГИС-систем, систем управления базами данных и маршрутизации сетевого трафика, операционных систем, микроконтроллеров, браузеров, серверов, игр, игровых движков и графических редакторов.

УДК 004.43
ББК 32.973.26-018.1

© Светлин Наков и колектив, 2019 г.
Перевод на русский язык: ООО «Интеджер». .

Краткое содержание

Предисловие 9
Глава 1. Первые шаги в программировании 28
Глава 2.1. Простые вычисления 59
Глава 2.2. Простые вычисления — экзаменационные задачи 93
Глава 3.1. Простые проверки 115
Глава 3.2. Простые проверки — экзаменационные задачи 145
Глава 4.1. Более сложные проверки 162
Глава 4.2. Более сложные проверки — экзаменационные задачи 187
Глава 5.1. Повторы (циклы) 210
Глава 5.2. Повторы (циклы) — экзаменационные задачи 227
Глава 6.1. Вложенные циклы 249
Глава 6.2. Вложенные циклы — экзаменационные задачи 267
Глава 7.1. Более сложные циклы 288
Глава 7.2. Более сложные циклы — экзаменационные задачи 310
Глава 8.1. Подготовка к практическому экзамену — часть I 322
Глава 8.2. Подготовка к практическому экзамену — часть II 358
Глава 9.1. Задачи для чемпионов — часть I 378
Глава 9.2. Задачи для чемпионов — часть II 397
Глава 10. Функции 415
Глава 11. Трюки и хаки 458
Заключение 472

Содержание

Предисловие 9
Для кого эта книга? 9
Почему мы выбрали именно C++? 10
Книга на других языках программирования:
C#, Java, JavaScript, Python, PHP 11
Программирование изучается с помощью написания,
а не чтения! 11
Университет программного обеспечения (SoftUni) 12
Как стать программистом? 15
Книга в помощь преподавателям 23
История этой книги 24
Официальный сайт книги 25
Форум для ваших вопросов 25
Лицензия и распространение 26
Сообщение об ошибках 26

Глава 1. Первые шаги в программировании 28
Что означает «программировать»? 28
Как написать консольную программу? 33
Среда разработки (IDE) 34
Пример: создание консольной программы «Hello C++» 39
Тестирование примеров программ 47
Типичные ошибки в программах на языке C++ 47
Что мы узнали из этой главы? 51
Упражнения: первые шаги в написании кода 52
Консольные, графические и веб-приложения 57

Глава 2.1. Простые вычисления 59
Системная консоль 59
Вычисления в программировании 60
Типы данных и переменные 61
Чтение чисел из консоли 61
Чтение дробного числа из консоли 63
Считывание текста 64
Вывод на экран текста и чисел 65
Арифметические операции 66

Объединение текста и чисел ... 68
Числовые выражения ... 69
Что мы узнали из этой главы? ... 72
Упражнения: простые вычисления ... 73

Глава 2.2. Простые вычисления — экзаменационные задачи ... 93
Чтение чисел из консоли ... 93
Отображение текста по шаблону (placeholder) ... 94
Арифметические операторы ... 94
Конкатенация ... 95
Экзаменационные задачи ... 95
Задача: учебный класс ... 95
Задача: овощной рынок ... 99
Задача: ремонт плитки ... 103
Задача: конверсия валют ... 107
Задача: ежедневная прибыль ... 111

Глава 3.1. Простые проверки ... 115
Сравнение чисел ... 115
Простые проверки ... 117
Проверки с помощью конструкции if-else ... 117
Использование фигурных скобок { } с if-else ... 118
Область видимости переменной ... 121
Серии проверок ... 121
Упражнения: простые проверки ... 123
Отладка — простые операции с отладчиком ... 128
Упражнения: простые проверки ... 130

Глава 3.2. Простые проверки — экзаменационные задачи ... 145
Экзаменационные задачи ... 145
Задача: транспортные расходы ... 146
Задача: трубы в бассейне ... 149
Задача: сонный кот Том ... 152
Задача: сбор урожая ... 155
Задача: компания ... 159

Глава 4.1. Более сложные проверки ... 162
Вложенные проверки ... 162
Более сложные проверки ... 165
Логическое «И» ... 166
Логическое «ИЛИ» ... 168
Логическое отрицание ... 170
Оператор скобок () ... 171
Более сложные логические условия ... 171

Условная конструкция switch-case ... 176
Что мы узнали из этой главы? ... 179
Упражнения: более сложные проверки ... 180

Глава 4.2. Более сложные проверки — экзаменационные задачи ... 187
Вложенные проверки ... 187
Проверки switch-case ... 187
Экзаменационные задачи ... 188
Задача: успеть к экзамену ... 188
Задача: путешествие ... 194
Задача: операции над числами ... 198
Задача: билеты на матч ... 202
Задача: гостиничный номер ... 206

Глава 5.1. Повторы (циклы) ... 210
Повторения блоков кода (цикл for) ... 210
Сниппет (шаблон) кода для цикла for в Visual Studio ... 212
Пример: наибольшее число ... 215
Пример: суммирование гласных букв ... 220
Что мы узнали из этой главы? ... 222
Упражнения: повторения (циклы) ... 222

Глава 5.2. Повторы (циклы) — экзаменационные задачи ... 227
Экзаменационные задачи ... 228
Задача: умная Лили ... 234
Задача: вернуться в прошлое ... 237
Задача: больница ... 240
Задача: деление без остатка ... 243
Задача: логистика ... 246

Глава 6.1. Вложенные циклы ... 249
Вложенные циклы ... 251
Рисование более сложных фигур ... 258
Что мы узнали из этой главы? ... 266

Глава 6.2. Вложенные циклы — экзаменационные задачи ... 267
Экзаменационные задачи ... 267
Задача: нарисовать крепость ... 268
Задача: бабочка ... 271
Задача: знак «Стоп» ... 275
Задача: стрелка ... 279
Задача: топор ... 283

Глава 7.1. Более сложные циклы ... 288
Циклы с шагом ... 288
Цикл while ... 291
Наибольший общий делитель (GCD) ... 293
Цикл do-while ... 295
Бесконечные циклы и оператор break ... 297
Вложенные циклы и оператор break ... 301
Задачи с циклами ... 303
Что мы узнали из этой главы? ... 308

Глава 7.2. Более сложные циклы — экзаменационные задачи ... 310
Экзаменационные задачи ... 310
Задача: генератор паролей ... 310
Задача: магические числа ... 313
Задача: стоп-число ... 316
Задача: особые числа ... 318
Задача: разряды числа ... 320

Глава 8.1. Подготовка к практическому экзамену — часть I ... 322
Практический экзамен по «Основам программирования» ... 322
Система онлайн-оценки (судейская) ... 322
Задачи с простыми вычислениями ... 323
Задачи с однократной проверкой ... 327
Задача: точка на рисунке ... 330
Задачи с более сложными проверками ... 332
Задача: сумма трех чисел ... 334
Задачи с одним циклом ... 337
Задачи по рисованию фигур на консоли ... 344
Задачи на использование вложенных циклов с более сложной логикой ... 351

Глава 8.2. Подготовка к практическому экзамену — часть II ... 358
Типы экзаменационных задач ... 358
Задача: расстояние ... 359
Задача: замена плитки ... 363
Задача: цветочный магазин ... 366
Задача: оценки ... 369
Задача: рождественская шапка ... 372
Задача: сочетания букв ... 375

Глава 9.1. Задачи для чемпионов — часть I ... 378
Более сложные задачи по изученному материалу ... 378
Задача: пересекающиеся последовательности ... 378
Задача: магические даты ... 384
Задача: пять особых букв ... 390

Глава 9.2. Задачи для чемпионов — часть II ... 397
Более сложные задачи по изученному материалу ... 397
Задача: дни страстного шопинга ... 397
Задача: числовое выражение ... 403
Задача: быки и коровы ... 409

Глава 10. Функции ... 415
Что такое функция? ... 415
Функции с параметрами ... 425
Возвращаемый результат функций ... 431
Вложенные функции (локальные функции) ... 442
Именование функций.
Лучшие практики при работе с функциями ... 443
Что мы узнали из этой главы? ... 446
Упражнения ... 446

Глава 11. Трюки и хаки ... 458
Форматирование кода ... 458
Именование элементов кода ... 461
Клавиатурные сокращения в Visual Studio ... 462
Шаблоны кода (code snippets) ... 463
Приемы отладки кода ... 465
Полезные приемы ... 468

Заключение ... 472
Эта книга — лишь первый шаг! ... 473
Куда двигаться дальше? ... 473
Онлайн-сообщества для начинающих программистов ... 478

Предисловие

Данная книга является официальным учебником по курсу «Основы программирования» для начинающих в Университете программного обеспечения (SoftUni): https://softuni.bg/courses/programming-basics. Она знакомит читателей с написанием программного кода на начальном уровне (базовые навыки программирования), работой с IDE, использованием переменных и данных, операторов и выражений, работой с консолью (чтение ввода и вывод результатов), использованием условных операторов (`if`, `if-else`, `switch-case`), циклов (`for`, `while`, `do-while`), работой с функциями (объявление и вызов функций, передача параметров и возврат значения). Используются язык программирования C++ и среда разработки Visual Studio. Изложенный материал обеспечивает базовую подготовку к углубленному изучению программирования и готовит читателя к сдаче вступительного экзамена в этот университет.

Эта книга дает вам только *начальные основы программирования*. Она охватывает базовые навыки, которые нужно развивать годами, чтобы достичь достаточно высокого уровня для начала работы программистом.

Кроме того, книга используется в качестве неофициального учебника для школьных курсов по программированию в профессиональных средних школах Болгарии по программам обучения «программирование», «прикладное программирование» и «системное программирование», а также в качестве дополнительного учебника на начальных курсах по программированию в средних школах, специализированных и математических гимназиях, для классов с профилем «Информатика и информационные технологии».

Для кого эта книга?

Эта книга предназначена для *начинающих программистов*, которые хотят узнать, что такое программирование, и изучить основные конструкции для создания программного кода, используемые

при разработке программного обеспечения, независимо от используемых языков программирования и технологий. Книга закладывает *прочный фундамент практических навыков*, которые используются для дальнейшего обучения программированию и разработке программного обеспечения.

Всем тем, кто еще не прошел бесплатный курс по основам программирования для начинающих в университете SoftUni, мы настоятельно рекомендуем пройти его бесплатно, ведь программированию учатся практикуясь, а не читая!

На курсе вы получите бесплатный доступ к учебным пособиям, очным или онлайн-объяснениям и демонстрациям (как в видеоуроках), много практики в написании кода, помощь в решении задач после изучения каждой темы, доступ к наставникам, помощникам и менторам, возможность участия в форумах и дискуссионных группах для обсуждения возникших вопросов, доступ в многотысячное сообщество начинающих программистов и всевозможную помощь новичкам.

Бесплатный курс SoftUni для начинающих подходит школьникам (с 5-го класса), студентам и людям других профессий, которые хотят получить технические знания и понять, нравится ли им программирование и хотят ли они в будущем серьезно заниматься разработкой программного обеспечения.

Каждый месяц набирается новая группа. Курс «Основы программирования» в SoftUni организуется регулярно и включает в себя несколько различных языков программирования, так что попробуйте. Курс бесплатный, и вы можете отказаться от него в любой момент, если он вам не понравится. Записаться на бесплатное очное или онлайн-обучение начинающим программистам можно через форму заявки в SoftUni: https://softuni.bg/apply.

Почему мы выбрали именно C++?

Для этой книги мы выбрали язык C++, хотя он и не самый подходящий для начинающих, зато это *мощный и эффективный* язык программирования с открытым исходным кодом, который широко используется в тех случаях, когда требуется эффективность. Во многих школах изучение программирования начинается именно с C++ (а не с более простого языка), чтобы научиться низкоуровневой рабо-

те (ближе к аппаратной части), управлению памятью и написанию эффективного кода. C++ часто используется *на олимпиадах и конкурсах по программированию*, в том числе на Международной олимпиаде по информатике (International Olympiad in Informatics, IOI).

Язык C++ *широко распространен* и поддерживается на всех платформах, имеет развитую экосистему и поэтому является прочной основой для разработок в области программирования и программных технологий. C++ — статически типизированный компилируемый язык, сочетающий в себе парадигмы процедурного, объектно-ориентированного и общего программирования.

В этой книге мы будем использовать язык C++ и среду разработки Visual Studio, которые бесплатно предоставляются компанией Microsoft. Как мы объясним позже, *язык программирования, с которого мы начинаем, не имеет принципиального значения*, но все же необходимо использовать какой-либо язык программирования, и в данной книге мы выбрали C++. У этой книги также есть «зеркальное отражение» на других языках программирования, таких как C#, Java, Python и JavaScript.

Книга на других языках программирования: C#, Java, JavaScript, Python, PHP

Эта книга по программированию для начинающих доступна на нескольких языках программирования (или находится в процессе адаптации для них):

- Основы программирования на языке C#
- Основы программирования на языке Java
- Основы программирования на JavaScript
- Основы программирования на языке Python
- Основы программирования на PHP

Программирование изучается с помощью написания, а не чтения!

Если кто-то думает, что, прочитав книгу, научится программировать без написания кода и решения задач, то он, безусловно, заблуждается. Программирование осваивается с помощью постоянной

практики, путем ежедневного написания кода и решения сотен, а то и тысяч задач, серьезно и упорно, в течение многих лет.

Приходится решать проблемы, возиться, что-то исправлять, искать решения и информацию в Интернете, пробовать, экспериментировать, находить лучшие решения, привыкать к коду, синтаксису, языку программирования, среде разработки, проводить поиск ошибок и отладку неработающего кода, учиться обдумыванию задач, алгоритмическому мышлению, разбиению задач на шаги и реализации каждого шага, каждый день набираться опыта и повышать свою квалификацию, ведь научиться писать код — это только первый шаг на пути к профессии «инженер-программист». Вам предстоит еще многому научиться, действительно многому!

Как минимум мы рекомендуем читателю решить все приведенные в книге примеры, «поиграть» с ними, модифицировать их и протестировать. Однако еще более важными, чем примеры, являются практические задачи, поскольку они развивают практические навыки.

Решите все задачи, приведенные в книге, потому что программирование познается с практикой! Задачи после каждой темы тщательно подобраны для глубокого охвата материала, а цель решения всех задач из всех пройденных тем состоит в том, чтобы дать исчерпывающие навыки написания кода для программирования начального уровня (что также является целью данной книги). Не зря на курсах SoftUni мы делаем упор на практику и решение задач, и на большинстве курсов написание кода в классе занимает более 70 % всего времени.

Решите все упражнения в книге. Иначе вы ничему не научитесь! Программирование изучается путем написания большого количества кода и решения тысяч задач!

Университет программного обеспечения (SoftUni)

Университет программного обеспечения (SoftUni, наш адрес — https://softuni.org/) — крупнейший образовательный центр для инженеров-программистов в Болгарии. Ежегодно через него проходят десятки тысяч студентов. SoftUni был основан в 2014 году

как продолжение большой работы доктора Светлина Накова по массовой подготовке квалифицированных специалистов в области программного обеспечения через реальное, современное и качественное образование, сочетающее фундаментальные знания с современными программными технологиями и большим количеством практики.

Университет SoftUni предоставляет возможность получить качественное образование, профессию, работу и степень бакалавра для программистов, инженеров-программистов и IT-специалистов. SoftUni добился огромного успеха в построении прочных отношений между образованием и промышленностью, сотрудничая с сотнями компаний — разработчиков программного обеспечения, предоставляя рабочие места и стажировки своим студентам, поставляя квалифицированных специалистов в индустрию программного обеспечения и непосредственно ориентируясь на потребности работодателей в процессе обучения.

Бесплатные курсы программирования в SoftUni

Компания SoftUni организует бесплатные курсы программирования для начинающих по всей Болгарии — очно и онлайн. Цель — чтобы каждый, кто интересуется программированием и технологиями, мог попробовать себя в программировании и понять, интересно ли это ему и хочет ли он всерьез заняться разработкой программного обеспечения.

Записаться на бесплатный курс по основам программирования можно на сайте SoftUni: https://softuni.bg/apply.

Бесплатные курсы по основам программирования в университете SoftUni призваны познакомить вас с основными конструкциями программирования из мира разработки программного обеспечения, которые вы сможете применить в любом языке программирования. Это работа с данными, переменными и выражениями, использование проверок, построение циклов, определение и вызов функций, а также другие приемы построения логики программы. Обучение очень практичное, то есть большое внимание уделяется практике, и вы получаете возможность применить полученные знания еще в процессе обучения.

Данная книга по программированию сопровождает бесплатные курсы программирования для начинающих в университете SoftUni и служит дополнительным учебным пособием для поддержки процесса обучения.

Судейская система для проверки заданий

Система SoftUni Judge (https://judge.softuni.bg) представляет собой автоматизированную веб-систему для проверки решений задач по программированию с помощью серии тестов. Передача и проверка задач происходят в реальном времени: вы отправляете решение и через несколько секунд получаете ответ, если оно правильное. За каждый успешно пройденный тест начисляются предусмотренные для него баллы. Если решение правильное, то вы получаете все баллы за задачу. В случае частично верного решения вы получаете часть баллов за задание. В случае полностью неверного решения вы получаете 0 баллов.

Все задачи, приведенные в книге, доступны для тестирования в системе SoftUni Judge, и мы настоятельно рекомендуем протестировать их после решения, чтобы понять, не упустили ли вы чего-нибудь и действительно ли ваше решение работает правильно, в соответствии с требованиями задачи.

Отметим также некоторые особенности системы SoftUni Judge.

- Для каждого задания система судейства сохраняет наибольшее количество набранных баллов. Если вы загрузите решение с неправильным кодом или с меньшим количеством баллов, чем у ранее представленного решения, система не снимет баллы.
- Вывод вашей программы сравнивается системой строго с ожидаемым результатом. Любой лишний символ, пропущенная запятая или пробел могут привести к 0 баллов на соответствующем тесте. Выходные данные, которые ожидает получить судейская система, описаны в условии каждой задачи, и ничего больше к ним добавлять не следует.

Пример: если для вывода требуется вывести число (например `25`), не выводите описательных сообщений типа `Результат: 25`, а выведите именно то, что требуется, то есть только число.

Доступ к системе SoftUni Judge можно получить в любое время на сайте: https://judge.softuni.bg.

- Для входа в систему используйте свою аутентификацию с сайта SoftUni: https://softuni.bg.
- Использование системы является бесплатным и не связано с обучением на курсах SoftUni.

Мы уверены, что уже после нескольких сданных задач вам понравится получать мгновенные отзывы о правильности выполненного вами решения, и судейская система станет вашим любимым помощником при изучении программирования.

Как стать программистом?

Уважаемые читатели, наверняка у многих из вас есть желание стать программистом, зарабатывать на жизнь разработкой программного обеспечения или работать в сфере информационных технологий. Именно поэтому мы подготовили краткое руководство «Как стать программистом», чтобы сориентировать вас на пути к этой желанной профессии.

Программистом (на уровне начала работы в софтверной компании) становятся как минимум через 1–2 года упорного обучения и ежедневного написания кода, решения нескольких тысяч задач по программированию, разработки нескольких серьезных практических проектов и приобретения большого опыта в написании кода и разработке программного обеспечения. Это не происходит за месяц или два! Профессия инженера-программиста требует большого объема знаний, подкрепленных многими и многими практическими действиями.

Существует четыре основных набора навыков, которыми должны обладать все программисты. Большинство из этих навыков стабильны во времени и не подвержены существенному влиянию эволюции конкретных технологий (которые постоянно меняются). Это те навыки, которыми обладает каждый хороший программист и к которым должен стремиться каждый новичок:

- написание кода (20 %)
- алгоритмическое мышление (30 %)
- фундаментальные знания о профессии (25 %)
- языки и технологии разработки (25 %)

Навык № 1 — написание кода (20 %)

Умение писать код составляет около 20 % минимальных навыков, необходимых программисту для начала работы в компании, занимающейся разработкой программного обеспечения. Умение писать код включает в себя следующие компоненты:

- работа с переменными, проверками, циклами
- использование функций, методов, классов и объектов
- работа с данными: массивы, списки, хеш-таблицы, строки

Навыком кодирования можно овладеть за несколько месяцев упорной учебы и практического решения задач путем ежедневного написания кода. В этой книге рассматривается только первый пункт навыка кодирования: работа с переменными, проверки и циклы. Остальное предстоит узнать на последующих тренингах, курсах и в книгах.

Эта книга (и основанные на ней курсы) — лишь начало долгого и серьезного обучения на пути к профессиональному программированию. Если вы не овладеете в совершенстве материалом, изложенным в этой книге, то не сможете стать программистом. Вы упустите фундаментальные основы, и с каждым разом постигать основы программирования будет все труднее. Поэтому уделите достаточно внимания основам: решайте трудные задачи и пишите много кода в течение нескольких месяцев, пока не научитесь с легкостью решать все задачи из этой книги. Затем двигайтесь дальше.

Мы специально отмечаем, что язык программирования не является обязательным условием умения писать код. Либо вы умеете, либо нет. Если вы умеете «кодить» на C++, то вы легко научитесь писать код на Java, C# или любом другом языке. Именно поэтому навыки кодирования достаточно серьезно изучаются на курсах начинающих программистов в SoftUni (см. программу на https://softuni.bg/curriculum), и с них начинается любая книга по программированию для начинающих, в том числе и наша.

Навык № 2 — алгоритмическое мышление (30 %)

Алгоритмическое (логическое, инженерное, математическое, абстрактное) мышление составляет около 30 % минимальных навыков программиста, необходимых для начала профессиональ-

ной деятельности. Алгоритмическое мышление — это способность разложить задачу на логическую последовательность шагов (алгоритм) и найти решение для каждого шага, а затем собрать все шаги в рабочее решение исходной задачи. Это самый важный навык программиста.

Как сформировать алгоритмическое мышление?

- Алгоритмическое мышление развивается путем решения множества (1000+) задач по программированию, причем как можно более разнообразных. Это и есть рецепт: решить тысячи практических задач, придумать для них алгоритм, реализовать алгоритм, попутно находя и исправляя ошибки.
- Физика, математика и/или близкие к ним естественные науки полезны, но не обязательны! Людям с инженерными и техническими наклонностями обычно легче научиться логически мыслить, поскольку у них уже сформированы навыки решения задач, пусть и не алгоритмические.
- Умение решать задачи программирования (что требует алгоритмического мышления) чрезвычайно важно для программиста. Многие компании проверяют этот навык сразу на собеседованиях.

Эта книга развивает начальный уровень алгоритмического мышления, но этого недостаточно для того, чтобы стать хорошим программистом. Чтобы стать квалифицированным специалистом, необходимо добавить навыки логического мышления и решения задач, выходящие за рамки данной книги, такие как работа со структурами данных (массивы, списки, матрицы, хеш-таблицы, древовидные структуры) и базовыми алгоритмами (поиск, сортировка, обход древовидных структур, рекурсия и др.).

Навыки алгоритмического мышления серьезно развиваются на вводных курсах для инженеров-программистов в SoftUni (см. учебный план на https://softuni.bg/curriculum), а также на специализированных курсах по структурам данных (https://softuni.bg/opencourses/data-structures) и алгоритмам (https://softuni.bg/opencourses/algorithms).

Как вы уже догадались, язык программирования не имеет никакого отношения к развитию алгоритмического мышления. Логическое

мышление универсально, оно даже не связано только с программированием. Именно благодаря высокоразвитому логическому мышлению программисты считаются достаточно умными, а человек, не обладающий таким мышлением, не может стать программистом.

Навык № 3 — фундаментальные знания о профессии (25 %)

Фундаментальные знания и навыки в области программирования, разработки программного обеспечения, программной инженерии и информатики составляют около 25 % от минимальной квалификации программиста начального уровня. Вот наиболее важные из этих знаний и навыков:

- основные математические понятия, связанные с программированием: системы координат, векторы и матрицы, дискретные и недискретные математические функции, конечные автоматы и автоматы состояний, понятия из комбинаторики и статистики, сложность алгоритмов, математическое моделирование и др.;
- навыки программирования — написание кода, работа с данными, использование условных операторов и циклов, работа с массивами, списками и ассоциативными массивами, строками и обработкой текста, работа с потоками и файлами, использование API, работа с отладчиками и др.;
- структуры данных и алгоритмы — списки, деревья, хеш-таблицы, графы, поиск, сортировка, рекурсия, обход древовидных структур и т. д.;
- объектно-ориентированное программирование (ООП) — работа с классами, объектами, наследование, полиморфизм, абстракция, интерфейсы, инкапсуляция данных, управление исключениями, шаблоны проектирования;
- функциональное программирование (ФП) — работа с лямбда-функциями, функциями высшего порядка, функциями, возвращающими в качестве результата функцию, закрытие состояния в функции (closure) и др.;
- базы данных — реляционные и нереляционные базы данных, моделирование баз данных (таблицы и связи между

ними), язык запросов SQL, объектно-реляционные технологии доступа к данным (ORM), транзакционность и управление транзакциями;

- сетевое программирование — сетевые протоколы, сетевое взаимодействие, TCP/IP, концепции, средства и технологии построения компьютерных сетей;
- клиент-серверное взаимодействие, связь между системами, back-end технологии, технологии front-end, архитектуры MVC;
- технологии разработки на стороне сервера (back-end) — архитектура веб-сервера, протокол HTTP, архитектура MVC, архитектура REST, фреймворки веб-разработки, шаблонизаторы;
- фронтенд-технологии (клиентская разработка) — HTML, CSS, JS, HTTP, DOM, AJAX, связь с back-end, вызов REST API, фронтенд-фреймворки, базовые концепции дизайна и UX (user experience);
- мобильные технологии — мобильные приложения, разработка под Android и iOS, мобильный пользовательский интерфейс (UI), вызов серверной логики;
- встраиваемые системы — микроконтроллеры, управление цифровыми и аналоговыми входами и выходами, доступ к датчикам, управление периферийными устройствами;
- операционные системы — работа с операционными системами (Linux, Windows и др.), установка, настройка и основы системного администрирования, работа с процессами, памятью, файловой системой, пользователями, многозадачность, виртуализация и контейнеры;
- параллельное программирование и асинхронность — управление потоками, асинхронные задачи, общие ресурсы и синхронизация доступа;
- программная инженерия — системы контроля исходных текстов, управление разработкой, планирование и управление задачами, методологии разработки ПО, требования

к ПО и прототипы, проектирование ПО, архитектуры ПО, документация ПО;

- тестирование программного обеспечения — модульное тестирование, разработка, управляемая тестами, QA-инжиниринг, создание отчетов об ошибках и баг-трекеров, автоматизация тестирования, процессы сборки и непрерывной интеграции.

На этот раз следует уточнить, что для овладения всеми этими навыками язык программирования неважен. Они накапливаются постепенно, в течение многих лет практики в профессии. Некоторые знания являются фундаментальными и могут быть усвоены теоретически, но для их полного понимания и глубокой реализации требуются годы практики.

Фундаментальные знания и навыки в области программирования, разработки программного обеспечения, программной инженерии и информатики преподаются в рамках комплексной программы «Инженер-программист» в SoftUni (см. https://softuni.bg/curriculum), а также на ряде факультативных курсов (см. https://softuni.bg/trainings/opencourses). Работа с разнообразными программными библиотеками, интерфейсами прикладного программирования (API), технологическими фреймворками (Frameworks), программными технологиями и их взаимодействием постепенно формирует эти знания и навыки, поэтому не стоит ожидать, что они будут получены в результате изучения одного курса, книги или проекта.

Для начала работы программистом, как правило, достаточно лишь базовых знаний в перечисленных областях, а углубление происходит в процессе работы в соответствии с технологиями и средствами разработки, используемыми в конкретной компании и коллективе.

Навык № 4 — языки программирования и программные технологии (25 %)

Языки программирования и технологии разработки программного обеспечения составляют около 25 % минимальных навыков программиста. Они занимают наибольшую часть в программах обучения, но очень быстро меняются с течением времени. Если по-

смотреть на объявления о вакансиях в сфере программного обеспечения, то там часто упоминаются всевозможные «жужжащие» слова (например, перечисленные ниже), но на самом деле в объявлениях подразумеваются первые три навыка: умение кодировать, алгоритмически мыслить и владеть основами информатики и программной инженерии.

Для этих чисто технологических навыков язык программирования уже имеет значение.

Но обратите внимание: только в 25 % требований язык программирования имеет значение! Для остальных 75 % требуемых навыков язык программирования неважен, и эти навыки устойчивы во времени и переносимы между различными языками и технологиями.

Вот некоторые широко используемые языки и технологии (стеки разработки программного обеспечения), востребованные софтверными компаниями:

- C# + ООП + ФП + классы .NET + базы данных SQL Server + Entity Framework (EF) + ASP.NET MVC + HTTPS + HTML + CSS + JS + DOM + jQuery
- Java + классы Java API + ООП + ФП + базы данных + MySQL + HTTPS + веб-разработка + HTML + CSS + JS + DOM + jQuery + JSP/Servlets + Spring MVC или Java EE/JSF
- PHP + ООП + базы данных + MySQL + HTTPS + веб-разработка + HTML + CSS + JS + DOM + jQuery + Laravel / Symfony / другой MVC-фреймворк для PHP
- JavaScript (JS) + ООП + ФП + базы данных + MongoDB или MySQL + HTTPS + веб-программирование + HTML + CSS + JS + DOM + jQuery + Node.js + Express + Angular или React
- Python + ООП + ФП + базы данных + MongoDB или MySQL + HTTPS + веб-программирование + HTML + CSS + JS + DOM + jQuery + Django
- C++ + ООП + STL + Boost + нативная разработка + базы данных + HTTPS + другие языки
- Swift + macOS + iOS + Cocoa + Cocoa Touch + XCode + HTTPS + REST + другие языки

Если перечисленные слова и аббревиатуры кажутся вам страшными и абсолютно непонятными, значит, вы находитесь в самом начале своей карьеры, и до получения профессии «инженер-программист» вам предстоит учиться не один год. Не волнуйтесь, каждый программист проходит через один или несколько технологических стеков и вынужден изучать набор взаимосвязанных технологий, но в основе всего этого лежит умение писать логику программы («кодить»), которое подробно изучается в этой книге, и умение алгоритмически мыслить (решать задачи программирования). Без них никак нельзя!

Язык программирования не имеет значения!

Как уже было сказано, разница между языками программирования, а точнее, между навыками программистов на разных языках и технологиях, составляет порядка 10–20 % навыков.

- Все программисты обладают примерно 80–90 % одних и тех же навыков, независимо от языка! Это навыки программирования и разработки программного обеспечения, которые очень похожи для всех языков программирования и технологий разработки.
- Чем больше языков и технологий вы освоите, тем быстрее вы выучите новые и тем меньше будете чувствовать разницу между ними.

На самом деле язык программирования практически не имеет значения, нужно просто научиться программировать, а это начинается с кодирования (данная книга), продолжается более сложными концепциями программирования (такими как структуры данных, алгоритмы, ООП и ФП) и включает в себя освоение фундаментальных знаний и навыков разработки программного обеспечения, программной инженерии и компьютерных наук.

Только в конце, когда вы начнете использовать конкретные технологии в программном проекте, вам понадобятся определенный язык программирования и знания о конкретных библиотеках программирования (API), фреймворках и программных технологиях (технологии front-end UI, back-end, ORM и т. д.). Расслабьтесь, вы их изучите, все программисты их изучают, но сначала нужно изучить основы программирования, и изучить их хорошо.

В книге используется язык C++, но это не принципиально, он может быть заменен на C#, JavaScript, Python, PHP, Java, Ruby, Swift, Go, Kotlin или другой язык. Чтобы освоить профессию «разработчик программного обеспечения», необходимо научиться «кодить» (20 %), научиться алгоритмически мыслить и решать задачи (30 %), обладать фундаментальными знаниями в области программирования и информатики (25 %), владеть конкретным языком программирования и сопутствующими технологиями (25 %). Наберитесь терпения, через год-два все это можно освоить на хорошем начальном уровне при условии серьезного подхода.

Книга в помощь преподавателям

Если вы преподаете программирование, информатику или информационные технологии либо хотите преподавать программирование, то эта книга даст вам больше, чем просто хорошо структурированный учебный материал с большим количеством примеров и задач.

Вместе с книгой вы получите качественный учебный материал для школы, адаптированный к ее требованиям.

- Учебные презентации (слайды PowerPoint) по каждой теме, адаптированные к 45-минутным урокам в школах — бесплатно.
- Хорошо разработанные задачи для аудиторных занятий и домашних заданий с подробными условиями и примерами входных и выходных данных — бесплатно.
- Автоматизированную систему проверки заданий и домашних работ (Online Judge System), которой студенты могут пользоваться также бесплатно.
- Видеоуроки с методическими указаниями из бесплатного курса для преподавателей программирования, который регулярно проводит SoftUni.

Все эти бесплатные учебные ресурсы, а также учебные материалы для целого ряда курсов по программированию и программным технологиям можно найти на сайте университета SoftUni. Скачать их можно здесь: http://softuni.foundation/projects/applied-software-developer-profession.

История этой книги

Основной движущей силой и руководителем проекта по созданию этой книги по программированию с открытым кодом для начинающих является доктор Светлин Наков (http://www.nakov.com/). Он является главным идеологом и создателем учебного контента курса «Основы программирования» (https://softuni.bg/courses/programming-basics) в университете SoftUni, который положен в основу книги.

Все началось с массовых бесплатных курсов по основам программирования, которые проводятся по всей Болгарии с 2014 года, когда была запущена инициатива SoftUni. Вначале эти курсы были более масштабными и включали больше теории, но в 2016 году доктор Светлин Наков полностью переработал их, обновил, упростил и сделал основной упор на практику. Так было создано основное учебное наполнение этой книги.

Бесплатные тренинги SoftUni по основам программирования — пожалуй, самые масштабные из всех, которые когда-либо проводились в Болгарии. К 2018 году курс «Основы программирования» прошел более 200 раз почти в 40 городах Болгарии очно и многократно онлайн, в нем приняли участие более 70 000 человек.

Вполне естественно, что возникла необходимость написать учебник для десятков тысяч слушателей курсов по программированию для начинающих от SoftUni. Исходя из принципа свободного программного обеспечения и свободных знаний, Светлин Наков возглавил команду добровольцев и продвинул этот проект с открытым исходным кодом, сначала для создания учебника по основам программирования на языке C#, а затем и на других языках программирования.

Проект является частью деятельности Software University Foundation (http://softuni.foundation/) по созданию и распространению открытого образовательного контента для подготовки инженеров-программистов и IT-специалистов.

Авторский коллектив

Эта книга, «Основы программирования на языке C++», была подготовлена широким авторским коллективом добровольцев, отдавших свое время для того, чтобы дать вам эти систематизированные

знания и руководство по началу работы в программировании. Список всех авторов и редакторов:

Андрей Царев, Божидар Колев, Борис Горанов, Венцислав Петров, Георгий Лацев, Георгий Шавов, Денис Миланов, Димитар Кацарский, Димо Димов, Димо Чанев, Зорница Йоханова, Игнес Симеонова, Йордан Добрев, Любомир Господинов, Мирела Дамьянова, Николай Костов, Пламен Динев, Цветан Иванов.

В основу книги положена ее оригинальная версия на языке C# (https://csharp-book.softuni.bg/), которая была разработана широким авторским коллективом, внесшим свой вклад в создание данной книги:

Александр Крастев, Александр Лазаров, Ангел Димитриев, Васко Викторов, Венцислав Петров, Даниил Цветков, Димитар Татарский, Димо Димов, Диан Тончев, Елена Роглева, Живко Недялков, Джульетта Атанасова, Захария Пехливанова, Ивелин Кирилов, Искра Николова, Калин Примов, Кристиан Памидов, Любослав Любенов, Николай Банкин, Николай Димов, Павлин Петков, Петр Иванов, Росица Ненова, Руслан Филиппов, Светлин Наков, Стефка Васильева, Теодор Куртев, Тоньо Желев, Кристиан Христов, Христо Христов, Цветан Илиев, Юлиан Линев, Яница Валева.

Книга была написана в период с декабря 2018 по март 2019 года.

Официальный сайт книги

Эта книга по основам программирования на языке C++ для начинающих доступна для свободного использования в Интернете по адресу:

https://cpp-book.softuni.bg

Это официальный сайт книги, на котором будет выложена последняя версия. Книга является «зеркальным отражением» книг по другим языкам программирования, перечисленным на сайте.

Форум для ваших вопросов

Задавайте свои вопросы об этой книге на форуме SoftUni:

https://softuni.bg/forum

На этом дискуссионном форуме вы сможете бесплатно получить адекватные ответы на любые вопросы по учебному материалу данной книги, а также на другие вопросы по программированию. Сообщество SoftUni для начинающих программистов настолько велико, что ответ на вопрос обычно приходит в течение нескольких минут. Кроме того, на ваши вопросы постоянно отвечают тьюторы, ассистенты и менторы SoftUni.

Благодаря большому количеству студентов, изучающих этот учебник, на форуме можно найти решение практически любой задачи, которым поделился кто-то из коллег. Тысячи студентов до вас уже решали те же самые задачи, поэтому если вы «застряли», поищите решение на форуме. Хотя задачи курса «Основы программирования» время от времени меняются, на SoftUni всегда приветствуется обмен мнениями, поэтому вы легко найдете решения и рекомендации по всем задачам.

Если у вас есть конкретный вопрос, например, почему не запускается программа, над которой вы ломали голову несколько часов, задайте его на форуме, и вы получите ответ. Вы будете удивлены, насколько дружелюбны и отзывчивы люди на форуме SoftUni.

Лицензия и распространение

Книга распространяется бесплатно в электронном виде под открытой лицензией CC-BY-SA (https://creativecommons.org/licenses/by-sa/4.0).

Исходный код книги можно найти в репозитории GitHub проекта:

https://github.com/SoftUni/Programming-Basics-Book-CPP-BG.

Сообщение об ошибках

Если вы обнаружите в книге какие-либо ошибки, неточности или недочеты, вы можете сообщить о них на официальном трекере проекта:

https://github.com/SoftUni/Programming-Basics-Book-CPP-BG/issues.

Мы не обещаем, что будем исправлять все присланные вами ошибки, но у нас есть желание постоянно улучшать качество этой книги, поэтому сообщения о неоспоримых ошибках и любые разумные предложения будут рассмотрены.

Приятного чтения!

И не забывайте писать код в больших количествах, пробовать примеры из каждой темы и особенно решать задачи из упражнений. Просто читая, вы не научитесь кодить, поэтому решайте задачи упорно!

Глава 1.
Первые шаги в программировании

В этой главе мы выясним, что такое программирование по своей сути. Мы введем понятие о языках программирования, рассмотрим среды разработки программного обеспечения (IDE) и работу с ними, в частности Visual Studio. Мы напишем и запустим нашу первую программу на языке программирования C++, а затем отработаем несколько задач по созданию консольных программ. Мы научимся проверять решения задач из этой книги на корректность в автоматической судейской системе SoftUni, а также узнаем о типичных ошибках, которые часто допускаются при написании кода, и о том, как их избежать.

Что означает «программировать»?

Программирование означает подачу компьютеру команд, например, *«воспроизвести звук»*, *«напечатать что-то на экране»* или *«перемножить два числа»*. Если команд несколько и они следуют подряд, то они называются компьютерной программой. Текст компьютерных программ называется программным кодом (или исходным кодом, или сокращенно кодом).

Компьютерные программы

Компьютерные программы представляют собой последовательность команд, написанных на заранее выбранном языке программирования, например C++, Java, JavaScript, Python, Ruby, PHP, C, C#, Swift, Go или другом языке. Для написания команд нам необходимо знать синтаксис и семантику языка, с которым мы будем работать, в нашем случае C++. Поэтому в данной книге мы познакомимся с синтаксисом и семантикой языка C++ и программированием в целом, шаг за шагом изучая, как писать код, от простых конструкций программирования к более сложным.

Алгоритмы

Компьютерные программы обычно выполняют алгоритм. Алгоритм — это последовательность шагов, необходимых для выполнения определенной работы и получения некоторого ожидаемого результата, своего рода «рецепт». Например, если мы жарим яичницу, то выполняем некоторый рецепт (алгоритм): разогреваем масло на сковороде, разбиваем яйца, ждем, пока они поджарятся, снимаем с огня. Аналогично в программировании компьютерные программы выполняют алгоритмы — серии команд, необходимых для выполнения определенной работы. Например, чтобы расположить последовательность чисел в порядке возрастания, требуется алгоритм, скажем, найти наименьшее число и вывести его, из оставшихся чисел снова найти наименьшее и вывести его, и так повторять, пока числа не закончатся.

Для удобства создания программ, написания программного кода (команд), выполнения программ и для других операций, связанных с программированием, нам также необходима среда разработки, например Visual Studio.

Языки программирования, компиляторы, интерпретаторы и среды разработки

Язык программирования — это искусственный язык (синтаксис выражений), предназначенный для задания команд, чтобы компьютер прочитал, обработал и выполнил их. С помощью языков программирования мы пишем последовательности команд (программы), которые указывают компьютеру, что делать. Выполнение компьютерных программ может быть реализовано с помощью компилятора или интерпретатора.

Компилятор переводит код с языка программирования в машинный код, выбирая для каждого оператора (команды) кода соответствующий, заранее подготовленный фрагмент машинного кода и проверяя при этом текст программы на наличие ошибок. В совокупности скомпилированные фрагменты составляют программу в машинном коде, которую ожидает увидеть микропроцессор компьютера. После компиляции программы она может быть непосредственно выполнена микропроцессором совместно с операционной системой.

В компилируемых языках программирования компиляция программы перед ее выполнением является обязательной, а синтакси-

ческие ошибки (неправильно указанные инструкции) выявляются в процессе компиляции. С компилятором работают такие языки, как C++, C#, Java, Swift и Go.

Некоторые языки программирования не используют компилятор, а интерпретируются непосредственно специализированным программным обеспечением, называемым интерпретатором. Интерпретатор является «исполнителем» программ, написанных на каком-либо языке программирования. Он выполняет команды программы одну за другой, «понимая» не только отдельные команды и последовательности команд, но и другие языковые конструкции (проверки, повторения, функции и т. д.). Такие языки, как PHP, Python и JavaScript, работают с интерпретатором и запускаются без компиляции. Из-за отсутствия предварительной компиляции ошибки в интерпретируемых языках обнаруживаются не заранее, а во время выполнения, после начала работы программы.

Интегрированная среда разработки (IDE) — это набор традиционных инструментов для разработки программных приложений. В среде разработки мы пишем код, компилируем и выполняем программы. Среда разработки включает в себя текстовый редактор для написания кода, язык программирования, компилятор или интерпретатор и среду для выполнения программ, отладчик для отслеживания программы и поиска ошибок, средства проектирования пользовательского интерфейса и другие инструменты и надстройки.

Среды разработки удобны тем, что они объединяют в себе все необходимое для разработки программы, не выходя из среды. Если не использовать среду разработки, то придется писать код в текстовом редакторе, компилировать консольной командой, выполнять другой консольной командой, писать дополнительные команды, когда это необходимо, и терять время. Именно поэтому большинство программистов используют в своей повседневной работе среды разработки.

Для программирования на языке C++ наиболее распространенной средой разработки является Visual Studio, которая разрабатывается и бесплатно распространяется компанией Microsoft и может быть загружена по адресу: https://www.visualstudio.com/downloads. Другой распространенной средой разработки является Code::Blocks (https://www.codeblocks.org/downloads). В этой книге мы будем использовать среду разработки Visual Studio.

Языки низкого и высокого уровня, среды выполнения

Программа — это, по сути, набор инструкций, которые предписывают компьютеру выполнить определенную задачу. Они вводятся программистом и безоговорочно выполняются машиной.

Существуют различные типы языков программирования. Языки самого низкого уровня могут использоваться для написания собственно инструкций, управляющих процессором, например, язык ассемблера. Языки более высокого уровня, такие как C и C++, могут использоваться для создания операционных систем, драйверов для управления аппаратным обеспечением (например, драйвера видеокарты), веб-браузеров, компиляторов, графических и игровых движков, а также других системных компонентов и программ. Еще более высокоуровневые языки, такие как C#, Python и JavaScript, используются для создания прикладных программ, например, программы для чтения почты или чата.

Языки низкого уровня управляют аппаратной частью напрямую и требуют больших усилий и огромного количества команд для выполнения единицы работы. Языки более высокого уровня требуют меньше кода на единицу работы, но не имеют прямого доступа к аппаратной части. Они используются для разработки прикладного программного обеспечения, такого как веб-приложения и мобильные приложения.

Большинство программ, которыми мы пользуемся ежедневно, например, музыкальный проигрыватель, видеоплеер, GPS-программа и т. д., написаны на языках прикладного программирования высокого уровня, таких как C++, Java, Python, C#, JavaScript, PHP и др.

Язык C++ является компилируемым, то есть мы пишем команды, которые перед выполнением компилируются в машинный код. Именно эти команды с помощью утилиты (компилятора) преобразуются в файл, который можно выполнить (исполняемый файл). Для написания кода на таком языке, как C++, необходим текстовый редактор или среда разработки.

Компьютерные программы — компиляция и исполнение

Как мы уже говорили, программа — это последовательность инструкций, другими словами, она описывает ряд вычислений, про-

верок, итераций и любых других подобных операций, направленных на достижение некоторого результата.

Программа записывается в текстовом формате, а сам текст программы называется исходным кодом. Он компилируется в исполняемый файл (например, Program.cpp компилируется в Program.exe).

Процесс компиляции кода перед выполнением присущ только компилируемым языкам, таким как C++, Java, C# и др. В скриптовых и интерпретируемых языках, таких как JavaScript, Python и PHP, исходный код последовательно выполняется интерпретатором.

Компьютерные программы — примеры

Начнем с очень простого примера короткой программы на языке C++.

Пример: программа, выводящая текст на экран

Наша первая программа будет состоять из одной команды на языке C++, которая выводит текст «Hello»:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello";

    return 0;
}
```

Вы можете протестировать пример в Интернете: https://repl.it/@nakov/CPlusPlus-Hello.

Как выполнить эту команду и увидеть выведенный на экран текст, мы узнаем чуть позже, а пока давайте просто посмотрим, что такое команды в программировании. Рассмотрим еще несколько примеров.

Пример: программа, выводящая английский алфавит

Мы можем усложнить предыдущую программу, назначив команды вывода текста для многократного выполнения в цикле:

```
for (char ch = 'A'; ch <= 'Z'; ch++) {
    cout << ch << endl;
}
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/EnglishAlphabet.

В приведенном примере мы заставляем компьютер выводить множество символов один за другим, на новых строках, начиная с A и заканчивая Z. Результатом работы программы является вывод на экран всего английского алфавита.

О том, как работают повторы (циклы) в программировании, мы узнаем в соответствующей главе, а пока предположим, что мы просто многократно повторяем какую-либо команду.

Пример: программа, конвертирующая левы в евро

Рассмотрим еще одну простую программу, которая считывает у пользователя некоторую сумму в левах (целое число), переводит ее в евро (делит на курс евро) и выводит результат. Эта программа состоит из четырех последовательных команд:

```
double leva;
cin >> leva;
double euro = leva / 1.95583;
cout << euro << endl;
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/LevaToEuro.

Мы рассмотрели три примера компьютерных программ: одиночная команда, серия команд в цикле и последовательность из четырех команд. Теперь перейдем к более интересному: как писать собственные программы на языке C++, как их компилировать и исполнять.

Как написать консольную программу?

Рассмотрим шаги по созданию и выполнению компьютерной программы, которая считывает и записывает свои данные в текстовую консоль (окно ввода и вывода текста) и из нее. Такие программы называются консольными. Однако перед этим необходимо установить и подготовить среду разработки, в которой мы будем писать и выполнять программы на языке C++, описанные в этой книге и упражнениях к ней.

Среда разработки (IDE)

Как уже говорилось, для программирования нам необходима среда разработки — интегрированная среда разработки (IDE). Это фактически редактор программ, в котором мы пишем код программы и можем его выполнить, увидеть ошибки, исправить их и запустить программу снова.

- Для программирования на языке C++ мы используем среду Visual Studio для операционной системы Windows и Eclipse для Linux или Mac OS X.
- Если вы программируете на языке Java, то подходящими средами являются IntelliJ IDEA, Eclipse или NetBeans.
- Если мы собираемся писать на языке Python, то можно воспользоваться средой PyCharm.

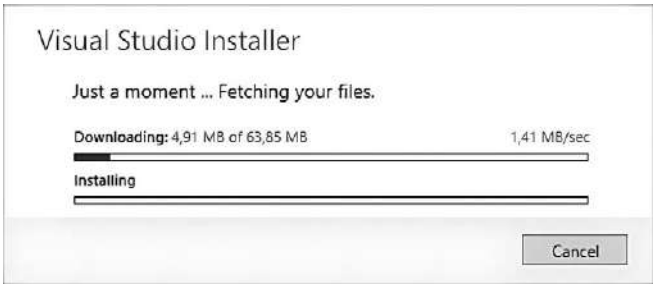
Установка Visual Studio Community

Начнем с установки интегрированной среды Microsoft Visual Studio Community.

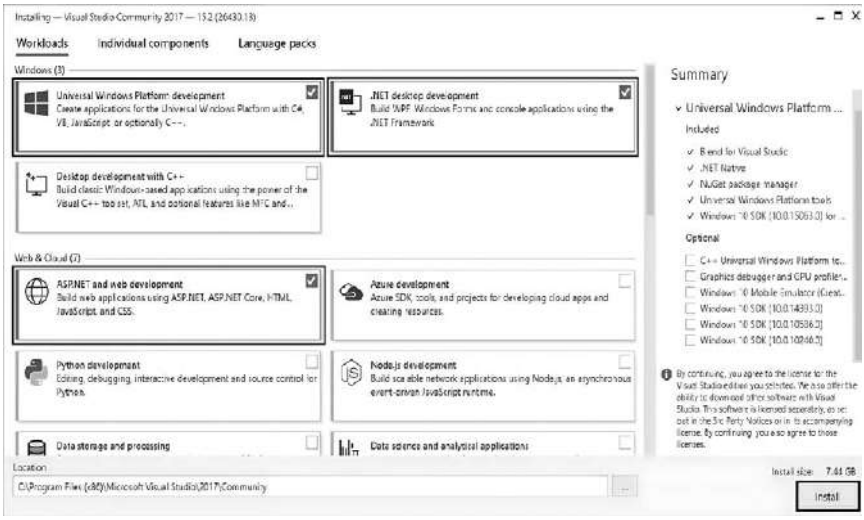
Community-версия Visual Studio (VS) распространяется бесплатно компанией Microsoft и может быть загружена по адресу: https://www.visualstudio.com/vs/community. Установка типична для Windows: нажимаем [Next], [Next] и [Finish], но важно включить компонент «Десктоп-разработка с использованием C++». Остальные параметры установки нам менять не нужно.

В следующих строках подробно описаны шаги установки Visual Studio (версия Community 2017). После загрузки установочного файла и его запуска появляется следующее окно:

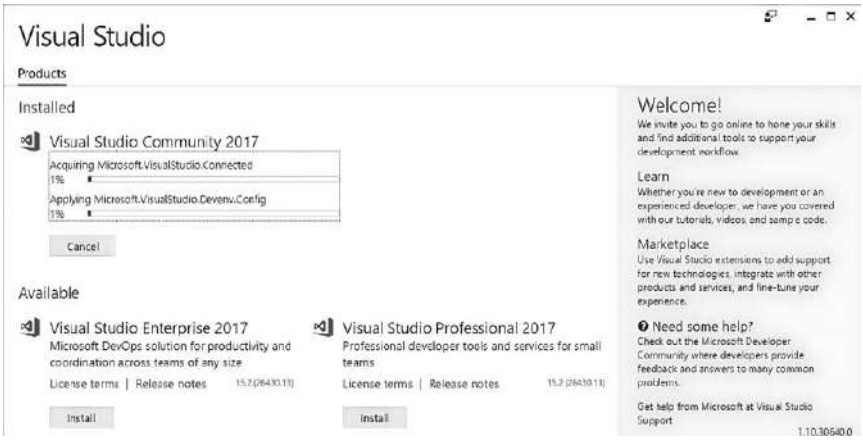
Нажмите кнопку [Continue], после чего появится окно, показанное ниже:



Загружается окно панели установки Visual Studio. Установите флажок [Desktop development with C++], затем нажмите [Install]:



Начнется установка Visual Studio, и на экране появится окно, подобное показанному ниже:



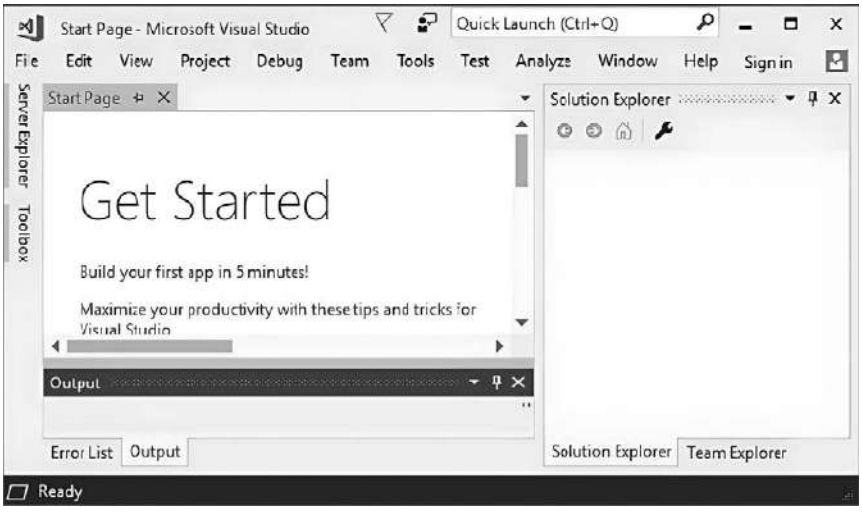
После установки Visual Studio система попросит перезагрузить компьютер перед началом работы. Мы можем нажать кнопку [Not now], если не хотим перезагружать компьютер сразу, но мы обязательно должны это сделать перед использованием Visual Studio. Если же мы готовы перезагрузиться немедленно, то нажимаем кнопку [Restart]. После этого компьютер перезагрузится. Когда он снова включится, мы запустим Visual Studio. Если мы устанавливаем более старую или другую версию, перезагрузка может не потребоваться. Тогда для запуска Visual Studio достаточно нажать кнопку [Launch]:

После запуска VS появляется окно, подобное приведенному ниже. В нем мы можем выбрать, входить ли нам в систему под учетной записью Microsoft. Пока мы хотим работать без входа в систему под учетной записью Microsoft, поэтому выбираем вариант [Not now, maybe later].

Позже, если у вас есть такая учетная запись, вы сможете войти в систему, а если ее нет и вы испытываете затруднения при ее создании, то вы можете написать на форуме SoftUni: https://softuni.bg/forum.

Следующим шагом является выбор цветовой темы (Blue, Dark и Light), в которой будет отображаться Visual Studio. Здесь выбор полностью зависит от предпочтений пользователя, и не имеет значения, какой вариант будет выбран.

Последнее, что нам нужно сделать, это нажать кнопку [Start Visual Studio], после чего загрузится начальное представление Visual Studio Community:



Старые версии Visual Studio

Можно использовать и старые версии Visual Studio (например, версии 2015, или 2013, или даже 2010 либо 2005), но это нежелательно, так как в них отсутствуют некоторые новые возможности разработки, и нет гарантии, что все примеры, приведенные в книге, будут работать.

Онлайн-среды разработки

Существуют также альтернативные среды разработки в режиме онлайн, непосредственно в браузере. Эти среды не очень удобны, но если у вас нет другого выхода, вы можете начать обучение с них, установив Visual Studio позже. Вот некоторые ссылки:

- Для языка C++ можно использовать: https://cpp.sh.

- Для языка C# сайт .NET Fiddle позволяет писать код и выполнять его в режиме онлайн: https://dotnetfiddle.net.
- Для Java мы можем использовать следующую онлайновую Java IDE: https://www.compilejava.net.
- Для JavaScript мы можем писать JS-код непосредственно в консоли браузера, нажав [F12].

Проектирование решений и проектов в Visual Studio

Прежде чем приступить к работе с Visual Studio, необходимо ознакомиться с понятиями Visual Studio Solution и Visual Studio Project, которые являются ее неотъемлемой частью.

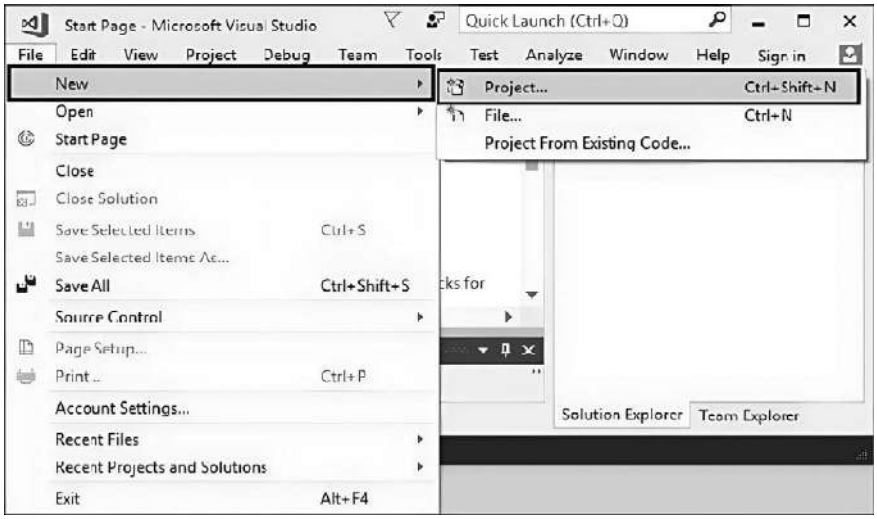
Visual Studio Project представляет собой «проект», над которым мы работаем. На первых порах это будут наши консольные программы, которые мы научимся писать, используя эту книгу, приведенные в ней ресурсы и курс «Основы программирования» на SoftUni. С более глубоким изучением, временем и практикой эти проекты перейдут в приложения, игры и другие разработки. Проект в VS логически группирует несколько файлов, составляющих приложение или компонент. Проект C++ содержит один или несколько исходных файлов C++, файлы конфигурации и другие ресурсы. Каждый исходный файл C++ содержит одно или несколько определений типов (классов или других определений). Классы содержат методы (действия), которые состоят из последовательностей команд. Это выглядит сложно, но в больших проектах такая структура очень удобна и позволяет хорошо организовать рабочие файлы.

Visual Studio Solution — это контейнер (рабочее решение), в котором логически объединены несколько проектов. Цель объединения этих VS Projects состоит в том, чтобы код любого из проектов мог взаимодействовать с кодом других VS Projects, чтобы приложение работало корректно. Если разрабатываемый программный продукт или сервис большой, то он строится как VS Solution, а Solution разбивается на проекты (VS Projects), и в каждом проекте есть папки с исходными текстами. Такая иерархическая организация очень удобна для более серьезных проектов (скажем, свыше 50 000 строк кода).

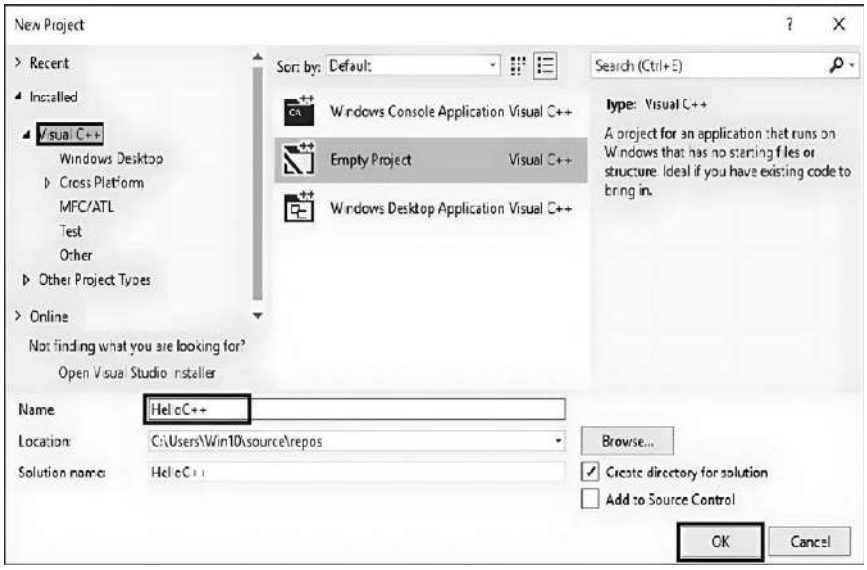
Для небольших проектов VS Solutions и VS Projects больше усложняют работу, чем помогают, но к этому быстро привыкаешь.

Пример: создание консольной программы «Hello C++»

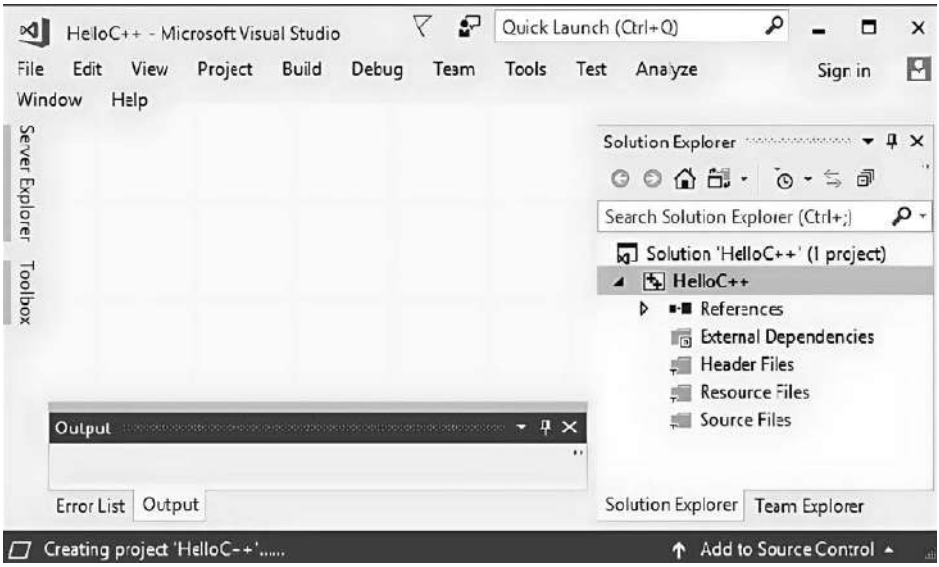
Вернемся к нашей консольной программе. У нас уже есть Visual Studio, и мы можем запустить его. Затем создайте новый консольный проект: [File] → [New] → [Project...]:



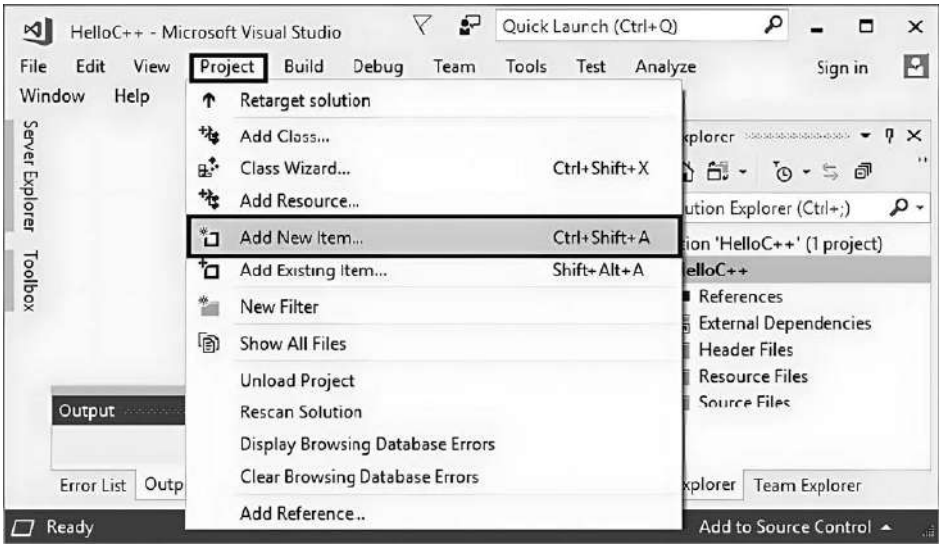
Откроется окно, которое будет выглядеть так, как показано на рисунке ниже. Нам необходимо выбрать [Visual C++] → [Empty Project]. После этого зададим осмысленное имя нашему проекту, например, «HelloC++»:



После этого мы нажмем [OK], и Visual Studio создаст для нас VS Solution с пустым проектом на языке C++:



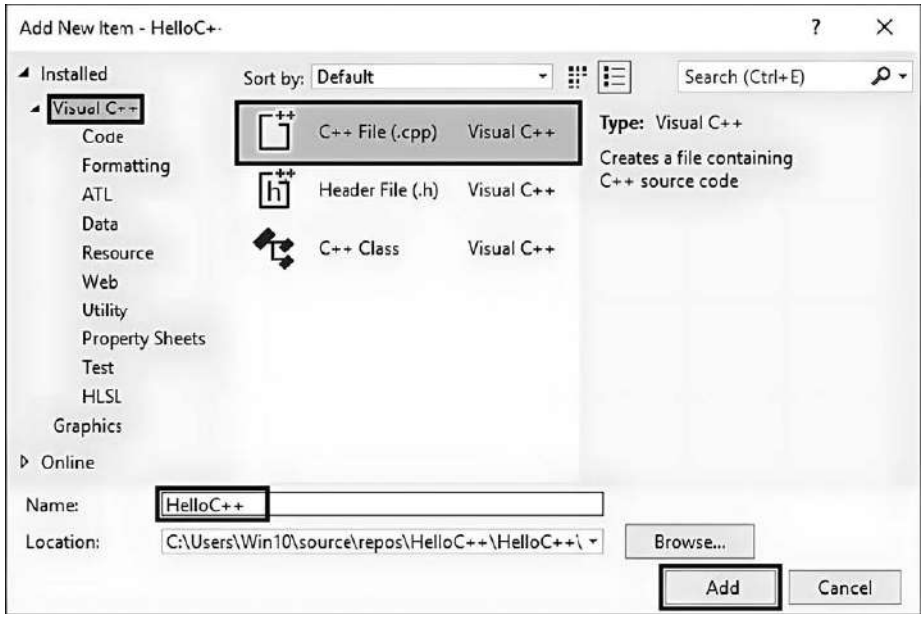
Чтобы добавить пустую программу на языке C++ в созданный проект, необходимо выбрать [Проект] → [Добавить новый элемент...]:



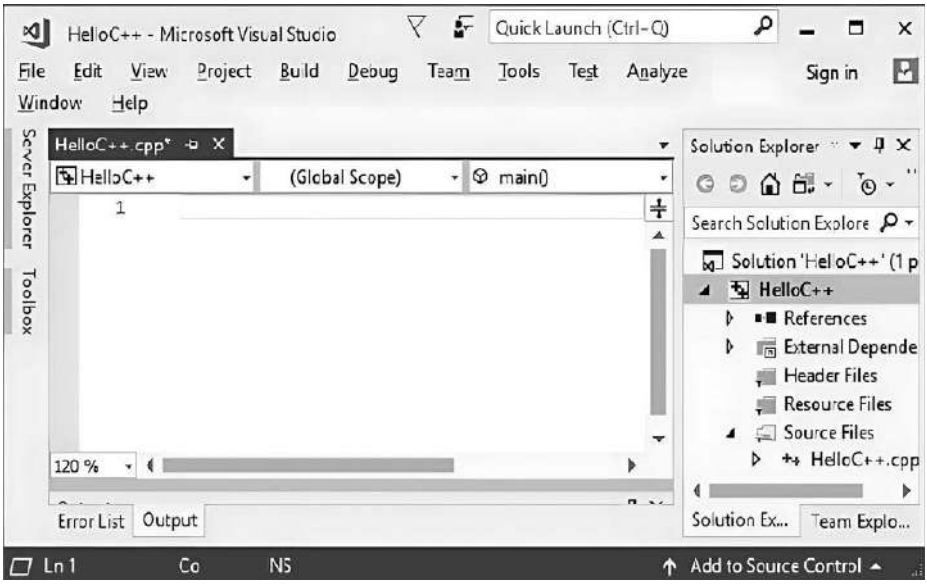
После этого откроется окно, в котором необходимо выбрать [Visual C++] → [C++ file (.cpp)].

Укажите осмысленное имя для нашей программы, например `HelloC++.cpp`.

Если мы забудем написать расширение `.cpp`, Visual Studio добавит его:

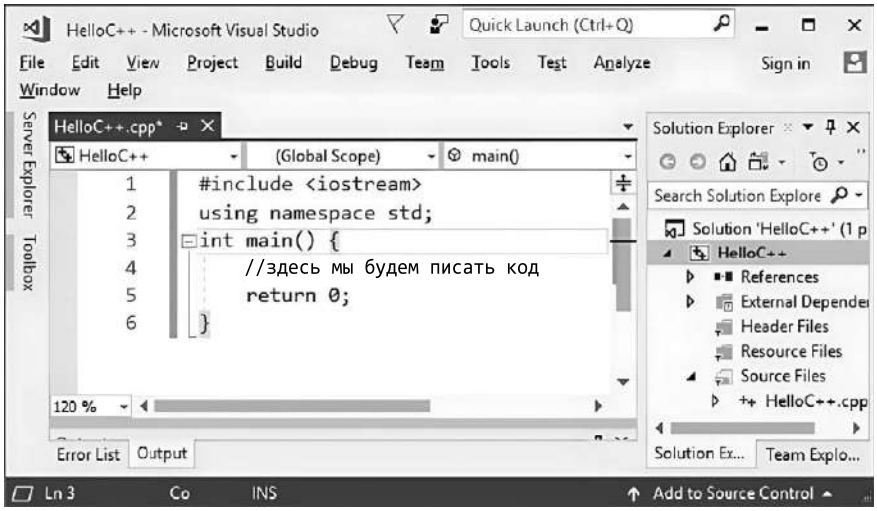


Нажмите кнопку [Add], и откроется пустая программа на языке C++:



Написание программного кода

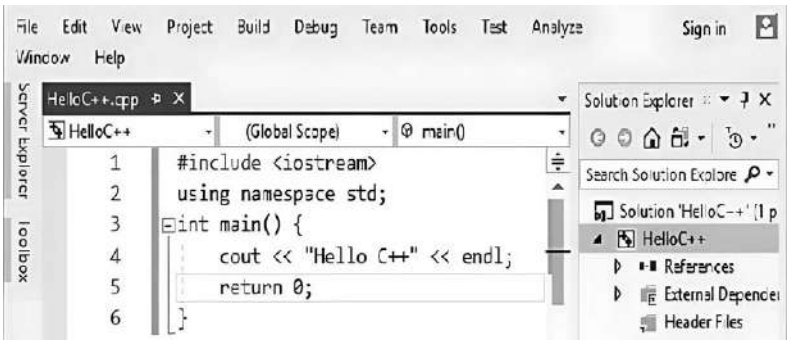
Пока что мы должны начинать каждую программу со следующего кода:



Кроме того, пока мы будем записывать команды только в функции `int main()`, между открывающей и закрывающей фигурными скобками `{ }`. Это основная функция (действие), которая выполняется при запуске консольной программы на языке C++. Позже в книге в главе «Функции» мы более подробно рассмотрим, что такое функции и как их можно использовать.

Нажмите [Enter] после открывающей скобки { и начните ввод. Код программы набирается с отступом, так как это является частью разметки текста, для удобства при повторном просмотре и/или отладке. Напишем следующую команду:

```
cout << "Hello C++" << endl;
```



Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/HelloCPP1.

Команда `cout << "Hello C++" << endl;` означает вывод на консоль текстового сообщения `Hello C++`, которое мы должны окружить кавычками, чтобы пояснить, что это текст. Далее, `endl` означает переход на новую строку, а `cout << endl;` — вывод новой строки. Эти команды можно записать и в две строки:

```
cout << "Hello C++";
cout << endl;
```

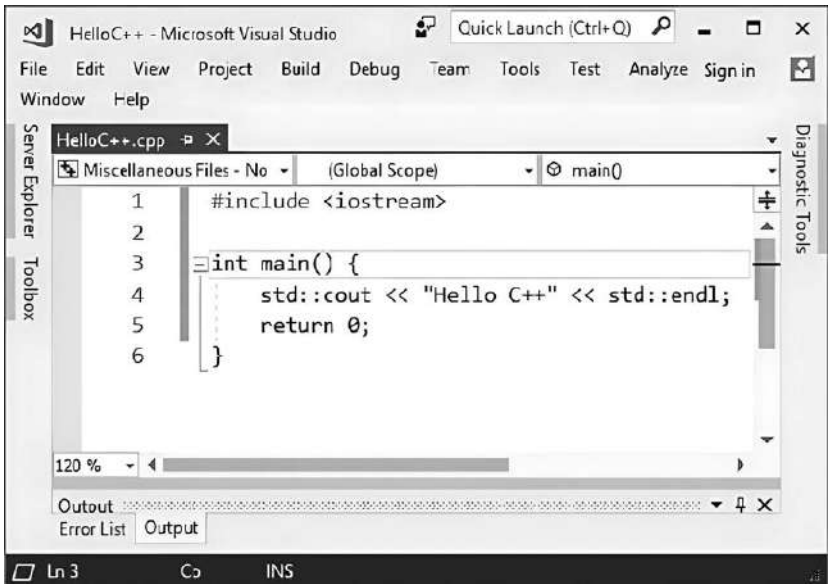
Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/HelloCPP2.

В конце каждой команды языка C++ ставится символ `;`. Он указывает на то, что команда заканчивается в этом месте (то есть не переходит на следующую строку).

Строка `#include<iostream>` означает включение библиотеки `iostream`. Библиотеки — это написанный кем-то код, который мы можем использовать в готовом виде. Например, из библиотеки `iostream` мы используем `cin` и `cout`. Существует множество других библиотек, а также мы можем писать свои собственные библиотеки, но это уже выходит за рамки данной книги.

Строка `using namespace std;` означает использование пространства имен `std`. Что такое пространство имен? Представьте себе, что над проектом работает много людей, в том числе Петр и Георгий. Петр хочет назвать переменную `count`, и Георгий также хочет использовать переменную `count`. Для компилятора эти две переменные будут означать одно и то же, независимо от того, кто их использует — Петр или Георгий. Для решения этой задачи используется пространство имен. Тогда у Петра будет пространство имен `Peter`, а у Георгия — пространство имен `George`. Для использования своей переменной `count` Петр будет писать `Peter::count`, а для использования своей переменной `count` Георгий будет писать `George::count` соответственно. Тогда компилятор будет различать переменную `count` Георгия и переменную `count` Петра соответственно. Символы `::` означают, что переменная, записанная справа от `::`, берется из пространства имен, записанного слева от `::`.

Существуют и готовые пространства имен, например `std`. Аббревиатура `std` происходит от Standard Template Library и конкретно от слова standard. Несколько библиотек объединены в пространство имен `std`. То есть чтобы использовать что-то из них, нужно везде писать `std::...`. Как мы уже говорили, строка `using namespace std;` означает использование пространства имен `std`. То есть если мы напишем ее в своей программе, то включим в нее все содержимое из соответствующего пространства имен, и нам не нужно будет везде писать `std::...`. Приведенный выше пример без этой строки выглядел бы следующим образом:



Для простоты в этой книге мы будем использовать первый вариант с `using namespace std;`.

Начало программы

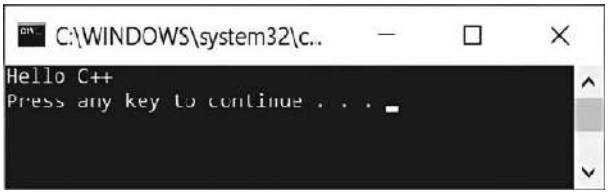
Для запуска программы нажмите [Ctrl + F5]. Если ошибок нет, то программа будет запущена. Результат будет выведен на консоль (в черное окно).

Обратите внимание, что мы запускаем программу с помощью [Ctrl + F5], а не просто [F5] или кнопки запуска Visual Studio. Если мы используем [F5], то программа выполнится на короткое время, но сразу после этого консоль исчезнет, и мы не увидим результата.

Фактически на выходе программы получается следующее текстовое сообщение:

```
Hello C++
```

Сообщение «`Press any key to continue...`» дополнительно выводится в самой нижней строке консоли Visual Studio после завершения выполнения программы, чтобы предложить нам нажать клавишу для закрытия консоли:

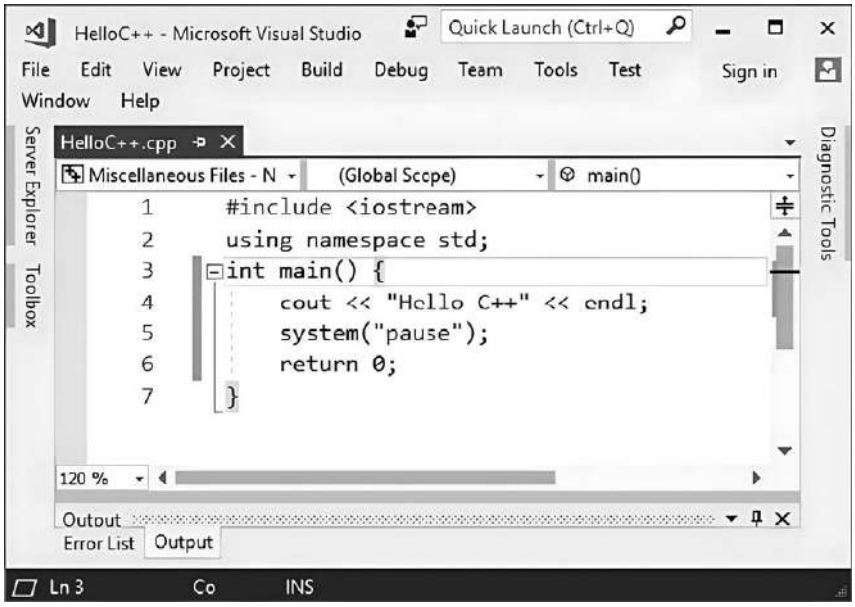


Возможна ситуация, когда [Ctrl + F5] не срабатывает, и консоль сразу закрывается.

Чтобы устранить эту проблему, нам необходимо добавить следующую строку кода в строку перед `return 0;`:

```
system("pause");
```

Наш код будет выглядеть следующим образом:



Если у нас возникла такая проблема, то не имеет значения, запустим мы [Ctrl + F5] или [F5].

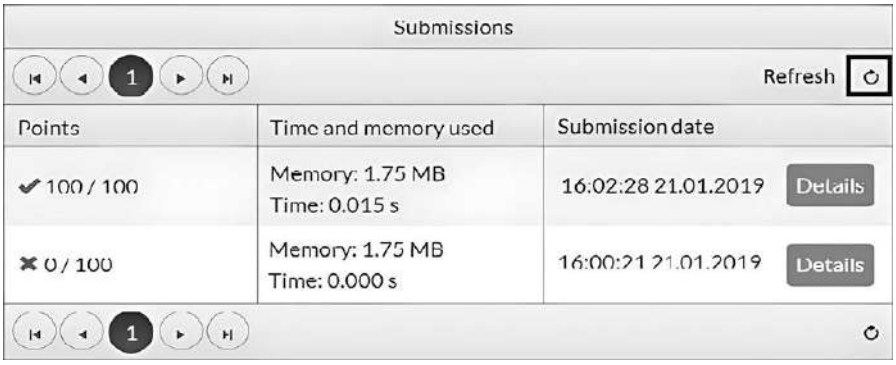
Тестирование программы в судейской системе

Тестирование задач, приведенных в книге, автоматизировано и осуществляется через Интернет, на сайте судейской системы: https://judge.softuni.bg. Оценка заданий производится системой мгновенно. Каждое задание проходит через серию тестов, и за каждый успешно пройденный тест начисляются предусмотренные баллы. Тесты, выставляемые по заданиям, скрыты.

Приведенная выше программа может быть протестирована здесь: https://judge.softuni.bg/Contests/Practice/Index/1357#0.

Поместите весь исходный код программы в черное окно и выберите код C++, как показано на рисунке.

Отправляем оценочное решение с помощью кнопки [Submit]. Через несколько секунд система возвращает результат в виде таблицы представленных решений. При необходимости мы можем нажать кнопку [Refresh], расположенную в правом верхнем углу таблицы представленных решений:



Судейская система отобразит в таблице представленных решений один из следующих возможных результатов:

- Количество баллов (от 0 до 100), когда представленный код успешно компилируется (без синтаксических ошибок) и может быть протестирован.
  - Если решение правильное, то все тесты отмечаются зеленым цветом, и мы получаем 100 баллов.
  - В случае неправильного решения часть тестов отмечается красным цветом, и мы получаем неполные баллы или 0 баллов.
- Если программа неверна, то во время компиляции мы получим сообщение об ошибке.

Как зарегистрироваться в судейской системе SoftUni?

Мы используем вашу идентификацию (имя пользователя + пароль) с сайта softuni.bg. Если вы еще не зарегистрированы в SoftUni, сделайте это. Регистрация займет всего минуту.

Тестирование примеров программ

Теперь, когда вы знаете, как запускать программы, вы можете протестировать приведенные выше примеры программ. Повеселитесь, попробуйте изменить их и «поиграйте» с ними.

Типичные ошибки в программах на языке C++

Отсутствие точки с запятой (;) в конце команд — одна из вечных проблем начинающего программиста. Пропуск этого символа приводит к сбою или невыполнению программы, причем часто проблема остается незамеченной. Приведем пример неправильного кода:

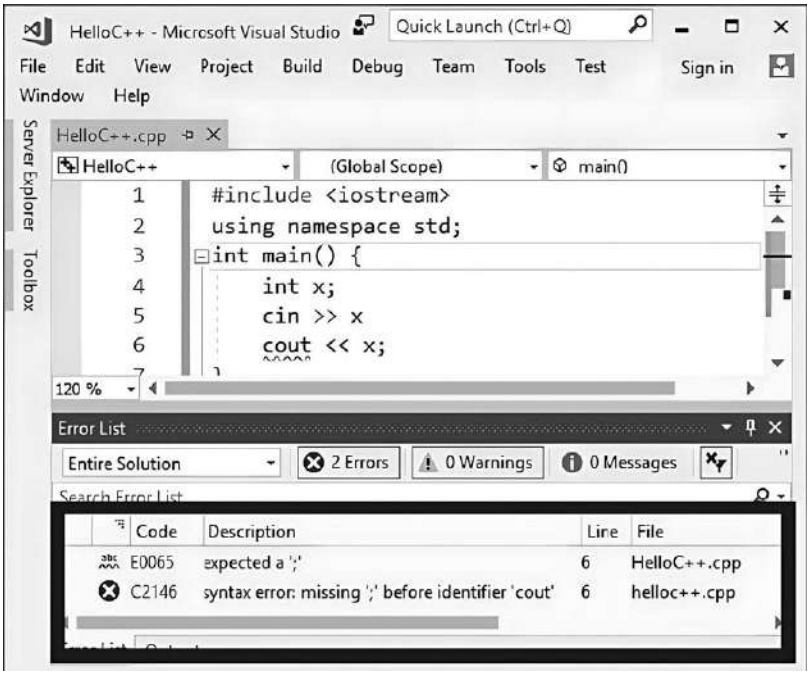
```
#include <iostream>
using namespace std;

int main() {
    int x;
```

```
    cin >> x
    cout << x;
}
```

При попытке компиляции Visual Studio выдает окно, в котором сообщается, что существует проблема:

Если мы нажмем [Yes], то будет выполнена последняя версия программы, которая успешно скомпилировалась.

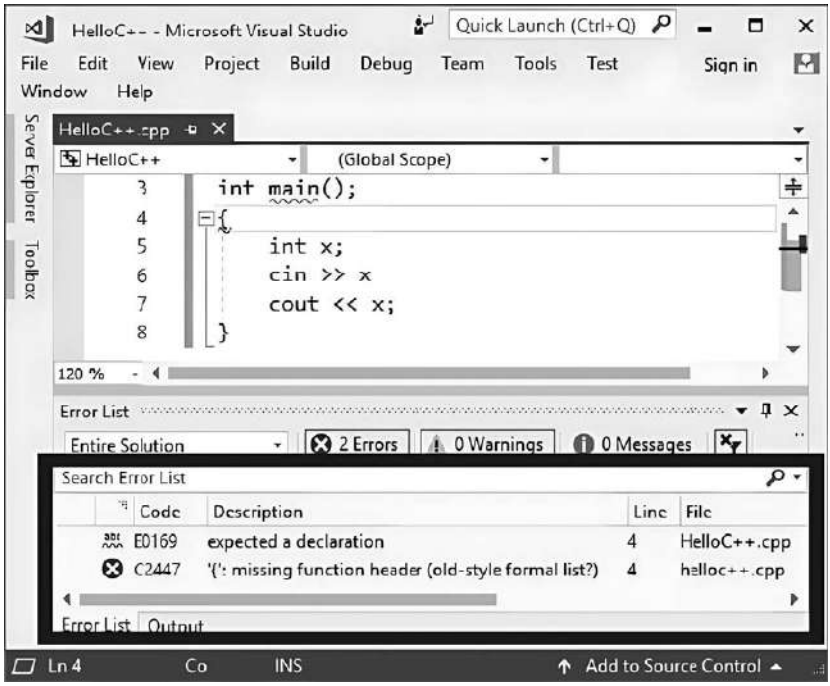
Однако если мы выберем [No], то сможем увидеть список ошибок, который находится в окне Error List, под кодом программы:



Лишняя точка с запятой (`;`) также вызывает проблему. Она приводит к сбою или невыполнению программы. Пример неправильного кода:

```
#include <iostream>
using namespace std;

int main() ;
{
    int x;
    cin >> x
    cout << x;
}
```

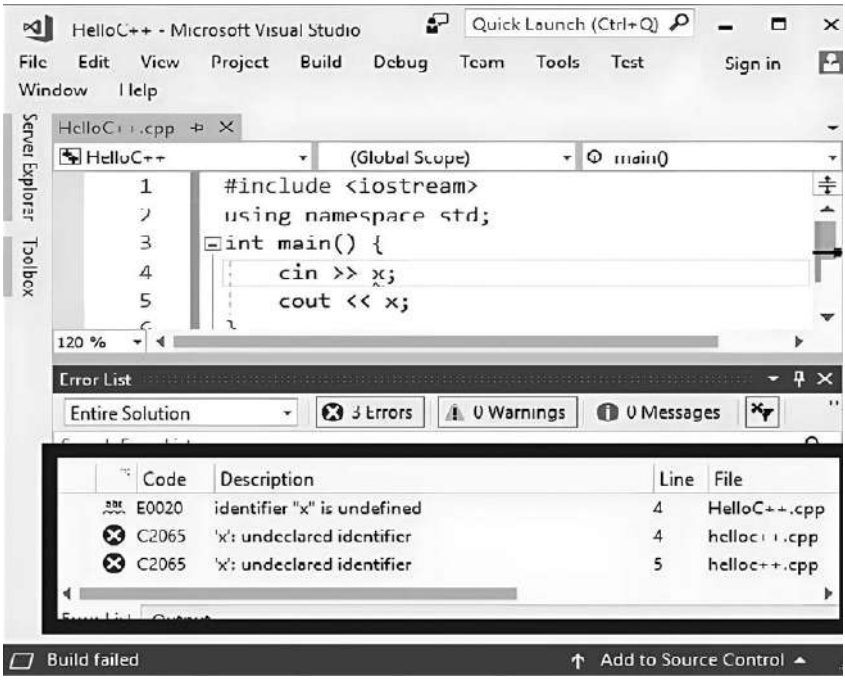
В Visual Studio появится следующая ошибка:



Необъявленные переменные. В следующем примере компилятор не знает, что такое x, поскольку сначала должен быть задан тип переменной. Программа не будет выполняться:

```
#include <iostream>
using namespace std;

int main() {
    cin >> x;
    cout << x;
}
```

В Visual Studio появится следующая ошибка:

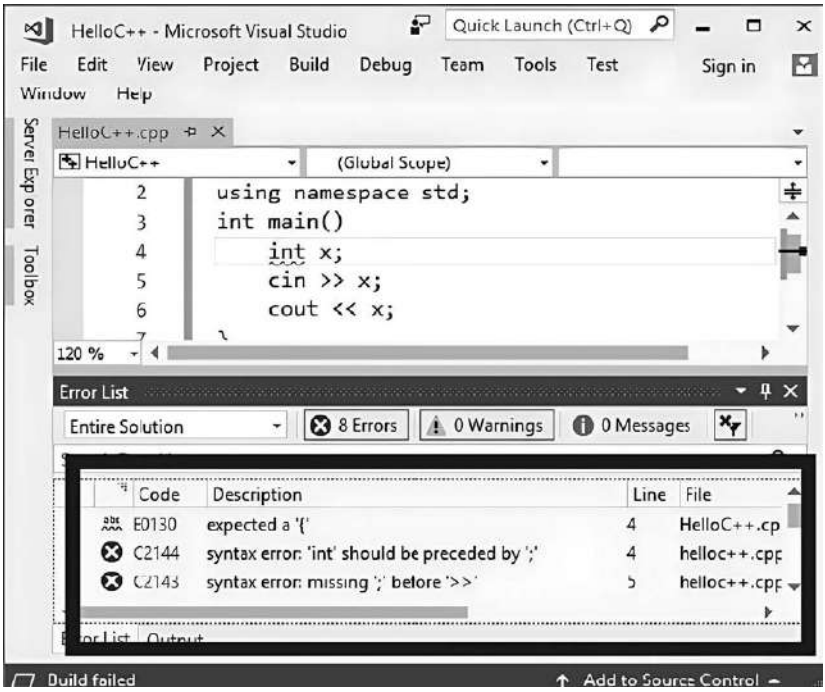


Проблемой также может быть *пропущенная кавычка или отсутствие открывающей или закрывающей скобки (неправильное количество скобок)*. Как и в случае с точкой с запятой, проблема приводит к сбою в работе программы или ее невыполнению. Такое упущение трудно заметить в более громоздком коде.

Вот пример неправильной программы:

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cin >> x;
    cout << x;
}
```

При попытке компиляции и запуска эта программа выдаст ошибку, причем еще до этого код будет подсвечен, чтобы привлечь внимание программиста к допущенной им ошибке (пропущенной открывающей скобке). Здесь список ошибок будет длиннее, что поначалу может несколько сбить с толку:



Что мы узнали из этой главы?

Во-первых, мы узнали, что такое программирование — это задание команд, написанных на компьютерном языке, которые компьютер понимает и может выполнять. Также мы узнали, что такое компьютерная программа — это последовательность команд, расположенных друг за другом. Мы познакомились с языком программирования C++ на базовом уровне и научились создавать простые консольные программы с помощью Visual Studio. Также мы проследили структуру программного кода на языке C++, например, расположение команд в секции `int main()` между открывающими и закрывающими фигурными скобками. Мы увидели, как выводить на экран с помощью `cout << ... ;` и как запускать нашу программу с помощью [Ctrl + F5]. Мы научились тестировать наш код в судейской системе SoftUni, а также узнали о типичных ошибках в программах на языке C++.

Молодцы! Перейдем к упражнениям. Не забыли ли вы, что программирование изучается с помощью написания кода и решения задач? Давайте решим несколько задач, чтобы закрепить пройденный материал.

Упражнения: первые шаги в написании кода

Добро пожаловать в раздел упражнений. Сейчас мы напишем несколько консольных программ, чтобы сделать еще несколько шагов в программировании.

Задача: программа «Выражение»

Напишите программу на языке C++, которая вычисляет и выводит на экран значение следующего числового выражения:

```
(3522 + 52353) * 23 - (2336 * 501 + 23432 - 6743) * 3
```

Примечание: не допускается вычислять значение заранее (например, с помощью Калькулятора Windows).

Рекомендации и советы

Создаем новый пустой проект на C++ с именем «Expression». Пишем наш начальный код, располагаем тело `int main()` между { и }. Затем необходимо написать код для вычисления приведенного выше числового выражения и вывода его значения на консоль. Запишем приведенное выше числовое выражение вместо многоточия в команде `cout << ... ;`.

Запустите предложенную программу с помощью [Ctrl + F5] и проверьте, совпадает ли результат с приведенным на рисунке:

Задача: числа от 1 до 20

Напишите программу на языке C++, которая выводит на консоль числа от 1 до 20 в отдельных строках.

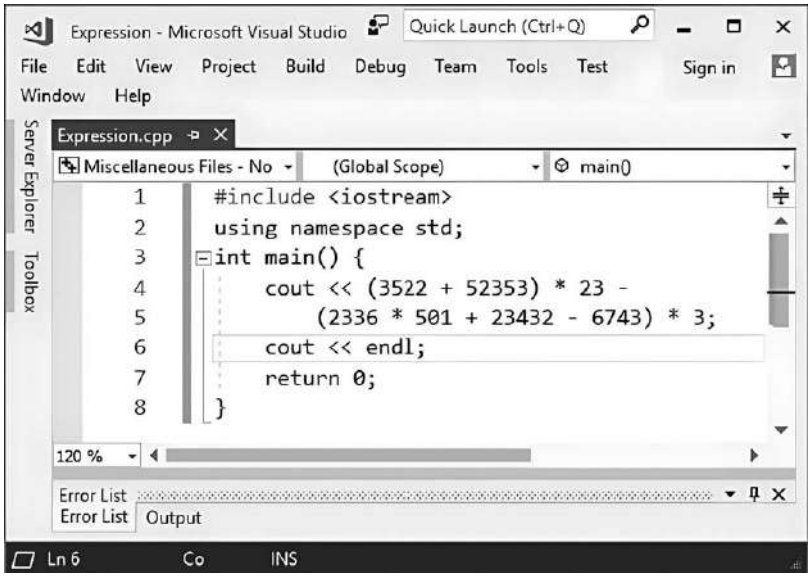
Рекомендации и советы

Создаем новый пустой проект с именем «Nums1To20». В функции `int main()` напишем 20 команд `cout << ... << endl;`, каждую в отдельной строке, для вывода последовательно чисел от 1 до 20:

```
cout << 1 << endl;
cout << 2 << endl;
...
```

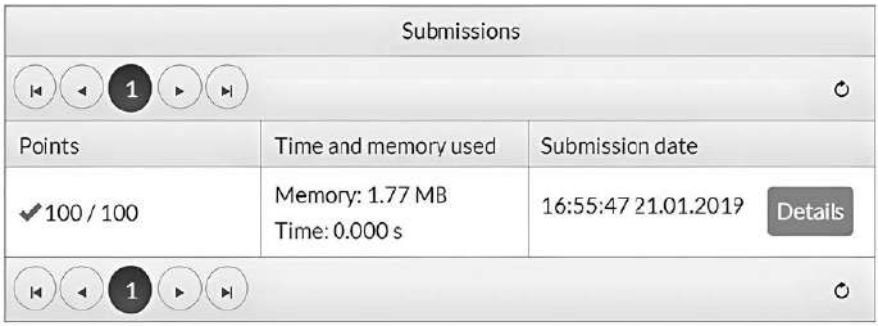
Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1357#2.



Теперь мы запускаем программу и проверяем, будет ли результат соответствовать ожиданиям:

```
1
2
...
20
```



Теперь подумаем, можно ли написать программу более «умным» способом, чтобы не повторять одну и ту же команду 20 раз. Найдите в Интернете информацию о «for loop C++».

Задача: треугольник из 55 звезд

Напишите программу на языке C++, которая выводит треугольник из 55 звездочек, расположенных в 10 рядов:

```
*
**
***
****
*****
******
*******
********
*********
**********
```

Рекомендации и советы

Создадим новый пустой проект на языке C++ с именем «TriangleOf55Stars». В нем нам необходимо написать код, который выводит треугольник из звездочек, например, с помощью десяти команд, подобных приведенным ниже:

```
cout << "*" << endl;
cout << "**" << endl;
...
```

Вы можете протестировать пример в Интернете: https://repl.it/@vncpetrov/TriangleOf55Stars.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1357#3.

Попробуйте усовершенствовать решение так, чтобы в нем не было много повторяющихся команд. Можно ли это сделать с помощью цикла `for`? Смогли ли вы найти «умное» решение (например, с помощью цикла) предыдущей задачи? В этой задаче можно использовать нечто подобное, но немного сложнее (два цикла, один внутри другого). Если у вас не получилось, не беда, мы будем изучать циклы в нескольких главах, и тогда вы вспомните эту задачу.

Задача: площадь прямоугольника

Напишите программу на языке C++, которая считывает с консоли два числа `a` и `b`, вычисляет и выводит на экран площадь прямоугольника со сторонами `a` и `b`.

Примеры ввода и вывода

a	b	Площадь
2	7	14

a	b	Площадь
12	5	60

a	b	Площадь
7	8	56

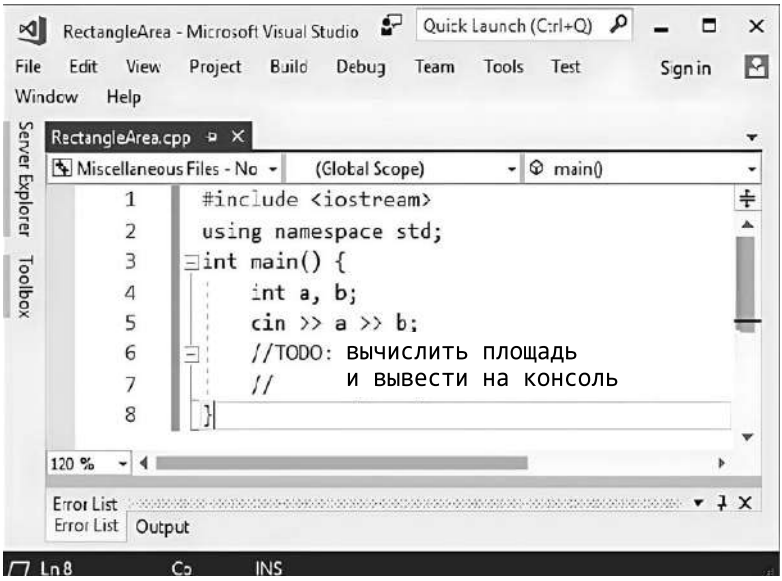
Рекомендации и советы

Создадим новый пустой проект на языке C++. Для считывания двух чисел используем следующие команды:

```
int a, b;
cin >> a >> b;
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/RectangleArea.

Остается дополнить программу вычислением площади прямоугольника и выводом на экран. Используйте ставшую уже привычной команду `cout < < ... ;`, где вместо многоточия запишите произведение чисел `a` и `b`. В программировании умножение выполняется с помощью оператора *.



Проверьте свое решение

Проверьте свое решение на нескольких примерах. Вы должны получить результат, подобный этому (на вход вводятся 2 и 7, а в качестве результата программа выводит 14 — их произведение):

```
2
7
14
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1357#4.

Задача*: квадрат из звездочек

Напишите программу на языке C++, которая считывает с консоли целое положительное число `N` и выводит на консоли квадрат из `N` звездочек, как показано в примерах ниже.

Пример ввода и вывода

Ввод	Вывод
3	*** * * ***

Ввод	Вывод
4	**** * * * * ****

Ввод	Вывод
5	***** * * * * * * *****

Рекомендации и советы

Создадим новый пустой проект на языке C++. Для чтения числа `N` (`2 ≤ N ≤ 100`) используем следующий код:

```
int n;
cin >> n;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/SquareOfStars.

Доработайте приведенную выше программу для вывода квадрата, состоящего из звездочек. Возможно, потребуется использование циклов `for`. Поищите информацию в Интернете.

Внимание: это задание сложнее остальных, оно специально дано сейчас и помечено звездочкой, чтобы побудить вас к поиску информации в Интернете. Это один из самых важных навыков, который необходимо развивать при изучении программирования: поиск информации в Интернете. Это вам предстоит делать каждый день, если вы работаете программистом, поэтому не бойтесь, а пробуйте. Если у вас возникнут трудности, можете обратиться за помощью на форум SoftUni: https://softuni.bg/forum.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1046#5.

Консольные, графические и веб-приложения

В консольных приложениях, как можно догадаться, все операции по чтению входных и выводу выходных данных выполняются через консоль. Входные данные, считываемые приложением, вво-

дятся туда, а выходные данные выводятся там после или во время выполнения программы.

Если консольные приложения используют текстовую консоль, то веб-приложения — веб-интерфейс пользователя. Для обеспечения их работоспособности необходимы две вещи — веб-сервер и веб-браузер, причем браузер играет основную роль в визуализации данных и взаимодействии с пользователем. Веб-приложения гораздо удобнее для пользователя, визуально выглядят гораздо лучше, используют мышь и прикосновения пальцев (на планшетах и телефонах), но за всем этим стоит программирование. И именно поэтому нам нужно учиться программировать, и мы уже сделали свои первые, очень маленькие шаги.

Графические (GUI) приложения имеют визуальный пользовательский интерфейс, непосредственно на компьютере или мобильном устройстве, без необходимости использования веб-браузера. Графические приложения (настольные приложения, или, другими словами, десктопные приложения) состоят из одного или нескольких графических окон, содержащих определенные элементы управления (текстовые поля, кнопки, картинки, таблицы и т. д.), которые служат для диалога с пользователем в более интуитивно понятной форме. Мобильные приложения на телефоне и планшете устроены аналогично: мы используем формы, текстовые поля, кнопки и другие элементы управления и управляем ими с помощью программного кода. Именно поэтому мы сейчас учимся писать код: в разработке программного обеспечения код присутствует везде.

Звучит пугающе? Не бойтесь! Нам предстоит пройти долгий путь, чтобы достичь уровня знаний и умений, позволяющего свободно писать код на языке C++ в гораздо более крупных и сложных программах. Если у вас не получается, ничего страшного, смело двигайтесь дальше. Со временем вы будете с улыбкой вспоминать, каким непонятным и захватывающим было ваше первое знакомство с программированием. Если у вас возникнут трудности с приведенными примерами, попросите помощи на форуме SoftUni: https://softuni.bg/forum. Вам предстоит еще многому научиться.

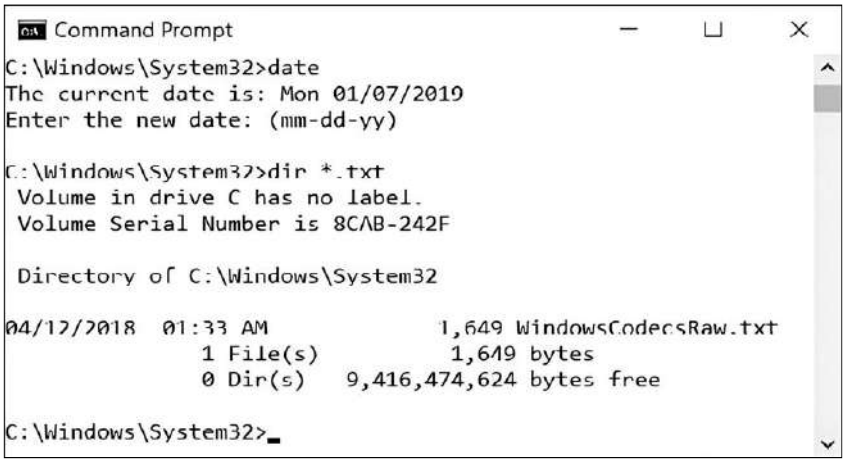
Глава 2.1.
Простые вычисления

В этой главе мы познакомимся со следующими понятиями и приемами программирования.

- Мы узнаем, что такое системная консоль.
- Узнаем, как считать числа из системной консоли.
- Как работать с типами данных и переменными, которые необходимы нам при обработке чисел и операциях с ними.
- Как вывести результат (число) на системную консоль.
- Как выполнить простые арифметические действия: сложение, вычитание, умножение, деление, объединение строк.

Системная консоль

Системная консоль, или консоль компьютера, обычно называется просто консолью и представляет собой устройство, с помощью которого мы отдаем компьютеру команды в текстовом виде и получаем результаты их выполнения опять же в виде текста:



В большинстве случаев системная консоль является текстовым терминалом, то есть принимает и отображает только текст, без графических элементов — кнопок, меню и т. д. Обычно она выглядит как черно-белое окно.

В большинстве операционных систем консоль доступна в виде отдельного приложения, в котором мы пишем консольные команды. В Windows она называется Command Prompt, а в Linux и Mac — Terminal. На консоли запускаются консольные приложения. Они считывают текстовый ввод из командной строки и выводят свои результаты в виде текстового вывода консоли. В этой книге мы будем изучать программирование, создавая в основном консольные приложения. Обязательно настройте консоль (из меню [Project] -> [Properties] -> [Linker] -> [System] -> Console(/SUBSYSTEM:CONSOLE)).

В языке C++ `cin` используется для ввода данных, а `cout` — для их вывода на экран. Это объекты, определенные в пространстве имен `std`. Для удобства пространство имен можно добавить в наш проект. В противном случае, как мы узнали в предыдущей главе, каждый раз перед использованием `cin` и `cout` (а также других объектов, определенных в `std`) необходимо набирать `std::`, то есть:

```
std::cin >> name;
std::cout << name;
```

Протестировать пример можно в Интернете: https://repl.it/@vncpetrov/CinCout.

Вычисления в программировании

Мы знаем, что компьютеры — это машины, обрабатывающие данные. Все данные хранятся в оперативной памяти компьютера (RAM) в виде переменных. Переменные — это именованные области памяти, в которых хранятся данные определенного типа, например, число или текст. Каждая переменная в языке C++ имеет имя, тип и значение. Вот как мы определяем переменную и одновременно с ее объявлением присваиваем ей значение:

Данные после их обработки записываются обратно в переменные (то есть куда-то в память, выделенную нашей программой).

Типы данных и переменные

В программировании каждая переменная хранит значение заданного типа. Типами данных могут быть, например, число, буква, текст (строка), дата, цвет, рисунок, список и т. д.

Приведем несколько примеров типов данных:

- целое число: 1, 2, 3, 4, 5, ...
- дробное число: 0.5, 3.14, –1.5, ...
- тип буквы (символа) алфавита: ‘a’, ‘b’, ‘c’, ...
- текстовый тип (строка): «Привет», «Hi», «Пиво», ...

Язык C++ является сильно типизированным языком, поэтому тип значения всегда должен быть указан (2 — целое число, 0.5 — дробное число, «Hi» — строка).

В следующих примерах и задачах мы рассмотрим такие типы данных:

- `int` — для целых чисел;
- `double` — для дробных чисел;
- `string` — для текста.

Чтение чисел из консоли

Для чтения целого (не дробного) числа с консоли необходимо объявить переменную, указать тип числа и использовать >> (оператор побитового сдвига вправо) для чтения информации с системной консоли:

```
int num;
cin >> num;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ReadIntNumber.

Пример: вычисление стороны квадрата со стороной a

В качестве примера рассмотрим следующую программу, которая считывает с консоли целое число, умножает его на себя (возводит

в квадрат) и выводит результат умножения. Таким образом, мы можем вычислить площадь квадрата по заданной длине его стороны:

```
cout << "a = ";
int a;
cin >> a;

int area = a * a;
cout << "Square = ";
cout << area << endl;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/SquareArea.

На следующем рисунке показано, как будет работать программа для квадрата со стороной 3:

Попробуйте ввести неправильные данные, например, `hello`.

Как работает пример?

В первой строке, с `int a`, объявлена переменная типа `int` (то есть целое число).

Во второй строке при помощи `cout << "a = ";` выводится информационное сообщение, предлагающее пользователю ввести сторону квадрата `a`. Для вывода информации на системную консоль используется `<<` (побитовый сдвиг влево). После вывода курсор остается на той же строке. Оставаться на той же строке удобнее для пользователя чисто визуально. Для вывода информации на экран используйте не `cout <<"...";`, а `cout << "..." << endl;`, и таким образом курсор остается на той же строке (`endl` также является частью пространства имен `std`, для перехода на новую строку можно использовать `"\n"`): `cout << area << "\n"`.

В третьей строке `cin >> a;` целое число считывается с консоли и сохраняется в уже определенной переменной `a`.

В четвертой строке `int area = a * a;` создается новая переменная `area`, в которую записывается результат умножения `a` на `a`.

В пятой строке `cout << "Square area = ";` выводит указанный текст без перехода на новую строку. Снова используется `cout << "...",` а не `cout << "..." << endl;`, и поэтому курсор остается на той же строке, чтобы затем можно было вывести вычисленную площадь квадрата.

В шестой строке последняя команда `cout << area;` выводит значение переменной `area`.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#0.

Чтение дробного числа из консоли

Для того чтобы считать дробное число с консоли, необходимо снова объявить переменную, указать числовой тип и использовать `>>` (оператор побитового сдвига вправо):

```
double num;
cin >> num;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ReadDoubleNumber.

Пример: перевод из дюймов в сантиметры

Напишем программу, которая считывает дробное число в дюймах и переводит его в сантиметры:

```
cout << "Inches = ";
double inches;
cin >> inches;
double centimeters = inches * 2.54;
cout << "Centimeters = ";
cout << centimeters << endl;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/InchesToCentimeters.

Запустим программу и убедимся, что при вводе значения в дюймах мы получаем правильный результат в сантиметрах.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#1.

Считывание текста

При работе с текстом необходимо включить библиотеку `<string>`. Для чтения текста из системной консоли мы объявляем новую переменную (типа string) и используем >> (оператор побитового сдвига вправо):

```
#include <iostream>
#include <string>

using namespace std;

int main {
  string str;
  cin >> str;

  return 0;
}
```

Вы можете протестировать пример в Интернете: https://repl.it/@vncpetrov/ReadText.

Пример: приветствие по имени

Напишем программу, которая вводит имя пользователя и приветствует его текстом «`Hello, `*имя*».

```
string name;
getline(cin, name);
cout << "Hello, " << name << "!\n";
```

Вы можете протестировать пример в Интернете: https://repl.it/@vncpetrov/GreetingByName.

В данном примере используется другой способ чтения входных данных, поскольку неизвестно, сколько слов будет введено для имени. Поэтому для получения всех символов, записанных в строке (включая пробелы), используется функция C++ `getline(...)`. В главе «Функции» мы рассмотрим функции более подробно, а пока давайте посмотрим, что нужно записать в данную функцию.

Сначала мы заключаем в круглые скобки объект `cin`, который «берет» входные данные и сохраняет их в переменной. Чтобы указать, какая именно переменная, во втором месте пишем ее имя (в данном случае — `name`).

Как видите, в последней строчке строка «Hello, » находится перед переменной name, а строка «!» — после. Все они будут выведены именно в таком порядке благодаря оператору << (оператор побитового сдвига влево), который связывает их в единый текст:

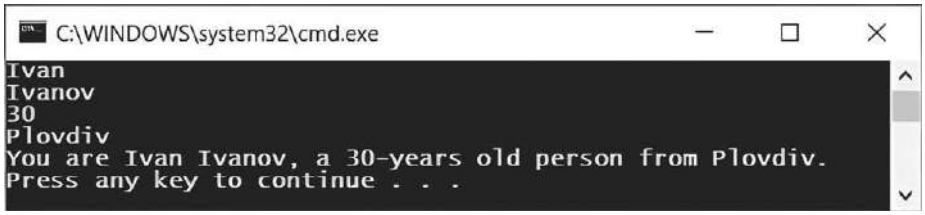
Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#2.

Вывод на экран текста и чисел

```
string firstName, lastName, town;
int age;
cin >> firstName;
cin >> lastName;
cin >> age;
cin >> town;
cout << "You are " << firstName << " " << lastName << ", a "
<< age << "-years old person from " << town << "." <<
endl;
```

Вы можете протестировать пример в Интернете: https://repl.it/@vncpetrov/ConcatenateData. Вот результат, который мы получим после выполнения этого примера:



Вы можете заметить, что firstName, lastName, town записаны в одной строке. Это объясняется тем, что если переменные имеют одинаковый тип, то они могут быть объявлены в одной строке.

Кроме того, в этом примере следует обратить внимание на то, что каждая переменная должна передаваться в том порядке, в котором мы хотим ее вывести. Это относится и к тем местам, где нужно поставить пробел (нужно явно указать, какие именно — в данном случае между `firstName` и `lastName` используется пустая строка `" "`, но возможны и другие варианты).

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#3.

Арифметические операции

Рассмотрим основные арифметические операции в программировании.

Сложение (оператор +)

Мы можем складывать числа с помощью оператора +:

```
int a = 5;
int b = 7;
int sum = a + b; // 12
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/SumOperator.

Вычитание (оператор -)

Вычитание чисел производится с помощью оператора `-`.

```
int a, b;
cin >> a;
cin >> b;
int result = a - b;
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/SubtractionOperator. Вот результат выполнения программы (при числах 10 и 3):

Умножение (оператор *)

Для умножения чисел используется оператор *:

```
int a = 5;
int b = 7;
int product = a * b; // 35
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/MultiplyOperator.

Деление (оператор /)

Деление чисел выполняется с помощью оператора `/`. Он по-разному работает для целых и дробных чисел.

- При делении двух целых чисел выполняется *целочисленное деление*, в результате которого получается целое число с усеченной дробной частью. Например, 11 / 3 = 3.
- При делении двух чисел, хотя бы одно из которых является дробью, выполняется *дробное деление*, и результатом является дробное число, как в математике. Например, 11 / 4,0 = 2,75. Когда точное деление невозможно, результат округляется, например, 11,0 / 3 = 3,66666666666667.
- *Целочисленное деление на 0* вызывает исключение времени выполнения.
- *Дробное деление на 0* не приводит к ошибке, а результатом является +/- бесконечность или специальное значение `inf`. Например, 5 / 0,0 = ∞.

Приведем несколько примеров использования оператора деления:

```
int a = 25;
cout << a / 4 << endl; // 6 - целочисленное деление, дает
// целочисленный результат.
// При делении целых чисел дробная часть усекается.
cout << a / 4.0 << endl; // 6,25 - дробное деление, дает
// в результате дробное число. Мы указали, что число 4
// должно интерпретироваться как дробное, путем добавления
// десятичной точки, за которой следует ноль.
// (если бы f имело тип int, то дробная часть все равно была
// бы усечена, и результатом было бы 6)
cout << a / 0 << endl;   // Ошибка: целочисленное деление на 0
```

Вы можете протестировать этот пример онлайн: https://repl.it/@vncpetrov/DivisionOperator1.

Рассмотрим несколько примеров целочисленного деления (напомним, что при делении целых чисел в C++ результат является целым числом):

```
int a = 25;
cout << a / 7 << endl; // Целочисленный результат: 3
cout << a / 9 << endl; // Целочисленный результат: 2
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/DivisionOperator2.

Рассмотрим несколько примеров деления на дробные числа. В этом случае результат всегда является дробным числом, и деление никогда не дает ошибки и корректно работает со специальными значениями +∞ и -∞:

```
int a = 15;
cout << a / 2.0 << endl; // Дробный результат: 7,5
cout << a / 0.0 << endl; // Результат: inf
cout << -a / 0.0 << endl; // Результат: -inf
cout << 0.0 / 0.0 << endl; // Результат: -nan(Not a Number),
// результат операции не является допустимым числовым
значением
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/DivisionOperator3.

Объединение текста и чисел

Оператор +, помимо сложения чисел, используется также для конкатенации текста (объединения двух строк друг за другом). В программировании объединение текста с текстом или с числом называется *конкатенацией*. Приведем пример, как мы можем объединить текст и число с помощью оператора +. Не забудьте добавить библиотеку <string>:

```
string firstName = "Мария";
string lastName = "Иванова";
string age = "19";
string str = firstName + " " + lastName + " @ " + age;
cout << str << endl; // Мария Иванова @ 19
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/Concatenation.

Цифры также могут быть представлены в виде текста, и поэтому конкатенация возможна. Приведем еще один пример:

```
string a = "1.5";
string b = "2.5";
string concatenation = "Сумма: " + a + b;
cout<< concatenation << endl;  // Сумма: 1.52.5
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/Concatenation2.

Числовые выражения

В программировании мы также можем вычислять числовые выражения, например:

```
int expr = (3 + 5) * (4 - 2);
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/Expr.

Действует стандартное правило приоритета арифметических операций: умножение и деление всегда выполняются перед сложением и вычитанием. Если в скобках стоит выражение, то оно вычисляется первым, но все это мы знаем из школьной математики.

Пример: вычисление площади трапеции

Напишем программу, которая принимает на входе длины двух оснований трапеции и ее высоту (по одному дробному числу в строке) и вычисляет площадь трапеции по стандартной математической формуле:

Если `(b1 + b2) * h` делится на 2,0 без остатка, то десятичная точка и ноль игнорируются и не выводятся на консоль.

```
#include <iostream>
#include <string>

using namespace std;

int main() {
```

```
    string firstName, lastName, town;
    double b1, b2, h;
    cin >> b1;
    cin >> b2;
    cin >> h;
    double area = (b1 + b2) * h / 2.0;
    cout << "Площадь трапеции = " << endl;

    return 0;
```

Если мы запустим программу и введем для сторон соответственно 3, 4 и 5, то получим следующий результат:

```
3
4
5
Площадь трапеции = 17.5
```

Проверка в судейской системе

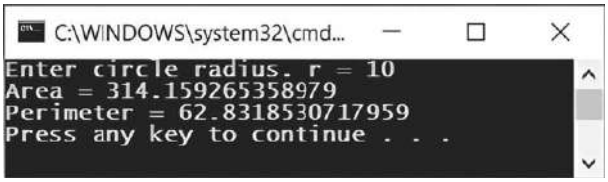
Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#4.

Пример: длина окружности и площадь круга

Напишем программу, которая при заданном радиусе r окружности вычисляет площадь круга и длину окружности.

- Площадь (`Area`) = π * r * r
- Длина окружности (`Perimeter`) = 2 * π * r
- π ≈ 3.14159265358979323846...

```
#include <iostream>
using namespace std;

int main() {
    cout << "Enter circle radius, r =";
    double r;
    cin >> r;
    double pi = 3.14159265359;
    cout << "Area = " << pi * r * r << endl;
    cout << "Perimeter = " << 2 * pi * r << endl;

    return 0;
}
```



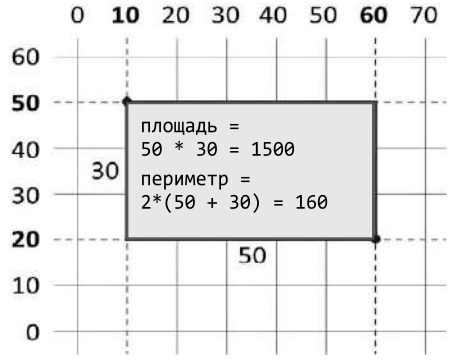
Опробуем программу с радиусом `r = 10`:

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#5.

Пример: площадь прямоугольника на плоскости

Прямоугольник задан координатами двух его противоположных углов. Вычислите его площадь и периметр:



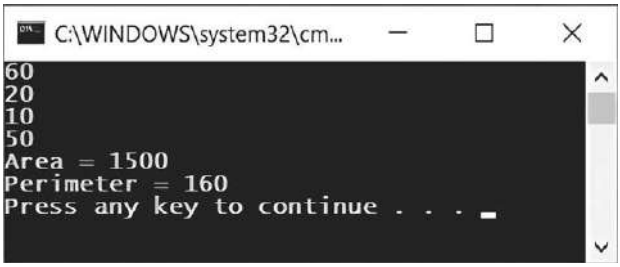
В данной задаче необходимо учесть, что если из большего x вычесть меньший x, то получим длину прямоугольника. Аналогично, если из большего y вычесть меньшее y, то получим высоту прямоугольника. Остается перемножить две стороны. Приведем пример реализации описанной логики:

```
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    double x1, x2, y1, y2;
    cin >> x1;
    cin >> y1;
```

```
    cin >> x2;
    cin >> y2;

    double width = fabs(x2 - x1);
    double height = fabs(y2 - y1);

    cout << "Area = " << width * height << endl;
    cout << "Perimeter = " << 2 * (width + height) << endl;

    return 0;
}
```

Для нахождения абсолютного значения полученной разности мы используем функции `fabs(x2 - x1)` и `fabs(y2 - y1)`, поскольку мы не знаем, какое из двух чисел больше. Эта функция содержится в библиотеке `<cmath>`.

При выполнении программы со значениями системы координат, указанными в условии, получаем следующий результат:



Проверка в судейской системе

Проверьте программу в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1358#6.

Что мы узнали из этой главы?

Давайте подведем итог тому, что мы изучили в этой главе:

- Ввод текста, целых и дробных чисел через `cin`: `cin >> age;`.
- Выполнение вычислений с использованием соответствующих арифметических операторов [`+`, `-`, `*`, `/`, `()`]: `int sum = 5 + 3;`.
- Вывод текста с помощью `cout`: `cout << "My names is" << firstName << " " << lastName << endl;`.

Упражнения: простые вычисления

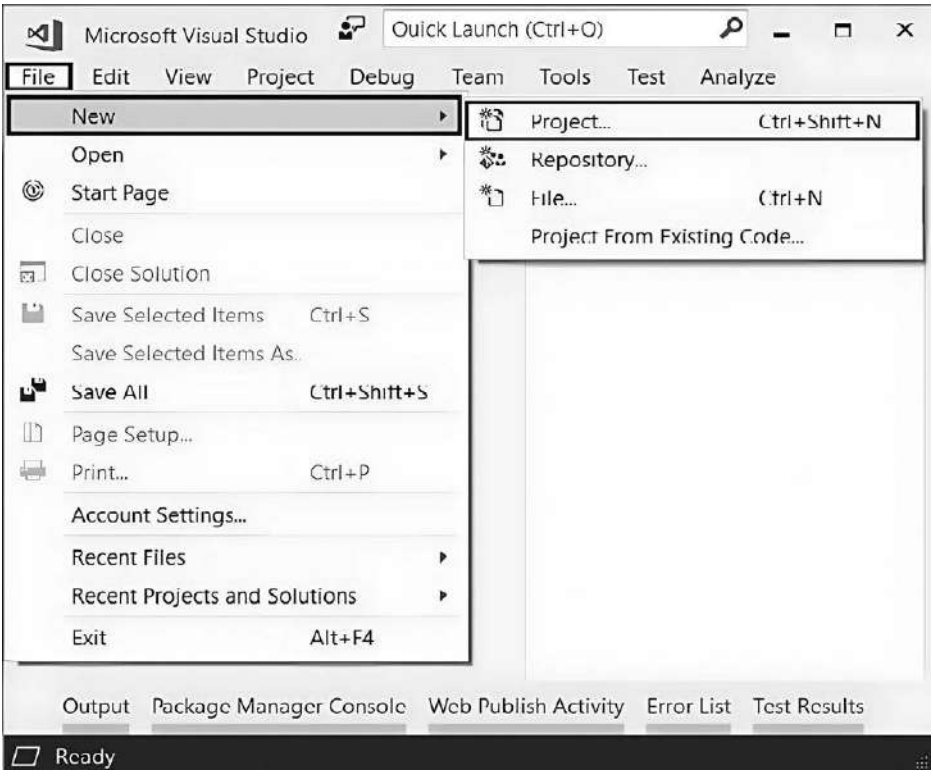
Давайте закрепим полученные в этой главе знания с помощью нескольких задач.

Пустое решение Visual Studio (пустой проект)

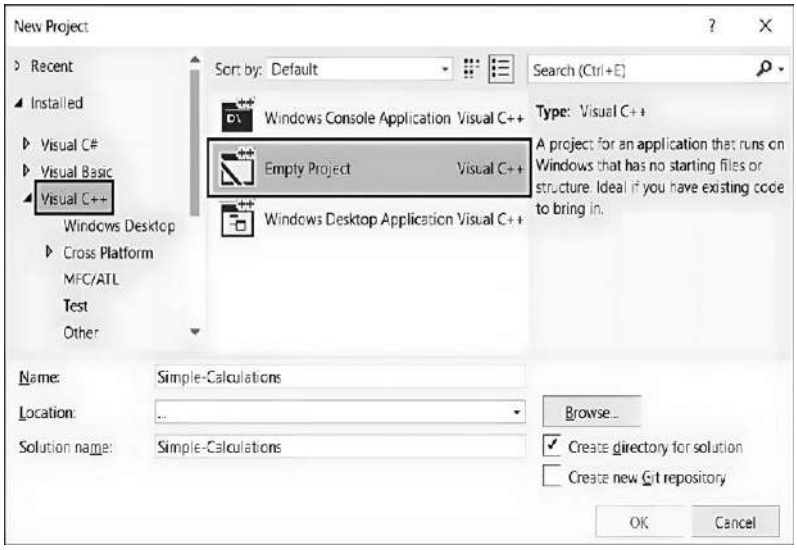
В этом практическом занятии мы будем использовать Solution с несколькими проектами для организации решений упражнений, то есть каждая задача будет находиться в отдельном проекте, а все проекты — в общем VS Solution.

Начнем с создания пустого проекта (Empty Project) в Visual Studio.

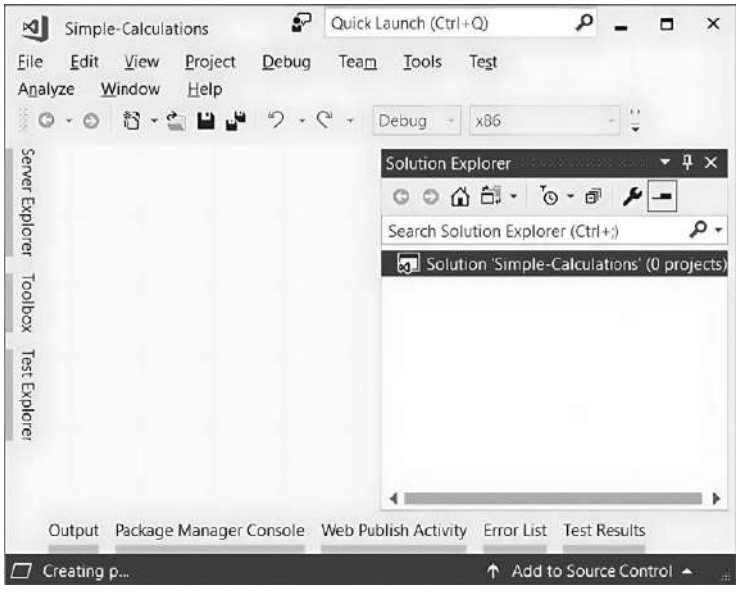
- Запустите Visual Studio.
- Создайте новый пустой проект: [File] -> [New] -> [Project...].



В диалоговом окне выберите [Visual C++] -> [Empty Project] и дайте проекту соответствующее имя, например, «Simple-Calculations»:



Теперь мы создали пустое решение Visual Studio Solution (с 0 проектов в нем):



Задача: вычисление площади квадрата

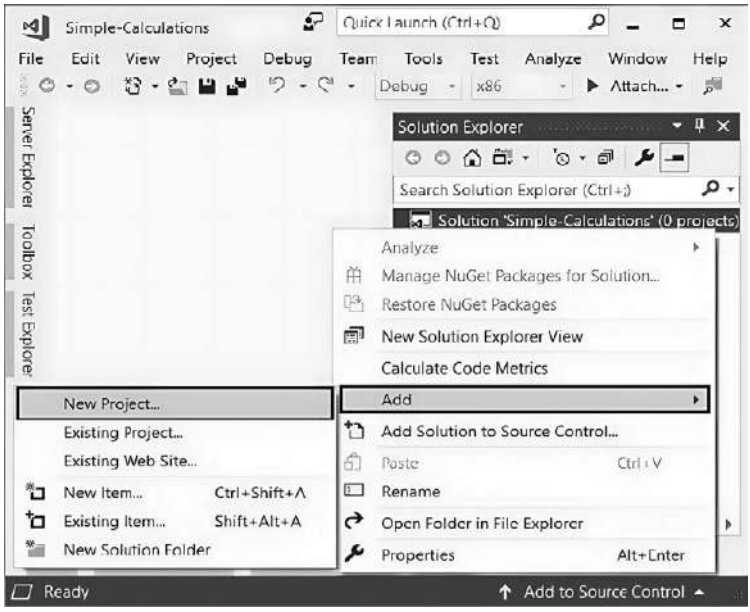
Первая задача этой главы состоит в следующем: написать консольную программу, которая вводит целое число a и вычисляет площадь квадрата со стороной a. Задача тривиальна: считать с консоли число, умножить его на себя и вывести результат на консоль.

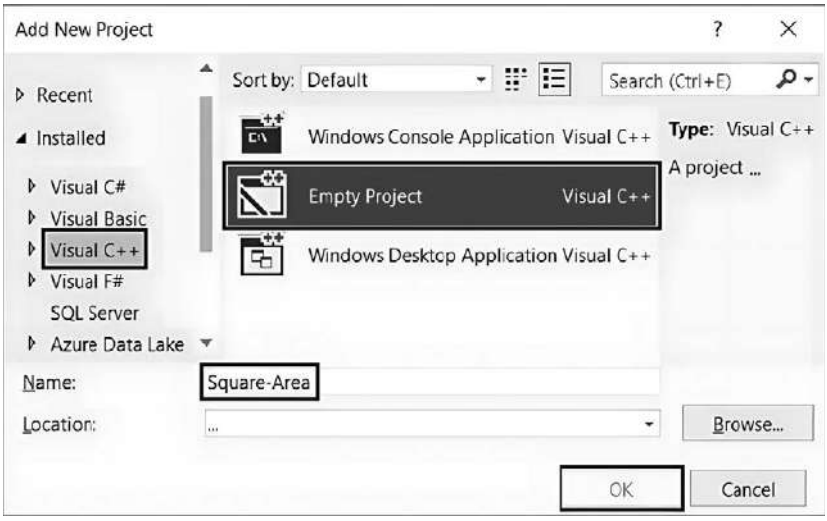
Рекомендации и советы

Создадим новый проект в существующем решении Visual Studio. В Solution Explorer щелкните правой кнопкой мыши на решении ‘Simple-Calculations’ и выберите [Add] -> [New Project...].

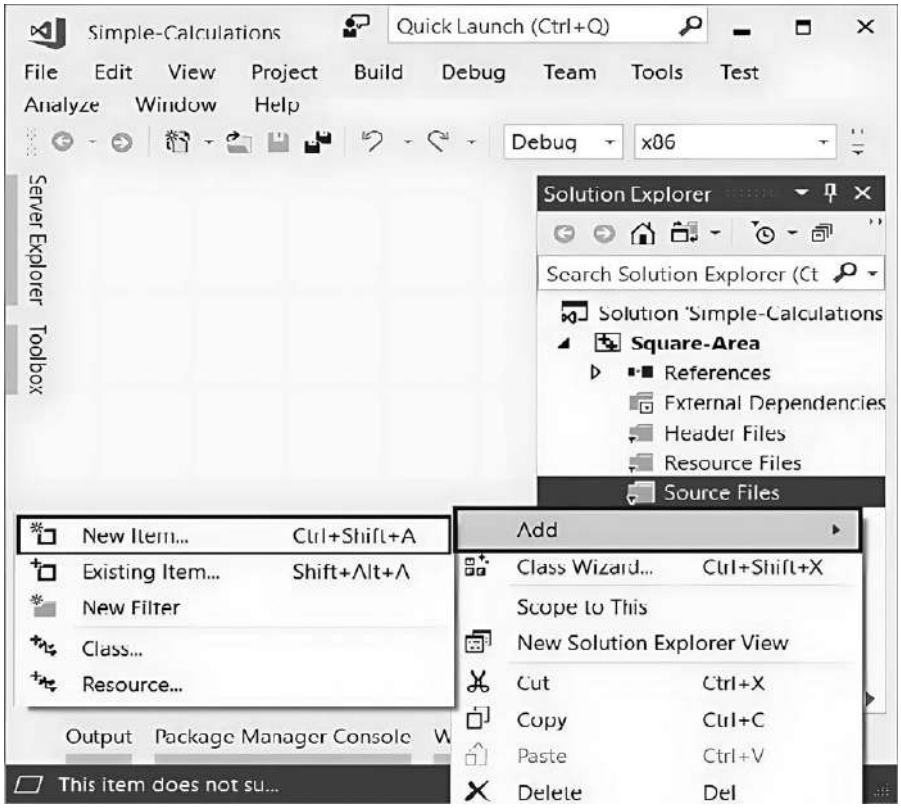
После этого откроется диалоговое окно для выбора типа создаваемого проекта. Выберите [Visual C++] -> [Empty Project] и дайте ему осмысленное имя, например «Square-Area».

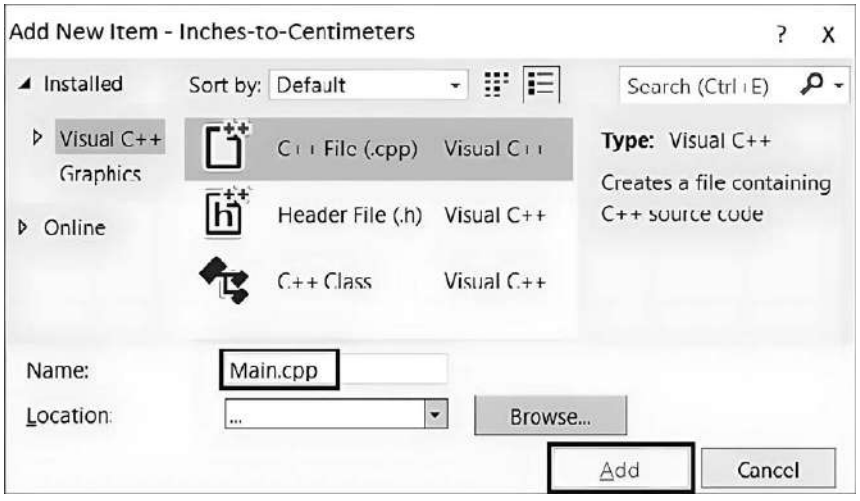
Следующие два скриншота иллюстрируют описанное:



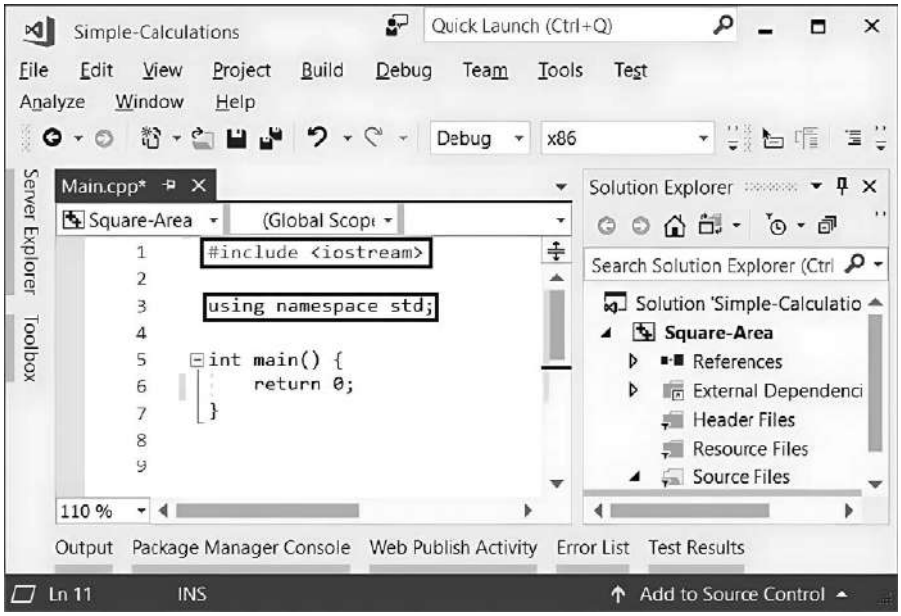


Затем добавьте новый файл C++ (.cpp), который должен иметь имя Main.cpp (из меню [Source Files] -> [Add] -> [New Item...]):





В файле ничего не записано. Как мы знаем, в первой строке необходимо добавить библиотеку `<iostream>`, а затем включаем и пространство имен `std`. Далее мы напишем функцию `main()`, которая дает нашей программе возможность работать:



Следующий шаг — написание кода для решения задачи. Для этого мы переходим в тело программы — функцию `main()` — и пишем следующий код:

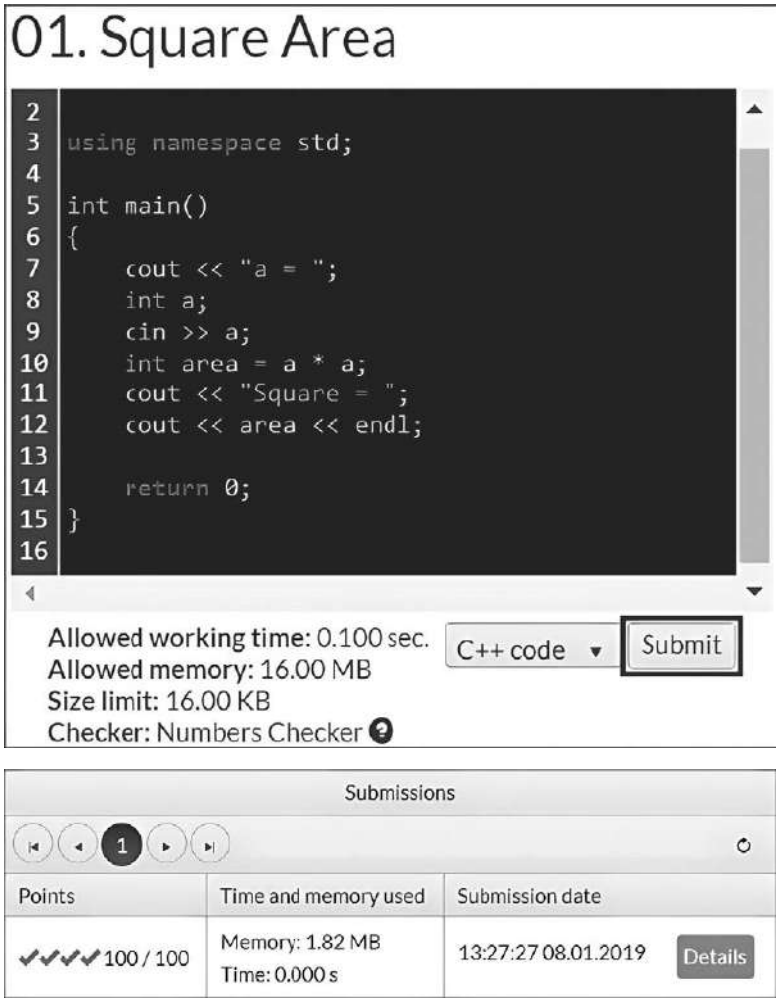
```
cout << "a = ";
int a;
cin >> a;
int area = a * a;
cout << "Square = ";
cout << area << endl;
```

Код принимает на вход целое число с помощью команды `cin >> a;`, затем вычисляет площадь квадрата (`area = a * a;`) и, наконец, выводит значение переменной `area` на консоль. Запустите программу с помощью [Ctrl + F5]:

Проверьте, правильно ли программа работает при различных значениях входных данных.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#0. Вы должны получить 100 баллов (полностью правильное решение):

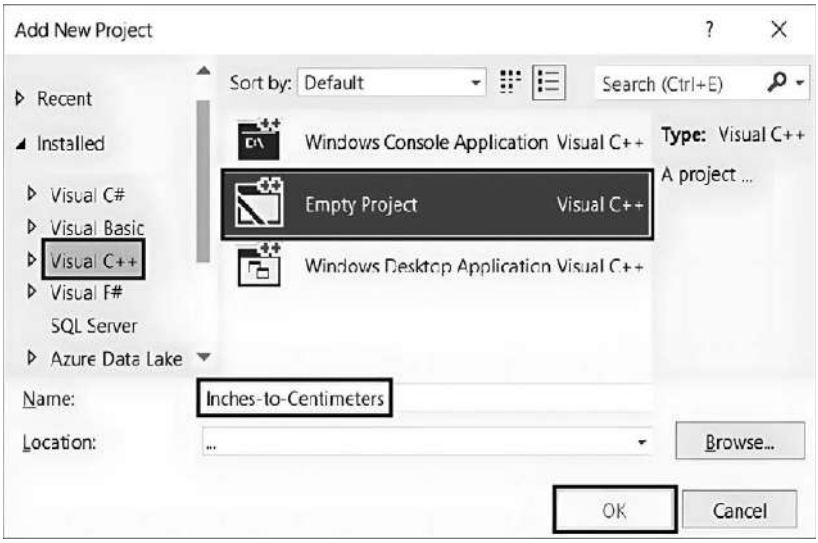


Задача: от дюймов к сантиметрам

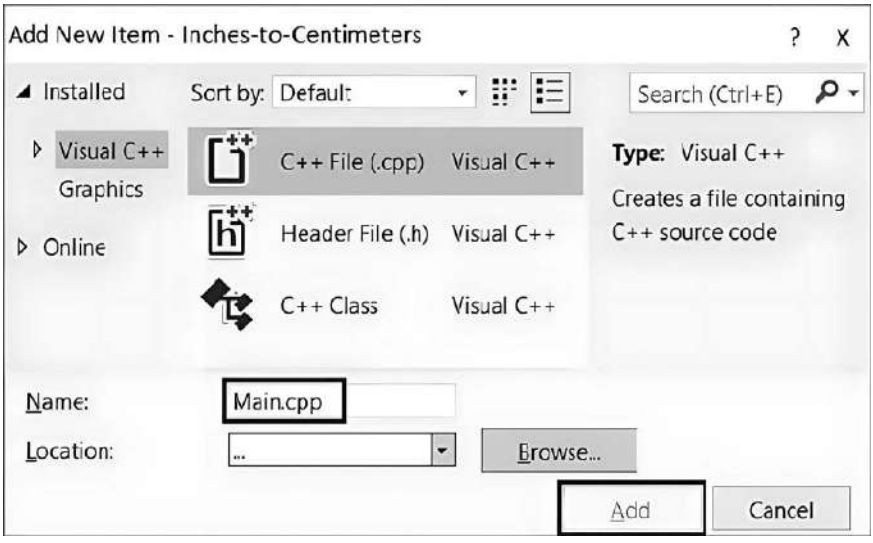
Напишите программу, которая считывает с консоли число (не обязательно целое) и переводит его из дюймов в сантиметры. Для этого дюймы умножаются на 2,54 (так как 1 дюйм = 2,54 сантиметра).

Рекомендации и советы

Сначала мы создадим новый проект на языке C++ в решении «Простые вычисления». Для этого щелкнем правой кнопкой мыши на решении в Solution Explorer и выберем [Add] -> [New Project...]. Выбираем [Visual C++] -> [Empty Project] и задаем имя «Inches-to-Centimeters»:



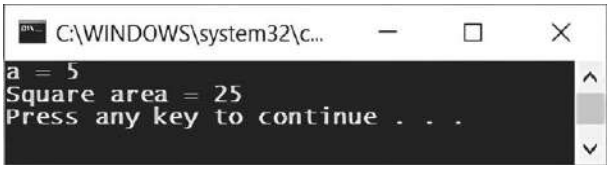
Затем добавляем файл Main.cpp (из меню [Source Files] -> [Add] -> [New Item...]):



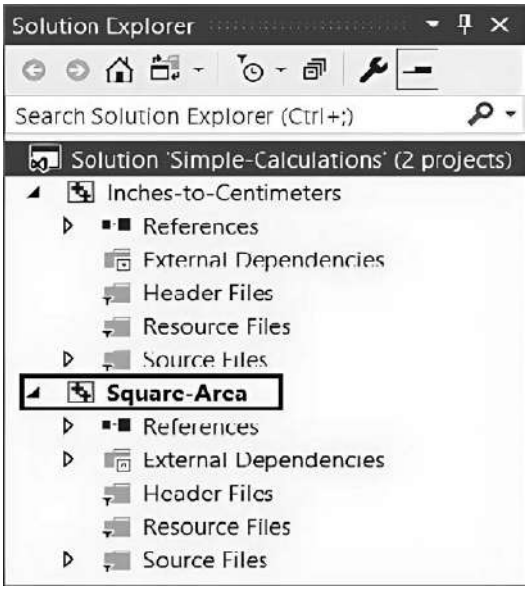
Далее необходимо написать программный код:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Inches = ";
    double inches;
    cin >> inches;
    double centimeters = inches * 2.54;
    cout << "Centimeters =
    cout << centimeters << endl;
    return 0;
}
```

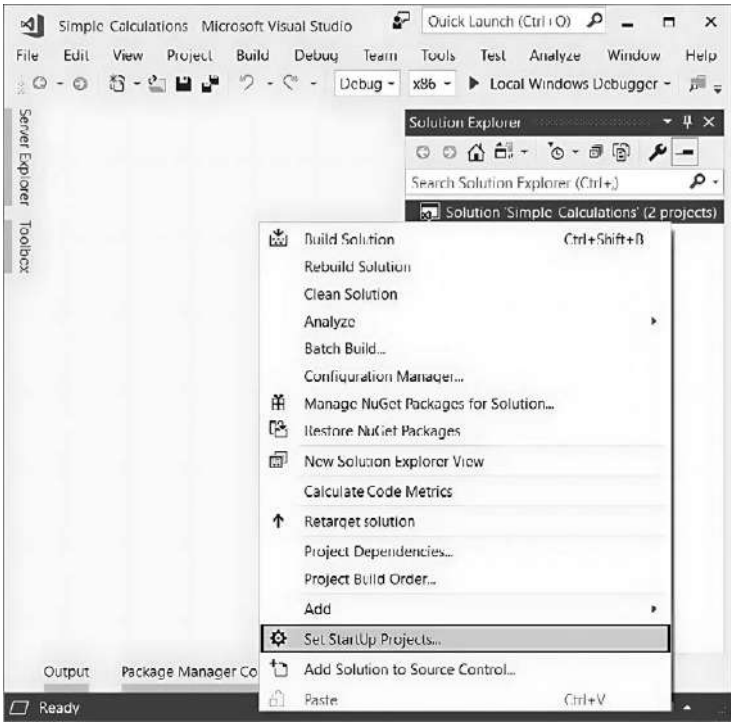
Запустите программу с помощью [Ctrl + F5]:



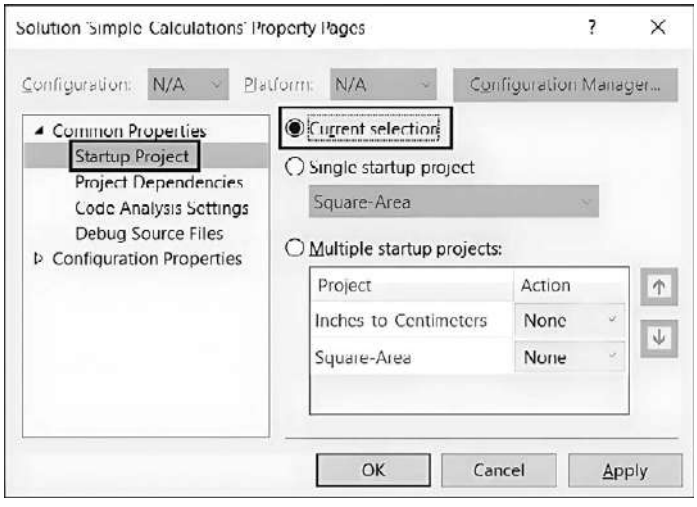
Сюрприз! Что происходит? Программа работает некорректно... А не является ли это на самом деле предыдущей программой? В Visual Studio *текущий активный проект* в Solution выделяется жирным шрифтом и может быть изменен. Как мы видим, выделенным проектом является «Square-Area»:



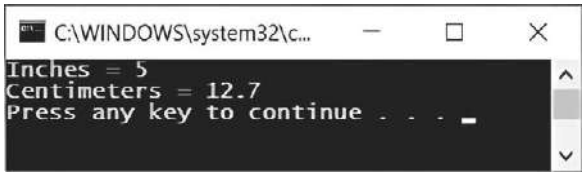
Чтобы включить *автоматическое переключение на текущий проект*, щелкните правой кнопкой мыши на решении и выберите [Set StartUp Projects...]:



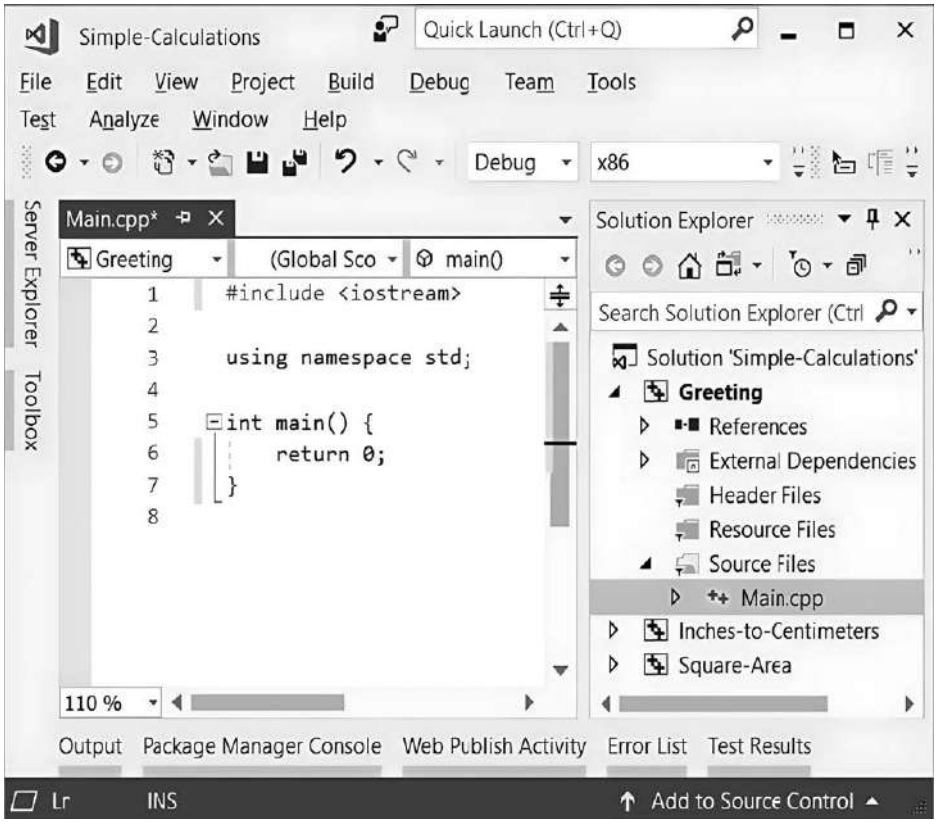
Появится диалоговое окно, в котором необходимо выбрать [Startup Project] -> [Current Selection]:



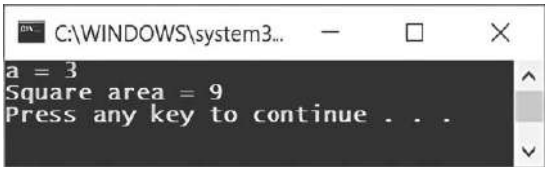
Теперь снова запустите программу, как обычно, с помощью [Ctrl + F5]. На этот раз будет запущена текущая открытая программа, которая преобразует дюймы в сантиметры. Кажется, что все работает правильно:



Теперь перейдем к программе, которая вычисляет площадь квадрата. Для этого необходимо дважды щелкнуть на файле Main.cpp из предыдущего проекта «Square-Area» в панели [Solution Explorer] Visual Studio:

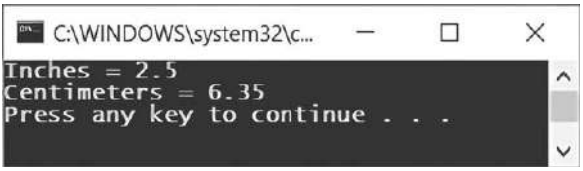


Снова нажмите [Ctrl + F5]. На этот раз должен быть запущен другой проект, как показано на скриншоте:



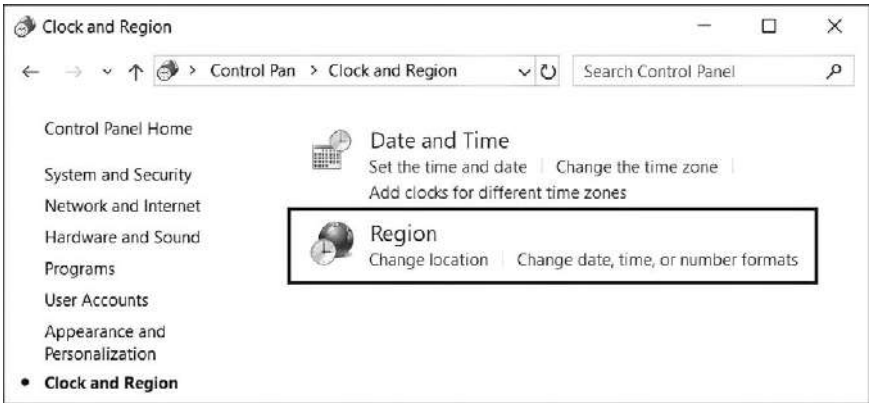
Вернитесь к проекту «Inches-to-Centimeters» и запустите его с помощью [Ctrl + F5]:

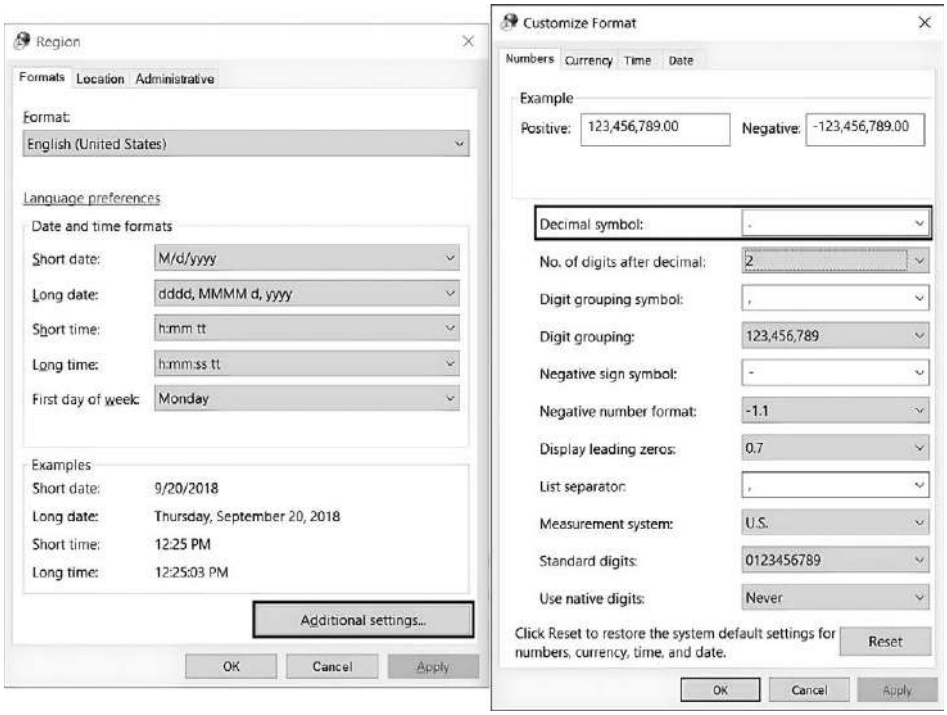
Переключаться между проектами очень просто, не правда ли? Достаточно выбрать файл с исходным кодом программы, дважды щелкнуть по нему мышью, и программа при запуске стартует из этого файла. Проведем тест с дробными числами, например `2.5`:



В зависимости от региональных настроек операционной системы возможно использование десятичной запятой (настройки BG) вместо десятичной точки (настройки US).

Поэтому рекомендуется изменить настройки компьютера таким образом, чтобы использовалась десятичная точка:



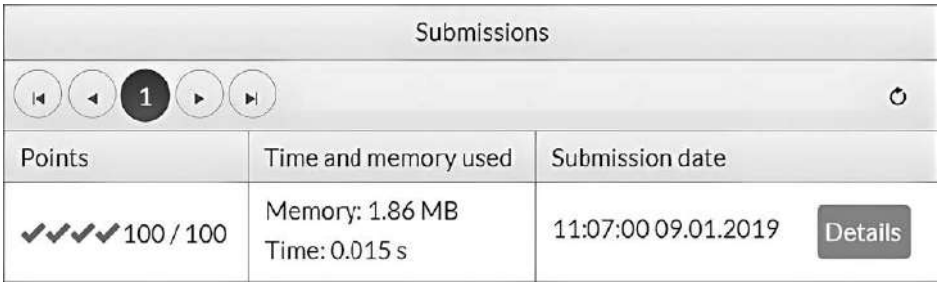


Если программа ожидает число с десятичной запятой, а вводится число с десятичной точкой или наоборот (десятичная запятая вместо десятичной точки), то возможно возникновение ошибки, которая будет выведена на консоль.

Проверка в судейской системе

Проверьте пример программы в судейской системе SoftUni, которую можно найти по адресу: https://judge.softuni.bg/Contests/Practice/Index/1358#1.

Решение должно быть полностью правильным:

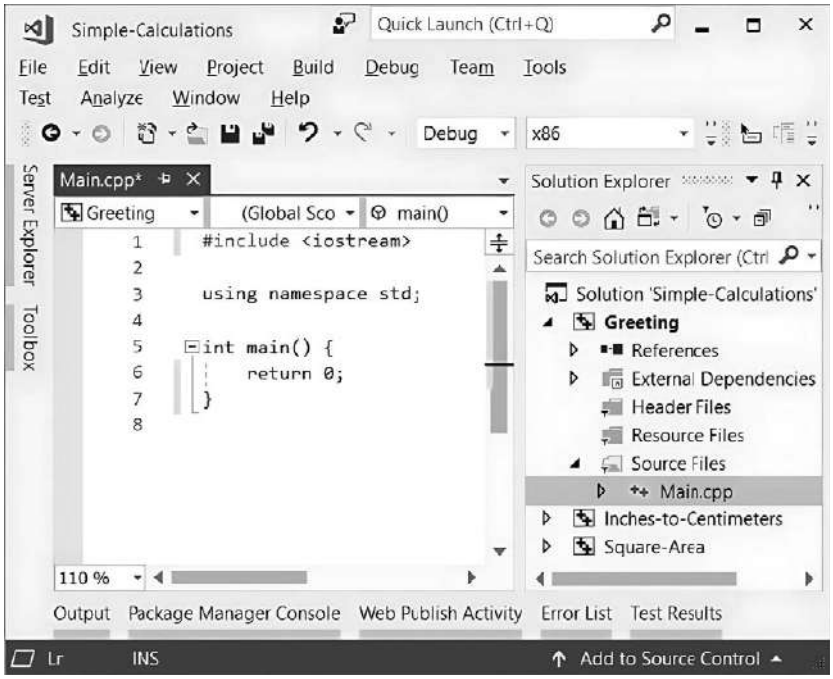


Задание: приветствие по имени

Напишите программу, которая считывает имя человека с консоли и выводит `Hello, <name>!`, где `<name>` — ранее введенное имя.

Рекомендации и советы

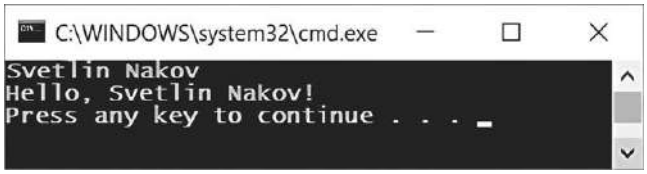
Сначала в решении «Simple-Calculations» создадим новый проект на языке C++ с именем «Greeting»:



Далее необходимо написать код программы. Для этого можно воспользоваться приведенным ниже примером:

```
#include <iostream>
#include <istream>
#include <string>

using namespace std;

int main() {
    string name;
    getline(cin, name);
    cout << "Hello, " << name << endl;
return 0;
}
```

Запустите программу с помощью [Ctrl + F5] и проверьте, работает ли она:



Проверка в судейской системе

Проверьте программу в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1358#2.

Задача: конкатенация текста и чисел

Напишите программу на языке C++, которая считывает с консоли имя, фамилию, возраст и город и выводит сообщение следующего вида: `You are <firstName> <lastName>, a <age>-years old person from <town>`.

Рекомендации и советы

Добавим к текущему решению Visual Studio еще один проект на языке C++ с именем «Concatenate-Data». Напишем код, считывающий входные данные с консоли:

```
string firstName, lastName, town;
int age;
cin >> firstName;
cin >> lastName;
cin >> age;
cin >> town;
```

Код, выводящий сообщение, описанное в постановке задачи, **намеренно сделан нечетким и должен быть дописан читателем**:

Далее мы тестируем решение локально с помощью [Ctrl + F5] и выборки входных данных.

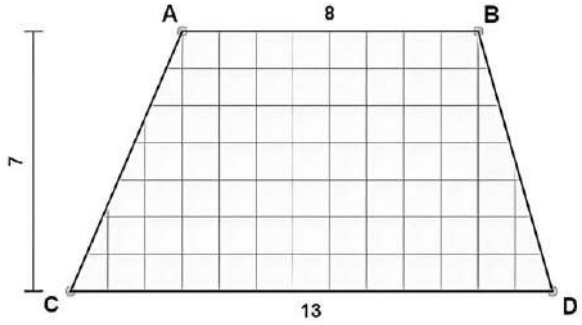
Проверка в судейской системе

Проверьте программу в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1358#3.

Задача: площадь трапеции

Напишите программу, которая считывает с консоли три числа `b1`, `b2` и `h` и вычисляет площадь трапеции с основаниями `b1` и `b2` и высотой `h`. Напомним, что формула для площади трапеции имеет вид `(b1 + b2) * h / 2`.

На рисунке изображена трапеция со сторонами 8 и 13 и высотой 7. Ее площадь (8 + 13) * 7 / 2 = 73,5.



Рекомендации и советы

Снова нужно добавить к текущему решению Visual Studio еще один проект на языке C++ с именем «Trapezoid-Area» и написать *код, который считывает входные данные с консоли, вычисляет площадь трапеции и выводит ее на экран*:

```
double b1, b2, h;

[illegible]

cout << "Площадь трапеции = " << area << endl;

return 0;
```

Код на картинке **намеренно размыт**, чтобы вы могли подумать и дописать его самостоятельно.

Проверьте решение локально с помощью [Ctrl + F5] и введите данные выборки.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#4.

Задача: длина окружности и площадь круга

Напишите программу, которая считывает с консоли число `r`, вычисляет и выводит площадь круга и длину окружности радиуса `r`.

Примеры ввода и вывода

Ввод	Вывод
3	Area = 28,2743 Perimeter = 18,8496

Ввод	Вывод
4	Area = 63,6173 Perimeter = 28,2743

Рекомендации и советы

Для расчетов можно использовать следующие формулы:

- `Area = pi * r * r`
- `Perimeter = 2 * pi * r.`

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#5.

Задача: площадь прямоугольника на плоскости

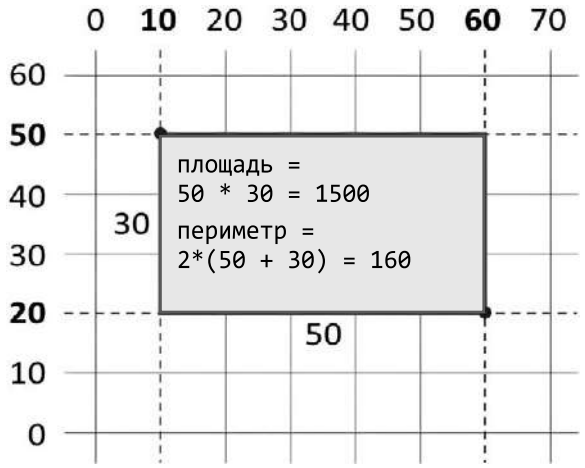
Прямоугольник задан координатами двух его противоположных углов `(x1, y1) - (x2, y2)`. Вычислите его площадь и периметр. Входные данные считываются с консоли. Числа `x1`, `y1`, `x2` и `y2` задаются по одному в строке. Выходные данные выводятся на консоль и должны содержать две строки с одним числом в каждой (площадь и периметр).

Примеры ввода и вывода

Ввод	Вывод
60 20 10 50	1500 160

Ввод	Вывод
30 40 70 -10	2000 180

Ввод	Вывод
600.25 500.75 100.50 -200.5	350449.6875 2402



Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#6.

Задача: площадь треугольника

Напишите программу, которая считывает с консоли стороны и высоту треугольника и вычисляет его площадь. Используйте формулу для площади треугольника: `area = a * h / 2`.

Округлите результат до двух знаков после запятой с помощью `fixed << setprecision(2)` за счет включения библиотеки `<iomanip>`.

Примеры ввода и вывода

Ввод	Вывод
20 30	Triangle area = 300
7.75 8.45	Triangle area = 32.74

Ввод	Вывод
15 35	Triangle area = 262.5
1.23456 4.56789	Triangle area = 2.82

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1358#7.

Задача: консольный преобразователь из градусов °C в градусы °F

Напишите программу, которая считывает градусы Цельсия (°C) и преобразует их в градусы Фаренгейта (°F). Найдите в Интернете подходящую формулу для выполнения вычислений. Округлите результат до двух знаков после запятой.

Примеры ввода и вывода

Ввод	Вывод
25	77

Ввод	Вывод
0	32

Ввод	Вывод
-5.5	22.1

Ввод	Вывод
32.3	90.14

Проверка в судейской системе

Проверьте программу в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1358#8.

Задача: консольный преобразователь из радиан в градусы

Напишите программу, которая считывает угол в радианах (`rad`) и преобразует его в градусы (`deg`). Найдите в Интернете подходящую формулу.

Округлите полученный результат до ближайшего целого числа с помощью функции `round(degree)`, которая находится в библиотеке `<cmath>`.

Примеры ввода и вывода

Ввод	Вывод
3.1416	180
6.2832	360

Ввод	Вывод
0.7854	45
0.5236	30

Проверка в судейской системе

Проверьте программу в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1358#9.

Задача: консольный конвертер из USD в BGN

Напишите программу для конвертации долларов США (USD) в болгарские левы (BGN). Результат округлите до двух знаков после запятой. Используйте фиксированный курс доллара США к болгарским левам: 1 доллар США = 1,79549 лева.

Примеры ввода и вывода

Ввод	Вывод
20	35.91 BGN

Ввод	Вывод
100	179.55 BGN

Ввод	Вывод
12.5	22.44 BGN

Проверка в судейской системе

Проверьте программу в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1358#10.

Задача*: консольный конвертер валют

Напишите программу для перевода денежной суммы из одной валюты в другую. Должны поддерживаться следующие валюты : болгарские левы, доллары США, евро, фунты стерлингов. Используйте фиксированные курсы валют:

Курс	USD	EUR	GBP
1 BGN	1.79549	1.95583	2.53405

На вход подаются сумма для конвертации, валюта ввода и валюта вывода. На выходе получается одно число — пересчитанная сумма по указанным курсам, округленная до двух цифр после запятой.

Примеры ввода и вывода

Ввод	Вывод
20 USD BGN	35.91 BGN

Ввод	Вывод
12.35 EUR GBP	9.53 GBP

Ввод	Вывод
100 BGN EUR	51.13 EUR

Ввод	Вывод
150.35 USD EUR	138.02 EUR

Проверка в судейской системе

Проверьте программу в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/504#11.

Глава 2.2.
Простые вычисления — экзаменационные задачи

В предыдущей главе мы познакомились с системной консолью и работой с ней — как считать число с консоли и как вывести результат на консоль. Мы рассмотрели основные арифметические операции и кратко упомянули о типах данных. В этой главе мы отработаем и закрепим полученные знания, рассмотрев несколько более сложных задач, задаваемых на экзаменах.

Чтение чисел из консоли

Прежде чем перейти к решению задач, повторим наиболее важный материал предыдущей главы. Начнем с чтения чисел из консоли.

Чтение целого числа

Нам нужна переменная, в которой мы будем хранить число (например `num`), и стандартная команда чтения с консоли `cin`:

```
int num;
cin >> num;
```

Протестировать пример можно в Интернете: https://repl.it/@vncpetrov/ReadingNumber.

Чтение дробного числа

Эта опреация аналогична чтению целого числа, но на этот раз нам нужно использовать переменную типа `double`:

```
double num;
cin >> num;
```

Запустить и протестировать этот код можно в Интернете: https://repl.it/@vncpetrov/ReadingFloatingPointNumber.

Отображение текста по шаблону (placeholder)

Заполнитель, или шаблон (placeholder), — это выражение, которое при выводе будет заменено определенным значением. Функция `printf(...)` поддерживает вывод текста по шаблону, где первым передаваемым аргументом является строка формата, за которой следует количество аргументов, равное количеству плейсхолдеров. Функция находится в библиотеке `stdio.h`, и на нее необходимо ссылаться в начале программы:

```
printf("Вы - %s %s, человек в возрасте %d лет из %s",
firstName.c_str(), secondName.c_str(), age, town.c_str());
```

Вы можете протестировать пример в Интернете: https://repl.it/@vncpetrov/Placeholders.

Арифметические операторы

Вспомним основные арифметические операторы для вычислений.

Оператор +

```
int result = 3 + 5; // результат равен 8
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/Sum.

Оператор -

```
int result = 3 - 5; // результат равен -2
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/Difference.

Оператор *

```
int result = 3 * 5; // результат равен 15
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/Multiplication.

Оператор /

```
int result = 7 / 3; // результат 2 (целочисленное деление)
double result2 = 5 / 2.0; // результат 2.5 (дробное деление)
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/Division.

Конкатенация

При использовании оператора «+» между переменными текстового типа выполняется так называемая конкатенация (объединение строк).

```
string firstName = "Ivan";
string lastName = "Ivanov";
int age = 19;
string str = firstName + " " + lastName + " is " +
to_string(age) + " years old";
// Ivan Ivanov is 19 years old
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ConcatenationExample.

Функция `to_string(...)` преобразует годы в текст, чтобы можно было выполнить конкатенацию.

Экзаменационные задачи

Теперь, когда мы вспомнили, как выполнять вычисления, как читать и выводить числа на консоль, перейдем к решению задач. Мы решим несколько задач из вступительного экзамена для поступления в SoftUni.

Задача: учебный класс

Аудитория имеет прямоугольную форму размером `l` на `w` метров, без колонн внутри. Зал разделен на две части, левую и правую, с коридором примерно посередине. В левой и правой частях расположены ряды парт. В задней части зала находится большой дверной проем. В передней части зала находится кафедра с трибуной для лектора. Одно рабочее место занимает площадь 70 на 120 см (стол 70 на 40 см + стул 70 на 80 см и место для прохода). Ширина коридора составляет не менее 100 см. Рассчитано, что из-за входной двери (проем которой составляет 160 см) теряется ровно одно

рабочее место, а из-за кафедры (размер которой составляет 160 см на 120 см) — ровно два рабочих места. Напишите программу, которая принимает размеры аудитории и рассчитывает количество рабочих мест в ней для описанной планировки (см. рисунок).

Входные данные

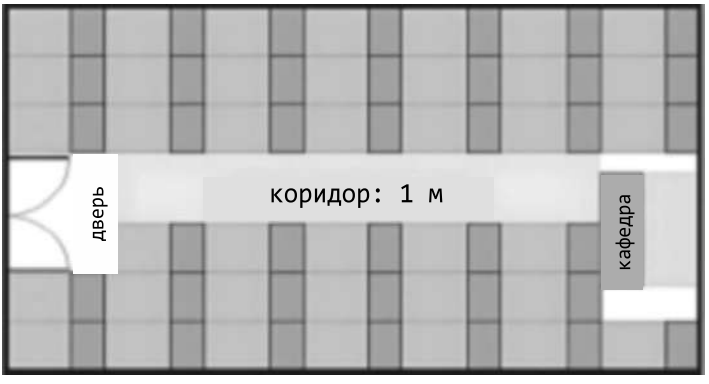
Консоль принимает два числа, по одному на строку: `l` (длина в метрах) и `w` (ширина в метрах).

Ограничения: $3 \le w \le l \le 100$.

Выходные данные

Выведите на консоль целое число: количество мест в учебном зале (аудитории).

Примеры ввода и вывода

Ввод	Вывод	Чертеж
15 8.9	129	
8.4 5.2	39	

Пояснения к примерам

В первом примере длина зала составляет 1500 см. В нем можно разместить 12 рядов (12 * 120 см = 1440 + 60 см остатка). Ширина зала составляет 890 см. Из них 100 см приходится на коридор посередине. На оставшихся 790 см можно разместить 11 парт в каждом ряду (11 * 70 см = 770 см + 20 см остаток). Количество мест = 12 * 11 - 3 = 132 - 3 = 129 (имеем 12 рядов по 11 мест = 132 минус 3 места для кафедры и входной двери).

Во втором примере длина зала составляет 840 см. В нем можно разместить 7 рядов (7 * 120 см = 840, без остатка). Ширина зала составляет 520 см. Из них 100 см приходится на коридор посередине. На оставшихся 420 см можно разместить по 6 парт на ряд (6 * 70 см = 420 см, без остатка). Количество мест = 7 * 6 - 3 = 42 - 3 = 39 (у нас 7 рядов по 6 мест = 42 минус 3 места для кафедры и дверного проема).

Рекомендации и советы

Сначала попробуйте решить задачу самостоятельно. Если это не удается, обратитесь к подсказкам.

Идея решения

Как и в любой другой задаче по программированию, прежде чем приступить к написанию кода важно иметь представление о решении. Давайте внимательно посмотрим на заданное условие. Нам необходимо написать программу, которая вычисляет количество рабочих мест в помещении, причем это количество зависит от длины и ширины помещения. Обратите внимание, что на вход нам будут подаваться метры, а информация о том, сколько места занимают рабочие места и коридор, дается в сантиметрах. Для выполнения вычислений нам необходимо использовать одни и те же единицы измерения, причем неважно, будем ли мы переводить высоту и длину в сантиметры или остальные данные в метры. Для представленного здесь решения выбран первый вариант.

Далее необходимо рассчитать, сколько мест рядом и сколько рядов столов поместится. Количество мест рядом мы можем рассчитать, вычтя из ширины пространство, необходимое для коридора (100 см) и разделив остаток на 70 см (ширина рабочего места). Количество рядов мы найдем, разделив длину на 120 см. Обе операции могут дать вещественное число с целой и дробной частью, одна-

ко нам нужно хранить в переменной только целую часть. Наконец, умножим количество рядов на количество мест рядом и вычтем из него 3 (места, которые теряются из-за входной двери и кафедры). Это даст нам искомое значение.

Выбор типов данных

Из примера входных данных видно, что на вход может быть подано вещественное число с целой и дробной частью, поэтому выбирать тип `int` нецелесообразно, используем для них `double`. Выбор типа для следующих переменных зависит от выбранного нами метода решения. Как и любая задача по программированию, эта также имеет несколько методов решения. Здесь будут показаны два из них.

Решение

Настало время перейти к решению. Мы можем мысленно разделить его на три подзадачи.

- Чтение входных данных.
- Выполнение расчетов.
- Консольный вывод.

Первое, что необходимо сделать, это считать входные данные из консоли. С помощью функции `cin` мы считываем значения с консоли, при этом тип данных будет `double`:

```
double length;
cin >> length;

double width;
cin >> width;
```

Приступим к вычислениям. Особенность заключается в том, что после выполнения деления нам достаточно сохранить весь результат в переменной.

Ищите в Google! Когда у нас есть идея, как решить задачу, но мы не знаем, как написать ее на C++, или когда мы сталкиваемся с проблемой, которая, как мы предполагаем, есть у многих других людей, самый простой способ решить ее — поискать информацию в Интернете.

В этом случае мы можем попробовать следующий поиск: «C++ get integer part of double». Находим, что одной из возможностей является использование функции `trunc(...)`. Поскольку она работает с переменными типа `double`, то для количества строк и столбцов мы также создадим переменные этого типа. Приведенный ниже код намеренно обфусцирован (скрыт) и должен быть доработан читателем:

Второй вариант: как мы уже знаем, оператор деления «`/`» по-разному действует на целые и вещественные числа. При делении целого числа на целочисленный тип (например `int`) возвращаемый результат снова является целым числом. Поэтому мы можем поискать, как преобразовать вещественные числа, которые мы имеем в качестве значений высоты и ширины, в целые числа, а затем выполнить деление.

Поскольку в этом случае возможны потери данных при удалении дробной части, необходимо сделать преобразование явным (explicit typecasting). Для этого мы используем оператор преобразования данных (`type`), заменяя слово `type` на требуемый тип данных и помещая его перед переменной:

```
int cols = ((int)width - 100) / 70;
int rows = (int)length / 120;
int seats = rows * cols - 3;
```

С помощью `cout` выводим результат на консоль:

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1359#0.

Задача: овощной рынок

Садовод-огородник продает урожай со своего огорода на овощном рынке. Овощи он продает по цене N левов за килограмм, а фрукты — по цене M левов за килограмм. Напишите программу, которая

вычисляет доход от урожая в евро (при условии, что один евро равен 1,94 лева).

Исходные данные

Консоль считывает четыре числа, по одному на строку:

- Первая строка — цена за кг овощей — число с плавающей точкой.
- Вторая строка — цена за кг фруктов — число с плавающей точкой.
- Третья строка — всего килограммов овощей — целое число.
- Четвертая строка — общее количество килограммов фруктов — целое число.

Все числа будут находиться в диапазоне от 0,00 до 1000,00.

Исходные данные

Выведите на консоль число с плавающей точкой: выручку от продажи всех фруктов и овощей в евро.

Примеры ввода и вывода

Ввод	Вывод
0.194 19.4 10 10	101

Ввод	Вывод
1.5 2.5 10 10	20.6185567010309

Пояснение к первому примеру:

- Стоимость овощей: 0,194 лв. * 10 кг = 1,94 лв.
- Стоимость фруктов: 19,4 лв. * 10 кг = 194 лв.
- Итого: 195,94 лв. = 101 евро.

Рекомендации и советы

Сначала мы приведем некоторые соображения, а затем дадим конкретные рекомендации по решению задачи, а также существенную часть кода.

Идея решения

Рассмотрим сначала условие задачи. В данном случае нас просят рассчитать, сколько составляет общий доход от собранного урожая.

Он равен сумме прибыли от продажи фруктов и овощей, и мы можем рассчитать ее, умножив цену за килограмм на количество. Входные данные даны в левах, а выходные — в евро. По условию, 1 евро равен 1,94 лева, поэтому для получения требуемой величины выхода нужно разделить сумму на 1,94.

Выбор типов данных

После уточнения идеи решения задачи можно переходить к выбору подходящих типов данных. Рассмотрим исходные данные: даны два целых числа для общего количества килограммов овощей и фруктов, переменные, которые мы объявим для хранения их значений, будут иметь тип `int`. Для цен на овощи и фрукты указано, что будут передаваться два числа с плавающей точкой, то есть переменные будут иметь тип `double`.

Мы также можем объявить две переменные, чтобы хранить значение прибыли от продажи овощей и фруктов отдельно. Поскольку мы умножаем переменную типа `int` (общее количество килограммов) на переменную типа `double` (цена), то и результат должен быть типа `double`. Поясним: операторы обычно работают с аргументами одного типа. Поэтому, чтобы выполнить такую операцию как умножение над двумя разными типами данных, необходимо привести их к одному типу. Если в выражении присутствуют типы с разными областями видимости, то преобразование всегда выполняется к типу с наибольшей областью видимости, в данном случае это `double`. Поскольку при этом не возникает опасности потери данных, преобразование является неявным и выполняется компилятором автоматически.

В качестве выхода нам также требуется число с плавающей точкой, то есть результат мы будем хранить в переменной типа `double`.

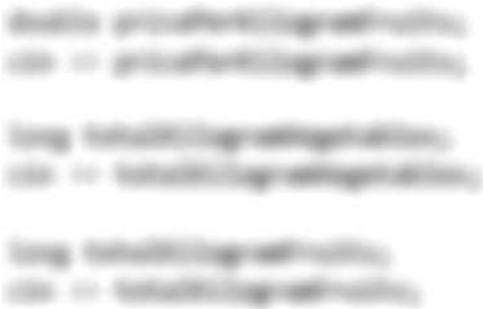
Решение

Пришло время перейти к решению задачи. Мы можем мысленно разделить его на три подзадачи:

- Чтение входных данных.
- Выполнение расчетов.
- Консольный вывод.

Для считывания входных данных мы объявляем переменные, стараясь использовать правильный тип данных и называть их так, чтобы было понятно, какие значения содержат переменные. С помощью функции `cin` мы считываем значения с консоли, а типы данных, которые мы будем использовать, будут `int` и `double`:

```
double pricePerKilogramVegetables;
cin >> pricePerKilogramVegetables;
```



Затем выполняем необходимые расчеты, как показано здесь:

```
double priceVegetables =
    pricePerKilogramVegetables * totalKilogramVegetables;
double priceFruits = pricePerKilogramFruits * totalKilogramFruits;
double totalPrice = priceVegetables + priceFruits;
```

В условии задачи специальное форматирование вывода не задано, но для того, чтобы судейская система приняла наше решение, необходимо использовать `setprecision` из-за неточностей, возникающих при вычислениях.

Как в математике, так и в программировании деление имеет приоритет над сложением. Однако в данной задаче мы должны сначала вычислить сумму двух полученных значений, а затем разделить на 1,94. Чтобы отдать предпочтение сложению, можно использовать круглые скобки. С помощью `cout` мы выводим результат на консоль:

```
cout << setprecision(15) << totalPrice << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1359#1.

Задача: ремонт плитки

Плитка должна быть уложена на детской площадке перед многоквартирным домом. Площадка должна быть квадратной со стороной N метров. Ширина плитки — W метров, длина — L метров. На детской площадке есть скамейка шириной M метров и длиной O метров. Под нее не нужно класть плитку. Каждая плитка укладывается за 0,2 минуты.

Напишите программу, которая считывает с консоли размеры детской площадки, плитки и скамейки и вычисляет, сколько плиток необходимо для покрытия площадки, а также рассчитывает время, необходимое для укладки всех плиток.

Пример: площадка размером 20 м имеет площадь 400 кв. м. Скамейка шириной 1 м и длиной 2 м занимает площадь 2 кв. м. Плитка шириной 5 м и длиной 4 м имеет площадь 20 кв. м. Площадь, которую необходимо покрыть, составляет 400 - 2 = 398 кв. м. Требуется 398 / 20 = 19,90 плитки. Необходимое время составляет 19,90 * 0,2 = 3,98 минуты.

Входные данные

С консоли считывается пять чисел:

- N — длина стороны площадки в интервале [1 ...100].
- W — ширина плитки в интервале [0,1 ...10,00].
- L — длина плитки в интервале [0,1 ...10,00].
- M — ширина скамьи в интервале [0 ...10].
- O — длина скамьи в интервале [0 ...10].

Выходные данные

Выведите на консоль два числа:

- количество плиток, необходимых для ремонта
- время укладки

Каждое число должно быть записано с новой строки и округлено до второго десятичного знака.

Примеры ввода и вывода

- Общая площадь = 20 * 20 = 400.
- Площадь скамьи = 1 * 2 = 2.

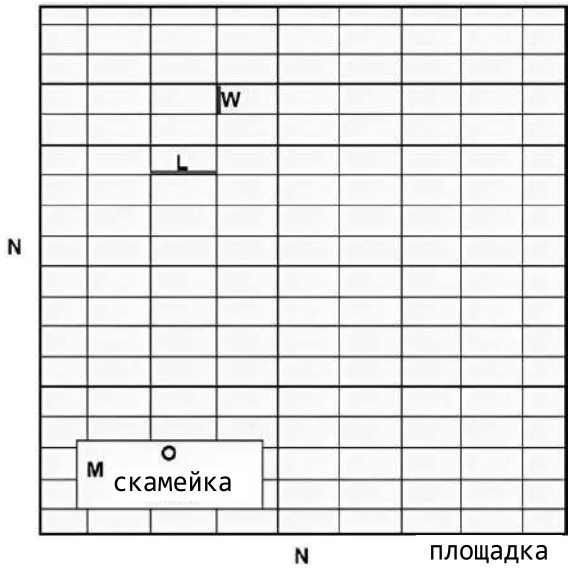
- Площадь покрытия = 400 - 2 = 398.
- Площадь плитки = 5 * 4 = 20.
- Требуется плитки = 398 / 20 = 19,9.
- Требуемое время = 19,9 * 0,2 = 3,98.

Ввод	Вывод
20 5 4 1 2	19.9 3.98

Ввод	Вывод
40 0.8 0.6 3 5	3302.08333333333 660.416666666667

Рекомендации и советы

Для уточнения условия задачи сделаем рисунок. Он может выглядеть так, как показано ниже.



Идея решения

От нас требуется рассчитать количество укладываемых плиток и время, которое потребуется для этого. Чтобы рассчитать количество, необходимо вычислить площадь покрытия и разделить ее на сторону одной плитки. По условию, площадка имеет квадратную форму, поэтому общую площадь мы найдем, умножив ее сторону на значение `N * N`. Затем вычисляем площадь, занимаемую скамейкой, также умножая две ее стороны — `M * O`. Вычтя площадь скамейки из площади всего участка, получим площадь, подлежащую ремонту.

Площадь одной плитки мы вычисляем путем умножения одной стороны на другую — `W * L`. Как отмечалось ранее, теперь нам нужно разделить площадь участка на площадь плитки. Это даст нам необходимое количество плиток. Умножаем его на 0,2 (время, необходимое для укладки одной плитки, по условию). Таким образом, мы уже будем иметь необходимые выходные значения.

Выбор типов данных

Длина стороны игровой площадки, ширина и длина скамейки будут заданы как целые числа, поэтому для хранения их значений можно объявить переменные типа `int`. Для ширины и длины плитки нам будут даны вещественные числа (с целой и дробной частью), поэтому для их хранения мы будем использовать тип `double`. Выходом задачи снова будет вещественное число, поэтому переменные также будут иметь тип `double`.

Решение

Как и в предыдущих задачах, мы можем мысленно разделить решение на три части:

- Чтение входных данных.
- Выполнение расчетов.
- Консольный вывод.

Прежде всего необходимо рассмотреть исходные данные для решения задачи. Важно внимательно отнестись к порядку их подачи. С помощью `cin` мы считываем значения с консоли:

```
int groundLength;
cin >> groundLength;
```

```
[illegible]
[illegible]

[illegible]
[illegible]

[illegible]
[illegible]

[illegible]
[illegible]
```

После инициализации переменных и сохранения в них соответствующих значений мы приступаем к вычислениям. Поскольку значения переменных `groundLength`, `benchWidth` и `benchLength`, с которыми мы работаем, хранятся в переменных типа `int`, мы можем определить переменные этого типа для результатов вычислений. Приведенный ниже код намеренно дан размыто, чтобы читатель мог самостоятельно додуматься до него:

```
[illegible]
[illegible]
[illegible]
```

Переменные `tileWidth` и `tileLength` имеют тип `double`. В завершение вычисляем значения для вывода на консоль. Необходимое количество плиток мы получаем, разделив площадь покрытия на площадь одной плитки. При делении чисел, одно из которых вещественное, в результате получается вещественное число с целой и дробной частью. Поэтому для корректности вычислений мы храним результат в переменной типа `double`. В постановке задачи не указано специальное форматирование или округление результата, поэтому мы просто выводим значения с помощью `cout`:

```
double tiles = areaToRepair / (tileWidth * tileLength);
double time = tiles * 0.2;

cout << tiles << endl;
cout << time << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1359#2.

Задача: конверсия валют

Некоторое время назад Петр купил биткоины. Теперь он отправится в путешествие по Европе, и ему понадобятся евро. Кроме биткоинов у него есть еще китайские юани. Петр хочет обменять свои деньги на евро для поездки. Напишите программу, которая вычислит, сколько евро он сможет купить по следующим курсам:

- 1 биткойн = 1168 левов
- 1 китайский юань = 0,15 долл.
- 1 доллар = 1,76 лева
- 1 евро = 1,95 лева

Комиссия обменного пункта составляет от 0 % до 5 % от конечной суммы в евро.

Входные данные

С консоли считываются три числа:

- В первой строке — количество биткоинов. Целое число в диапазоне [0...20].
- Во второй строке — количество китайских юаней. Вещественное число в диапазоне [0.00...50 000.00].
- В третьей строке — комиссия. Вещественное число в диапазоне [0.00...5.00].

Выходные данные

Выведите на консоль одно число — результат обмена валюты. Результат не нужно округлять.

Примеры ввода и вывода

Ввод	Вывод
1 5 5	569.668717948718

Пояснение:

- 1 биткойн = 1168 левов
- 5 юаней = 0,75 долл.
- 0,75 доллара = 1,32 лева
- 1168 + 1,32 = 1169,32 лева = 599,651282051282 евро
- Комиссия: 5 % от 599.651282051282 = 29.9825641025641
- Результат: 599.651282051282 - 29.9825641025641 = 569.668717948718 евро

Ввод	Вывод
20 5678 2.4	12442.2442010256

Ввод	Вывод
7 50200.123	10659.4701177436

Рекомендации и советы

Опять же, прежде чем приступать к написанию кода, давайте сначала подумаем, как можно решить задачу.

Идея решения

Мы видим, что нам будет дано количество биткоинов и количество китайских юаней. Выходное значение указано в евро. В условии также указаны курсы валют, с которыми нам необходимо работать. Мы видим, что конвертировать в евро можно только сумму в левах, поэтому сначала нужно вычислить общую сумму, которой владеет Петр, в левах, а затем рассчитать выходную стоимость.

Поскольку нам предоставлена информация о курсе биткойнов по отношению к леву, мы можем напрямую выполнить эту конвертацию. С другой стороны, для того чтобы получить стоимость китайских юаней в левах, нам необходимо сначала перевести их в доллары, а затем перевести доллары в левы. Наконец, мы сложим две полученные величины и вычислим, скольким евро они соответствуют.

Остается последний шаг: рассчитать, сколько составит комиссия, и вычесть полученную сумму из общей. В качестве комиссионных нам будет предоставлено реальное число, которое будет представлять собой определенный процент от общей суммы. Для расчета процентного значения разделим полученное число на 100. Умножим его на сумму в евро и вычтем полученный результат из этой же суммы. Полученную сумму выведем на консоль.

Выбор типов данных

Биткоины представлены в виде целого числа, поэтому для их значения мы можем использовать переменную типа `int`. Если взять количество китайских юаней и комиссионных, то получится вещественное число, поэтому для них мы используем `double`. Поскольку `double` — это тип данных с большим диапазоном, а выходное значение также будет вещественным числом, мы будем использовать его и для других создаваемых переменных.

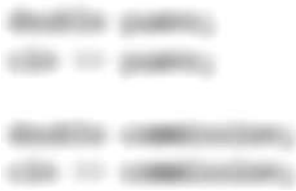
Решение

После того как мы построили идею решения задачи и выбрали структуры данных, с которыми будем работать, пора приступать к написанию кода. Как и в предыдущих задачах, мы можем разделить решение на три подзадачи:

- Чтение входных данных.
- Выполнение расчетов.
- Консольный вывод.

Мы объявляем переменные, которые будем использовать, опять же тщательно выбирая осмысленные имена, позволяющие понять, что они содержат. Инициализируем их значения — с помощью `cin` считываем переданные на консоль числа:

```
double bitcoins;
cin >> bitcoins;
```



Выполняем необходимые вычисления. Приведенный ниже код намеренно завуалирован и должен быть выполнен читателем самостоятельно:

```
double bitcoinsToLeva = bitcoins * 1168;
double yuansToDollars = yuans * 0.1S;
double dollarsToLeva = yuansToDollars * 1.76;

[illegible]
[illegible]
```

Наконец, мы вычисляем величину комиссии и вычитаем ее из суммы в евро. Обратим внимание на то, как это можно записать: `euro -= commission * euro` — это сокращенный вариант написания `euro = euro - (commission * euro)`. В данном случае мы используем составной оператор присваивания (`-=`), который вычитает значение операнда справа из операнда слева. Оператор умножения (`*`) имеет больший приоритет, чем комбинированный оператор, поэтому сначала выполняется выражение `commission * euro`, а затем его значение вычитается.

Никакого специального форматирования или округления результата в постановке задачи не указано, поэтому нам нужно просто вычислить результат и вывести его на консоль:

```
euro -= euro * commission
cout << euro << endl;
```

Отметим то, что справедливо для всех задач такого типа: в таком виде решение задачи получается достаточно подробным. Поскольку условие в общем случае несложное, то теоретически можно написать большое выражение для вычисления выходного значения непосредственно после получения входных данных. Например, такое выражение будет выглядеть следующим образом:

```
double euro = (bitcoins * 1168 + yuans * 0.15 * 1.76) / 1.95 -
((bitcoins * 1168 + yuans * 0.15 * 1.76) / 1.95 * commission)
```

Такой код даст правильный результат, но его трудно читать. Нелегко будет понять, что он делает, содержит ли он ошибки, и если да, то как их исправить. Лучше написать несколько простых выражений вместо одного сложного и записать результаты в переменные с соответствующими именами.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1359#3.

Задача: ежедневная прибыль

Иван работает в среднем N дней в месяц, зарабатывая в среднем M долларов в день. В конце года он получает премию, равную 2,5 месячной зарплаты. С заработанной за год суммы у него удерживается 25 % налогов. Напишите программу, которая вычислит, сколько долларов составляет чистый средний заработок Ивана в день. В году 365 дней. В функцию будет подаваться курс доллара по отношению к леву.

Входные данные

С консоли считываются три числа:

- В первой строке — рабочие дни в месяце. Целое число в диапазоне [5 ...30].
- Во второй строке — деньги, заработанные за день. Вещественное число [10.00 ... 2000.00].
- В третьей строке — курс доллара по отношению к леву (1 доллар = X левов). Это вещественное число в интервале [0,99 ...1,99].

Выходные данные

Выведите на консоль одно число — среднюю прибыль за день в левах. Отформатируйте результат до второй цифры после точки.

Примеры ввода и вывода

Ввод	Вывод
21 75.00 1.59	74.61

Ввод	Вывод
15 105 1.71	80.24

Ввод	Вывод
22 199.99 1.50	196.63

Пояснение к первому примеру:

- Заработная плата за 1 месяц = 21 * 75 = 1575 долл.
- Годовой доход = 1575 * 12 + 1575 * 2,5 = 22837,5 долл.
- Налог = 25 % от 22837,5 = 5709,375 лв.
- Чистый годовой доход = 17128,125 долл. = 27233,71875 лева.
- Средняя прибыль в день = 27233,71875 / 365 = 74,61 лв.

Рекомендации и советы

Сначала проанализируем задачу и выясним, как ее решить. Затем выберем типы данных и, наконец, напишем код решения.

Идея решения

Сначала рассчитаем, сколько составляет месячная зарплата Ивана. Для этого умножим количество рабочих дней в месяце на сумму, которую он зарабатывает за день. Полученный результат умножим сначала на 12, чтобы вычислить, сколько составляет его зарплата за 12 месяцев, а затем на 2,5, чтобы вычислить премию. Сложив два полученных значения, мы получим его общий годовой доход. Из него необходимо вычесть 25 %. Это можно сделать, умножив общую сумму дохода на 0,25 и вычтя полученный результат. Переводим доллары в евро по указанному курсу, затем делим полученный результат на количество дней в году, которое, как мы предполагаем, равно 365.

Выбор типов данных

Рабочие дни в месяце задаются в виде целого числа, поэтому для их значения мы можем объявить переменную типа `int`. Для заработанных денег, а также для курса доллара по отношению к леву мы получим вещественное число, поэтому для них мы используем `double`. Поскольку `double` — тип данных с большим диапазоном,

а выходное значение также требует вещественного числа (с целой и дробной частями), мы будем использовать его и для остальных создаваемых переменных.

Решение

И снова, представив себе, как решить задачу, и продумав типы данных, с которыми будем работать, переходим к написанию программы. Как и в предыдущих задачах, мы можем разделить решение на три подзадачи:

- Чтение входных данных.
- Выполнение расчетов.
- Консольный вывод.

Мы объявляем переменные, которые будем использовать, опять же стараясь выбрать подходящие имена. С помощью функции `cin` считываем переданные числа на консоль. Типы данных, которые мы будем использовать, это `int` и `double`:

Выполняем необходимые вычисления. Приведенный выше код намеренно скрыт и должен быть выполнен читателем самостоятельно.

```
double monthSalary = workDays * moneyPerDay;
double moneyPerYear = (monthSalary * 12) + (monthSalary * 2.5);
double taxes = moneyPerYear * 0.25;
double netSalary = moneyPerYear - taxes;
double salarylnLeva = netSalary * currencyRate;
```

Выражение, которое мы используем для расчета общего годового дохода, можно записать и без скобок. Поскольку умножение является более приоритетной операцией, чем сложение, оно будет выполнено первым. Однако при использовании больше-

го числа операторов рекомендуется писать круглые скобки, так как это облегчает чтение кода и снижает вероятность совершения ошибки.

Наконец, необходимо вывести результат на консоль. Обратите внимание, что требуется отформатировать числовое значение до второго десятичного знака. Для этого мы воспользуемся функцией `setprecision(...)`, которая находится в библиотеке `iomanip`:

```
double average = salaryInLeva / 365;
cout << fixed << setprecision(2) << average << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1359#4.

Глава 3.1.
Простые проверки

В этой главе мы рассмотрим условные операторы в языке C++, которые можно использовать для того, чтобы заставить нашу программу вести себя по-разному в зависимости от какого-либо условия. Мы объясним синтаксис условных операторов для проверок (`if`, `if-else`) на подходящих примерах и посмотрим, в каком диапазоне «живет» переменная (ее область видимости). Наконец, мы рассмотрим методы отладки, позволяющие последовательно проследить путь, пройденный нашей программой в процессе ее выполнения.

Сравнение чисел

В программировании мы можем сравнивать значения с помощью следующих операторов:

- Оператор < (меньше)
- Оператор > (больше)
- Оператор <= (меньше или равно)
- Оператор >= (больше или равно)
- Оператор == (равно)
- Оператор != (не равно)

При сравнении результат сводится к булевому значению — `true (1)` или `false (0)`, в зависимости от того, является ли результат сравнения истинным или ложным.

Примеры сравнения чисел

```
int a = 5;
int b = 10;
cout << boolalpha << (a < b);  // true
```

```
cout << boolalpha << (a > 0);         // true
cout << boolalpha << (a  > 100);      // false
cout << boolalpha << (a  < a);        // false
cout << boolalpha << (a  <= 5);       // true
cout << boolalpha << (b  == 2 * a);   // true
```

Обратите внимание, что для вывода булевых значений `true` и `false` с помощью оператора `cout` необходимо вызвать функцию `boolalpha`. В противном случае будет выведено `1` для истинного значения или `0` для ложного.

Сравнение чисел типов float и double

При сравнении чисел целого типа все происходит предсказуемо. Однако при сравнении чисел с плавающей точкой необходимо иметь в виду одну особенность. Рассмотрим следующий пример, который приведет к неожиданному результату:

```
double d1 = 100 - 99.99;
double d2 = 10 - 9,99;

cout << boolalpha << (d1 == d2) << endl;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ComparingFloatsAndDoubles.

При выполнении приведенной выше программы мы ожидали бы, что она выведет `true`, поскольку значения переменных `d1` и `d2` должны быть равны (0,01), но вместо этого она выводит `false`. Это связано с тем, что эти числа вычисляются со значением, достаточно близким к 0,01, но не равным в точности 0,01 — `d1` равно `0.010000000000000005116`, а `d2` равно `0.0099999999999997868`. Мы можем найти это, если выводить числа с достаточно высокой точностью после запятой:

```
double d1 = 100 - 99.99;
double d2 = 10 - 9,99;

cout << setprecision(17) << d1 << endl;
cout << setprecision(17) << d2 << endl;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ComparingFloatsAndDoubles2.

Простые проверки

В программировании мы часто проверяем заданные условия и выполняем различные действия в зависимости от результата проверки. Проверки выполняются с помощью предложений `if`, которые имеют следующую конструкцию:

```
if (boolean expression) {
    // тело условной конструкции;
}
```

Пример: отличная оценка

Мы вводим в консоль оценку (в виде десятичной дроби) и проверяем, является ли она отличной (≥ `5,50`).

```
double grade;
cin >> grade;

if (grade >= 5.50) {
    cout << "Отлично!" << endl;
}
```

Протестируйте код примера локально. Попробуйте ввести различные оценки, например 4.75, 5.49, 5.50 и 6.00. При оценке менее 5.50 программа ничего не выдаст, а при оценке 5.50 и более — выдаст сообщение “`Отлично!`”.

Проверка в судейской системе

Проверьте свою программу в автоматической судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#0.

Проверки с помощью конструкции if-else

Оператор `if` может также содержать предложение `else`, позволяющее выполнить определенное действие в случае, если булево выражение (которое указывается в начале `if (булево выражение)`) вернет отрицательный результат (`false`). Построенный таким образом условный оператор называется `if-else`, и его поведение тако-

во: если результат условия положительный (`true`) — мы выполняем одни действия, а если отрицательный (`false`) — другие. Формат утверждения следующий:

```
if (булево условие) {
    // тело условной конструкции;
}
else {
    // тело else-конструкции;
}
```

Пример: отличная оценка или нет

Аналогично приведенному выше примеру мы вводим оценку, проверяем, отличная ли она, но выводим результат в обоих случаях:

```
double grade;
cin >> grade;

if (grade >= 5.50) {
    cout << "Отлично!" << endl;
}
else {
    cout << "Не отлично." << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#1.

Использование фигурных скобок { } с if-else

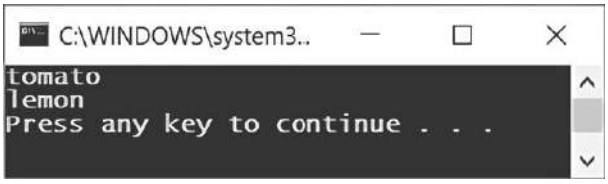
Если в теле оператора `if` содержится только одно утверждение, то фигурные скобки, обозначающие тело условного оператора, можно опустить. Когда мы хотим выполнить блок кода (группу команд), фигурные скобки обязательны. Если их опустить, то будет выполнена только первая строка после оператора `if`.

Это хорошая практика — *всегда ставить фигурные скобки*, потому что это делает наш код более читабельным и «чистым».

Приведем пример, в котором отказ от фигурных скобок приводит к путанице:

```
string color = "red";
if (color == "red")
    cout << "tomato" << endl;
else if (color == "yellow")
    cout << "banana" << endl;
cout << "lemon" << endl;
```

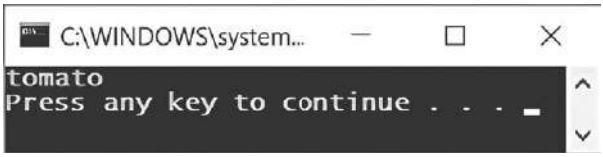
Выполнение приведенного выше кода выведет на консоль следующий результат:



С фигурными скобками:

```
string color = "red";
if (color == "red") {
    cout << "tomato" << endl;
}
else if (color == "yellow") {
    cout << "banana" << endl;
    cout << "lemon" << endl;
}
```

На консоль будет выведено следующее сообщение:



Обратите внимание, что для объявления и работы с переменными типа `string` в начале программы необходимо добавить следующую строку: `#include <string>`.

Безусловно, оба варианта корректны и могут быть использованы в зависимости от конкретного случая, требований документации и/или других требований, но при этом всегда следует проявлять абсолютную осторожность и учитывать результаты.

Пример: четное или нечетное

Напишите программу, которая проверяет, является ли целое число четным или нечетным.

Решить задачу можно с помощью оператора `if-else` и оператора %, который возвращает остаток при делении на два числа:

```
int num;
cin >> num;

if (num % 2 == 0) {
     cout << "Четное" << endl;
}
else {
    cout << "Нечетное" << endl;
}
```

Проверка в судейской системе

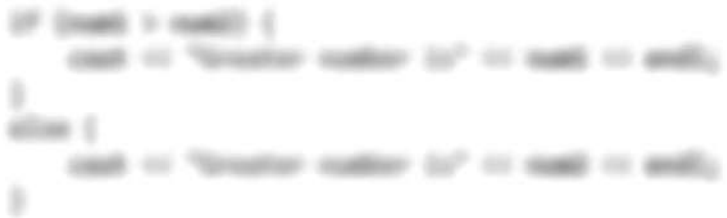
Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#2.

Пример: большее число

Напишите программу, которая считывает два целых числа и выводит большее из них.

Наша первая подзадача — прочитать два числа. Затем, используя простой оператор `if-else` в сочетании с оператором «больше» (>), мы выполняем проверку. Код намеренно размыт и должен быть дописан читателем:

```
int num1, num2
cin >> num1;
cin >> num2;
```



Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#3.

Область видимости переменной

Каждая переменная имеет область видимости, в которой она существует, называемую областью видимости переменной. Эта область определяет, где переменная может быть использована. В языке C++ область видимости переменной начинается со строки, в которой мы ее определили, и заканчивается первой закрывающей фигурной скобкой } (функции, оператора `if` и т. д.). В связи с этим важно знать, что любая переменная, определенная в теле оператора `if`, будет недоступна за его пределами, если мы не определили ее далее в коде.

В приведенном ниже примере в последней строке, где мы пытаемся вывести переменную `salary`, которая определена в операторе `if`, мы получим ошибку, поскольку у нас нет доступа к ней (в случае, если сама IDE предупреждает нас об области видимости переменной):

```
int myMoney = 500;
int payDayDate =7;
int todayDate = 10;

if (todayDate > payDayDate) {
    int salary = 5000;
    myMoney = myMoney + salary;
}

cout << myMoney << endl;
cout << salary << endl; // Error!
```

Серии проверок

Иногда перед принятием решения о том, какие действия будет выполнять наша программа, необходимо провести серию проверок. В таких случаях мы можем последовательно применять конструкции `if-else` `if...-else`. Для этого используем следующий формат:

```
if (условие) {
    // тело условной конструкции;
}
```

```
else if (условие2) {
    // тело условной конструкции;
}
else if (условие3) {
    // тело условной конструкции;
}
...
else {
    // тело else-конструкции;
}
```

Пример: числа от 1 до 9 на английском языке

Выведите число в диапазоне от 1 до 9 текстом на английском языке (число считывается с консоли). Мы можем прочитать число, а затем при помощи ряда проверок вывести соответствующее английское слово:

```
int num;
cin >> num;

if (num == 1) {
    cout << "one" << endl;
}
else if (num == 2) {
    cout << "two" << endl;
}
else if (...) {
    ...
}
else if (num == 9) {
    cout << "nine" << endl;
}
else {
    cout << "число слишком велико" << endl;
}
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/NumberToText.

В приведенном примере программная логика последовательно сравнивает вводимое пользователем с консоли число с цифрами от 1 до 9, причем каждое последующее сравнение выполняется только в том случае, если предыдущее не было истинным.

В итоге если ни одно из условий `if` не выполнено, выполняется последнее условие `else`.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#4.

Упражнения: простые проверки

Для закрепления знаний об условных операторах `if` и `if-else` мы решим несколько практических задач.

Задача: бонусные баллы

Задается *целое число* — количество очков. К нему добавляются бонусные очки по правилам, описанным ниже. Напишите программу, которая вычисляет бонусные очки для данного числа и общее количество очков с учетом бонусов.

- Если число до 100 включительно, то бонусные очки равны 5.
- Если число больше 100, то бонусные баллы составляют 20 % от числа.
- Если число больше 1000, то бонусные баллы составляют 10 % от числа.
- Дополнительные бонусные баллы (начисляются отдельно от предыдущих бонусных баллов):
  - Для четного числа -> + 1 элемент.
  - За число, оканчивающееся на 5 -> +2 балла.

Примеры ввода и вывода

Ввод	Вывод
20	6 26

Ввод	Вывод
175	37 212

Ввод	Вывод
2703	270.3 2973.3

Ввод	Вывод
15875	1589.5 17464.5

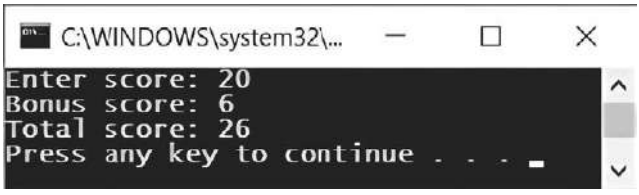
Рекомендации и советы

Вычислить основные и дополнительные бонусные баллы можно с помощью серии проверок `if-else-if-else`. Для основных бонусных баллов у нас есть три случая (когда введенное

число до 100, от 100 до 1000 и больше 1000), а для дополнительных бонусных баллов — еще 2 случая (когда число четное и нечетное):

```
int num;
cin >> num;
float bonusScore = 0.0f;

if (num > 1000) {
    bonusScore = num * 0.10f;
}
else // TODO: дополнительная логика программы...

if (num % 10 == 5) {
    bonusScore += 2;
}
else // TODO: дополнительная логика программы...

// счет бонусов
cout << bonusScore << endl;

// общий счет
cout << num + bonusScore << endl;
```

Вот как это может выглядеть в действии:



Обратите внимание, что для этого задания судейская система SoftUni настроена на игнорирование всего, что не является числом, поэтому мы можем выводить не только числа, но и поясняющий текст.

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#5.

Задача: суммирование секунд

Три спортсмена финишируют через некоторое количество секунд (от 1 до 50). Напишите программу, которая принимает время спортсменов и вычисляет их суммарное время в формате «минуты:секунды». Секунды должны выводиться с ведущим нулем (2 -> «02», 7 -> «07», 35 -> «35»).

Примеры ввода и вывода

Ввод	Вывод
35 45 44	2:04

Ввод	Вывод
22 7 34	1:03

Ввод	Вывод
50 50 49	2:29

Ввод	Вывод
14 12 10	0:36

Рекомендации и советы

Сначала мы складываем три числа, чтобы получить общий результат в секундах. Поскольку 1 минута = 60 секундам, нам нужно вычислить количество минут и количество секунд в диапазоне от 0 до 59:

- Если результат находится в диапазоне от 0 до 59, выведите 0 минут + вычисленные секунды.
- Если результат находится в диапазоне от 60 до 119, выведите 1 минуту + вычисленные секунды минус 60.
- Если результат находится в диапазоне от 120 до 179, выведите 2 минуты + вычисленные секунды минус 120.
- Если количество секунд меньше 10, то перед ними выводится ведущий ноль.

```
int firstCompetitor;
cin >> firstCompetitor;
// TODO: считайте также время второго и третьего участников

int seconds = firstCompetitor + secondCompetitor +
thirdCompetitor
int minutes = 0;

if (seconds > 59) {
    minutes++;
    seconds = seconds - 60;
```

```
}
// TODO: дополнительная логика программы...

if (seconds < 10) {
    cout << minutes << ":" << "0" << seconds << endl;
}
else {
    cout << minutes << ":" << seconds << endl;
}
```

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#6.

Задача: конвертер единиц измерения

Напишите программу, которая переводит расстояние в следующие восемь единиц: m, mm, cm, mi, in, km, ft, yd (м, мм, см, мили, дюймы, км, футы, ярды). Используйте соответствия из таблицы ниже:

Входной показатель	Выходной показатель
1 метр (м)	1000 миллиметров (мм)
1 метр (м)	100 сантиметров (см)
1 метр (м)	0.000621371192 мили (mi)
1 метр (м)	39.3700787 дюйма (in)
1 метр (м)	0.001 километра (км)
1 метр (м)	3.2808399 фута (ft)
1 метр (м)	1.0936133 ярда (yd)

Входные данные состоят из трех строк.

- Первая строка: число для преобразования.
- Вторая строка: входная единица измерения.
- Третья строка: выходная единица измерения (для результата).

Примеры ввода и вывода

Ввод	Вывод
12 km ft	39370.0788

Ввод	Вывод
150 mi in	9503999.99393599

Ввод	Вывод
450 yd km	0.41147999937455

Рекомендации и советы

Если мы хотим, чтобы при чтении единиц измерения программа могла распознавать как прописные, так и строчные буквы, мы можем использовать функцию `transform(...)` (из пространства имен `std`) для преобразования всех строчных букв:

```
transform(data.begin(), data.end(), data.begin(), ::tolower);
```

Чтобы воспользоваться этой возможностью, необходимо подключить библиотеку `algorithm` в начале программы:

```
#include <algorithm>
```

Не забудем также включить файл `string`, чтобы иметь возможность объявлять и использовать переменные типа `string`.

Как видно из таблицы в условии, мы можем преобразовывать только между метрами и какой-либо другой единицей измерения. Поэтому сначала необходимо вычислить число, которое нужно перевести в метры. Таким образом, нам необходимо выполнить ряд проверок для определения входной единицы измерения, а затем для выходной единицы измерения:

```
double size;
cin >> size;

string sourceMetric;
getline(cin, sourceMetric);
transform(sourceMetric.begin(), sourceMetric.end(),
   sourceMetric.begin(), ::tolower);

string destMetric;
getline(cin, destMetric);
transform(destMetric.begin(), destMetric.end(),
   destMetric.begin(), ::tolower);

if (sourceMetric == "km") {
```

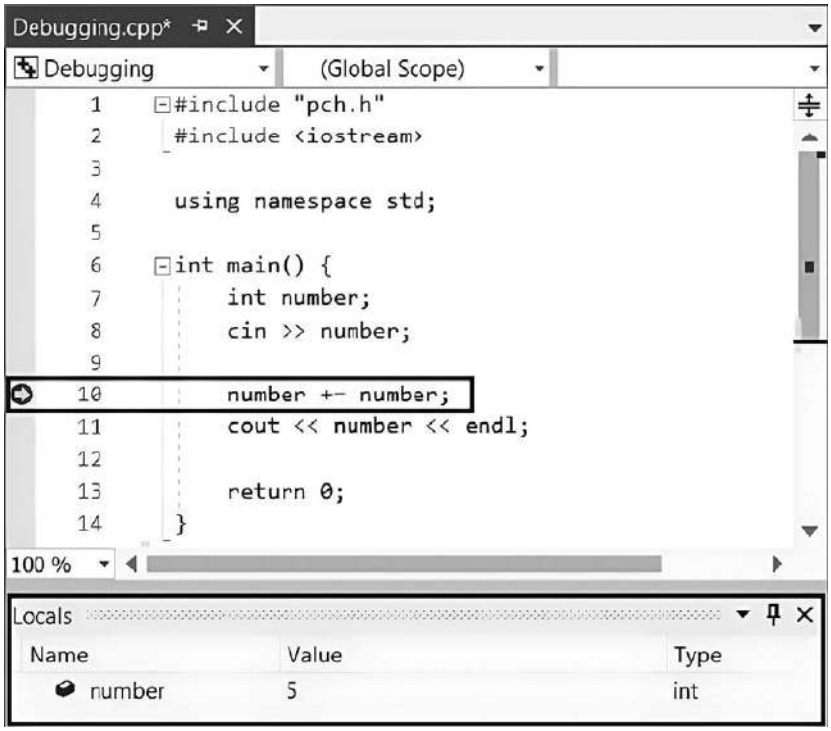
```
    size = size / 0.001;
}
// TODO: Проверить на другие единицы измерения: m; cm, ft, yd

if (destMetric == "ft") {
    size = size * 3.2808399;
}
// TODO: Проверить на другие единицы измерения: m, cm, ft, yd

cout << size << " " << destMetric << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#7.

Отладка — простые операции с отладчиком

К этому моменту мы написали немало кода, и в нем часто встречались ошибки, не так ли? Сейчас мы изучим инструмент, который поможет нам находить ошибки легче.

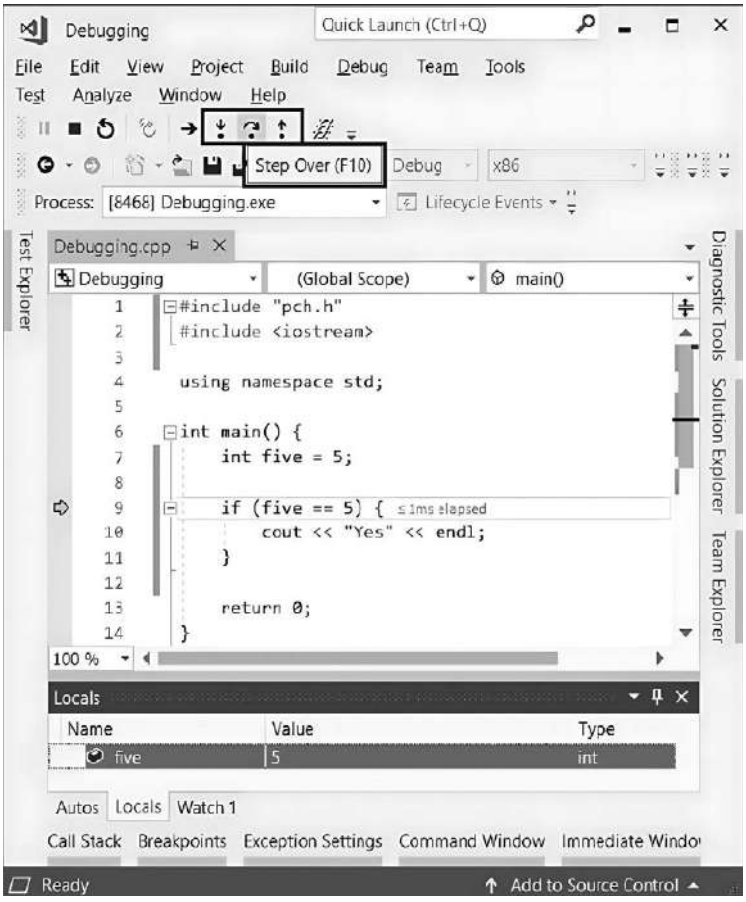


Что такое отладка?

Отладка — это процесс «подключения» к выполнению программы, позволяющий отслеживать поэтапно процесс ее выполнения. Мы можем отслеживать построчно, что происходит с нашей программой, по какому пути она движется, какие значения имеют определенные переменные на каждом шаге выполнения программы и многое другое, что позволяет обнаружить ошибки (баги):

Отладка в Visual Studio

Нажав кнопку [F10], мы запускаем программу в режиме отладки. Переход к следующей строке снова осуществляется с помощью [F10]. С помощью [F9] мы создаем остановки — так называемые точки останова, к которым мы можем перейти непосредственно с помощью [F5] при запуске программы. Протестируйте описанные действия, чтобы получить полную картину и выработать полезную привычку.



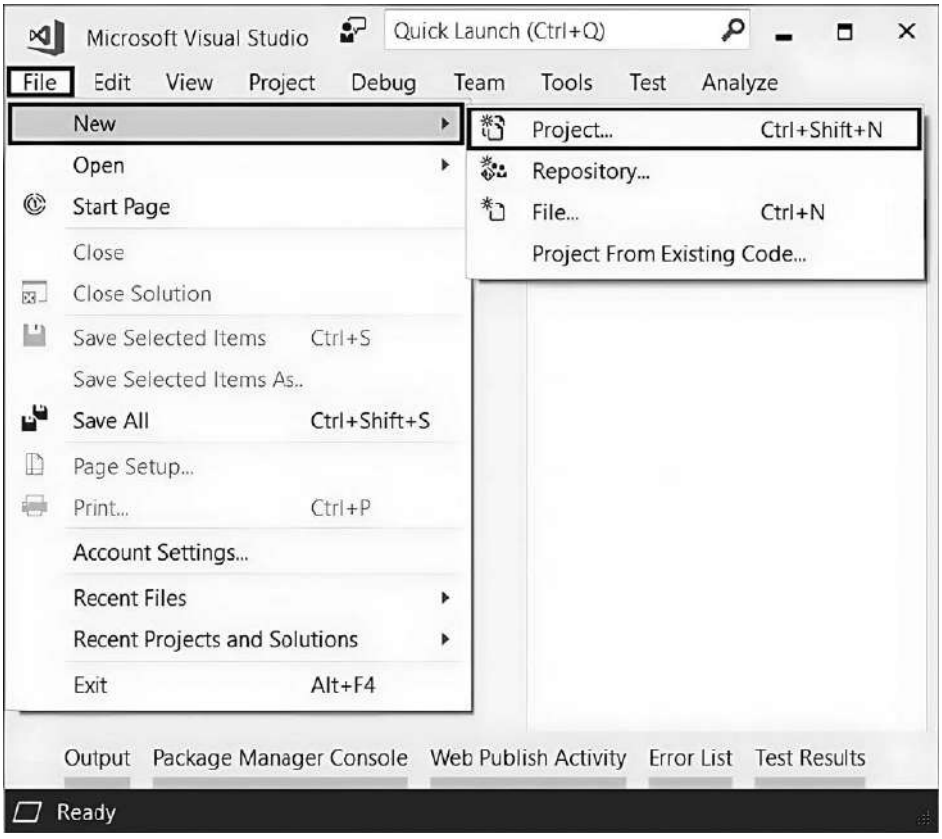
Упражнения: простые проверки

Давайте закрепим полученные в этой главе знания с помощью нескольких задач.

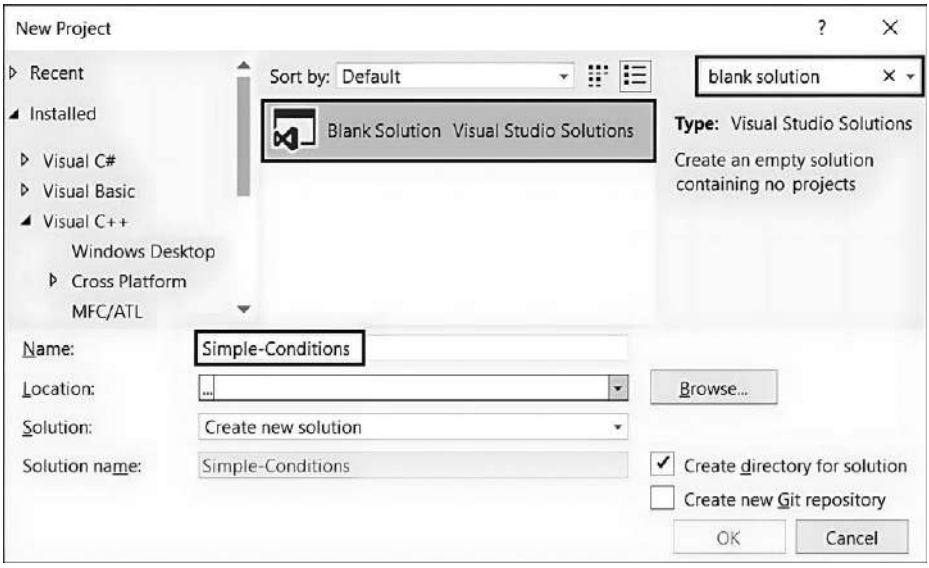
Пустое решение Visual Studio (Blank Solution)

Вспомним, как создать пустое решение (Blank Solution) в Visual Studio, чтобы лучше организовать решение упражнений — каждая задача будет находиться в отдельном проекте, а все проекты — в общем Solution.

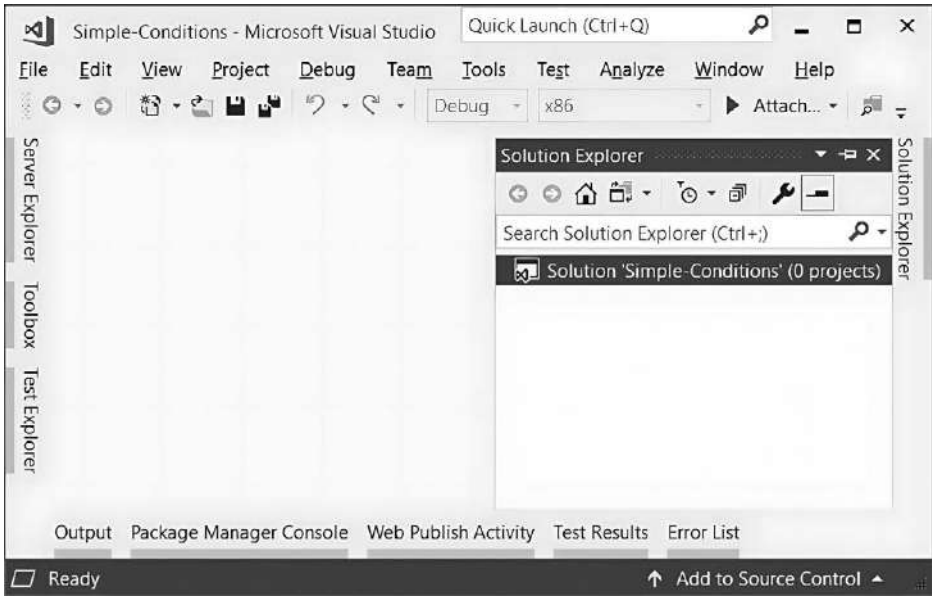
Запустите Visual Studio. Создайте новое пустое решение из меню [File] -> [New] -> [Project...]:



Выберите в диалоговом окне [Templates] -> [Other Project Types] -> [Visual Studio Solutions] -> [Blank Solution] или воспользуйтесь поисковой системой и дайте проекту подходящее имя, например «Simple-Conditions»:



Теперь мы создали пустое решение Visual Studio Solution (в котором нет ни одного проекта):



Задача: проверка на отличную оценку

Первое задание в упражнениях по этой теме — написать консольную программу, которая принимает на вход оценку (десятичное число) и выводит «`Отлично!`», если оценка `5.50` или выше.

Примеры ввода и вывода

Ввод	Вывод
6	Отлично!

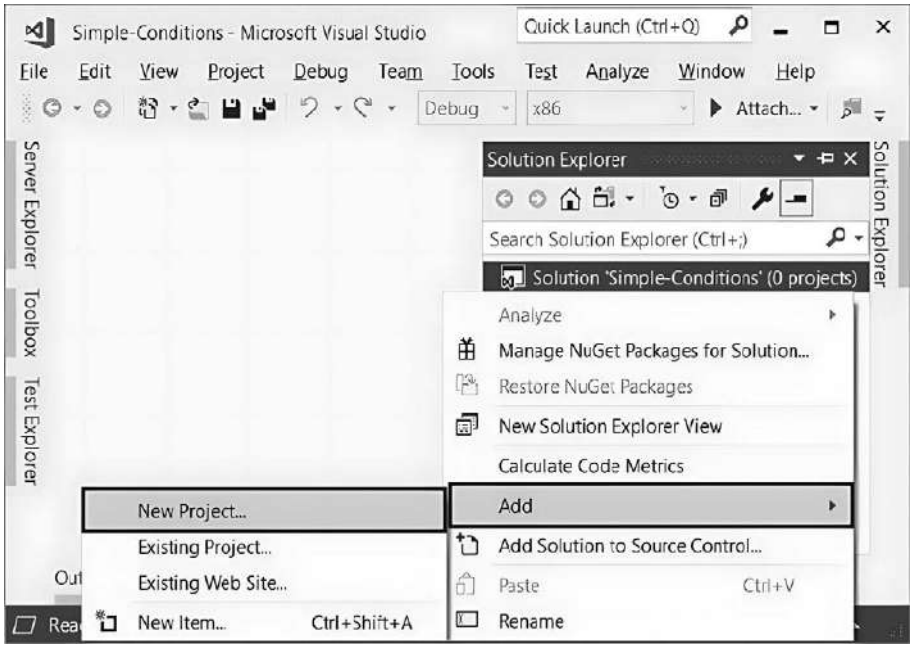
Ввод	Вывод
5	Не отлично

Ввод	Вывод
5.49	(нет вывода)

Рекомендации и советы

Создайте новый проект в существующем решении Visual Studio. В Solution Explorer щелкните правой кнопкой мыши на решении ‘Simple-Conditions’.

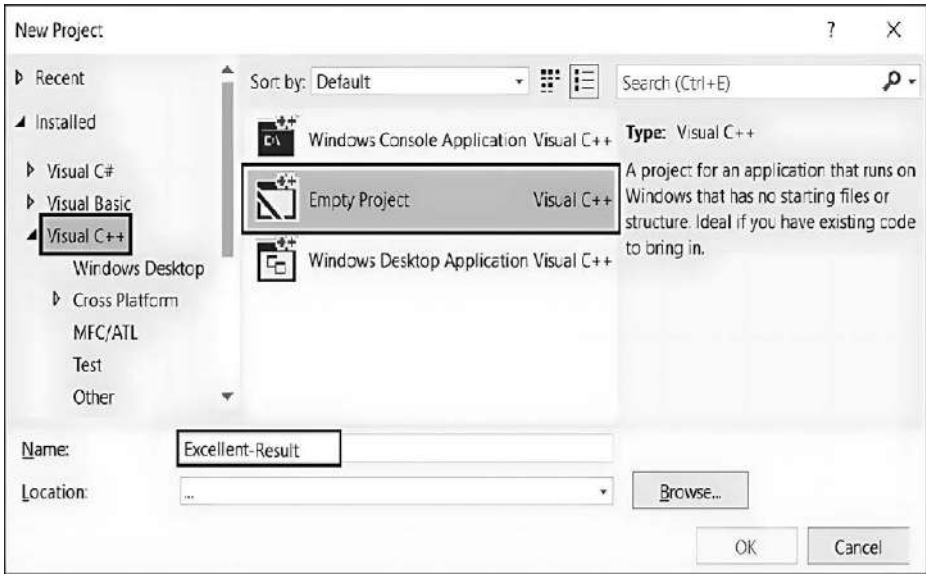
Выберите [Add] -> [New Project...]:

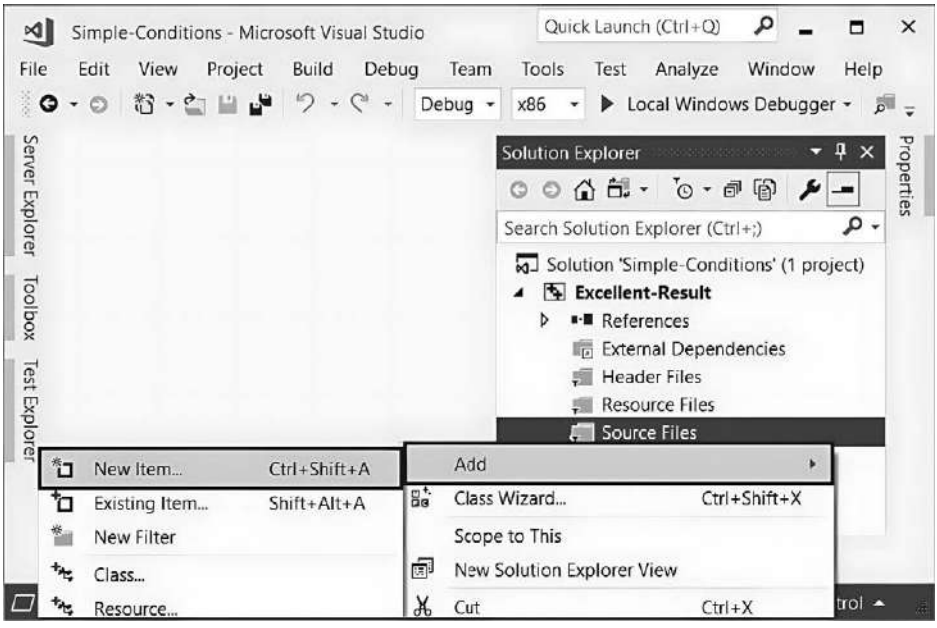


Откроется диалог выбора типа создаваемого проекта. В дереве слева выберите [Visual C++], а в центральном списке — [Empty Project] и укажите имя, например «Excellent-Result».

Затем добавьте новый файл C++ (.cpp), который должен иметь имя Main.cpp (из меню [Source Files] -> [Add] -> [New Item...]).

У нас есть решение с консольным приложением в нем, осталось только написать код.






```
double grade;
cin >> grade;

if (grade >= 5.50) {
    cout << "Отлично!" << endl;
}
```

Запустите программу с помощью [Ctrl + F5], чтобы протестировать ее с различными входными значениями:

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#0.

Задача: отличная оценка или нет

Следующая задача этой главы — написать консольную программу, которая принимает оценку (десятичное число) и выводит на экран «`Отлично!`», если оценка 5,50 или выше, или «`Не отлично`» в противном случае.

Примеры ввода и вывода

Ввод	Вывод
6	Отлично!

Ввод	Вывод
5	Не отлично

Ввод	Вывод
5.49	Не отлично

Рекомендации и советы

Сначала мы создадим в решении «Simple-Conditions» новый консольный проект на языке C++ с именем «Excellent-or-Not».

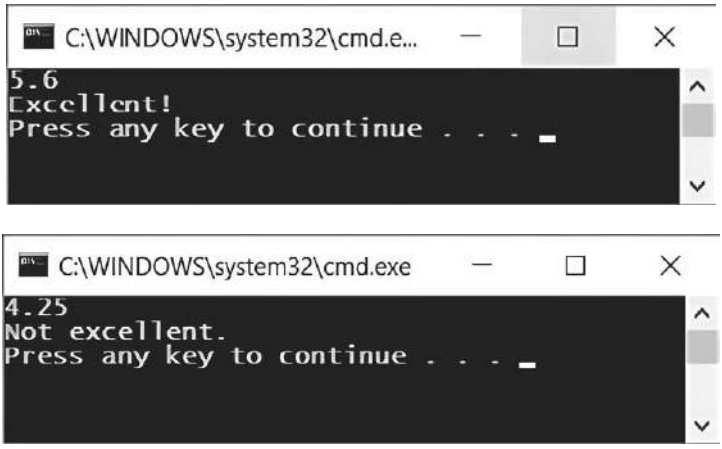
Далее необходимо написать код программы. Для этого можно использовать пример:

```
double grade;
cin >> grade;

if (grade >= 5.50) {
```

```
    cout << "Отлично!" << endl;
}
else {
    cout << "Не отлично" << endl;
}
```

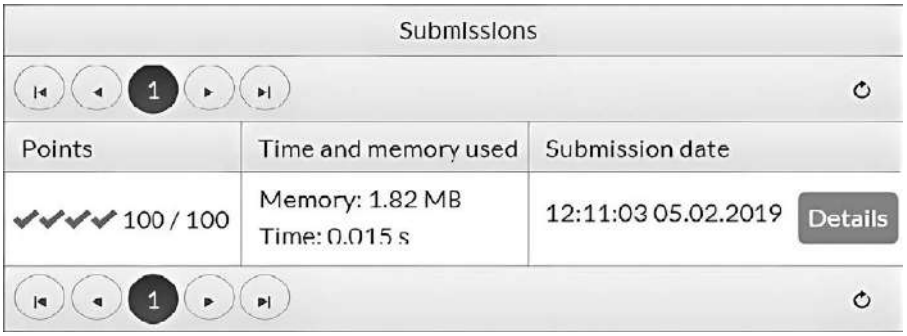
Напомним, что нам необходимо включить автоматический переход к текущему проекту, о котором мы подробно рассказывали в главе «Простые вычисления». Для этого нам необходимо щелкнуть правой кнопкой мыши на основном Solution и выбрать [Set StartUp Projects...]. Появится диалоговое окно, в котором необходимо выбрать [Startup Project] -> [Current selection].

Теперь запустим программу с помощью [Ctrl + F5] и проверим, правильно ли она работает:



Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#1.



Задача: чет или нечет

Напишите программу, которая принимает на вход целое число и выводит, четное оно или нечетное.

Примеры ввода и вывода

Ввод	Вывод
2	четное

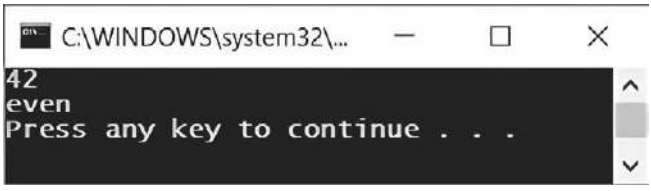
Ввод	Вывод
3	нечетное

Ввод	Вывод
25	нечетное

Ввод	Вывод
1024	четное

Рекомендации и советы

Опять же, сначала мы добавляем новый консольный проект на языке C++ к существующему Solution. Проверка четности числа может быть реализована с помощью оператора `%`, который возвращает остаток при делении целого числа на 2 следующим образом: `int isEven = (num % 2 == 0)`. Нам остается только запустить программу с помощью [Ctrl + F5] и протестировать ее:



Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#2.

Задача: найти большее число

Напишите программу, которая принимает на вход два целых числа и выводит большее из них.

Примеры ввода и вывода

Ввод	Вывод
5 3	5

Ввод	Вывод
3 5	5

Ввод	Вывод
10 10	10

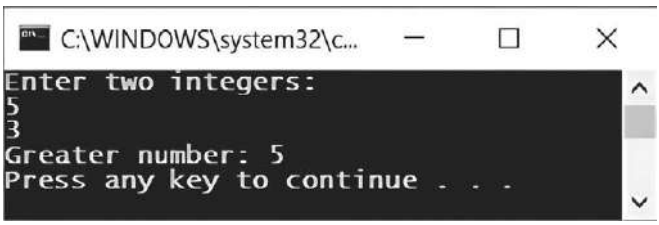
Ввод	Вывод
-5 5	5

Рекомендации и советы

Как обычно, сначала нам необходимо добавить новый консольный проект на языке C++ к существующему Solution. Для программного кода нам нужен один оператор `if-else`. Вам может помочь код на изображении, которое намеренно размыто, чтобы вы подумали, как дописать его самостоятельно:

```
int num1, num2;
cin >> num1;
cin >> num2;
```

После того как мы закончили реализацию решения, запускаем программу с помощью [Ctrl + F5] и тестируем ее:



Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#3.

Задача: запись числа до 9 словом

Напишите программу, которая вводит целое число в диапазоне [0 ... 9] и записывает его на английском языке. Если число выходит за пределы диапазона, то выводится сообщение «`число слишком велико`».

Примеры ввода и вывода

Ввод	Вывод
2	two

Ввод	Вывод
3	three

Ввод	Вывод
9	nine

Ввод	Вывод
1024	число слишком велико

Рекомендации и советы

Мы можем использовать серию операторов `if-else` для рассмотрения 11 возможных случаев.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#4.

Задача: угадать пароль

Напишите программу, которая принимает на вход пароль (одну строку случайного текста) и проверяет, совпадает ли введенное слово с фразой «`s3cr3t!P@ssw0rd`». Если совпадает, то выводится сообщение «`Welcome`», а если не совпадает, то «`Wrong password!`».

Примеры ввода и вывода

Ввод	Вывод
qwerty	Wrong password!

Ввод	Вывод
s3cr3t!P@ssw0rd	Welcome

Ввод	Вывод
25	Wrong password!

Рекомендации и советы

Мы можем использовать оператор `if-else`.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#8.

Задача: число между 100 и 200

Напишите программу, которая принимает на вход целое число и проверяет, меньше ли оно 100, между 100 и 200 или больше 200. Выведите соответствующие сообщения, как показано в примерах ниже.

Примеры ввода и вывода

Ввод	Вывод
95	Меньше 100

Ввод	Вывод
120	От 100 до 200

Ввод	Вывод
210	Больше 200

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1360#9.

Задача: одинаковые слова

Напишите программу, которая принимает на вход два слова и проверяет, одинаковые ли они. Прописные и строчные буквы не различать. Выводить «да» или «нет».

Примеры ввода и вывода

Ввод	Вывод
Hello Hello	да

Ввод	Вывод
SoftUni softuni	да

Ввод	Вывод
Soft Uni	нет

Ввод	Вывод
beer vodka	нет

Рекомендации и советы

Прежде чем сравнивать слова, необходимо перевести их в нижний регистр, чтобы размер букв (верхний/нижний регистр) не влиял на них. Для этого воспользуемся функцией `transform(...)`, которая сделает все буквы строчными. Функция имеет следующий формат:

```
transform(data.begin(), data.end(), data.begin(), ::tolower);
```

Чтобы иметь возможность использовать эту функцию, необходимо включить файл `algorithm` в начало программы: `#include <algorithm>`.

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#10.

Задача: информация о скорости

Напишите программу, которая вводит скорость (десятичное число) и выводит информацию о скорости:

- Для скоростей до 10 включительно выведите «медленная».
- Для скоростей свыше 10 и до 50 выведите «средняя».
- Для скоростей свыше 50 и до 150 — «быстрая».
- Для скоростей свыше 150 и до 1000 — «сверхбыстрая».
- При более высоких скоростях — «чрезвычайно быстрая».

Примеры ввода и вывода

Ввод	Вывод
8	медленная
49.5	средняя

Ввод	Вывод
126	быстрая
160	сверхбыстрая

Ввод	Вывод
3500	чрезвычайно быстрая

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#11.

Задача: площадь фигур

Напишите программу, которая по размерам геометрической фигуры вычисляет ее площадь. Существует четыре типа фигур: квадрат (`square`), прямоугольник (`rectangle`), круг (`circle`) и треугольник (`triangle`).

В первой строке ввода читается тип фигуры (`square`, `rectangle`, `circle`, `triangle`).

- Если фигура представляет собой квадрат, то в следующей строке считывается одно число — длина его стороны.
- Если фигура представляет собой прямоугольник, то в следующих двух строках считываются два числа — длины его сторон.
- Если фигура представляет собой круг, то в следующей строке считывается одно число — радиус круга.
- Если фигура представляет собой треугольник, то в следующих двух строках считываются два числа — длина его стороны и длина высоты, проведенной к ней.

Округлите полученный результат до трех знаков после запятой.

Примеры ввода и вывода

Ввод	Вывод
`square` `5`	`25`

Ввод	Вывод

`rectangle`
`7`
`2.5`
	`17.5`

Ввод	Вывод
`circle` `6`	`113.097`

Ввод	Вывод

`triangle`
`4.5`
`20`
	`45`

Проверка в судейской системе

Округление выходного значения может быть выполнено путем передачи в `cout` функций `fixed` и `setprecision(n)`, где `n` указывает, до какого десятичного знака должно быть выполнено округление. В начале программы мы также включаем следующую библиотеку: `#include <iomanip>`.

Приведем пример использования, где в качестве выходных данных будет выведено 3.142:

```
double f = 3.14159;
cout << fixed << setprecision(3) << f << endl;
```

Запустить и протестировать пример можно в Интернете: https://repl.it/@vncpetrov/SampleRounding.

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#12

Задача: время + 15 минут

Напишите программу, которая принимает на входе часы и минуты 24-часового дня и вычисляет, сколько времени будет через 15 минут. Выведите результат в формате чч:мм. Часы всегда находятся в диапазоне от 0 до 23, а минуты всегда находятся в диапазоне от 0 до 59.

Часы записываются одной или двумя цифрами. Минуты всегда записываются двумя цифрами и, при необходимости, с ведущим нулем.

Примеры ввода и вывода

Ввод	Вывод
1 46	2:01

Ввод	Вывод
0 01	0:16

Ввод	Вывод
23 59	0:14

Ввод	Вывод
11 08	11:23

Рекомендации и советы

Добавьте 15 минут и выполните несколько проверок. Если минуты превышают 59, увеличьте часы на 1 и уменьшите минуты на 60. Аналогично рассмотрим случай, когда часы превышают 23.

При выводе на экран минут проверьте наличие ведущего нуля.

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#13.

Задача: равенство трех чисел

Напишите программу, которая принимает три числа и выводит, одинаковы ли они («да» / «нет»).

Примеры ввода и вывода

Ввод	Вывод
5 5 5	да

Ввод	Вывод
5 4 5	нет

Ввод	Вывод
1 2 3	нет

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1360#14.

Задача*: запишите число от 0 до 100 словами

Напишите программу, которая преобразует число в диапазоне [0 ... 100] в текст на английском языке.

Примеры ввода и вывода

Ввод	Вывод
25	twenty five

Ввод	Вывод
42	forty two

Ввод	Вывод
6	six

Рекомендации и советы

Сначала проверьте однозначные числа, и если число однозначное, выведите для него соответствующее слово. Затем проверьте двузначные числа. Мы выводим их на экран в двух частях: левая часть (десятки = число / 10) и правая часть (единицы = число % 10). Если число состоит из трех цифр, то оно должно быть равно 100 и может рассматриваться как частный случай.

Проверка в судейской системе

Проверьте программу в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1360#15.

Глава 3.2.
Простые проверки — экзаменационные задачи

В предыдущей главе мы рассмотрели условные операторы в языке C++, которые могут использоваться для выполнения различных действий в зависимости от некоторого условия. Мы также рассказали о том, что такое *область видимости* переменной и как последовательно отслеживать выполнение нашей программы (это называется *отладкой*). В этой главе мы потренируемся работать с логическими проверками, рассмотрев несколько задач, приведенных на экзаменах. Для этого сначала вспомним, как строится логическая проверка:

```
if (булево выражение) {
    // тело условной конструкции;
}
else {
    // тело else-конструкции;
}
```

Проверки `if` состоят из:

- условия `if`;
- булева выражения — переменной булева типа (`bool`) или булева логического выражения (выражения, возвращающего результат `true`/`false`);
- тела оператора — оно содержит произвольный блок исходного кода;
- условия `else` и блока его исходного кода (необязательно).

Экзаменационные задачи

После того как мы вспомнили правила применения условных операторов, давайте решим несколько задач, чтобы получить практический опыт работы с оператором `if-else`.

Задача: транспортные расходы

Студенту необходимо проехать n километров. Он может выбрать один из трех видов транспорта:

- *Такси.* Начальная плата: 0,70 BGN. 0,79 BGN/км. Ночной тариф: 0,90 лв/км.
- *Автобус.* Стоимость проезда днем/ночью: 0,09 BGN/км. Может использоваться на расстояниях не менее 20 км.
- *Поезд.* Тариф день/ночь: 0,06 BGN/км. Может использоваться на расстояниях не менее 100 км.

Напишите программу, которая принимает количество километров n и время суток (день или ночь) и находит и рассчитывает стоимость самого дешевого транспорта из трех.

Исходные данные

Из консоли считываются две строки:

- Первая строка содержит число n — количество километров — целое число в диапазоне [1 ... 5000].
- Вторая строка содержит слово «день» или «ночь» — путешествие днем или ночью.

Выходные данные

Выведите на консоль самую низкую цену за указанное количество километров.

Примеры ввода и вывода

Ввод	Вывод
5 день	4.65

Ввод	Вывод
7 ночь	7

Ввод	Вывод
25 день	2.25

Ввод	Вывод
180 ночь	10.8

Рекомендации и советы

Мы будем считывать входные данные и, в зависимости от расстояния, выбирать наиболее дешевый транспорт. Для этого мы будем использовать несколько проверок.

Обработка входных данных

В условии задачи дана информация о входных и выходных данных. Соответственно, первые три строки решения будут содержать объявление и инициализацию двух переменных, в которых мы будем хранить значения входных данных. В первой строке указано, что она содержит целое число, поэтому объявляемая переменная имеет тип `int`. Для второй строки утверждается, что она содержит слово, поэтому переменная имеет тип `string`:

```
int distance;
string dayOrNight;
cin >> distance >> dayOrNight;
```

Перед началом проверок нам также необходимо объявить переменную, в которой мы будем хранить значение транспортной стоимости:

```
double price = 0;
```

Проведение проверок и соответствующих расчетов

После того как мы объявили и инициализировали входные данные и переменную, в которой будет храниться значение цены, нам необходимо решить, какие условия задачи будут проверяться в первую очередь.

Из условия следует, что тарифы двух транспортных средств не зависят от того, день это или ночь, а тариф одного транспортного средства (такси) зависит. Поэтому первая проверка будет заключаться именно в проверке на день или ночь, чтобы понять, какой тариф будет использоваться для такси. Для этого мы объявляем еще одну переменную, в которой будем хранить значение тарифа такси:

```
double taxi Rate = 0;
```

Для определения стоимости проезда в такси мы будем использовать проверку типа `if-else`:

```
if (dayOrNight == "день") {
    taxiRate = 0.79;
}
else {
    taxiRate = 0.90;
}
```

После этого можно переходить к расчету собственно стоимости проезда. Ограничения, присутствующие в постановке задачи, касаются расстояния, которое студент хочет проехать. Поэтому мы применим еще один оператор `if-else` для определения стоимости перевозки в зависимости от заданного километража:

```
if (distance < 20) {
    price = 0.70 + distance * taxiRate;
}
else if (distance < 100) {
    price = distance * 0.09;
}
else {
    price = distance * 0.06;
}
```

Сначала мы проверяем количество километров, так как из условия ясно, что при расстоянии меньше 20 км студент мог воспользоваться только такси. Если условие проверки истинно (возвращает `true`), то переменной, хранящей значение стоимости перевозки (`price`), мы присвоим соответствующее значение. Это значение равно начальному тарифу, который мы добавим к его стоимости, умноженному на расстояние, которое должен проехать студент.

Если условие переменной не истинно (возвращает `false`), то следующим шагом нашей программы будет проверка того, не меньше ли количество километров 100. Это делается потому, что из условия ясно, что в этом диапазоне можно использовать автобус в качестве средства передвижения. Стоимость одного километра для автобуса ниже, чем для такси. Поэтому, если результат проверки верен, то в теле блока `else if` переменной цены транспорта (`price`) должно быть присвоено значение, равное результату умножения стоимости проезда на автобусе на расстояние.

Если и эта проверка не возвращает `true`, то в теле оператора `else` остается присвоить переменной `price` значение, равное результату умножения расстояния на стоимость проезда на поезде. Это делается потому, что поезд является самым дешевым видом транспорта на данном расстоянии.

Вывод выходных данных

После того как мы выполнили проверку расстояния и рассчитали самую дешевую цену перевозки, необходимо вывести ее на экран.

В постановке задачи нет требований к форматированию результата, поэтому мы выведем только переменную:

```
cout << price << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1361#0.

Задача: трубы в бассейне

Бассейн объемом V имеет две трубы, из которых он наполняется. Каждая труба имеет определенный расход (литры воды, проходящие через трубу за один час). Рабочий одновременно впускает и выпускает воду из труб в течение N часов. Напишите программу, которая выводит состояние бассейна на момент возвращения рабочего.

Входные данные

Из консоли считываются четыре строки:

- Первая строка содержит число V — объем бассейна в литрах — целое число в диапазоне [1 ... 10000].
- Вторая строка содержит число P1 — расход первого трубопровода в час — целое число в диапазоне [1 ... 5000].
- В третьей строке записано число P2 — расход второго трубопровода в час — целое число в диапазоне [1 ... 5000].
- Четвертая строка содержит число H — часы отсутствия работника — число с плавающей точкой в диапазоне [1.0 ... 24.00].

Выходные данные

Выведите на консоль одно из двух возможных состояний:

- На сколько заполнился резервуар и какая труба внесла сколько процентов. Приведите все проценты к целому числу (без округления):
  - «`Бассейн заполнен на [x]%. Труба 1: [y]%. Труба 2: [z]%`».

- Если бассейн переполнился — на сколько литров он переполнился за указанное время, число с плавающей точкой:
  - «За [x] часов бассейн переполняется на [y] литров».

Обратите внимание, что при округлении до целого числа происходит потеря данных, и нормальным является тот факт, что сумма процентов составляет не 100 %, а 99 %.

Примеры ввода и вывода

Ввод	Вывод
1000 100 120 3	Бассейн заполнен на 66 %. Труба 1: 45 %. Труба 2: 54 %.

Ввод	Вывод
100 100 100 2.5	За 2,5 часа бассейн переполняется на 400 литров.

Рекомендации и советы

Для решения задачи мы считаем входные данные, выполним несколько проверок и вычислений и выведем результат.

Обработка входных данных

Из постановки задачи видно, что в нашей программе должно быть четыре строки, из которых мы будем считывать входные данные. Первые три будут использоваться для ввода целых чисел, поэтому переменные, в которых мы будем хранить значения, будут иметь тип `int`. Для четвертой строки известно, что это будет число с плавающей точкой, поэтому переменная, которую мы будем использовать, будет иметь тип `double`:

```
int volume;
int pipe1;
int pipe2;
double hours;
```

Следующим шагом будет объявление и инициализация переменной, в которой мы рассчитаем, сколько литров было заполнено в бассейне за время отсутствия работника. Вычисления будут производиться путем сложения значений расхода воды в двух трубах и умножения их на количество часов, которое дается нам в качестве входных данных:

```
int pipes = [illegible]
double water = [illegible]
```

Выполнение проверок и обработка выходных данных

После получения значений количества воды, прошедшей по трубам, следующим шагом будет сравнение этого количества с объемом самого бассейна.

Мы сделаем это с помощью простого оператора `if-else`, в котором условие будет проверять, меньше ли количество воды, чем объем бассейна. Если проверка возвращает `true`, то необходимо вывести строку, содержащую отношение количества воды, прошедшей по трубам, к объему бассейна, а также отношение количества воды из каждой трубы к объему бассейна.

Отношение необходимо выразить в процентах, поэтому все вычисления до сих пор будут умножаться на 100. Поскольку существует условие форматирования результата в процентах до двух цифр после десятичной запятой без округления, то для этой цели воспользуемся функцией `trunc(...)`:

```
if (water <= volume) {
    double sumPercent = trunc([illegible]);
    double pipe1Percent = trunc([illegible]);
    double pipe2Percent = trunc([illegible]);
cout << "Бассейн заполнен на " << sumPercent << " %. Труба 1: "
  << pipe1Percent << " %. Труба 2: " << pipe2Percent << " %."
  << endl;
}
else {
    double difference = water - volume;
    cout << "За " << hours << " часа бассейн переполняется на" <<
       difference << " литров" << endl;
}
```

Если же проверка возвращает результат `false`, то это означает, что количество воды больше объема бассейна, следовательно, он переполнился. И снова вывод должен быть в виде одной строки, но на этот раз он содержит только два значения — часы отсутствия работника и количество воды, которое представляет собой разницу между количеством набранной воды и объемом бассейна.

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1361#1.

Задача: сонный кот Том

Кот Том любит спать целыми днями, но, к сожалению, его хозяин играет с ним, когда у него есть свободное время. Для того чтобы Том хорошо высыпался, норма его игры должна составлять 30 000 минут в год.

Время игр Тома зависит от выходных дней его хозяина:

- Когда хозяин на работе, он играет с Томом 63 минуты в день.
- Во время выходных хозяин играет с ним 127 минут в день.

Напишите программу, которая принимает количество выходных дней и выводит, может ли Том хорошо выспаться и на сколько количество часов его сна отличается от нормы для текущего года, предполагая, что в году 365 дней.

Пример: 20 выходных дней -> 345 рабочих дней (365 - 20 = 345). Фактическое время за игру составляет 24 275 минут (345 * 63 + 20 * 127). Отличие от нормы составляет 5725 минут (30 000 - 24 275 = 5 725), или 95 часов 25 минут.

Входные данные

Входные данные считываются с консоли и представляют собой целое число — количество выходных дней в интервале [0 ... 365].

Выходные данные

На консоль должны быть выведены две строки.

- Если игровое время Тома превышает норму для текущего года:

    - В первой строке выведите «`Том сбежит`».
    - Во второй строке выведите отличие от нормы в формате «`На {H} часов и{M} минут больше на игру`».
- Если игровое время Тома ниже нормы для текущего года:
    - В первой строке выведите: «`Том выспится`».
    - Во второй строке выведите отличие от нормы в формате «`На {H} часов и {M} минут меньше на игру`».

Примеры ввода и вывода

Ввод	Вывод
20	Том выспится На 95 часов и 25 минут меньше на игру

Ввод	Вывод
113	Том сбежит 3 часа и 47 минут больше на игру

Рекомендации и советы

Для решения задачи мы прочитаем входные данные, выполним несколько проверок и вычислений и выведем результат.

Обработка входных данных и соответствующие расчеты

Из постановки задачи видно, что входные данные будут считываться только из одной строки, которая будет содержать целое число в диапазоне [0 ... 365]. Поэтому мы будем использовать переменную типа `int`:

```
int holidays;
```

Для решения задачи сначала нужно подсчитать, сколько всего минут играет с Томом его хозяин. Из условия видно, что, кроме выходных, сонный кот вынужден играть и в рабочие дни своего хозяина. Число, которое мы считаем с консоли, относится к выходным дням.

Следующим шагом будет использование этого числа для расчета количества рабочих дней у владельца кота, поскольку без них мы не сможем получить общее количество минут игры. Поскольку общее количество дней в году равно 365, а количество выходных дней равно X, это означает, что количество рабочих дней равно 365 - X. Разницу мы сохраним в новой переменной, которую будем использовать только для этого значения:

```
int workingDays = [illegible]
```

Теперь, когда у нас есть количество игровых дней, мы можем рассчитать игровое время Тома в минутах. Его значение равно результату умножения рабочих дней на 63 минуты (в условии указано, что в рабочие дни время игры составляет 63 минуты в день), прибавленному к результату умножения выходных дней на 127 минут (в условии указано, что в выходные дни время игры составляет 127 минут в день):

```
int totalPlayMinutes = [illegible]
```

В условии вывода задачи мы видим, что нам необходимо вывести разницу между двумя значениями в часах и минутах. Для этого из общего времени игры вычтем норму в 30 000 минут и запишем полученную разницу в новую переменную. Затем разделим эту переменную на 60, чтобы получить часы, а чтобы узнать количество минут, воспользуемся модульным делением (оператор %) на 60.

Здесь следует отметить, что если полученное Томом количество игрового времени меньше 30 000, то вычитание из него нормы даст число со знаком минус. Чтобы в дальнейшем нейтрализовать знак в двух делениях, при нахождении разности мы будем использовать функцию `abs(...)`:

```
int difference = abs([illegible]);
int hours = [illegible]
int minutes = [illegible]
```

Проведение проверок

Время игры уже вычислено, что приводит нас к следующему шагу — сравнению времени игры Тома с нормой, которая определяет, будет ли кот высыпаться. Для этого мы используем проверку

`if-else`, в условии `if` мы проверим, не превышает ли время игры 30 000 (норму).

Обработка исходных данных

Какой бы результат ни выдала проверка, нам необходимо вывести, сколько составляет разница в часах и минутах. Сделаем это с помощью переменных, в которых мы вычисляли значения часов и минут, отформатировав их в соответствии с условием вывода:

```
if (totalPlayMinutes > 30000) {
    cout << "Том сбежит" << endl;
    cout << "На " << hours << " часов и " <<
        minutes << " минут больше на игру" << endl;
>
else {
    cout << "Том выспится" << endl;
    cout << "На " << hours << " часов и " <<
        minutes << " минут меньше на игру" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1361#2.

Задача: сбор урожая

С виноградника площадью X кв. м 40 % урожая отводится на производство вина. С 1 кв. м виноградника получают Y кг винограда. Для производства 1 литра вина требуется 2,5 кг винограда. Желаемое количество вина для продажи — Z литров.

Напишите программу, которая вычисляет, сколько вина можно произвести и достаточно ли этого количества. Если достаточно, то остаток делится поровну между работниками виноградника.

Входные данные

Входные данные считываются с консоли и состоят ровно из четырех строк:

- Первая строка: X кв. м — площадь виноградника — целое число в интервале [10 ...5000].
- Вторая строка: Y винограда на квадратный метр — вещественное число в интервале [0,00 ... 10,00].
- Третья строка: требуется Z литров вина — целое число в диапазоне [10 ... 600].
- Четвертая строка: количество работников — целое число в диапазоне [1 ... 20].

Входные данные

На консоль должно быть выведено следующее сообщение:

- Если количество произведенного вина меньше требуемого:
  - «`Это будет суровая зима! Нужно больше {не хватает вина} литров вина`». Результат должен быть округлен до меньшего целого числа.
- Если произведенного вина больше, чем требуется:
  - «`Хороший урожай в этом году! Общее количество вина:литров`». Результат должен быть округлен до меньшего целого числа.
  - «`{Осталось вина} литров -> {Вино на 1 работника} литров на человека`». Оба результата должны быть округлены до большего целого числа.

Примеры ввода и вывода

Ввод	Вывод
650 2 175 3	Хороший урожай в этом году! Общий объем вина: 208 литров. 33 литра осталось -> 11 литров на человека.

Ввод	Вывод
1020 1.5 425 4	Это будет суровая зима! Нужно еще 180 литров вина.

Рекомендации и советы

Для решения задачи мы прочитаем входные данные, выполним несколько проверок и вычислений и выведем результат.

Обработка входных данных и соответствующие расчеты

Сначала необходимо проверить, какими будут входные данные, и выбрать, какие переменные мы будем использовать. Приведенный ниже код намеренно размыт и должен быть доработан читателем:

```
double vineyardArea;
double grapePerSquare;
double neededliters;
int workers;
```

Для решения задачи необходимо рассчитать, сколько литров вина мы получим, на основе входных данных. Из условия задачи видно, что для вычисления количества вина в литрах нам сначала нужно узнать, какое количество винограда в килограммах будет получено от данного урожая. Для этого объявим переменную, которой присвоим значение, равное 40 % от результата умножения площади виноградника и количества винограда, получаемого с 1 кв. м.

Проведя эти расчеты, мы готовы вычислить количество вина в литрах, которое будет произведено из этого винограда. Для этого мы объявляем еще одну переменную, в которой будем хранить это количество, и из условия приходим к выводу, что для его расчета нужно разделить количество винограда в кг на 2,5:

```
double harvestPerWine =
double wine =
```

Выполнение проверок и обработка выходных данных

Мы уже произвели необходимые расчеты, и следующим шагом будет проверка достаточности полученных литров вина. Для этого воспользуемся простым условным оператором типа `if-else`, а в условии проверим, больше ли литров вина из урожая или равно необходимому количеству литров.

Если проверка возвращает `true`, то из условия задачи видно, что в первой строке мы должны вывести количество вина, полученное в результате сбора урожая. Чтобы выполнить условие округления этого значения до младшего целого числа, воспользуемся функцией `floor(...)`.

Во второй строке требуется вывести результаты, округлив их до большего целого числа, что мы и сделаем с помощью функции `ceil(...)`. Нам нужно вывести следующие значения: количество оставшегося вина и количество вина на одного работника. Оставшееся количество вина равно разнице между полученными и требуемыми литрами вина. Значение этого количества мы вычислим в новой переменной, которую объявим и инициализируем в теле блока в условии `if` перед выводом первой строки. Количество вина на одного работника мы рассчитаем, разделив остаток вина на количество работников:

```
if (                                    ) {
    double totalWine = floor(      );
    double litersLeft = ceil(                    );
    double litersPerPerson = ceil(                                  );

    cout << "Хороший урожай в этом году! Общий объем вина: " <<
        totalWine << " литров." << endl;
    cout << litersLeft << " литра осталось -> " <<
        litersPerPerson << " литров на человека." << endl;
}
```

Если проверка возвращает результат `false`, из условия задачи видно, что нам нужно вывести разность искомых литров и литров вина, полученного из данного винограда. При этом есть условие, чтобы результат был округлен до меньшего целого числа, что мы и сделаем с помощью функции `floor(...)`:

```
else {
    double litersNeeded = floor(                                  );
    cout << "Это будет суровая зима! Нужно еще " <<
        litersNeeded << " литров вина." << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1361#3.

Задача: компания

IT-компания получает запрос на разработку проекта, требующего определенного количества часов. У компании есть определенное количество дней. В течение 10 % дней сотрудники находятся на обучении и не могут работать над проектом. Нормальный рабочий день в компании составляет 8 часов. Проект важен для компании, и каждый сотрудник должен работать над проектом в нерабочее время по 2 часа в день.

Количество часов должно быть округлено до меньшего целого числа (например, 6,98 часа округляется до 6 часов).

Напишите программу, которая вычисляет, сможет ли компания завершить проект в срок и сколько часов не хватает или остается.

Входные данные

Входные данные считываются с консоли и содержат ровно три строки:

- В первой строке указываются требуемые часы — целое число в диапазоне [0 ... 200 000].
- Вторая строка — количество дней, имеющихся в распоряжении компании — целое число в диапазоне [0 ... 20 000].
- Третья строка — это количество всех сотрудников — целое число в диапазоне [0 ... 200].

Выходные данные

Для вывода строки на консоль:

- Если времени достаточно: «`Да! {оставшиеся часы} часов осталось`».
- Если времени недостаточно: «`Недостаточно времени! Необходимо {недостаточно часов} часов`».

Примеры ввода и вывода

Ввод	Вывод
90 7 3	Да! 99 часов осталось.

Ввод	Вывод
99 3 1	Недостаточно времени! Необходимо 75 часов.

Рекомендации и советы

Для решения задачи мы прочитаем входные данные, выполним несколько проверок и вычислений и выведем результат.

Обработка входных данных

Для решения поставленной задачи сначала необходимо определиться, какие типы переменных мы будем использовать для входных данных. Приведенный ниже код намеренно завуалирован и должен быть доработан читателем:

```
int projectHours;
int availableDays;
int overtimeWorkers;
```

Вспомогательные расчеты

Следующим шагом является расчет количества рабочих часов путем умножения рабочих дней на 8 (каждый день отводится 8 часов) на количество работников и сложения их с отгулами. Количество рабочих дней равно 90 % дней, имеющихся в распоряжении фирмы. Нерабочее время равно результату умножения числа работников на 2 (возможные часы нерабочего времени), и оно также умножается на количество дней, имеющихся в распоряжении фирмы.

Из условия задачи видно, что существует условие округления часов до меньшего целого числа, что мы и сделаем с помощью функции `floor(...)`:

```
double workDays =
double overtimeHours =
double workHours =
double totalHours =
```

Проведение проверок

После проведения расчетов нам необходимо выяснить величину рабочего времени и проверить, хватает ли этих часов.

Если времени достаточно, то мы выводим результат, требуемый по условию задачи, а именно разницу между рабочим временем и требуемым для завершения проекта.

Если времени недостаточно, мы выводим дополнительные часы, необходимые для завершения проекта, которые равны разнице между часами, необходимыми для завершения проекта, и часами работы:

```
if (                                    ) {
    int result =
    cout << "Да!" << result << " часов осталось." << endl;
}
else {
    int result =
    cout << "Недостаточно времени! Необходимо " << result <<
" часов." << endl;
}
```

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1361#4.

Глава 4.1.
Более сложные проверки

В этой главе мы рассмотрим вложенные проверки в языке C++, которые могут быть использованы для того, чтобы применить в программе условные операторы, внутри которых вложены другие условные операторы. Мы называем их вложенными, поскольку помещаем оператор `if` внутри другого оператора `if`. Мы также рассмотрим более сложные логические условия на соответствующих примерах.

Вложенные проверки

Довольно часто логика программы требует использования операторов `if` или `if-else`, которые содержатся друг в друге. Такие операторы называются вложенными `if` или `if-else`. Как следует из названия «вложенные», это операторы `if` или `if-else`, которые вложены в другие операторы `if` или `else`:

```
if (условие1) {
    if (условие2) {
        // тело
    }
    else {
        // тело
    }
}
```

Глубокая вложенность

Вложение *более трех условных операторов* друг в друга не считается хорошей практикой и *должно быть исключено*, в основном за счет оптимизации структуры/алгоритма кода и/или использования другого типа условного оператора, который мы рассмотрим ниже.

Пример: обращение по возрасту и полу

В соответствии с введенным возрастом (десятичная дробь) и полом (m/f) для вывода обращения:

- «`Mr.`» — мужчина (пол «`m`», то есть «male», мужской) — 16 и более лет.
- «`Master`» — юноша (пол «`m`») в возрасте до 16 лет.
- «`Ms.`» — женщина (пол «`f`», «female», женский) — 16 и более лет.
- «`Miss`» — девушка (пол «`f`») в возрасте до 16 лет.

Примеры ввода и вывода

Ввод	Вывод
12 f	Miss

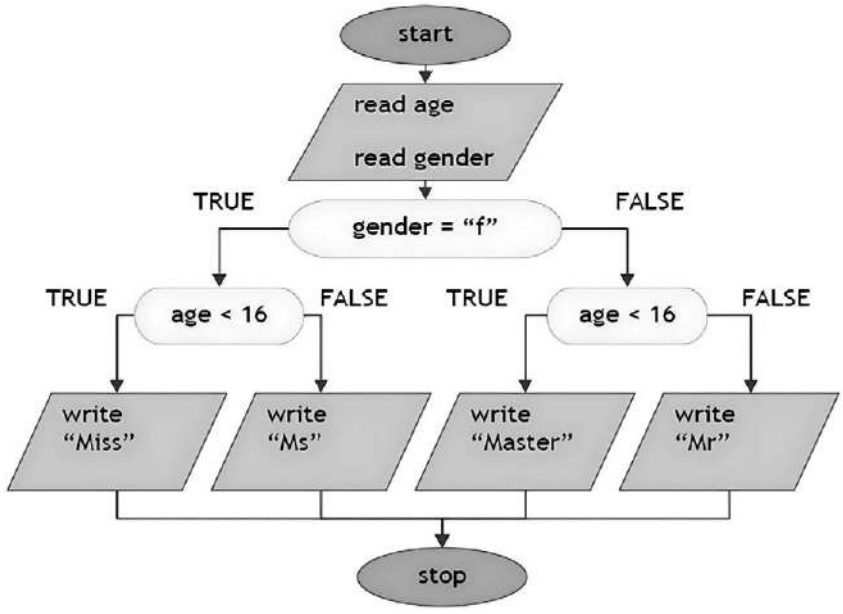
Ввод	Вывод
17 m	Mr.

Ввод	Вывод
25 f	Ms.

Ввод	Вывод
13.5 m	Master

Решение

Можно заметить, что результат работы программы зависит от нескольких вещей. Сначала нам необходимо проверить, какой пол введен, а затем проверить возраст. Соответственно, мы будем использовать несколько блоков `if-else`. Эти блоки будут вложенными, то есть результат выполнения первого из них будет определять, какой из остальных выполнить:



После чтения входных данных с консоли необходимо выполнить следующий пример логики программы:

```
double age;
string gender;
cin >> age >> gender
if (gender == "m") {
    if (age < 16) {
        cout << "Master" << endl;
    }
    else {
        cout << "Mr." << endl;
    }
}
else if (gender == "f") {
    if (age < 16) {
        cout << "Miss" << endl;
    }
    else {
        cout << "Ms." << endl;
    }
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1362#0.

Пример: магазин в микрорайоне

Предприимчивый болгарин открывает в нескольких городах магазины шаговой доступности с разными ценами на следующие товары:

продукт /город	София	Пловдив	Варна
кофе	0.50	0.40	0.45
вода	0.80	0.70	0.70
пиво	1.20	1.15	1.10
сладости	1.45	1.30	1.35
арахис	1.60	1.50	1.55

Рассчитайте цену по заданному *городу* (строка), *товару* (строка) и *количеству* (десятичное число).

Примеры ввода и вывода

Ввод	Вывод
кофе Варна 2	0.9

Ввод	Вывод
арахис Пловдив 1	1.5

Ввод	Вывод
пиво София 6	7.2

Ввод	Вывод
вода Пловдив 3	2.1

Решение

Примерный алгоритм решения:

```
string product;
string town;
double quantity;

cin >> product >> town >> quantity;

if (town == "София") {
    if (product == "кофе") {
         cout << 0.50 + quantity << endl;
    }
    // TODO: Завершите этот блок...
}
if (town == "Пловдив") {
    // TODO: Завершите этот блок...
}
if (town == "Варна") {
    // TODO: Завершите этот блок...
```

Код специально не завершен, чтобы стимулировать самостоятельное мышление и принятие решений.

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1362#1.

Более сложные проверки

Рассмотрим, как можно выполнять более сложные логические проверки. Мы можем использовать логическое «И» (`&&`), логическое «ИЛИ» (`||`), логическое отрицание (`!`) и скобки (`()`).

Логическое «И»

Как мы видели, в некоторых задачах приходится выполнять много проверок одновременно. Но как быть, если для выполнения некоторого кода необходимо одновременное выполнение нескольких условий, и мы не хотим делать отрицание (`else`) для каждого из них? Вариант с вложенными блоками `if` допустим, но код будет выглядеть очень запутанным и трудным для чтения и сопровождения.

Логическое «И» (оператор &&) означает одновременное выполнение нескольких условий. При этом применяется следующая таблица истинности:

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

Как работает оператор && ?

Оператор && принимает несколько булевых (условных) выражений, имеющих значение `true` или `false`, и возвращает в качестве результата одно булево выражение (0 — `false` и 1 — `true`).

Использование этого оператора вместо нескольких вложенных блоков `if` делает код более читабельным, аккуратным и удобным для сопровождения. Но как это работает, когда мы вставляем несколько условий одно за другим? Как мы видели выше, логическое «И» возвращает `true` только тогда, когда принимает в качестве аргументов выражения со значением `true`.

Соответственно, когда у нас есть последовательность аргументов, логическое «И» проверяет их либо до тех пор, пока они не закончатся, либо пока не встретит аргумент со значением `false`.

Пример:

```
bool a = true;
bool b = true;
```

```
bool c = false;
bool d = true;

bool result = a && b && c && d;
// 0 - false (так как d не проверяется)
```

Протестировать пример можно в Интернете: https://repl.it/@vncpetrov/OperatorAND.

Программа будет выполняться следующим образом: она начинает проверку с аргумента a, считывает его и сообщает, что он имеет значение true, затем проверяет b.

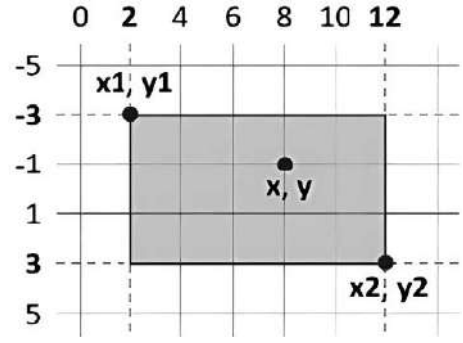
После того как программа сообщила, что a и b возвращают true, она проверяет следующий аргумент. Она доходит до c и сообщает, что переменная имеет значение false. Как только программа сообщает, что аргумент c имеет значение false, она оценивает выражение до c, независимо от того, какое значение имеет d. При этом проверка d пропускается, и все выражение оценивается как false.

```
if (x >= x1 && x <= x2 && y >— y1 && y <= y2)
```

Пример: точка в прямоугольнике

Проверьте, находится ли точка {x, y} внутри прямоугольника {x1, y1} - {x2, y2}. Входные данные считываются с консоли и состоят из шести строк: десятичные числа x1, y1, x2, y2, x и y (при этом должно быть обеспечено, что x1 < x2 и y1 < y2).

Пример ввода и вывода

Ввод	Вывод	Визуализация
2 -3 12 3 8 -1	Внутри	

Решение

Точка является внутренней по отношению к прямоугольнику, если одновременно выполняются следующие четыре условия:

- Точка находится справа от левой стороны прямоугольника.
- Точка находится слева от правой стороны прямоугольника.
- Точка находится внизу от вершины прямоугольника.
- Точка находится вверху от нижней стороны прямоугольника.

```
double x1, y1, x2, y2, x, y;
cin >> x1 >> y1
    >> x2 >> y2
    >> x >> y;

if (x >= x1 && x <= x2
    && y >= y1 && y <= y2) {
    cout << "Внутри" << endl;
}
else {
    cout << "Снаружи" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1362#2.

Логическое «ИЛИ»

Логическое «ИЛИ» (оператор `||`) означает, что выполняется хотя бы одно из нескольких условий. Как и оператор `&&`, логическое «ИЛИ» принимает несколько аргументов булевого (условного) типа и возвращает `true` или `false`. Легко догадаться, что значение `true` мы получаем всякий раз, когда хотя бы один из аргументов имеет значение `true`. Типичным примером логики этого оператора является следующий.

В школе учитель говорит: «Иван или Петр должны вымыть доску». Для того чтобы это условие было выполнено (доска была вымыта), можно, чтобы ее вымыл только Иван, только Петр или оба.

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

Как работает оператор || ?

Мы уже узнали, что такое логическое «ИЛИ». Но как оно реализуется на практике? Как и в случае с логическим «И», программа проверяет слева направо указанные аргументы. Для того чтобы получить из выражения `true`, необходимо, чтобы только один аргумент имел значение `true`, поэтому проверка продолжается до тех пор, пока не встретится аргумент с таким значением или пока не закончатся аргументы.

Приведем пример оператора `||` в действии:

```
bool a = false;
bool b = false;
bool c = true;
bool d = true;

bool result = a || b || c || d;
// 1 - true (так как c и d не проверяются)
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/OperatorOr.

Программа проверяет `a`, сообщает, что оно имеет значение `false`, и продолжает работу. Достигнув `b`, она сообщает, что имеет значение `true`, и все выражение получает значение `true` без проверки `c` и `d`, поскольку их значения не изменят результат выражения.

Пример: фрукт или овощ

Давайте проверим, является ли тот или иной продукт фруктом или овощем. К фруктам относятся банан, яблоко, киви, вишня, лимон и виноград. Овощами являются помидор, огурец, перец и морковь. Все остальные — «неизвестно».

Ввод	Вывод
банан	фрукт

Ввод	Вывод
помидор	овощ

Ввод	Вывод
java	неизвестно

Решение

Нам необходимо использовать несколько условных проверок с логическим «ИЛИ» (||):

```
string input;
cin >> input;

if (input == "банан" || input == "яблоко" ||
    input == "киви" || input -- "вишня" ||
    input == "лимон" || input == "виноград") {
    cout << "фрукт" << endl;
}
else if (input == "помидор" || input == "огурец"
    input == "перец" || input -- "морковь") {
    cout << "овощ" << endl;
}
else {
    cout << "неизвестно" << endl;
}
```

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1362#3.

Логическое отрицание

Логическое отрицание (оператор !) означает, что условие не выполняется.

a	!a
True	False
False	True

Оператор ! принимает в качестве аргумента булеву переменную и меняет ее значение на противоположное (истина становится ложью, а ложь — истиной).

Пример: недопустимое число

Число считается допустимым, если оно находится в диапазоне [100 ... 200] или равно 0. Проведите проверку на недопустимое число.

Примеры ввода и вывода

Ввод	Вывод
75	недопустимое
150	(без вывода)
220	недопустимое

Решение

```
bool inRange = (number >= 100 && number <= 200) || number == 0;
if (!inRange) {
    cout << "недопустимое" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1362#4.

Оператор скобок ()

Как и другие операторы в программировании, операторы `&&` и `||` подчиняются правилам приоритета, в данном случае `&&` имеет больший приоритет, чем `||`. Оператор `()` используется для изменения приоритета операторов и оценивается первым, как и в математике. Использование круглых скобок также делает код более читабельным и считается хорошей практикой.

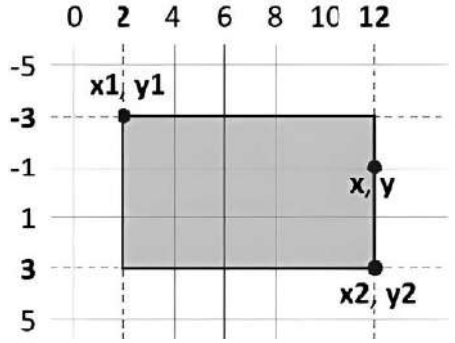
Более сложные логические условия

Иногда условия могут быть достаточно сложными и требовать длинного булева выражения или ряда проверок. Рассмотрим несколько таких примеров.

Пример: точка на стороне прямоугольника

Напишите программу, которая проверяет, находится ли точка `{x, y}` на одной из сторон прямоугольника `{x1, y1} - {x2, y2}`. Входные данные считываются с консоли и состоят из шести строк:

десятичные числа x1, y1, x2, y2, x и y (при этом необходимо, чтобы x1 < x2 и y1 < y2). Выведите “Сторона” (точка находится на одной из сторон) или “Внутри/снаружи” (в противном случае).



Примеры ввода и вывода

Ввод	Вывод
2 -3 12 3 12 -1	Сторона

Ввод	Вывод
2 -3 12 3 8 -1	Внутри/ снаружи

Решение

Точка лежит на одной из сторон прямоугольника, если:

- x совпадает с x1 или x2 и в то же время y находится между y1 и y2 или
- y совпадает с y1 или y2 и в то же время x находится между x1 и x2.

```
if (((x == x1 || x == x2) && (y >= y1) && (y <= y2)) ||
    ((y == y1 || y == y2) && (x >= x1) && (x <= x2))) {
    cout << "Граница" << endl;
}
```

Предыдущую проверку можно упростить способом, предложенным далее:

```
int onLeftSide = (x == x1) && (y >= y1) && (y <= y2);
int onRightSide = (x = x2) && (y >= y1) && (y <= y2);
```

```
int onUpSide = (y == y1) && (x >= x1) && (x <= x2);
int onDownSide = (y == y2) && (x >= x1) && (x <= x2);

if (onLeftSide || onRightSide || onUpSide || onDownSide) {
    cout << "Сторона" << endl;
}
```

Второй способ с дополнительными булевыми переменными длиннее, но он гораздо более понятен, чем первый, не так ли? Мы рекомендуем при написании булевых терминов делать их не короткими, а легкими для чтения и понимания. При необходимости используйте дополнительные переменные с осмысленными именами. Имена булевых переменных должны подсказывать, какое значение в них хранится.

Осталось добавить код для вывода «`Внутри/снаружи`», если точка не находится ни на одной из сторон прямоугольника.

Проверка в судейской системе

После завершения работы над решением вы можете протестировать его здесь: https://judge.softuni.bg/Contests/Practice/Index/1362#5.

Пример: магазин фруктов

Фруктовый магазин продает фрукты в будние дни по ценам, указанным во втором столбце таблицы, а в выходные дни — по ценам, указанным в третьем столбце:

Фрукты	Цена в будние дни	Цена выходного дня
банан	2.50	2.70
яблоко	1.20	1.25
апельсин	0.85	0.90
грейпфрут	1.45	1.60
киви	2.70	3.00
ананас	5.50	5.60
виноград	3.85	4.20

Напишите программу, которая считывает с консоли фрукт (банан / яблоко / ...), день недели (понедельник / вторник / ...) и количество (десятичное число) и вычисляет цену в соответствии с ценами из таблицы, приведенной выше. Выводимый на экран результат

округляется до двух цифр после запятой. В случае неверного дня недели или неверного названия фрукта выведите «ошибка».

Примеры ввода и вывода

Ввод	Вывод
апельсин воскресенье 2	2.70

Ввод	Вывод
киви понедельник 2.5	6.75

Ввод	Вывод
помидор понедельник 0.5	ошибка

Решение

```
if (day == "суббота" || day == "воскресенье") {
    if (fruit == "банан") {
        price = 2.70;
    }
    else if (fruit == "яблоко") {
        price = 1.25;
    }
    // TODO: добавить остальные фрукты...
}
else if (day == "понедельник" || day == "вторник" ||
    day == "среда" || day == "четверг" || day == "пятница") {
    if (fruit == "банан") {
    price = 2.50;
    }
    // TODO: добавить остальные фрукты...
```

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1362#6.

Пример: комиссионное вознаграждение

Компания устанавливает следующие комиссионные для своих продавцов в зависимости от города, в котором они работают, и объема продаж s:

Город	0 <= s <= 500	500 < s <= 1000	1000 < s <= 10000	s > 10000
София	5 %	7 %	8 %	12 %
Варна	4.5 %	7.5 %	10 %	13 %
Пловдив	5.5 %	8 %	12 %	14.5 %

Напишите программу, которая считывает *название города* (строка) и *объем продаж* (десятичное число) и вычисляет размер комиссионного вознаграждения. Результат должен выводиться с округлением до двух цифр после точки. Если город или объем продаж неверны (отрицательное число), выведите «ошибка».

Примеры ввода и вывода

Ввод	Вывод
София 1500	120.00

Ввод	Вывод
Пловдив 499.99	27.50

Ввод	Вывод
Бургас -50	ошибка

Решение

Первоначально мы устанавливаем комиссию равной -1. Она будет изменена, если в таблице комиссионных будут найдены город и ценовой диапазон. Для расчета комиссии в зависимости от города и объема продаж необходимо несколько вложенных проверок if, как в приведенном ниже примере кода:

```
double comission = -1.0;

if (town == "София") {
    if (0 <= sales && sales <= 500) {
        comission = 0.05;
    }
    else if (500 < sales && sales <= 1000) {
        comission = 0.07;
    }

// TODO: проверить остальные ценовые диапазоны...
```

```
}
else if (town == "Варна") {
    //TODO: проверить остальные ценовые диапазоны...
}
else if (town == "Пловдив") {
    // TODO: проверить остальные ценовые диапазоны...
}
if (comission >= 0) {
    cout << setprecision(2) << fixed << [illegible]
}
else {
    cout << "ошибка";
}
```

Хорошей практикой является использование блоков, которые мы заключаем в фигурные скобки `{ }` после `if` и `else`. Также рекомендуется делать отступ после `if` и `else` на одну табуляцию, чтобы код было легче читать.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1362#7.

Условная конструкция switch-case

Конструкция `switch-case` работает как серия блоков `if-else`. Когда работа программы зависит от значения переменной, вместо последовательных проверок с помощью блоков `if-else` можно использовать условный оператор `switch`. Он используется для выбора из списка возможных вариантов. Конструкция сравнивает значение с определенными константами и в зависимости от результата выполняет то или иное действие.

Переменная, которую мы хотим сравнить, помещается в круглые скобки после оператора `switch` и называется «селектором». Здесь тип должен быть сравниваемым. Последовательное сравнение начинается с каждого значения, идущего после метки `case`. При совпадении выполнение кода начинается с соответствующего места и продолжается до тех пор, пока не достигнет оператора

break. В C++ и некоторых других языках программирования оператор break может быть опущен для выполнения кода из другого оператора case до тех пор, пока он не достигнет рассматриваемого оператора. Если совпадения нет, то выполняется оператор default, если таковой существует:

```
switch (селектор) {
    case значение1:
        конструкция;
        break;
    case значение2:
        конструкция;
        break;
    case значение3:
        конструкция;
        break;
    ...
    default:
        конструкция;
        break;
}
```

В языке C++ значения селектора и метки case должны быть целыми числами (int). В отличие от других языков программирования, в C++ не допускается использование конструкции switch-case строками (string) или дробными числами (double).

Пример: день недели

Напишите программу, которая выводит день недели (на английском языке) в соответствии с введенным числом (1 ... 7) или «Error!», если введен неправильный день.

Примеры ввода и вывода

Ввод	Вывод
1	Monday
7	Sunday
-1	Error!

Решение

```
int day;
cin >> day;

switch (day)
{
case 1:
    cout << "Monday" << endl;
    break;
case 2:
    cout << "Tuesday" << endl;
    break;
...
default:
    cout << "Error!" << endl;
    break;
}
```

Хорошей практикой является размещение тех условий, которые обрабатывают наиболее часто встречающиеся ситуации, первыми, а конструкции, обрабатывающие менее часто встречающиеся ситуации, оставлять в конце, перед выражением по умолчанию. Другая хорошая практика — располагать метки `case` в порядке возрастания, независимо от того, являются ли они целочисленными или символьными.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1362#8.

Пример: класс животного

Напишите программу, которая выводит класс животного в соответствии с его названием:

- `собака -> млекопитающее`
- `крокодил, черепаха, змея -> рептилия`
- `другие -> неизвестно`

Примеры ввода и вывода

Ввод	Вывод
черепаха	рептилия

Ввод	Вывод
собака	млекопитающее

Ввод	Вывод
слон	неизвестно

Решение

Мы не можем использовать `switch-case` в этом случае, поскольку нам нужно сравнивать строки, а не целые числа.

Решим задачу с помощью нескольких проверок типа `if-else`:

```
if (animal == "собака") {
    cout << "млекопитающее" << endl;
}
else if (animal == "крокодил" ||
        animal == "черепаха" ||
        animal == "змея") {
    cout << "рептилия" << endl;
}
else {
    cout << "неизвестно" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1362#9.

Что мы узнали из этой главы?

Вспомним новые конструкции и приемы программирования, с которыми мы познакомились в этой главе.

Вложенные проверки

```
if (условие1) {
    if (условие2) {
        // тело
    }
    else {
    // тело
    }
}
```

Более сложные проверки с помощью &&, ||, ! и ()

```
if (((x == left) || (x == right))
    && (y >= top) && (y <= bottom)) {
    // тело
}
```

Проверки switch-case

```
switch (селектор)
{
    case значение1:
        конструкция;
        break;
    case значение2:
        конструкция;
        break;
    case значение3:
        конструкция;
        break;
    ...
    default:
        конструкция;
        break;
}
```

Упражнения: более сложные проверки

Теперь потренируемся работать с более сложными элементами управления.

Решим несколько практических задач.

Задача: кинотеатр

В кинозале кресла расположены в форме прямоугольника с количеством `rows` рядов и `columns` мест в ряду. Существует три типа киносеансов с билетами разной стоимости:

- Премьера — премьерный показ, цена 12.00 BGN.
- Обычный — стандартный показ, цена 7,50 BGN.
- Скидка — показ для детей, школьников и студентов по льготной цене 5,00 BGN.

Напишите программу, которая принимает на вход *тип показа* (строка), *количество рядов* и *количество мест в ряду* (целые числа) и вычисляет *общий доход* от билетов *при полном зале*. Результат должен быть выведен в том же формате, что и в примерах ниже — с округлением до двух цифр после точки.

Примеры ввода и вывода

Ввод	Вывод
Премьера 10 12	1440.00

Ввод	Вывод
Обычный 21 13	2047.50

Рекомендации и советы

Произведем считывание входных данных. Затем создадим и инициализируем переменную, в которой будет храниться рассчитанная нами выручка. В другой переменной мы вычислим полную вместимость зала. Поскольку мы не можем использовать здесь условный оператор `switch-case`, мы будем использовать `if-else` для расчета выручки в зависимости от типа показа и выводить результат на консоль в заданном формате (поищите в Интернете необходимую функциональность C++).

Пример кода описанной идеи (части кода размыты для стимулирования самостоятельного мышления и принятия решения):

```
string type;
int rows;
int columns;
[illegible]

int full = [illegible]
double income;

if (type == "Premiere") {
    income = [illegible]
}
[illegible]
    [illegible]
}
[illegible]
    [illegible]
}

[illegible]
```

Проверка в судейской системе

Проверьте решение в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1362#10.

Задача: волейбол

Владимир — студент, живет в Софии и иногда ездит в свой родной город. Он очень любит волейбол, но в будние дни занят и играет в волейбол только по выходным и праздникам. Владимир играет в София каждую субботу, когда он не на работе и не ездит в родной город, а также в 2/3 праздничных дней. Раз в год он ездит в свой родной город, где по воскресеньям играет в волейбол со своими старыми друзьями. Владимир не бывает на работе 3/4 выходных, когда он находится в городе.

Отдельно отметим, что в високосные годы Владимир играет в волейбол на 15 % больше, чем обычно. Предполагается, что в году есть ровно 48 выходных дней, подходящих для игры в волейбол.

Напишите программу, которая вычисляет, сколько раз Владимир играл в волейбол в течение года. Результат округлите до ближайшего целого числа (например, 2.15 -> 2; 9.95 -> 9).

Входные данные считываются с консоли:

- Первая строка содержит слово «`leap`» (високосный год) или «`normal`» (обычный год с 365 днями).
- Вторая строка содержит целое число `p` — количество праздничных дней в году (которые не являются субботой или воскресеньем).
- Третья строка содержит целое число `h` — количество выходных, в которые Владимир ездит в свой родной город.

Примеры ввода и вывода

Ввод	Вывод
leap 5 2	45

Ввод	Вывод
normal 3 2	38

Ввод	Вывод
normal 11 6	44

Ввод	Вывод
leap 0 1	41

Рекомендации и советы

По умолчанию мы считываем входные данные из консоли. Последовательно вычисляются выходные дни, проведенные в Софии, время игры в Софии и общее время игры. Наконец, мы проверяем, является ли год високосным, при необходимости производим дополнительные вычисления и выводим результат на консоль, округлив его до ближайшего целого числа (поищите в Интернете функцию на языке C++ с подобной функциональностью).

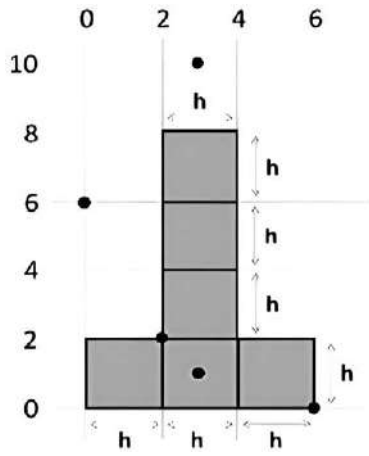
Пример кода описанной идеи (части кода размыты для стимулирования самостоятельного мышления и принятия решения):

```
string year;
int holidays;
int weekendsHome;
[illegible]


int sofiaWeekends = [illegible]
double playSofia =
    [illegible]
double playTotal = [illegible]

if ( [illegible] ) {
    playTotal =
        [illegible]
}
else if ( [illegible] ) {
    [illegible]
}

cout << playTotal;
```

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1362#11.

Задача*: точка на рисунке



Фигура состоит из шести блоков размером `h * h`, расположенных так, как показано на рисунке.

Левый нижний угол фигуры находится в позиции {0, 0}. Правый верхний угол фигуры находится в позиции {`2*h, 4*h`}. На рисунке координаты даны при `h = 2`.

Напишите программу, которая принимает целое число `h` и координаты точки {`x, y`} (целые числа) и выводит, находится ли эта точка внутри фигуры (`inside`), вне фигуры (`outside`) или на одной из сторон фигуры (`border`).

Примеры ввода и вывода

Ввод	Вывод
2 3 10	outside

Ввод	Вывод
2 3 1	inside

Ввод	Вывод
2 2 2	border

Ввод	Вывод
2 6 0	border

Ввод	Вывод
2 0 6	outside

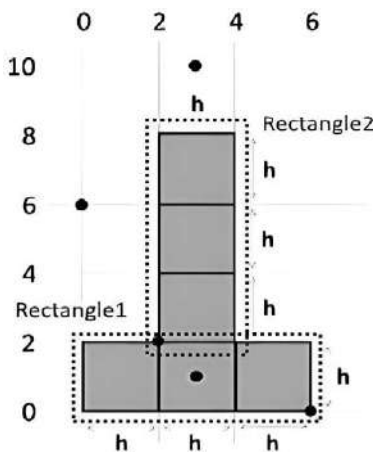
Ввод	Вывод
15 13 55	outside

Ввод	Вывод
15 29 37	inside

Ввод	Вывод
15 37 18	outside

Рекомендации и советы

Примерная логика решения задачи (не единственно верная):



- Мы можем разделить фигуру на два прямоугольника с общей стороной (см. рисунок).
- Точка является внешней по отношению к фигуре, если она одновременно находится вне обоих прямоугольников.
- Точка находится внутри фигуры, если она расположена внутри одного из прямоугольников (без учета их сторон) или лежит на их общей стороне.
- В противном случае точка лежит на стороне прямоугольника (`border`).

Пример кода (части кода размыты, чтобы стимулировать самостоятельное мышление и решение):

```
int h, x, y;
[illegible]

bool outRectangle1 = [illegible]
bool outRectangle2 = [illegible]

bool inRectangle1 = [illegible]
bool inRectangle2 = [illegible]

bool commonBorder = [illegible]

if ( [illegible] ) {
    cout << "outside" << endl;
}
```

```
else if ( [illegible] ) {
    cout << "inside" << endl;
}
else {
    cout << "border" << endl;
}
```

Проверка в судейской системе

Проверьте пример программы в автоматизированной судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1362#12.

Глава 4.2.
Более сложные проверки — экзаменационные задачи

В предыдущей главе мы познакомились с вложенными условными операторами в языке C++. С их помощью логика программы может быть представлена содержащимися друг в друге операторами `if`. Мы также рассмотрели условный оператор `switch-case`, который позволяет выбирать из списка возможных вариантов. Далее мы отработаем и закрепим полученные знания, рассмотрев несколько более сложных задач, приведенных на экзаменах. Прежде чем перейти к решению задач, вспомним условные операторы.

Вложенные проверки

```
if (условие1) {
    if (условие2) {
        // тело;
    } else {
        // тело;
    }
}
```

Помните, что не стоит писать глубоко вложенные условные операторы (с более чем тремя уровнями вложенности). Избегайте вложения более трех условных операторов друг в друга. Это усложняет код и затрудняет его чтение и понимание.

Проверки switch-case

Когда работа программы зависит от значения переменной, вместо последовательных проверок с помощью нескольких блоков `if-else` можно использовать условный оператор `switch-case`.

```
switch (селектор)
{
    case значение1:
        конструкция;
        break;
    case значение2:
        конструкция;
        break;
    default:
        конструкция;
        break;
}
```

Конструкция состоит из:

- Селектора — выражения, которое оценивается в некоторое определенное значение. Типом селектора может быть целое число или перечисление `enum`.
- Множества меток `case` с командами после них, которые могут заканчиваться оператором `break`, но, как мы уже знаем, его можно и опустить.

Экзаменационные задачи

Теперь, когда мы вспомнили, как используются условные операторы и как условные операторы вложены друг в друга для реализации более сложных проверок и программной логики, давайте решим несколько экзаменационных задач.

Задача: успеть к экзамену

Студент должен прийти на экзамен в определенное время (например в 9:30). Он заходит в экзаменационную аудиторию в заданное время (например в 9:40).

Студент считается пришедшим вовремя, если он пришел к моменту начала экзамена или не более чем за полчаса до него. Если он пришел раньше, более чем на 30 минут, то он пришел слишком рано. Если студент пришел после времени начала экзамена, то он опоздал.

Напишите программу, которая принимает время экзамена и время прихода студента и выводит, пришел ли студент вовремя, опоздал ли он или пришел слишком рано, на сколько часов или минут опоздал или пришел слишком рано.

Входные данные

Из консоли считываются четыре целых числа (по одному на строку):

- В первой строке указывается время экзамена — целое число от 0 до 23.
- Вторая строка содержит минуту экзамена — целое число от 0 до 59.
- Третья строка содержит время прибытия, целое число от 0 до 23.
- Четвертая строка содержит минуту прибытия, целое число от 0 до 59.

Выходные данные

Выведите на консоль в первой строке следующее сообщение:

- «`Опоздание`», если студент прибыл позже времени начала экзамена.
- «`Вовремя`» — если студент прибыл точно к началу экзамена или менее чем на 30 минут раньше.
- «`Слишком рано`», если студент прибыл более чем за 30 минут до начала экзамена.

Если студент прибыл с разницей более чем в одну минуту по сравнению с временем начала экзамена, выведите в следующей строке:

- «`За мм минут до начала`» в случае прибытия менее чем за час до начала.
- «`чч:мм часов до начала`» для прибытия за 1 час или раньше. Всегда выводите минуты, используя две цифры, например «`1:05`».
- «`мм минут после начала`» в случае прибытия с опозданием менее чем на час.

- «чч:мм часов после начала» для опоздавших на 1 час и более. Всегда выводите минуты, используя две цифры, например «1:03».

Примеры ввода и вывода

Ввод	Вывод
9 30 9 50	Опоздание 20 минут после начала

Ввод	Вывод
16 00 15 00	Слишком рано 1:00 до начала

Ввод	Вывод
9 00 8 30	Вовремя 30 минут до начала

Ввод	Вывод
9 00 10 30	Опоздание 1:30 после начала

Ввод	Вывод
14 00 13 55	Вовремя 5 минут до начала

Ввод	Вывод
10 00 10 00	Вовремя

Рекомендации и советы

Прежде чем приступить к написанию кода, следует несколько раз прочитать задание, делая заметки и зарисовывая схемы по мере осмысления его условий.

Обработка входных данных

Согласно заданию, мы ожидаем получить четыре последовательные строки с различными целыми числами. Учитывая заданные входные параметры, мы можем выбрать тип `int`, так как он удовлетворяет нашим ожидаемым значениям. Выбрав тип переменных, необходимо объявить их и считать их значения с консоли:

```
int examHours, examMinutes, arrivalHours, arrivalMinutes;

cin >> examHours;
```

```
cin >> examMinutes;
cin >> arrivalHours;
cin >> arrivalMinutes;
```

Посмотрев на ожидаемый вывод, мы можем создать переменные, содержащие различные типы вывода, чтобы избежать использования в коде так называемых «магических строк»:

```
string late = "Опоздание";
string onTime = "Вовремя";
string early = "Слишком рано";
```

Расчеты

Прочитав исходные данные, мы можем приступить к написанию логики вычисления результата. Для удобства и точности сравнения сначала вычислим время начала экзамена в минутах:

```
int examTime = (examHours * 60) + examMinutes;
```

Рассчитаем время прибытия студента, используя ту же логику:

```
int arrivalTime = (arrivalHours * 60) + arrivalMinutes;
```

Остается вычислить разницу между этими двумя значениями времени, чтобы определить, когда и к какому времени относительно начала экзамена пришел студент:

```
int totalMinutesDifference = arrivalTime - examTime;
```

Следующим шагом мы выполним необходимые проверки и вычисления и, наконец, выведем результат. Разделим полученный результат на две части:

- Сначала мы покажем, когда пришел студент — опоздал ли он, задержался или пришел вовремя. Для этого мы воспользуемся оператором `if-else`.
- Мы покажем разницу во времени, если студент прибыл в другое время, отличное от времени начала экзамена.

Чтобы не тратить лишнюю проверку (`else`), мы можем по умолчанию считать, что студент опоздал.

Затем, согласно условию, проверяем, не превышает ли разница во времени 30 минут, что означает, что он пришел более чем на 30 минут раньше. Если да, то мы предполагаем, что он пришел рано. Если мы не вводим первое условие, то нам достаточно проверить, меньше или равна ли разница нулю (`<= 0`), что проверяет условие, что студент пришел в пределах от 0 до 30 минут до начала экзамена.

Во всех остальных случаях мы предполагаем, что студент опоздал, что мы и сделали по умолчанию, и дальнейшая проверка не требуется:

```
string studentArrival = late;
if (totalMinutesDifference < -30) {
    studentArrival = early;
}
else if (totalMinutesDifference <= 0) {
    studentArrival = onTime;
```

В заключение необходимо выяснить и показать, с какой разницей во времени студент прибыл на экзамен, и указывает ли эта разница на время прибытия до или после экзамена.

Мы проверяем, превышает ли наша разница один час, чтобы отобразить часы и минуты в нужном формате, или меньше одного часа, чтобы отобразить только минуты. Осталось проверить еще один момент — время прибытия студента до или после начала экзамена:

```
string result;
if (totalMinutesDifference != 0) {
    int hoursDifference =
        abs(totalMinutesDifference / 60);
    int minutesDifference =
        abs(totalMinutesDifference % 60);

    if (hoursDifference > 0) {
        //result = "{hoursDifference}:"
        result = to_string(hoursDifference) + ":";

        //result = "{hoursDifference}:0" / "{hoursDifference}:"
        if (minutesDifference < 10) {
            result += "0";
    }

        //result = "{hoursDifference}:{minutesDifference} hours"
        result += to_string(minutesDifference) + " часов";

    }
    else {
        result = to_string(minutesDifference) + " минут";
```

```
    }

    if (totalMinutesDifference < 0) {
        result += " до начала";
    }
    else {
         result += " после начала";
    }
}
```

Вывод результата

Наконец, нам необходимо вывести результат на консоль. По заданию, если студент пришел точно в срок (без единой минуты разницы), мы не должны вычитать второй результат. Поэтому мы выполняем следующую проверку:

```
cout << studentArrival << endl;
if (result != "") {
    cout << result << endl;
}
```

Обязательно добавьте библиотеку `<string>`, если вы хотите работать с текстовыми данными.

Для целей данной задачи вывод результата на консоль может быть выполнен на более раннем этапе — во время самих вычислений. Как правило, это не очень хорошая практика. Почему?

Давайте подумаем о том, что наш код состоит не из 10 строк, а из 100 или 1000! Когда-нибудь вывод нужно будет делать не в консоль, а сохранять в файл или отображать в веб-приложении. Тогда сколько мест в коде придется исправить в связи с этим изменением? И не пропустим ли мы что-либо?

Всегда рассматривайте код, содержащий логические вычисления, как отдельную часть, отличную от той, что обрабатывает входные и выходные данные. Он должен работать независимо от того, как ему передаются данные и где будет отображаться результат.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1363#0.

Задача: путешествие

Как ни странно, большинство людей планируют свой отпуск заранее. У молодого болгарского программиста есть определенный бюджет и свободное время в конкретный сезон.

Напишите программу, которая принимает на вход бюджет и сезон и выводит, где будет отдыхать программист и сколько он потратит.

Бюджет определяет направление, а сезон — то, на что будет потрачен бюджет. Если сезон летний, программист отправится в кемпинг, если зимний — остановится в отеле. Если речь идет о Европе, независимо от сезона, программист остановится в отеле. Каждый кемпинг или отель, в зависимости от направления, имеет свою цену, которая соответствует определенному проценту от бюджета:

- Если 100 BGN или меньше — где-то в Болгарии.
  - Лето — 30 % бюджета.
  - Зима — 70 % бюджета.
- Если 1000 левов или меньше — где-то на Балканах.
  - Лето — 40 % бюджета.
  - Зима — 80 % бюджета.
- Если более 1000 BGN — где-то в Европе.
  - На путешествие по Европе, независимо от сезона, программист потратит 90 % бюджета.

Входные и выходные данные

Входные данные будут считываться с консоли и состоять из двух строк:

- В первой строке указывается бюджет — вещественное число в диапазоне `[10.00 ... 5000.00]`.
- Во второй строке указано одно из двух возможных времен года: «`лето`» или «`зима`».

Выходные данные

На консоль должны быть выведены две строки.

- В первой строке — «`Где-то {место назначения}`» из «`в Болгарии`», «`на Балканах`» и «`в Европе`».
- Во второй строке — «`{Тип отпуска} - {Сумма потраченных средств}`».
  - Отпуск может принимать значение «`Кемпинг`» или «`Отель`».
  - Сумма должна быть округлена до второй цифры после десятичной точки.

Примеры ввода и вывода

Ввод	Вывод
50 лето	Где-то в Болгарии Кемпинг - 15.00

Ввод	Вывод
75 зима	Где-то в Болгарии Отель - 52,50

Ввод	Вывод
312 лето	Где-то на Балканах Кемпинг - 124.80

Ввод	Вывод
1500 лето	Где-то в Европе Отель - 1350.00

Рекомендации и советы

Обычно, как и в случае с другими задачами, мы можем разделить решение на несколько частей:

- Чтение входных данных
- Расчеты
- Вывод результата

Обработка входных данных

Внимательно читая условие, мы понимаем, что ожидаем две строки входных данных. Первый параметр — вещественное число, для которого необходимо выбрать соответствующий тип переменной. В качестве типа для бюджета мы выберем `double`, а для сезона — `string`:

```
double budget;
cin >> budget;

string season;
cin >> season;
```

Всегда учитывайте, какой тип значений передается во входных данных, а также в какой тип их нужно преобразовать, чтобы условия программы работали правильно!

Расчеты

Создадим и инициализируем переменные, необходимые для логики и вычислений:

```
string destinationResult, holidaylnformation, restingPlace;
double moneySpent = 0;
```

Взглянув еще раз на условие задачи, мы замечаем, что базовое распределение того, где отдыхает программист, определяется величиной представленного бюджета, то есть наша базовая логика распадается на два случая:

- Если бюджет меньше заданного значения.
- Если он меньше другого значения или больше заданного предельного значения.

В зависимости от того, как мы выстроим нашу логическую схему (в каком порядке мы будем обходить граничные значения), мы будем иметь больше или меньше проверок в условиях. Подумайте, почему!

Затем нам необходимо провести проверку стоимости представленного сезона. По ней мы определим, какой процент бюджета будет потрачен, и где будет отдыхать программист — в отеле или в кемпинге.

Пример одного из возможных подходов к решению:

```
if (budget <= 100) {
    destinationResult = "в Болгарии";
    if (season == "лето") {
        moneySpent = 0.3 * budget;
        restingPlace - "Кемпинг";
    }
    else {
        moneySpent = 0.7 * budget;
        restingPlace = "Отель";
    }
}
else if (budget <= 1000) {
    destinationResult = "на Балканах";
    if (season == "лето") {
        moneySpent = 0.4 * budget;
        restingPlace = "Кемпинг";
    }
    else {
        moneySpent = 0.8 * budget;
        restingPlace = "Отель";
    }
}
else {
    destinationResult = "в Европе";
    moneySpent = 0.9 * budget;
    restingPlace = "Отель";
}
```

Мы всегда можем инициализировать значение параметра, а затем выполнить только одну проверку. Это позволяет сэкономить один логический шаг.

Например, следующий блок:

```
if (budget <= 100) {
    destinationResult = "в Болгарии";
    if (season == "лето") {
        moneySpent = 0.3 * budget;
        restingPlace = "Кемпинг";
    }
    else {
        moneySpent = 0.7 * budget;
        restingPlace = "Отель";
    }
}
```

можно сократить до:

```
destinationResult = "в Болгарии";
moneySpent = 0.7 * budget;
restingPlace = "Отель";

if (season == "лето") {
    moneySpent = 0.3 * budget
    restingPlace = "Кемпинг";
}
```

Вывод результата

Остается вывести на консоль рассчитанный результат:

```
cout << "Где-то в " << destinationResult << endl;
cout << restingPlace << " - " <<
    fixed << setprecision(2) << moneySpent << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1363#1.

Задача: операции над числами

Напишите программу, которая считывает два целых числа (`n1` и `n2`) и оператор для выполнения над ними математической операции. Возможные операции: сложение (`+`), вычитание (`-`), умножение (`*`), деление (`/`) и деление с остатком (`%`). При сложении, вычитании и умножении в консоли должны быть выведены результат и сведения о том, четное получилось число или нечетное. При простом делении — только результат, а при делении с остатком — остаток. Обратите внимание, что делитель может быть равен нулю (`= 0`), а деление на ноль невозможно. В этом случае должно быть выведено специальное сообщение.

Входные данные

Из консоли считываются три строки:

- `N1` — целое число в диапазоне `[0 ... 40 000]`.
- `N2` — целое число в диапазоне `[0 ... 40 000]`.
- Оператор — символ из следующих: «+», «-», «*», «/», «%».

Выходные данные

Выведите результат на консоль в виде *одной строки*:

- Если операция заключается в *сложении*, *вычитании* или *умножении*:
  - `"{N1} {оператор} {N2} = {вывод} - {четное/нечетное}"`.
- Если операция является *делением*:
  - `"{N1} / {N2} = {вывод}"` — результат форматируется до второй цифры после десятичной точки.
- Если операция представляет собой *деление с остатком*:
  - `"{N1} % {N2} = {остаток}"`.
- В случае *деления на 0 (ноль)*:
  - `"Невозможно разделить {N1} на ноль"`.

Примеры ввода и вывода

Ввод	Вывод
10 1 -	10 - 1 = 9 - нечетное

Ввод	Вывод
7 3 *	7 * 3 = 21 - нечетное

Ввод	Вывод
10 12 +	10 + 12 = 22 - четное

Ввод	Вывод
112 0 /	Невозможно разделить 112 на ноль

Ввод	Вывод
123 12 /	123 / 12 = 10.25

Ввод	Вывод
10 3 %	10 % 3 = 1

Рекомендации и советы

Задача несложная, но придется написать довольно много строк кода.

Как мы уже видели, хорошей практикой является разделение кода на части, но в данном случае это только усложнит задачу.

Обработка входных данных

Прочитав условие, мы понимаем, что ожидаем три строки входных данных. В первых двух строках нам даются целые числа (в указанном диапазоне), а в третьей — арифметический символ:

```
double n1, n2;
cin >> n1;
cin >> n2;

char nOperator;
cin >> nOperator;
```

Расчеты и вывод результата

Создадим и инициализируем переменную, в которой будем хранить результат вычислений:

```
double result = 0;
```

Внимательно прочитав условие, мы понимаем, что есть случаи, когда нам не нужно производить никаких вычислений, а достаточно вывести результат. Поэтому мы можем сначала проверить, равно ли второе число `0` (нулю), а также является ли операция делением или делением с остатком, а затем вывести нужное сообщение:

```
if (n2 == 0 && (nOperator == '/' || nOperator == '%')) {
    cout << "Невозможно разделить " << n1 << " на ноль" << endl;
}
```

Из условия видно, что для сложения (+), вычитания (-) или умножения (*) ожидаемый результат имеет одинаковую структуру: `{n1} {оператор} {n2} = {результат} - {четное/нечетное}`, в то время как для деления (/) и деления с остатком (%) результат имеет другую структуру.

В языке C++ мы не можем использовать оператор модульного деления (%) с вещественными числами, поэтому нам придется воспользоваться функцией модульного деления `fmod(...)`, а для этого нам понадобится библиотека `<cmath>`:

```
else if (nOperator == '/') {
    result = n1 / n2;
    cout << n1 << " " << nOperator << " " << n2 <<
         " = " << fixed << setprecision(2) << result << endl;
}
```

```
else if (nOperator == '%') {
    result = fmod(n1, n2);
    cout << n1 << " " << nOperator << " " << n2 <<
        " = " << result << endl;
}
```

В завершение мы проверяем сложение, вычитание и умножение:

```
else {
    if (nOperator == '+') {
        result = n1 + n2;
    }
    else if (nOperator == '-') {
        result = n1 - n2;
    }
    else if (nOperator == '*') {
        result = n1 * n2;
    }
    cout << n1 << " " << nOperator << " " << n2 <<
        " - " << result << " - " <<
        (fmod(result, 2) == 0 ? "четное" : "нечетное") << endl;
}
```

Для коротких и понятных проверок, как в приведенном выше примере для четных и нечетных чисел, можно использовать тернарный оператор. Рассмотрим возможную проверку с тернарным оператором и без него.

Без использования тернарного оператора код получается более длинным, но легко читаемым:

```
string numberIs;
if (fmod( result, 2) == 0) {
    numberIs = "четное";
}
else {
    numberIs = "нечетное";
}
```

При использовании тернарного оператора код получается значительно короче, но может потребовать дополнительных усилий для чтения и понимания логики:

```
string numberIs = fmod(result, 2) == 0 ? "четное" : "нечетное";
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1363#2.

Задача: билеты на матч

Группа футбольных болельщиков решила купить билеты на Еврокубок. Билеты продаются за болгарские левы (BGN) в двух ценовых категориях:

- VIP — 499,99 BGN (болгарских левов)
- Обычная (Normal) — 249,99 BGN (болгарских левов)

У футбольных фанатов есть общий бюджет, и количество человек в группе определяет, какой процент бюджета будет потрачен на транспорт:

- От 1 до 4 — 75 % бюджета
- От 5 до 9 — 60 % бюджета
- От 10 до 24 — 50 % бюджета
- От 25 до 49 — 40 % бюджета
- 50 и более — 25 % бюджета

Напишите программу, которая вычисляет, хватит ли оставшихся в бюджете денег, чтобы футбольные фанаты купили билеты в выбранной категории, а также то, сколько денег у них останется или будет недостаточно.

Входные данные

Входные данные считываются с консоли и содержат ровно три строки:

- В первой строке указывается *бюджет* — вещественное число в диапазоне `[1 000.00 ... 1 000 000.00]`.
- Во второй строке указывается *категория* — «VIP» или «Обычная».
- В третьей строке указывается *количество человек в группе* — целое число в диапазоне [1 ... 200].

Выходные данные

Выведите на консоль в виде одной строки следующее сообщение:

- Если бюджет достаточен:
  - `"Да! У вас осталось {N} лева"`, — где `N` — количество оставшихся денег у группы.
- Если бюджет недостаточен:
  - `"Не хватает денег! Вам нужно {M} лева"`, — где `M` — сумма, которой не хватает.

Суммы должны быть отформатированы до второй цифры после точки.

Примеры ввода и вывода

Ввод	Вывод	Пояснение
1000 Normal 1	Да! У вас осталось 0.01 лева.	1 человек: 75 % бюджета тратится на транспорт. Оставшаяся сумма: 1000 - 750 = 250. Категория Normal: цена билета составляет 249.99 * 1 = 249.99 249.99 < 250: у человека останется 250 - 249.99 = 0,01 лева

Ввод	Вывод	Пояснение
30000 VIP 49	Недостаточно денег! Вам нужно 6499.51 лева.	49 человек: 40 % бюджета тратится на транспорт. Оставшаяся сумма: 30000 - 12000 = 18000. Категория VIP: билет стоит 499.99*49. 24499.51 < 18000: сумма недостаточна, требуется больше денег: 24499.51 - 18000 = 6499.51

Рекомендации и советы

Прочитаем входные данные и выполним расчеты, описанные в постановке задачи, чтобы проверить, хватит ли денег.

Обработка входных данных

Давайте внимательно прочитаем требования и разберемся, что мы должны принимать в качестве входных данных, что ожидается вернуть в качестве результата, а также каковы основные шаги в решении задачи.

Для начала давайте обработаем и сохраним входные данные в соответствующих переменных:

```
double budget;
cin >> budget;

string ticketType;
cin >> ticketType;

int people;
cin >> people;
```

Расчеты

Создадим и инициализируем переменные, необходимые для вычислений:

```
double transportCharges = 0;
double moneyForTickets = 0;
double moneyDifference = 0;
```

Рассмотрим условие еще раз. Нам необходимо выполнить два разных блочных расчета.

Из первого расчета мы должны выяснить, какую часть бюджета нам необходимо выделить на транспорт. Для логики этих расчетов заметим, что значение имеет только количество человек в группе. Поэтому логическую разбивку мы сделаем в зависимости от количества болельщиков. Для этого мы будем использовать условную конструкцию — последовательность блоков `if-else`:

```
if (people <= 4) {
    transportCharges = 0.75 * budget;
}
else if (people <= 9) {
    transportCharges = 0.6 * budget;
}
else if (people <= 24) {
    transportCharges = 0.5 * budget;
}
else if (people <= 49) {
    transportCharges = 0.4 * budget;
```

```
}
else {
    transportCharges = 0.25 * budget
}
```

Из второго расчета нам необходимо найти сумму денег, которая потребуется для покупки билетов для группы. Согласно условию, это зависит только от типа билетов, которые нам необходимо приобрести. Воспользуемся условным оператором `if-else`:

```
if (ticketType == "VIP") {
    moneyForTickets = people * 499.99;
}
else {
    moneyForTickets = people * 249.99;
}
```

После того как мы подсчитали, каковы транспортные и билетные расходы, необходимо вычислить конечный результат и выяснить, сможет ли группа болельщиков поехать на Еврокубок или нет при заданных параметрах:

```
moneyDifference = budget - transportCharges - moneyForTickets;
```

Для получения результата, чтобы избавить себя от одной `else` проверки при построении — предположим, что группа по умолчанию сможет поехать на Еврокубок:

```
else {
    cout << "Да! У вас осталось " <<
        fixed << setprecision(2) << moneyDifference <<
        " лева." << endl;
}
```

Вывод результата

Наконец нам осталось вывести вычисленный результат на консоль.

Проверка в судейской системе

Проверьте свое решение в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1363#3.

Задача: гостиничный номер

Отель предлагает *два типа номеров: студия и апартаменты.* Цены указаны в болгарских левах (BGN / lv).

Напишите программу, которая рассчитывает стоимость всего проживания для студии и апартаментов. Цены зависят от *месяца* проживания:

Май и октябрь	Июнь и сентябрь	Июль и август
Студия - 50 BGN/ночь	Студия - 75.20 BGN/ночь	Студия - 76 BGN/ночь
Апартаменты - 65 BGN/ночь	Апартаменты - 68.70 BGN/ночь	Апартаменты - 77 BGN/ночь

Также предлагаются следующие скидки:

- При проживании в студии более 7 суток в мае и октябре: скидка 5 %.
- При проживании в студии более 14 суток в мае и октябре: скидка 30 %.
- При проживании в студии более 14 суток в июне и сентябре: скидка 20 %.
- При проживании в апартаментах более 14 суток, без ограничения месяца: скидка 10 %.

Входные данные

Входные данные считываются с консоли и содержат ровно две строки:

- В первой строке указывается *месяц* — `май`, `июнь`, `июль`, `август`, `сентябрь` или `октябрь`.
- Вторая строка — это время пребывания, целое число в диапазоне `[0 ... 200]`.

Выходные данные

На консоль выводятся следующие две строки:

- В первой: `"Апартаменты: { цена за все время проживания } lv."`

- Во второй: "Студия: {цена за все время проживания } lv."

Цена за все время пребывания должна быть отформатирована до двух символов после точки.

Примеры ввода и вывода

Ввод	Вывод	Пояснения
Май 15	Апартаменты: 877.50 lv. Студия: 525.00 lv.	В мае, при проживании более 14 суток, скидка на студию составляет 30 % (50 - 15 = 35), а на апартаменты - 10 % (65 - 6,5 = 58,5). За все время проживания в апартаментах: 877.50 lv. Полное проживание в студии: 525.00 lv.

Ввод	Вывод
Июнь 14	Апартаменты: 961.80 lv. Студия: 1052.80 lv.

Ввод	Вывод
Август 20	Апартаменты: 1386.00 lv. Студия: 1520.00 lv.

Рекомендации и советы

Посчитаем входные данные и выполним расчеты в соответствии с предоставленным прайс-листом и правилами скидок, а затем выведем результат.

Обработка входных данных

В соответствии с требованиями задачи считываем две строки входных данных: *месяц, в котором планируется пребывание* (первая строка), и *количество дней пребывания* (вторая строка).

Обработаем и сохраним входные данные в соответствующих переменных:

```
string month;
cin >> month;

int nights;
cin >> nights;
```

Расчеты

Создайте и инициализируйте переменные, необходимые для вычислений.

```
double studioPrice = 0;
double apartmentPrice = 0;
double studioRent = 0;
double apartnientRent = 0;
```

Рассматривая условие еще раз, мы замечаем, что наша основная логика зависит от того, какой месяц нам дан, а также от количества ночей.

Вообще говоря, существуют различные подходы и способы выполнения рассматриваемых проверок, и обычно мы использовали бы условный оператор `switch-case` с `if` и `if-else` в различных блоках `case`, но C++ не позволяет использовать для селектора переменную типа `string`, и нам придется использовать только операторы `if` и `if-else`.

Начнем с первой группы месяцев: май и октябрь. В эти два месяца цена одинакова для обоих типов размещения — студии и апартаментов.

Соответственно, нам достаточно выполнить внутреннюю проверку количества ночей, чтобы пересчитать соответствующую цену (при необходимости):

```
if (month == "май" || month == "октябрь") {
    studioPrice = 50;
    apartmentPrice = 65;

    studioRent = studioPrice * nights;
    apartmentRent = apartmentPrice * nights;

    if (nights > 14) {
        studioRent *= 0.7;
        apartmentRent *= 0.9;
    }
    else if (nights > 7) {
        studioRent *= 0.95;
    }
}
```

Для последующих месяцев логика и расчеты будут примерно одинаковыми:

```
else if (month == "июнь" || month == "сентябрь") {
    studioPrice = 75.2;
```

```
    apartmentPrice = 68.7;

    studioRent = studioPrice * nights;
    apartmentRent = apartmentPrice * nights;
    if (nights > 14) {
        studioRent *= 0.8;
        apartmentRent *= 0.9;
    }
}

else if (month == "июль" || month == "август") {
    studioPrice = 76;
    apartmentPrice = 77;

    studioRent = studioPrice * nights;
    apartmentRent = apartmentPrice * nights;
    if (nights > 14) {
        apartmentRent *= 0.9;
    }
}
```

Вывести результат

Наконец, необходимо вывести вычисленные результаты на консоль, не забыв при этом, что цена пребывания должна быть отформатирована с точностью до двух знаков после запятой:

```
cout << "Апартаменты: " <<
    fixed << setprecision(2) << apartmentRent <<
    " lv." << endl;
cout << "Студия: " <<
    fixed << setprecision(2) << studioRent <<
    " lv." << endl;
```

Проверка в судейской системе

Проверьте свое решение в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1363#4.

Глава 5.1.
Повторы (циклы)

В этой главе мы познакомимся с конструкциями для повторения группы инструкций, которые в программировании называются циклами. Мы напишем несколько циклов, используя оператор `for` в его классической форме. Наконец, мы решим несколько практических задач, требующих повторения последовательности действий с использованием циклов.

Повторения блоков кода (цикл for)

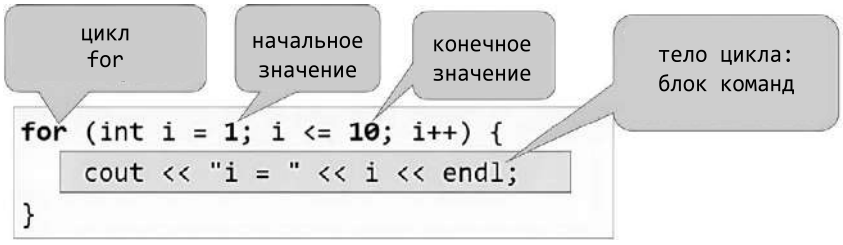
В программировании часто приходится выполнять блок инструкций несколько раз. Для этого используются так называемые циклы. Рассмотрим пример цикла `for`, который последовательно перебирает числа от 1 до 10 и выводит их на экран:

```
for (int i = 1; i <= 10; i++) {
    cout << "i = " << i << endl;
}
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/ForLoopExample.

Цикл начинается с оператора `for` и перебирает все значения переменной в заданном диапазоне, например, все числа от 1 до 10 включительно, и для каждого значения выполняет последовательность инструкций.

На рисунке ниже показана структура цикла `for`:



В объявлении цикла должны быть указаны начальное и конечное значения. Тело цикла обычно заключено в фигурные скобки `{ }` и представляет собой блок с одной или несколькими командами.

Цель цикла — последовательно перебрать числа 1, 2, 3, ..., n и выполнить для каждого из них некоторое действие. В приведенном примере мы создаем локальную переменную `i` с начальным значением 1 (`int i = 1`) и увеличиваем ее значение на 1 (`i++`) каждый раз, когда выполняется код между фигурными скобками. Цикл повторяется до тех пор, пока не будет выполнено условие `i <= 10` (`true`). Каждое такое повторение называется итерацией. В приведенном примере переменная `i` принимает значения от 1 до 10 включительно, а текущее значение выводится в теле цикла.

Пример: числа от 1 до 100

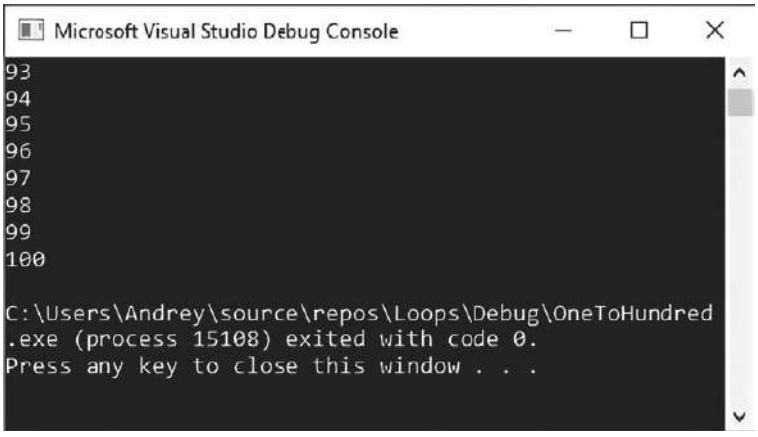
Напишите программу, которая выводит числа от 1 до 100. Эта программа не принимает никаких входных данных и выводит числа от 1 до 100 одно за другим, по одному в строке.

Рекомендации и советы

Решить задачу можно с помощью цикла `for`, в который мы передаем переменную `i` со значениями от 1 до 100:

```
for (int i = 1; i <= 100; i++) {
    cout << i << endl;
}
```

Запустите программу с помощью [Ctrl + F5] и протестируйте ее:



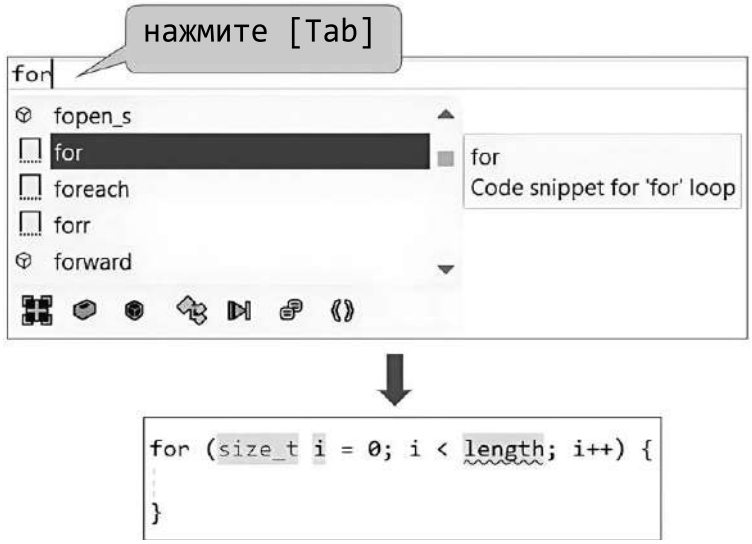
Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#0.

Вы должны получить 100 баллов (полностью правильное решение).

Сниппет (шаблон) кода для цикла for в Visual Studio

При программировании нам часто приходится писать циклы. Поэтому в большинстве сред разработки (IDE) имеются фрагменты (шаблоны, сниппеты) кода для написания циклов. Одним из таких шаблонов является шаблон цикла `for` в Visual Studio. Введите `for` в редакторе кода C++ в Visual Studio и нажмите один раз [Tab]. Visual Studio развернет для вас шаблон и напишет полный цикл `for`:



Попробуйте сами, чтобы овладеть навыками использования шаблона кода цикла `for` в Visual Studio.

Пример: числа до 1000, оканчивающиеся на 7

Напишите программу, которая находит все числа в интервале [1 ... 1000], оканчивающиеся на 7.

Рекомендации и советы

Мы можем решить эту задачу, используя цикл `for` для перебора чисел от 1 до1000 и проверяя для каждого числа, оканчивается

ли оно на 7. Конечно, есть и другие решения, но давайте решим задачу циклом + проверкой:

```
for (int i = 1; i <= 1000; i++) {
    if (i % 10 == 7) {
        // TODO: вывести i
    }
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#1.

Пример: все латинские буквы

Напишите программу, которая выводит на экран буквы латинского алфавита: a, b, ..., z.

Рекомендации и советы

Символы в C++ хранятся в памяти в виде чисел. Для их преобразования из числа в символ и обратно используется так называемая таблица ASCII. Например, символ `a` (строчная буква «a») хранится в памяти компьютера как число 97, а символ `z` (строчная буква «z») — как 122. Это означает, что мы можем создать цикл, использующий тип данных `char`, и изменять символ, увеличивая соответствующее ему число:

```
cout << "Latin alphabet: ";

for (char i = 'a'; i <= 'z'; i++) {
    cout << i << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#2.

Пример: сложение чисел

Напишите программу, которая принимает `n` целых чисел и складывает их.

- В первой строке вводится количество чисел `n`.

- Начиная со следующих n строк вводится по одному числу.
- Числа складываются, и в конце выводится результат.

Примеры ввода и вывода

Ввод	Вывод
2 10 20	30

Ввод	Вывод
3 -10 -20 -30	-60

Ввод	Вывод
4 45 -20 7 11	43

Ввод	Вывод
1 999	999

Ввод	Вывод
0	0

Рекомендации и советы

Задачу сложения можно решить следующим образом:

- Считываем с консоли входное число n.
- Начнем с того, что первоначально сумма `sum = 0`.
- Запускаем цикл от `0` до n. На каждом шаге цикла мы считываем число num и прибавляем его к сумме sum.
- Наконец, мы выводим полученную сумму sum.

Приведем исходный код описанной идеи:

```
int n, sum = 0;
cout << "Сколько чисел вы хотите сложить?";
cin >> n;

cout << "Введите числа:" << endl;
for (int i = 0; i < n; i++) {
    int num;
    cin >> num;
    sum = sum + num;
}

cout << "Сумма составляет " << sum << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#3.

Пример: наибольшее число

Напишите программу, которая принимает на вход n целых чисел (n > 0) и находит наибольшее из них. В первой строке вводится количество целых чисел n. Затем вводятся сами числа по одному в строку.

Примеры ввода и вывода

Ввод	Вывод
2 100 99	100

Ввод	Вывод
3 -10 20 -30	20

Ввод	Вывод
4 45 -20 7 99	43

Ввод	Вывод
1 999	999

Ввод	Вывод
2 -1 -2	-1

Рекомендации и советы

Задачу о наибольшем числе можно решить следующим образом:

- Введем количество чисел `n`.
- Прочитаем отдельно первое число и инициализируем переменную `max` с его значением, чтобы мы могли сравнить с ним следующие числа.
- Создадим цикл, начинающийся с 1 (чтобы пропустить первое отдельно прочитанное число) до `n`.
- На каждом шаге мы считываем число и проверяем, больше ли оно последнего найденного наибольшего (то есть переменной `max`) и если так и есть — сохраняем его значение в переменной `max`.

- Наконец, мы выводим переменную, в которой хранится наибольшее число.

```
int n, max;
cin >> n;
cin >> max;

for (int i = 1; i < n; i++) {
    int number;
    cin >> number;

    if (number > max) {
        max = number;
    }
}

cout << max << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#4.

Пример: наименьшее число

Напишите программу, которая вводит n целых чисел (n > 0) и находит среди них наименьшее. Сначала введите количество целых чисел n, затем n чисел по одному в строку.

Примеры ввода и вывода

Ввод	Вывод
2 100 99	99

Ввод	Вывод
3 -10 20 -30	-30

Ввод	Вывод
4 45 -20 7 99	-20

Рекомендации и советы

Задача полностью аналогична предыдущей, но с другой проверкой:

```
int n, min;
cin >> n;
cin >> min;

for (int i = 1; i < n; i++) {
    int number;
    cin >> number;

    if (number < min) {
        min = number;
    }
}

cout << min << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#5.

Задача: сумма левого и правого

Напишите программу, которая принимает `2 * n` целых чисел и проверяет, равна ли сумма первых `n` целых чисел (левая сумма) сумме вторых `n` чисел (правая сумма). Если суммы равны, выведите «`Да`» + сумму, в противном случае выведите «`Нет`» + разность. Разница вычисляется как положительное число (по абсолютной величине). Формат вывода должен соответствовать приведенному ниже образцу.

Примеры ввода и вывода

Ввод	Вывод
2 10 90 60 40	Да, сумма = 100

Ввод	Вывод
2 90 9 50 50	Нет, разность = 1

Рекомендации и советы

Сначала мы вводим число n, затем первые n чисел (левая половина) и складываем их. Далее вводим еще n чисел (правая половина) и находим их сумму.

Вычисляем разность между найденными суммами по абсолютной величине: `abs(leftSum - rightSum)`. Если разность равна 0, выводим «`Да, сумма = `» + сумму, иначе выводим «`Нет, разность = `» + разность:

```
int n, leftSum = 0, rightSum = 0;
cin >> n;

for (int i = 0; i < n; i++) {
    int num;
    cin >> num;
    leftSum += num;
}

// TODO: прочитать и вычислить rightSum

if (leftSum == rightSum) {
    cout << "Да, сумма = " << leftSum << endl;
}
else {
    int diff = abs(leftSum - rightSum);
    cout << "Нет, разность = " << diff << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#6.

Пример: четная/нечетная сумма

Напишите программу, которая вводит n целых чисел и проверяет, равна ли сумма чисел в четных позициях сумме чисел в нечетных позициях. Если равна, то выводит «Да» и сумму, иначе — «Нет» и разность. Разность вычисляется по абсолютному значению. Формат вывода должен быть таким, как показано в примерах ниже.

Примеры ввода и вывода

Ввод	Вывод
4 10 50 60 20	Да Сумма = 70

Ввод	Вывод
4 3 5 1 -2	Нет Разность = 1

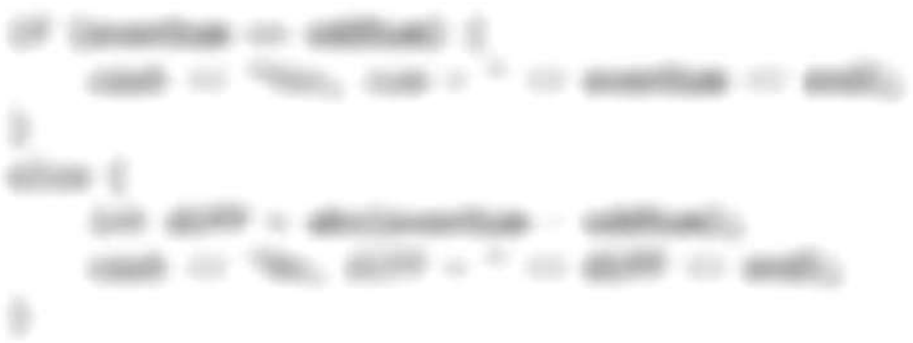
Ввод	Вывод
3 5 8 1	Нет Разность = 2

Рекомендации и советы

Мы вводим числа по одному и вычисляем две суммы (чисел на *четных* позициях и чисел на *нечетных* позициях). Идентично предыдущей задаче вычисляем абсолютное значение разности и выводим результат («Да» + *сумма* в случае разности, равной 0, или «Нет» + *разность* в любом другом случае):

```
int n, evenSum = 0, oddSum = 0;
cin >> n;

for (int i = 0; i < n; i++) {
    int num;
    cin >> num;

    if (i % 2 == 0) {
        evenSum += num;
}
```

```
    else {
        oddSum += num;
    }
}
```



Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#7.

Пример: суммирование гласных букв

Напишите программу, которая принимает на вход *текст* (строку), вычисляет и выводит *сумму значений гласных букв* в соответствии с таблицей ниже:

a	e	i	o	u
1	2	3	4	5

Примеры ввода и вывода

Ввод	Вывод	Пояснение
hello	6	e+o = 2+4 = 6
hi	3	(i = 3)
bamboo	9	a+o+o = 1+4+4 = 9
beer	4	e+e = 2+2 = 4

Рекомендации и советы

Мы считываем входной текст `text`, инициализируем переменную для суммы и создаем цикл, в котором перебираем все символы входного текста (от `0` до `text.length()`).

Перебирая символы, мы проверяем, является ли текущий символ (`text[i]`) гласной буквой, и если да, то добавляем соответствующее число к сумме:

```
string text;
getline(cin, text);

int sum = 0;

for (int i = 0; i < text.length(); i++) {
    char currentLetter = text[i];

    if (currentLetter == 'a') {
        sum += 1;
    }
    else if (currentLetter == 'e') {
        sum += 2;
    }
    else if (currentLetter == 'i') {
        sum += 3;
    }
    else if (currentLetter ==  'o') {
        sum += 4;
    }
    else if (currentLetter == 'u') {
        sum += 5;
    }
}

cout << "Сумма гласных = " << sum << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#8.

Что мы узнали из этой главы?

Мы можем выполнять итерации по блоку кода с помощью цикла `for`:

```
for (int i = 1; i <= 10; i++) {
    cout << "i = " << i << endl
}
```

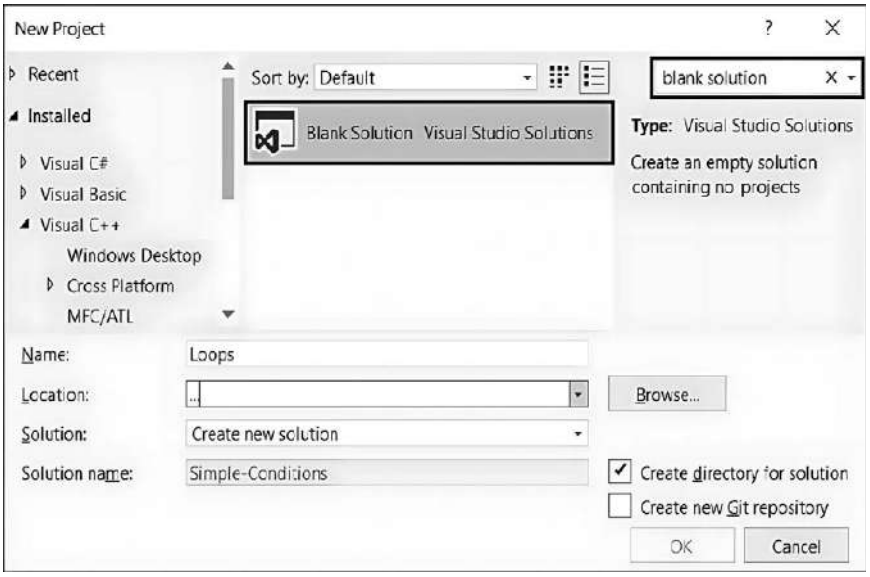
Мы можем считать с консоли последовательность из `n` чисел.

Упражнения: повторения (циклы)

Теперь, когда мы узнали о циклах, пришло время закрепить полученные знания на практике, а это, как известно, сопровождается написанием большого количества кода. Для тренировки давайте решим несколько задач.

Пустое решение Visual Studio (Blank Solution)

Для лучшей организации задач упражнения мы создаем в Visual Studio пустое решение. Это пустое решение должно содержать по одному проекту для каждой задачи упражнения:



Установим, что по умолчанию запускается текущий проект (не первый в решении). Щелкните правой кнопкой мыши на [Solution 'Loops'] -> [Set StartUp Projects...] -> [Current selection].

Примеры ввода и вывода

Ввод	Вывод	Пояснение
7 3 4 1 1 2 12 1	Да Сумма = 12	3 + 4 + 1 + 2 + + 1 + 1 = 12

Ввод	Вывод	Пояснение
3 1 1 10	Нет Разность = 8	|10 - (1 + + 1)| = 8

Ввод	Вывод
3 1 1 1	Нет Разность = 1

Ввод	Вывод
3 5 5 1	Нет Разность = 1

Ввод	Вывод
4 6 1 2 3	Нет Разность = 6

Рекомендации и советы

Нужно вычислить *сумму* всех элементов, найти *наибольший* из них и проверить условие.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1364#9.

Задача: четные/нечетные позиции

Напишите программу, которая считывает `n` чисел и вычисляет сумму, минимум и максимум чисел в четных и нечетных позициях (считая от 1). При отсутствии минимального/максимального элемента вывести «`Нет`».

Примеры ввода и вывода

Ввод	Вывод
6 2 3 5 4 2 1	OddSum=9, OddMin=2, OddMax=5, EvenSum=8, EvenMin=1, EvenMax=4

Ввод	Вывод
2 1.5 -2.5	OddSum=1.5, OddMin=1.5, OddMax=1.5, EvenSum=-2.5, EvenMin=-2.5, EvenMax=-2.5

Ввод	Вывод
1 1	OddSum=1, OddMin=1, OddMax=1, EvenSum=0, EvenMin=Нет, EvenMax=Нет

Ввод	Вывод
3 -1 -2 -3	OddSum=-4, OddMin=-3, OddMax=-1, EvenSum=-2, EvenMin=-2, EvenMax=-2

Ввод	Вывод
1 -5	OddSum=-5, OddMin=-5, OddMax=-5, EvenSum=0, EvenMin=Нет, EvenMax=Нет

Ввод	Вывод
1 1	OddSum=8, OddMin=-3, OddMax=8, EvenSum=9, EvenMin=-2, EvenMax=11

Рекомендации и советы

Задача объединяет некоторые из предыдущих: нахождение *минимального* и *максимального* значения и *суммы*, а также обработку элементов на *четных и нечетных позициях*.

В этой задаче лучше работать с *дробными числами* (а не с целыми). Сумма, минимум и максимум являются дробными числами.

Проверка в судейской системе

Проверьте свое решение в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1364#10.

Задача: идентичные пары

Имеется `2 * n` чисел. Первое и второе числа образуют *пару*, третье и четвертое — тоже, и так далее. Каждая пара имеет *значение* — сумму своих чисел. Напишите программу, которая проверяет, *все ли пары имеют одинаковое значение*.

Если значение одинаково, выведите «`Да, значение=...`» + *значение*, в противном случае выведите *максимальную разницу* между двумя соседними парами в формате: «`Нет, maxdiff=...`» + *максимальная разница.*

На вход подаются число `n`, затем `2 * n` целых чисел, по одному в строке.

Примеры ввода и вывода

Ввод	Вывод
3 1 2 0 3 4 -1	Да, значение = 3 Комментарии значения = {3, 3, 3} равные значения

Ввод	Вывод
2 1 2 2 2	Нет, maxdiff=1 Комментарии значения = {3, 4} различия = {1} максимальное различие = 1

Ввод	Вывод
2 -1 2 0 -1	Нет, maxdiff = 2 Комментарии значения = {2, 4, 4, 0} различия = {2, 0, 4} максимальное различие = 4

Ввод	Вывод
1 5 5	Да, значение=10 Комментарии значения = {10} одно значение равные значения

Ввод	Вывод
2 -1 0 0 -1	Да, значение=-1

Ввод	Вывод
4 1 1 3 1 2 2 0 0	Нет, maxdiff=4

Рекомендации и советы

Мы читаем входные числа *парами*. Для каждой пары вычисляем ее *сумму*. В процессе чтения входных пар для каждой пары, кроме первой, вычисляем *разницу по сравнению с предыдущей*. Для этого нам нужно сохранить в отдельной переменной сумму предыдущей пары. Наконец, мы находим *наибольшую разницу* между двумя парами. Если она равна 0, выводим «Да» + значение, иначе — «Нет» + разность.

Проверка в судейской системе

Проверьте свое решение в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1364#11.

Глава 5.2.
Повторы (циклы) — экзаменационные задачи

В предыдущей главе мы узнали, как выполнять блок команд более одного раза. Таким образом, мы познакомились с циклом `for` и рассмотрели некоторые основные способы его использования. Цель этой главы — закрепить полученные знания, решив несколько более сложных задач с циклами, которые даются на вступительных экзаменах. Для некоторых из них мы покажем подробные примеры решения, а для других оставим только подсказки. Прежде чем приступить к работе, нелишним будет вспомнить конструкцию цикла `for`:

Циклы `for` состоят из следующих элементов:

- Блок инициализации, в котором объявляется переменная-перечислитель (`int i`) и устанавливается ее начальное значение.
- Условие итерации (`i <= 10`), выполняемое один раз перед каждой итерацией цикла.
- Обновление счетчика (`i++`) — этот код выполняется после каждой итерации и указывает, на какой шаг следует обновить переменную счетчика.
- Тело цикла — содержит произвольный блок исходного кода.

Экзаменационные задачи

Давайте решим некоторые экзаменационные задачи SoftUni с циклами.

Задача: гистограмма

У нас есть `n` целых чисел в диапазоне [1 ... 1000]. Некоторый процент из них `p1` меньше 200, другая часть `p2` — от 200 до 399, `p3` — от 400 до 599, `p4` — от 600 до 799 и остальные `p5` — от 800 и выше. Напишите программу, которая вычисляет и выводит, сколько процентов составляют `p1`, `p2`, `p3`, `p4` и `p5`.

Пример: у нас есть `n = 20` чисел: 53, 7, 56, 180, 450, 920, 12, 7, 150, 250, 680, 2, 600, 200, 800, 799, 199, 46, 128, 65. Мы получаем следующее распределение и визуализацию:

Группа	Числа	Количество чисел	Процент
`< 200`	`53, 7, 56, 180, 12, 7, 150, 2, 199, 46, 128, 65`	12	`p1 = 12 / 20 * 100 = 60.00 %`
`200... 399`	`250, 200`	2	`p2 = 2 / 20 * 100 = 10.00 %`
`400... 599`	`450`	1	`p3 = 1 / 20 * 100 = 5.00 %`
`600... 799`	`680, 600, 799`	3	`p4 = 3 / 20 * 100 = 15.00 %`
`≥ 800`	`920, 800`	2	`p5 = 2 / 20 * 100 = 10.00 %`

Входные данные

В первой строке ввода находится целое число `n` ($1 \leq n \leq 1000$), представляющее собой количество строк с числами, которые будут переданы. В каждой из следующих `n` строк находится по одному целому числу в диапазоне [1... 1000] — числа, по которым мы должны рассчитать гистограмму.

Выходные данные

Выведите на консоль гистограмму, состоящую из пяти строк, каждая из которых содержит число в диапазоне [0 % ... 100 %], отформатированное до двух цифр после точки (например, 25.00 %, 66.67 %, 57.14 %).

Примеры ввода и вывода

Ввод	Вывод
3 1 2 999	66.67 % 0.00 % 0.00 % 0.00 % 33.33 %

Ввод	Вывод
4 53 7 56 999	75.00 % 0.00 % 0.00 % 0.00 % 25.00 %

Ввод	Вывод
7 800 801 250 199 399 599 799	14.29 % 28.57 % 14.29 % 14.29 % 28.57 %

Ввод	Вывод
9 367 99 200 799 999 333 555 111 9	33.33 % 33.33 % 11.11 % 11.11 % 11.11 %

Ввод	Вывод
14 53 7 56 180 450 920 12 7 150 250 680 2 600 200	57.14 % 14.29 % 7.14 % 14.29 % 7.14 %

Программу, решающую эту задачу, можно мысленно разделить на три части:

- *Считывание входных данных* — в данной задаче это чтение числа `n`, за которым следует `n` целых чисел, каждое в отдельной строке.
- *Обработка входных данных* — в данном случае это распределение чисел по группам и вычисление процентного соотношения по группам.
- *Вывод результата* — вывод гистограммы на консоль в указанном формате.

Чтение входных данных

Прежде чем перейти к чтению входных данных, необходимо объявить переменные, в которых мы будем их хранить.

Вспомним, что в программировании каждая переменная имеет некоторый тип данных. В данной задаче мы будем использовать числовые типы `int` для целых чисел и `double` для дробных.

Начнем с выбора соответствующего типа данных и подходящих имен для переменных:

```
int n;
// Переменные, в которых мы будем хранить
// процентное разделение для каждой из групп
```

```
double p1Percentage = 0;
double p2Percentage = 0;
double p3Percentage = 0;
double p4Percerttage = 0;
double p5Percentage = 0;

// Переменные для хранения количества чисел по группам
int cntP1 = 0
int cntP2 = 0
int cntP3 = 0
int cntP4 = 0
int cntP5 = 0
```

В переменной `n` мы будем хранить количество чисел, которые мы будем считывать с консоли. Тип `int` мы выбираем потому, что в условии указано, что `n` — целое число в диапазоне от 1 до 1000. Для переменных, в которых мы будем хранить проценты, мы выбрали тип `double`, так как не всегда ожидается, что они будут целыми числами. Кроме того, мы объявляем переменные `cntP1`, `cntP2` и т. д., в которых будем хранить количество чисел соответствующей группы, и для них снова выбираем тип `int`.

Объявив необходимые переменные, мы можем приступить к чтению числа `n` с консоли:

```
cin >> n;
```

Обработка входных данных

Для чтения и распределения каждого числа в соответствующую группу мы будем использовать цикл `for` от `0` до `n` (количество чисел). На каждой итерации цикла будет считываться и распределяться одно число (`currentNumber`) в соответствующую группу. Чтобы определить принадлежность числа к данной группе, мы проверяем его диапазон. Если оно принадлежит диапазону, увеличиваем количество чисел в этой группе (`cntP1`, `cntP2` и т. д.) на 1.

```
for (int i = 0; i < n; i++) {
    int currentNumber;
    cin >> currentNumber;

    if (currentNumber < 200) {
        cntP1++;
    }
```

```
    else if (currentNumber >= 200 && currentNumber < 400) {
        cntP2++;
    }
    else if (currentNumber >= 400 && currentNumber < 600) {
        cntP3++;
    }
    else if (currentNumber >= 600 && currentNumber < 800) {
        cntP4++;
    }
    else {
        cntP5++;
    }
}
```

Определив количество чисел в каждой группе, мы можем перейти к вычислению процентного соотношения, что и является основной целью данной задачи. Для этого воспользуемся следующей формулой:

```
(процент группы) = (количество чисел в группе) * 100 /
(количество всех чисел)
```

Эта формула в программном коде выглядит следующим образом:

```
p1Percentage = cntP1 * 100.0 / n;
```

Если делить не на 100 (число типа `int`), а на 100.0 (число типа `double`), то произойдет так называемое целочисленное деление, и в переменную будет записана только целая часть деления, что не является желаемым результатом. Например: 5 / 2 = 2, а 5 / 2,0 = 2,5.

С учетом этого формула для первой переменной будет выглядеть следующим образом:

```
p1Percentage = cntP1 * 100.0 / n;
// TODO: Также добавьте формулы для остальных переменных
```

Чтобы было еще понятнее, что происходит, рассмотрим следующий пример:

Ввод	Вывод
3 1 2 999	66.67 % 0.00 % 0.00 % 0.00 % 33.33 %

К примеру, `n = 3`. Для цикла имеем:

- `i = 0` — считываем число 1, которое меньше 200 и попадает в первую группу (`p1`), увеличиваем счетчик группы (`cntP1`) на 1.
- `i = 1` — считываем число 2, которое опять попадает в первую группу (`p1`), увеличиваем ее счетчик (`cntP1`) опять на 1.
- `i = 2` — считываем число 999, которое находится в последней группе (`p5`), так как оно больше 800, и увеличиваем счетчик группы (`cntP5`) на 1.

После прочтения чисел в группе `p1` у нас есть два числа, а в `p5` — одно число. В остальных группах у нас нет чисел. Применяя приведенную выше формулу, вычисляем процентное соотношение в каждой группе. Если в формуле вместо 100.0 умножить на 100, то получится 66 % для группы `p1` и 33 % для группы `p5` (дробной части не будет).

Вывод конечного результата

Остается только вывести результаты на экран. В условии сказано, что проценты должны быть точны до двух цифр после запятой. Этого мы добьемся с помощью функций `fixed` и `setprecision`. Для использования `setprecision` необходимо подключить библиотеку `iomanip`:

```
cout << fixed << setprecision(2) << plPercentage << "%" << endl;
cout << fixed << setprecision(2) << p1Percentage << "%" << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1365#0.

Задача: умная Лили

Лили исполнилось `N` лет. На каждый день рождения она получала подарок. На каждый нечетный день рождения (1, 3, 5, ..., n) она получала игрушки, а на каждый четный день рождения (2, 4, 6, ..., n) — деньги. На свой второй день рождения она получила 10.00 USD, и эта сумма увеличивается на 10.00 USD на каждый следующий четный день рождения (2 -> 10, 4 -> 20, 6 -> 30 и т. д.). Все эти годы Лили тайно копила деньги. Брат Лили в те годы, когда она получала деньги, брал по 1.00 USD из каждой суммы. Лилли продала игрушки, полученные за эти годы, каждую за `P` USD, и добавила эту сумму к сумме накопленных денег. На вырученные деньги она хотела купить стиральную машину за `X` USD.

Напишите программу, которая вычислит, сколько денег она собрала и достаточно ли их для покупки стиральной машины.

Входные данные

Читаем с консоли три числа, каждое на отдельной строке:

- Возраст Лили — целое число в диапазоне [1 ... 77].
- Цена стиральной машины — число в диапазоне [1.00 ... 10 000.00].
- Цена за единицу каждой игрушки — целое число в диапазоне [0 ... 40].

Выходные данные

Выведите на консоль одну строку:

- Если денег Лили будет достаточно:
  - `"Да! {N}"` — где `N` — оставшиеся после покупки деньги.
- Если денег не хватает:
  - `"Нет! {M}"` — где `M` — недостающая сумма.
  - Числа `N` и `M` должны быть отформатированы до второй цифры после десятичной точки.

Примеры ввода и вывода

Ввод	Вывод	Пояснения
10 170.00 6	Да! 5.00	На первый день рождения она получает игрушку; 2-й -> 10 USD; 3-й -> игрушку; 4-й - > 10 + 10 = 20 USD; 5-й -> игрушка; 6-й -> 20 + 10 = 30 USD; 7-й -> игрушка; 8-й -> 30 + 10 = 40 USD; 9-й -> игрушка; 10-й -> 40 + 10 = 50 USD. Она сэкономила -> 10 + 20 + 30 + 40 + 50 = 150 долларов. Она продала 5 игрушек по 6 USD каждая = 30 USD. Ее брат взял 1 USD 5 раз = 5 USD. Оставшаяся сумма -> 150 + 30 - 5 = 175 USD. 175 >= 170 (цена стиральной машины): она успела ее купить и осталась с 175 - 170 = 5 USD.

Ввод	Вывод	Пояснения
21 1570.98 3	Нет! 997.98	Лили накопила 550 USD. Она продала 11 игрушек по 3 USD каждая = 33 USD. Ее брат взял за 10 лет по 1 USD в год = 10 USD. Оставшаяся сумма: 550 + 33 - 10 = 573 USD. 573 < 1570,98: ей не удалось купить стиральную машину. Недостающая сумма составляет 1570,98 - 573 = 997.98 доллара США.

Рекомендации и советы

Решение этой задачи, как и предыдущей, также можно разделить на три части — *считывание* входных данных, их *обработка* и *вывод* на экран.

И снова мы начинаем с выбора соответствующих типов данных и имен переменных. Годы Лили (`age`) и цена полученной игрушки (`presentPrice`) условно заданы как целые числа. Поэтому мы будем использовать тип `int`. Мы знаем, что цена стиральной машины (`priceOfWashingMachine`) является дробным числом, выбираем тип

double. В приведенном коде мы объявляем переменные и инициализируем их (присваиваем им значение):

```
int age;
double priceOfWashingMachine;
int presentPrice;
cin >> age;
```

Для решения задачи нам потребуется несколько вспомогательных переменных — для количества игрушек (numberOfToys), для сэкономленных денег (savedMoney) и для денег, получаемых в каждый день рождения (moneyForBirthday). Присвоим moneyForBirthday начальное значение 10, так как по условию первая сумма, которую получила Лили, равна 10 USD:

```
int numberOfToys = 0;
int savedMoney = 0;
int moneyForBirthday = 10;
```

С помощью цикла for мы перебираем все дни рождения Лили. Когда счетчик в цикле (соответствующий годам Лили) становится четным числом, это означает, что она получила деньги, и мы прибавляем эти деньги к ее общим сбережениям. Одновременно мы вычитаем 1 USD — деньги, которые забирает брат. Затем увеличиваем значение переменной moneyForBirthday, то есть увеличиваем на 10 ту сумму, которую она получит на свой следующий день рождения. И наоборот, если счетчик (годы Лили) — нечетное число, то мы увеличиваем количество игрушек. Четность проверяется делением на остаток (%), равный 2, — если остаток равен 0, то число четное, а если остаток равен 1, то число нечетное:

```
for (int currentYear = 1; currentYear <= age; currentYear++) {
    if (currentYear % 2 == 0) {
        savedMoney += (moneyForBirthday - 1);
        moneyForBirthday += 10;
    }
    else {
        numberOfToys++;
    }
}
```

К сбережениям Лили мы добавляем деньги от проданных игрушек:

```
savedMoney += numberOfToys * presentPrice;
```

Наконец, необходимо вывести результаты с соблюдением форматирования, указанного в условии, то есть сумма должна быть округлена до двух цифр после точки:

```
if (savedMoney >= priceOfWashingMachine) {
    cout << fixed << setprecision(2) << "Да! "
         << (savedMoney - priceOfWashingMachine) << endl;
}
else {
    cout << fixed << setprecision(2) << "Нет! "
         << (priceOfWashingMachine - savedMoney) << endl;
}
```

Мы проверяем, достаточно ли денег, собранных Лили, для покупки стиральной машины. Если их больше или их количество равно цене стиральной машины, то проверка `savedMoney >= priceOfWashingMachine` вернет `true` и выведет `"Yes! ..."`, а если меньше, то результат будет `false` и выведет `"Нет! ..."`.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1365#1.

Задача: вернуться в прошлое

Иванчо 18 лет, и он получает наследство, состоящее из суммы денег X и машины времени. Он решает переместиться в 1800 год, но не знает, хватит ли ему денег, чтобы жить не работая. Напишите программу, которая вычисляет, хватит ли Иванчо денег, чтобы не работать до определенного года (включительно). Предположим, что за каждый четный (1800, 1802 и т. д.) год он потратит 12 000 долларов. В каждый нечетный (1801, 1803 и т. д.) год он потратит 12 000 + 50 * (возраст, которого он достигнет в данном году).

Входные данные

Входные данные считываются с консоли и содержат ровно две строки:

- *Унаследованные деньги* — вещественное число в диапазоне [1.00 ... 1 000 000.00].
- *Год, до которого он должен прожить в прошлом (включительно)* — целое число в диапазоне [1801 ... 1900].

Выходные данные

На консоль выводится одна строка. Сумма должна быть отформатирована до двух символов после десятичной точки:

- Если *денег достаточно:*
  - `"Да! Он будет жить беззаботно, и у него останется {N} долларов"`, где `N` — количество денег, которые останутся.
- Если *денег недостаточно:*
  - `"Чтобы выжить, ему потребуется {M} долларов"`, где `M` — сумма, которой не хватает.

Примеры ввода и вывода

Ввод	Вывод	Пояснения
50000 1802	Да! Он будет жить беззаботно, и у него останется 13050.00 долларов.	1800 → четный → Тратит 12000 долларов → Осталось 50000 - 12000 = 38000 1801 → нечетный → Потрачено 12000 + 19*50 = 12950 долларов → Осталось 38000 - 12950 = 25050 1802 → четный → Тратит 12000 долларов → Остаток 25050 - 12000 = 13050

Ввод	Вывод	Пояснения
100000.15 1808	Чтобы выжить, ему потребуется 12399.85 доллара.	1800 → четный → Остается 100000.15 - 12000 = 88000.15 1801 → нечетный → Остается 88000.15 - 12950 = 75050.15 ... 1808 → нечет → -399,85 - 12000 = - 12399.85 12399.85 недостаточно

Рекомендации и советы

Метод решения этой задачи ничем не отличается от предыдущих, поэтому приступаем к объявлению и инициализации необходимых переменных.

В условии сказано, что возраст Иванчо — 18 лет, поэтому при объявлении переменной `years` мы устанавливаем ее начальное значение равным 18. Остальные переменные мы считываем с консоли:

```
double heritage;
int yearToLive;
int years = 18;
[illegible]
[illegible]
```

Используя цикл `for`, пройдем по всем годам. Начнем с 1800 года — года, в который вернулся Иванчо, и дойдем до года, до которого он должен жить в прошлом. В цикле мы проверяем, является ли текущий год четным или нечетным. Четность проверяется делением на остаток (%) от 2. Если год четный, то из наследства (`heritage`) вычитается 12 000, а если нечетный, то из наследства (`heritage`) вычитается 12 000 + 50 * (лет Иванчо). На каждой итерации цикла мы увеличиваем количество лет Иванчо.

```
for (int currentYear = 1800; currentYear <= yearToLive;
             currentYear++) {
    if (currentYear % 2 == 0) {
        heritage -= 12000;
    }
    else {
        heritage -= (12000 + 50 * years);
    }
    years++;
}
```

Наконец, нам осталось вывести результаты, для чего мы проверяем, достаточно ли его наследства (`heritage`) для того, чтобы жить, не работая, или нет. Если наследство — положительное число, то выводим `"Да! Он будет жить беззаботно, и у него останется {N} долларов."`, а если отрицательное — `"Чтобы выжить, ему потребуется {M} долларов."`. Не забудьте отформатировать сумму с точностью до двух знаков после точки.

Подсказка: при выводе в случае, если наследства не хватает, следует использовать функцию `abs(...)` из библиотеки `cmath`.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1365#2.

Задача: больница

В течение определенного периода времени каждый день в больницу поступают пациенты для обследования. Изначально в ней работает семь врачей. Каждый врач может лечить только одного пациента в день, но иногда врачей не хватает, поэтому оставшихся пациентов отправляют в другие больницы. Каждый третий день в больнице проводятся подсчеты, и если количество непролеченных пациентов превышает количество пролеченных, назначается еще один врач. Назначение происходит до начала ежедневного приема пациентов.

Напишите программу, которая вычисляет количество пролеченных и непролеченных пациентов за определенный период времени.

Входные данные

Входные данные считываются с консоли и содержат:

- В первой строке — *период*, для которого необходимо произвести вычисления. Это целое число в диапазоне [1 ... 1000].
- В следующих строках (равных количеству дней) — *количество пациентов*, поступивших на лечение *за текущий день*. Целое число в диапазоне [0 ... 10 000].

Выходные данные

Выведите на консоль две строки:

- В первой строке: `"Пролеченные пациенты: {количество пролеченных пациентов}"`.
- Во второй строке: `"Непролеченные пациенты: {количество непролеченных пациентов}"`.

Примеры ввода и вывода

Ввод	Вывод	Пояснения
4 7 27 9 1	Пролеченные пациенты: 23. Непролеченные пациенты: 21.	День 1: 7 пролеченных и 0 непролеченных пациентов за день День 2: 7 пролеченных и 20 непролеченных пациентов за день День 3: К этому моменту пролеченных пациентов уже 14, а непролеченных - 20 -> Назначен новый врач. -> 8 пролеченных и 1 непролеченный пациент за день День 4: 1 пролеченный и 0 непролеченных пациентов за день Итого: 23 пролеченных и 21 непролеченный пациент.

Ввод	Вывод
3 7 7 7	Пролеченные пациенты: 21. Непролеченные пациенты: 0.

Ввод	Вывод
6 25 25 25 25 2	Пролеченные пациенты: 40. Непролеченные пациенты: 87.

Рекомендации и советы

Начнем с того, что объявим и инициализируем необходимые переменные. Период, для которого нам необходимо произвести расчеты, мы считываем с консоли и сохраняем в переменной `period`. Также нам понадобятся несколько вспомогательных переменных: количество пролеченных пациентов (`treatedPatients`),

количество непролеченных пациентов (`untreatedPatients`) и количество врачей (`countOfDoctors`), которое изначально равно 7:

```
int period;
int treatedPatients = 0;
int untreatedPatients = 0;
int countOfDoctors = 7;
```

С помощью цикла `for` мы перебираем все дни за заданный период (`period`). Для каждого дня мы считываем из консоли количество пациентов (`currentPatients`). Увеличение количества врачей по условию можно производить *каждый третий день*, но только в том случае, если количество необследованных пациентов больше количества обследованных. Для этого проверяем, является ли день третьим — с помощью арифметического оператора деления на остаток (%): `day % 3 == 0`.

Например:

- Если день третий, то остаток от деления на 3 будет равен 0 (`3 % 3 = 0`), и проверка `day % 3 == 0` вернет `true`.
- Если день второй, то остаток от деления на 3 будет равен 2 (`2 % 3 = 2`), и проверка вернет `false`.
- Если день четвертый, то остаток от деления будет равен 1 (`4 % 3 = 1`), и проверка снова вернет `false`.

Если проверка `day % 3 == 0` возвращает `true`, то также будет проверено, превышает ли число непролеченных пациентов число пролеченных: `untreatedPatients > treatedPatients`. Если результат снова окажется `true`, то количество врачей (`countOfDoctors`) будет увеличено.

Затем мы проверяем, превышает ли количество пациентов за день (`currentPatients`) количество врачей (`countOfDoctors`). Если количество пациентов больше:

- Мы увеличиваем значение переменной `treatedPatients` на количество врачей (`countOfDoctors`).
- Мы увеличиваем значение переменной `untreatedPatients` на количество оставшихся пациентов, которое рассчитываем путем вычитания количества врачей из всех пациентов (`currentPatients - countOfDoctors`).

Если количество пациентов не больше, то мы только увеличиваем переменную `treatedPatients` с количеством пациентов за день (`currentPatients`):

```
for (int day = 1; day <= period; day++) {
    int currentPatients;
    cin >> currentPatients;

    if ((day % 3 == 0) && (untreatedPatients >
            treatedPatients)) {
        countOfDoctors++;
    }

    if (currentPatients > countOfDoctors) {
        treatedPatients += countOfDoctors;
        untreatedPatients += currentPatients - countOfDoctors;
    }
    else {
        treatedPatients += currentPatients;
    }
}
```

Наконец, нам нужно вывести только количество пролеченных и количество непролеченных пациентов.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1365#3.

Задача: деление без остатка

У нас есть `n` целых чисел в диапазоне [1 ... 1000]. Среди них определенный процент чисел `p1`, которые *делятся без остатка на 2*, процент чисел `p2`, которые *делятся без остатка на 3*, процент чисел `p3`, которые *делятся без остатка на 4*. Напишите программу, которая вычисляет и выводит процентное соотношение `p1`, `p2` и `p3`.

Пример: у нас есть `n` = 10 чисел: 680, 2, 600, 200, 800, 799, 199, 46, 128, 65.

Получаем распределение и визуализацию, приведенные в таблице на следующей странице:

Деление без остатка на	Числа	Кол-во	Процент
2	680, 2, 600, 200, 800, 46, 128	7	p1 = (7 / 10) * 100 = 70.00 %
3	600	1	p2 = (1 / 10) * 100 = 10.00 %
4	680, 600, 200, 800, 128	5	p3 = (5 / 10) * 100 = 50.00 %

Входные данные

В первой строке ввода находится целое число n ($1 \le n \le 1000$) — количество чисел. В каждой из следующих n строк находится по одному целому числу из диапазона [1 ... 1000] — числа, которые необходимо проверить на делимость.

Выходные данные

Выведите на консоль *три строки*, каждая из которых содержит процент от 0 % до 100 %, две цифры после точки, например 25.00 %, 66.67 %, 57.14 %.

- В *первой строке* — процент чисел, которые *делятся на 2*.
- Во *второй строке* — процент чисел, которые *делятся на 3*.
- В *третьей строке* — процент чисел, которые *делятся на 4*.

Примеры ввода и вывода

Ввод	Вывод
3 3 6 9	33.33 % 100.00 % 0.00 %

Ввод	Вывод
1 12	100.00 % 100.00 % 100.00 %

Ввод	Вывод
10 680 2 600 200 800 799 199 46 128 65	 70.00 % 10.00 % 50.00 %

Рекомендации и советы

Для этого и следующего задания необходимо самостоятельно написать программный код, следуя приведенным инструкциям.

Программа, решающая данную задачу, аналогична программе из задачи «Гистограмма», рассмотренной выше. Поэтому мы можем начать с объявления необходимых нам переменных. Примерные имена переменных могут быть следующими: `n` — количество чисел (которое мы должны считать с консоли) и `divisibleBy2`, `divisibleBy3`, `divisibleBy4` — вспомогательные переменные, хранящие количество чисел в соответствующей группе.

Для того чтобы прочитать и распределить каждое число в соответствующую группу, необходимо выполнить цикл от `0` до `n` (количество чисел). На каждой итерации цикла должно быть прочитано и распределено одно число. Отличие заключается в том, что одно число может попадать в несколько групп одновременно, поэтому для каждого числа необходимо выполнить три отдельные проверки `if` — делится ли оно на 2, 3 и 4 соответственно и увеличивать значение переменной, содержащей количество чисел в соответствующей группе. Конструкция `if-else` в данном случае не поможет, так как, обнаружив совпадение, она прерывает дальнейшую проверку условий).

Наконец, необходимо вывести результаты на экран, соблюдая формат, указанный в условии.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1365#4.

Задача: логистика

Вы отвечаете за логистику различных типов грузов. В зависимости от веса каждого груза вам потребуются разные автомобили, и перевозка будет иметь разную цену за тонну:

- До 3 тонн — микроавтобус (200 USD за тонну).
- От 3 до 11 тонн — грузовик (175 USD за тонну).
- Свыше 11 тонн — поезд (120 USD за тонну).

Ваша задача — рассчитать среднюю цену за тонну груза, а также то, какой процент груза перевозится на каждом виде транспорта.

Входные данные

Из консоли мы должны прочитать последовательность чисел, каждое в отдельной строке:

- Первая строка: количество грузов для перевозки — целое число в диапазоне [1 ... 1000].
- В следующих строках мы передаем тоннаж текущего груза — целое число в диапазоне [1 ... 1000].

Выходные данные

На консоль выводятся ровно четыре строки следующего содержания:

- Строка № 1 — средняя цена за тонну груза (округляется до второй цифры после точки).
- Строка № 2 — процент грузов, перевозимых микроавтобусом (от 0,00 % до 100,00 %, округление до второй цифры после точки).
- Строка № 3 — процент грузов, перевозимых грузовиком (от 0,00 % до 100,00 %).
- Строка № 4 — процент грузов, перевозимых по железной дороге (от 0,00 % до 100,00 %).

Примеры ввода и вывода

Ввод	Вывод	Пояснения
4 1 5 16 3	143.80 16.00 % 20.00 % 64.00 %	На микроавтобусе вы перевозите два груза 1 + 3, всего 4 тонны. На грузовике вы перевозите один груз: 5 тонн. Поездом вы перевозите один груз: 16 тонн. Сумма всех грузов: 1 + 5 + 16 + 3 = 25 тонн. Процент груза на микроавтобусе: 4/25*100 = 16.00 % Процент груза на грузовике: 5/25*100 = 20.00 % Процент груза на поезде: 16/25*100 = 64.00 % Средняя цена за тонну перевозимого груза: (4 * 200 + 5 * 175 + 16 * 120) / 25 = 143.80

Ввод	Вывод
5 2 10 20 1 7	149.38 7.50 % 42.50 % 50.00 %

Ввод	Вывод
4 53 7 56 999	143.80 16.00 % 20.00 % 64.00 %

Рекомендации и советы

Сначала мы считываем вес каждого груза и суммируем, сколько тонн перевозится микроавтобусом, грузовиком и поездом, а также подсчитываем общее количество тонн перевезенного груза. Мы рассчитаем цены на каждый вид транспорта в зависимости от количества перевезенных тонн и общей цены. Наконец, мы рас-

считаем и выведем общую среднюю цену за тонну и то, сколько грузов перевозится различными видами транспорта, в процентах.

Объявляем необходимые переменные, например следующие: `countOfLoads` — количество перевозимых грузов (считываем из консоли), `sumOfTons` — сумма тоннажа всех грузов, `microbusTons`, `truckTons`, `trainTons` — переменные, содержащие сумму тоннажа грузов, перевозимых соответственно микроавтобусом, грузовиком и поездом.

Для перебора всех грузов нам понадобится цикл `for` от `0` до `countOfLoads`. Для каждого груза мы считываем его вес (в тоннах) из консоли и сохраняем его в переменной, например `tons`.

Прибавляем вес текущего груза (`tons`) к сумме весов всех грузов (`sumOfTons`). После того как мы считали вес текущего груза, нам необходимо определить, на каком транспортном средстве он будет перевозиться (микроавтобус, грузовик или поезд). Для этого нам понадобятся проверки типа `if-else`:

- Если значение переменной `tons` меньше 3, то мы увеличиваем значение переменной `microbusTons` на величину `tons`:

  ```
  microbusTons += tons;
  ```

- В противном случае, если значение `tons` доходит до 11, мы увеличиваем `truckTons` на `tons`.
- Если количество `tons` больше 11, то увеличиваем `trainTons` на `tons`.

Перед выводом на экран нам необходимо рассчитать процентное соотношение тонн, перевезенных каждым транспортным средством, и среднюю стоимость одной тонны. Для расчета средней стоимости тонны мы объявим еще одну вспомогательную переменную `totalPrice`, в которой просуммируем общую стоимость всех перевезенных грузов (микроавтобусом, грузовиком и поездом). Среднюю цену мы получим, разделив `totalPrice` на `sumOfTons`. Осталось вычислить процентное соотношение тонн, перевезенных каждым транспортным средством, и вывести результаты, соблюдая формат, указанный в условии.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1365#5.

Глава 6.1.
Вложенные циклы

В этой главе мы рассмотрим вложенные циклы и использование циклов `for` для рисования различных консольных фигур, состоящих из символов и знаков, расположенных на консоли в строках и столбцах. Мы будем использовать одиночные и вложенные циклы (один цикл внутри другого), вычисления и проверки для построения различных фигур на консоли в соответствии с заданными размерами.

Пример: прямоугольник размером 10 x 10 звездочек

Нарисуйте в консоли прямоугольник размером 10 × 10 звездочек.

Ввод	Вывод
(нет)	********** ********** ********** ********** ********** ********** ********** ********** ********** **********

Рекомендации и советы

```
for (int i = 0; i < 10; i++) {
    cout << string(10, '*') << endl;
}
```

Как работает данный пример? Создаем переменную типа `string`. Инициализируем цикл с переменной `i = 0`, которая уве-

личивается на каждой итерации цикла до тех пор, пока ее значение меньше 10.

Таким образом, код в теле цикла выполняется 10 раз. К строке в теле цикла добавляется звездочка. Это означает, что в конце цикла строка будет содержать 10 звездочек. Инициализируется второй цикл, который также выполняется 10 раз. В теле второго цикла созданная ранее строка из 10 звездочек выводится на новую строку в консоль.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#0.

Пример: прямоугольник из N x N звездочек

Напишите программу, которая принимает целое положительное число `n` и выводит на консоль прямоугольник из `N x N` звездочек.

Примеры ввода и вывода

Ввод	Вывод
2	** **

Ввод	Вывод
3	*** *** ***

Ввод	Вывод
4	**** **** **** ****

Рекомендации и советы

Задача аналогична предыдущей, поэтому решение практически такое же:

```
int n;
cin >> n;

for (int i = 0; i < n; i++) {
    cout << string(n, '*') << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#1.

Вложенные циклы

Вложенный цикл — это конструкция, в которой *в теле одного цикла* (внешнего) *находится другой цикл* (внутренний). В каждой итерации внешнего цикла выполняется весь внутренний цикл. Это происходит следующим образом:

- Когда вложенные циклы начинают выполняться, первым запускается внешний цикл: инициализируется управляющая переменная и после проверки на завершение цикла выполняется код в его теле.
- После этого выполняется внутренний цикл. Инициализируется начальная позиция управляющих переменных, производится проверка на завершение цикла и выполняется код в его теле.
- При достижении заданного значения для завершения цикла программа возвращается на один шаг вверх и продолжает выполнение предыдущего (внешнего) цикла. Управляющая переменная внешнего цикла изменяется на один шаг, происходит проверка выполнения условия завершения цикла и начинается новое выполнение вложенного (внутреннего) цикла.
- Это повторяется до тех пор, пока переменная внешнего цикла не выполнит условие завершения цикла.

Приведем пример, иллюстрирующий использование вложенных циклов. Задача состоит в том, чтобы вывести прямоугольник из `N x N` звездочек, при этом для каждой строки выполняется цикл от `1` до `N`, а для каждого столбца — вложенный цикл от `1` до `N`:

```
int n;
cin >> n;

for (int r = 0; r < n; r++) {
    for (int c = 0; c < n; c++) {
        cout << "*";
    }
    cout << endl;
}
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/NestedLoopExample.

Рассмотрим приведенный пример. После инициализации первого (внешнего) цикла начинается выполнение его тела, которое содержит второй (вложенный) цикл. Сам он выводит в одной строке количество звездочек `n`. После завершения выполнения внутреннего цикла на первой итерации внешнего цикла внешний цикл продолжит работу, то есть выведет на консоль одну пустую строку. Затем обновится первая переменная цикла и снова будет выполнен весь второй (вложенный) цикл. Внутренний цикл будет выполнен столько раз, сколько раз будет выполнено тело внешнего цикла, в данном случае `n` раз.

Пример: квадрат из звездочек

Выведите на консоль квадрат из `N x N` звездочек:

Ввод	Вывод
2	* * * *

Ввод	Вывод
3	* * * * * * * * *

Ввод	Вывод
4	* * * * * * * * * * * * * * * *

Рекомендации и советы

Задача аналогична предыдущей. Отличие состоит в том, что необходимо продумать, как вывести пробел после звездочек так, чтобы не было лишних пробелов ни в начале, ни в конце:

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#2.

Пример: треугольник из символов доллара

Напишите программу, которая принимает целое число n и выводит треугольник, составленный из символов доллара, размером n.

Примеры ввода и вывода

Ввод	Вывод
2	$ $ $

Ввод	Вывод
3	$ $ $ $ $ $

Ввод	Вывод
4	$ $ $ $ $ $ $ $ $ $

Рекомендации и советы

Задача аналогична рисованию прямоугольника и квадрата. Мы снова будем использовать вложенные циклы, но здесь есть одна загвоздка. Разница в том, что количество столбцов, которые нам нужно вывести, зависит от строки, в которой мы находимся, а не от входного числа n.

Из примера ввода и вывода видно, что количество символов доллара зависит от того, в какой строке мы находимся в момент вывода, то есть 1 доллар означает первую строку, 3 доллара — третью и т. д. Рассмотрим приведенный ниже пример более подробно. Мы видим, что переменная вложенного цикла связана с переменной внешнего цикла:

```
int n;
cin >> n;

for (int r = 0; r < n; r++) {
    cout << "$";
    for (int c = 0; c < r; c++) {
        cout << " $";
    }
    cout << endl;
}
```

Таким образом, наша программа выводит нужный треугольник:

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#3.

Пример: квадратная рамка

Напишите программу, которая принимает целое число `n` и рисует на консоли квадратную рамку размером `n * n`.

Ввод	Вывод
4	+ - - + | - - | | - - | + - - +

Ввод	Вывод
5	+ - - - + | - - - | | - - - | | - - - | + - - - +

Ввод	Вывод
6	+ - - - - + | - - - - | | - - - - | | - - - - | | - - - - | + - - - - +

Рекомендации и советы

Мы можем решить эту задачу следующим образом:

- Считываем с консоли число `n`.
- Выводим верхнюю часть: сначала знак `+`, затем `n-2` раза `-` и в конце знак `+`.
- Средняя часть: выводим `n-2` строки, так как сначала выводим знак `|`, затем `n-2` раза `-` и в конце снова знак `|`. Мы можем сделать это с помощью вложенных циклов.
- Выводим нижнюю часть: сначала знак `+`, затем `n-2` раза `-` и в конце знак `+`.

Вот пример реализации описанной выше идеи с использованием вложенных циклов:

```
int n;
cin >> n;

// Вывод верхнего ряда: + - - - +
cout << "+";
for (int i = 0; i < n - 2; i++) {
    cout << " -";
}
cout << " + " << endl;

// Вывод средних рядов: | - - - |
for (int r = 0; r < n - 2; r++) {
    cout << "|";
    for (int c = 0; c < n - 2; c++) {
        cout << " -";
    }
    cout << " |" << endl;
}

// Вывод нижнего ряда: + - - - +
cout << "+";
for (int i = 0; i < n - 2; i++) {
    cout << " -";
}
cout << " +" << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#4.

Задача: ромб из звездочек

Напишите программу, которая принимает целое положительное число `n` и выводит *ромб из звездочек* размером `n`.

Ввод	Вывод
1	*

Ввод	Вывод
2	* * * *

Ввод	Вывод
3	* * * * * * * * *

Ввод	Вывод
4	* * * * * * * * * * * * * * * *

Рекомендации и советы

Чтобы решить эту задачу, нужно мысленно разделить ромб на две части — верхнюю, которая также включает в себя средний ряд, и нижнюю. Для вывода каждой части мы будем использовать два отдельных цикла, а зависимость между `n` и переменными циклов оставим на усмотрение читателя. Для первого цикла мы можем использовать следующие рекомендации:

- Выводим `n - r - 1` пробелов.
- Выводим *.
- Выводим `r` раз *.

Вторую (нижнюю) часть выводим аналогичным образом, оставляя реализацию для самостоятельной работы читателя.

```
int n;
cin >> n;

for (int r = 0; r++) {
    for (int c = 0; c++) {
        cout << " ";
    }

    cout << "*";

    for (int c = 0; c++) {
        cout << " *";
    }

    cout << endl;
}
// TODO: вывести нижнюю часть ромба
```

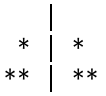
Проверка в судейской системе

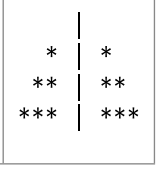
Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#5.

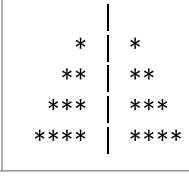
Задача: новогодняя елка

Напишите программу, которая принимает число `n` ($1 \leq n \leq 100$) и выводит на экран новогоднюю елку высотой `n+1`.

Ввод	Вывод
1	

Ввод	Вывод
2	

Ввод	Вывод
3	

Ввод	Вывод
4	

Рекомендации и советы

Из примеров видно, что новогоднюю елку можно разделить на три логические части. Первая часть — это звездочки и пробелы до и после них, средняя — |, и последняя — снова звездочки, но на этот раз пробелы только перед ними. Для вывода на экран достаточно одного цикла и операции умножения строк, которую мы будем использовать один раз для звездочек и один раз для пробелов:

```
for (int r = 0; r <= n; r++) {
    string stars, spaces;
    for (int i = 0; i < r; i++) {
        stars += '*';
    }

    for (int i = 0; i < n - r; i++) {
        spaces += ' ';
    }

    cout << spaces << stars;
```

```
    cout << " | ";
    cout << stars << spaces << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#6.

Рисование более сложных фигур

Рассмотрим, как можно рисовать фигуры на консоли с помощью более сложной логики построения, которую необходимо детально продумать перед началом написания.

Задача: солнечные очки

Напишите программу, которая принимает целое число `n` ($3 \leq n \leq 100$) и выводит на экран солнечные очки размером `5*n × n`, как показано в примерах:

Ввод	Вывод
3	``` ****** ****** *////*|||*////* ****** ******```

Ввод	Вывод
4	``` ******** ******** *//////*||||*//////* *//////* *//////* ******** ********```

Ввод	Вывод
5	``` ********** ********** *////////* *////////* *////////*|||||*////////* *////////* *////////* ********** **********```

Рекомендации и советы

Из примеров видно, что солнечные очки можно разделить на три части — верхнюю, среднюю и нижнюю. Часть кода, с помощью которого можно решить задачу, приведена ниже.

Вывод верхней и нижней строк

При рисовании верхней и нижней строк нам нужно вывести на экран `2 * n` звезд, `n` пробелов и `2 * n` звезд.

```
// Вывод верхней части
for (int i = 0; i < 2 * n; i++) {
    cout << "*";
}

for (int i = 0; i < n; i++) {
    cout << " ";
}

for (int i = 0; i < 2 * n; i++) {
    cout << "*";
}
cout << endl;

// TODO:  вывод средней части

// Вывод нижней части
for (int i = 0; i < 2 * n; i++) {
    cout << "*";
}

for (int i = 0; i < n; i++) {
    cout << " ";
}

for (int i = 0; i < 2 * n; i++) {
    cout << "*";
}
cout << endl;
```

При выводе средней части необходимо проверить, что порядок равен `(n - 1) / 2 - 1`, так как из примеров очевидно, что в этой строке вместо пробелов следует выводить вертикальные черточки:

```
// Вывод средней части
for (int i = 0; i < n - 2; i++) {
    // TODO: Вывести *//////*

    for (int j = 0; j < n; j++) {
        cout << ((i == (n - 1) / 2 - 1) ? '|' : ' ');
    }

    // TODO: Вывести *//////*
    cout << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#7.

Пример: дом

Напишите программу, которая принимает число n ($2 \le n \le 100$) и выводит на экран дом размером n x n, точно так же, как в примерах:

Ввод	Вывод
2	** ||

Ввод	Вывод
3	-*- *** |*|

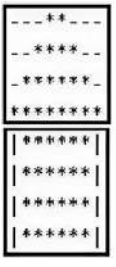
Ввод	Вывод
4	-**- **** |**| |**|

Ввод	Вывод
5	--*-- -***- ***** |***| |***|

Рекомендации и советы

Из условия задачи мы понимаем, что дом имеет размер n x n. Из входных и выходных данных примера мы видим, что:

- Дом состоит из двух частей: *крыши* и *основания*.
- Если n — четное число, то крыша дома не острая.
- Когда n нечетное, *крыша* у дома острая и на один ряд больше *основания*.



Крыша

- Состоит из звездочек и черточек.
- В верхней части находятся одна или две звездочки, в зависимости от того, четное или нечетное число n (также связано с черточками).

- В нижней части много звездочек, а черточек мало или нет.
- С каждым нижним рядом звездочки увеличиваются на 2, а черточки уменьшаются на 2.

Основание

- Состоит из `n` рядов.
- Сформировано из звездочек и вертикальных линий.
- Строки состоят из двух вертикальных линий, по одной в начале и в конце строки, и звездочек между линиями с длиной строки `n - 2`.

Мы считываем с консоли вводимое число `n` и записываем его значение в переменную:

```
int n;
cin >> n;
```

Очень важно проверить, являются ли входные данные корректными! В этих задачах не представляет проблемы прямое преобразование входных данных, считанных с консоли, к типу `int`, поскольку явно сказано, что мы получим корректные целые числа. Однако если речь идет о более серьезных приложениях, то проверка данных не помешает. Что произойдет, если вместо числа пользователь введет, например, букву «А»?

Для построения *крыши* определяем, какое изначальное количество звездочек будет в переменной `stars`:

- Если `n` — нечетное число, то оно будет равно 1.
- Если `n` четное, то будет 2 штуки.

```
int stars = 1;
if (n % 2 == 0) {
    stars++;
}
```

Вычисляем длину *крыши*. Она равна половине `n`. Записываем результат в переменную `roofLength`:

```
int roofLength = ceil(n / 2.0);
```

Примечание: Для применения функции `ceil()`, которая округляет до большего целого числа, независимо от дробной части, необходимо использовать библиотеку `math.h`.

Важно отметить, что если `n` — нечетное число, то длина крыши на порядок больше длины основания. В языке C++ при делении двух целых чисел с остатком результатом будет число без остатка.

Пример:

```
int result = 3 / 2; // Результат = 1
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/DivisionExample. Если мы хотим округлить значение в большую сторону, то необходимо использовать функцию `ceil(...)`:

```
#include <iostream>
#include <math.h>

using namespace std;

int main() {
    int result1 = ceil(3 / 2.0); // результат = 2
    int result2 = ceil(7 / 2.7); // результат = 3
{
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/ExmapleWithCeil.

В данном примере деление производится не на 2 "целочисленное". Когда целое число записывается с суффиксом `'.0'`, это означает, что оно имеет тип с плавающей точкой. Результат деления `3 / 2.0` равен `1.5`, а функция `ceil(...)` округляет результат деления в большую сторону. В нашем случае `1.5` превратится в `2.0`, которое преобразуется в целое число `2`.

После того как мы рассчитали длину крыши, мы выполняем цикл от `0` до `roofLength`. На каждой итерации вы:

- Вычисляете количество черточек, которые необходимо нарисовать. Это число будет равно `(n - stars) / 2`. Запишем его в переменной `padding`:

  ```
  int padding = (n - stars) / 2;
  ```

- Выводим на консоль: черточки (`padding` раз) + звездочки (`stars` times) + черточки (`padding` times):

  ```
  for (int i = 0; i < padding; i++) {
  ```

```
            cout << " ";
        }

        for (int i = 0; i < stars; i++) {
            cout << "*";
        }

        for (int i = 0; i < padding; i++) {
            cout << "-";
        }
```

- Перед завершением цикла увеличьте количество звездочек на 2:

```
stars += 2;
```

После того как мы закончили с крышей, настало время для основания. Его вывести проще:

- Начнем с цикла от 0 до `n` (не включая).
- Выведите на консоль: `|` + `*` (`n - 2` раза) + `|`.

```
for (int i = 0; i < n / 2; i++) {
    cout << "|";
    for (int j = 0; j < n - 2; j++) {
        cout << "*";
    }
    cout << "|" << endl;
}
```

Если мы все написали правильно, то наша задача решена.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1366#8.

Задача: бриллиант

Программа должна принимать целое число `n` ($1 \leq n \leq 100$) и выводить ромб размером `n`, как показано в примерах ниже.

Ввод	Вывод
1	*

Ввод	Вывод
2	**

Ввод	Вывод
3	-*- *-* -*-

Ввод	Вывод
4	-**- *--* -**-

Ввод	Вывод
5	--*-- -*-*- *---* -*-*- --*--

Рекомендации и советы

Из объяснения задачи мы знаем, что бриллиант имеет размер `n x n`. Из примера ввода и вывода можно сделать заключение, что все строки содержат ровно `n` символов, и все строки, за исключением верхней и нижней, имеют по две звездочки. Мы можем мысленно разделить бриллиант на две части:

- Верхняя часть. Начинается от верхней точки до середины.
- Нижняя часть. Начинается с ряда ниже среднего и спускается до нижней точки (включительно).

Верхняя часть

- Если `n` — нечетное число, построение бриллианта начинается с одной звездочки.
- Если `n` — четное число, построение начинается с двух звездочек.
- С каждым рядом вниз звездочки становятся все дальше друг от друга.
- Пространство между звездочками, до и после них заполняется черточками.

Нижняя часть

- С каждым рядом звездочки становятся ближе друг к другу. Это означает, что пространство (черточки) между ними становится меньше, а пространство (черточки) слева и справа — больше.
- Самая нижняя часть имеет одну или две звездочки, в зависимости от того, является ли `n` четным или нечетным числом.

Верхняя и нижняя части бриллианта

- В каждом ряду, кроме среднего, звездочки окружены внутренними и внешними черточками.
- В каждом ряду между двумя звездочками есть пространство, кроме первого и последнего ряда (в этом случае количество звездочек равно 1).

Чтение входных данных

Считываем n из консоли и сохраняем в переменной типа int:

```
int n;
cin >> n;
```

Начинаем выводить вершину ромба на консоль. Первое, что нам нужно сделать, это вычислить начальное значение внешнего числа черточек leftRight (черточки снаружи от звездочек). Оно равно (n - 1) / 2, с округлением "вниз":

```
int leftRight = (n - 1) / 2;
```

Вычислив leftRight, мы начинаем рисовать вершину ромба. Можем начать с цикла от 0 до (n + 1) // 2 (то есть округлить в меньшую сторону).

На каждой итерации цикла должны быть выполнены следующие действия:

- Рисуем на консоли левые черточки (длины leftRight) и сразу за ними первую звездочку:

  ```
  for (int j = 0; j < leftRight; j++) {
      cout << "-";
  }
  cout << "*";
  ```

- Вычисляем расстояние между двумя звездочками. Это можно сделать, вычтя из n длину внешних черточек и число 2 (количество звездочек, то есть контур ромба). Результат этой разности запишем в переменную mid:

  ```
  int mid = n - 2 * leftRight - 2;
  ```

- Если значение mid меньше 0, то мы знаем, что на этой строке должна быть одна звездочка. Если значение mid больше или равно 0, то мы должны нарисовать черточки длиной mid и одну звездочку после них.

- Мы выводим на консоль правые внешние черточки, длина которых также равна leftRight:

  ```
  for (int j = 0; j < leftRight; j++) {
      cout << "-";
  }
  ```

- В конце цикла уменьшаем leftRight на 1 (звездочки отодвигаются).

Отрисовка нижней части полностью аналогична отрисовке верхней части. Отличие состоит в том, что вместо того, чтобы уменьшать значение параметра `leftRight` на 1 в конце цикла, мы будем увеличивать его на 1 в начале цикла. Цикл будет от 0 до `(n - 1) // 2`.

Повторение кода считается плохой практикой, поскольку код становится довольно сложным для сопровождения. Представим, что у нас есть часть кода (например, логика построения линии из ромба) еще в нескольких местах, и мы решили внести изменения. Для этого придется пройтись по всем местам и внести изменения. Теперь представим, что код нужно использовать не 1, 2 или 3 раза, а десятки раз. Решить подобную задачу можно с помощью *функций*, что рассматривается далее.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1366#9.

Что мы узнали из этой главы?

Мы познакомились с одним из способов создания строк:

```
string printMe = string(10, '*');
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/PrintMe.

Мы научились рисовать фигуры с помощью вложенных циклов `for`:

```
for (int r = 0; r < 5; r++) {
    cout << "*";

    for (int c = 0; c < 5 - 1; c++) {
        cout << " *";
    }
    cout << endl;
}
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ForLoopFigure.

Глава 6.2.
Вложенные циклы — экзаменационные задачи

В предыдущей главе мы рассмотрели вложенные циклы и их использование для рисования различных фигур на консоли. Мы научились строить фигуры разных размеров, придумывая соответствующую логику построения с использованием одиночных и вложенных циклов `for` в сочетании с различными вычислениями и логикой программирования:

```
for (int r = 1; r <= 5; r++) {
    cout << '*';

    for (int c = 1; c < 5; c++) {
        cout << " *";
    }
    cout << endl;
}
```

Этот пример можно протестировать в Интернете: https://repl.it/@vncpetrov/nestedLoops.

Мы также познакомились с конструктором строк, который позволяет выводить заданный символ определенное количество раз:

```
string printMe(5, '*');
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/stringCtor.

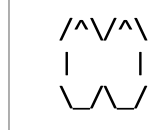
Экзаменационные задачи

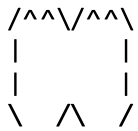
Теперь давайте вместе решим несколько экзаменационных задач, чтобы закрепить пройденный материал и развить алгоритмическое мышление.

Задача: нарисовать крепость

Напишите программу, которая принимает целое число n и рисует крепость шириной `2 * n` столбцов и высотой n строк, как показано в примерах ниже. Левый и правый столбцы внутри имеют ширину `n / 2`.

Примеры ввода и вывода

Ввод	Вывод
3	

Ввод	Вывод
4	

Ввод	Вывод
5	

Входные данные

Вход программы состоит из одного элемента (аргумента) — целого числа `n` в диапазоне [3 ... 1000].

Выходные данные

Выведите на консоль `n` строк текста, изображающих крепость точно так же, как в примерах.

Рекомендации и советы

Из объяснения видно, что входные данные будут состоять всего из одной строки, которая будет содержать целое число в диапазоне [3 ... 1000]. Поэтому мы будем использовать переменную типа `int`:

```
int n;
cin >> n;
```

После того как мы объявили и инициализировали входные данные, разделим крепость на три части:

- крыша
- корпус
- основание

Расчет и вывод на экран крыши

Из примеров видно, что крыша состоит из двух башен и средней части. Каждая башня состоит из начальной /, средней ^ и конечной \ частей.

\ — это специальный символ в языке C++, и если использовать его с `cout <<`, консоль его не выведет, поэтому мы показываем с помощью `\\`, что хотим вывести именно этот символ, не интерпретируя его как специальный (это называется «экранирование символов»).

Размер средней части равен `n / 2`, поэтому мы можем выделить эту величину в отдельную переменную. В ней будет храниться размер средней части башни:

```
int colSize = n / 2;
```

Мы также объявляем вторую переменную, в которой будем хранить значение части между двумя башнями. По условию мы знаем, что общая ширина крепости равна `n * 2`. Кроме того, у нас есть две башни с одной косой чертой в начале и в конце (всего 4 символа) и шириной `colSize`. Поэтому, чтобы получить количество символов между ними, нужно из ширины всей крепости вычесть размеры башен: `2 * n - 2 * colSize - 4`:

```
int midSize = [illegible];
```

Для вывода крыши на консоль мы будем использовать два цикла `for`, которые будут несколько раз выводить на экран символы ^ и _, соответственно `colSize` и `midSize`:

```
for (int i = 0; i < colSize; i++) {
    cout << '^';
}
cout << '\\';

for (int i = 0; i < midsize; i++) {
    cout << '_';
}
cout << '/';

for (int i = 0; i < colSize; i++) {
```

```
        cout << '^';
    }
    cout << '\\' << endl;
```

Напомним читателю, что \ — это специальный символ в языке C++, который мы экранируем при помощи \\.

Расчет и вывод на экран корпуса

Корпус крепости состоит из начала |, середины (пустого пространства) и конца |. Середина пустого пространства имеет размер `2 * n - 2`. Количество рядов для стен мы можем определить из приведенных примеров — `n - 3`:

```
for (int row = 1; row <= n - 3; row++) {
    cout << '|';

    for (int i = 0; i < [illegible]; i++) {
        cout << ' ';
    }
    cout << '|' << endl;
}
```

Чтобы нарисовать предпоследнюю линию, которая является частью корпуса, нам нужно вывести начальную |, среднюю (`пробел`)_(`пробел`) и конечную |. Для этого мы можем повторно использовать уже объявленные переменные `colSize` и `midSize`, так как из примеров видно, что они равны количеству _ в крыше.

```
cout << '|';
for (int i = 0; i < colSize + 1; i++) {
    cout << ' ';
}

for (int i = 0; i < [illegible]; i++) {
    cout << '_';
}

for (int i = 0; i < colSize + 1; i++) {
    cout << ' ';
}
cout << '|' << endl;
```

Мы прибавляем `+ 1` к значению пробелов, так как в примерах есть еще один пробел.

Расчет и вывод на экран корпуса

Структура основания крепости такая же, как и крыши. Оно состоит из двух башен и промежуточной части. Каждая башня имеет начало \, середину _ и конец /:

```
cout << '\\';

for (int i = 0; i < [illegible]; i++) {
    cout << '_';
}
cout << '/';

for (int i = 0; i < [illegible]; i++) {
    cout << ' ';
}
cout << '\\';

for (int i = 0; i < [illegible]; i++) {
    cout << '_';
}
cout << '/' << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1367#0.

Задача: бабочка

Напишите программу, которая принимает целое число `n` с консоли и рисует бабочку шириной `2 * n - 1` столбцов и высотой `2 * (n - 2) + 1` строк, как в примерах ниже. Левая и правая части имеют ширину `n - 1`.

Входные данные

На вход подается целое число `n` в диапазоне [3 ... 1000].

Выходные данные

Выведите на консоль `2 * (n - 2) + 1` текстовых строк, изображающих бабочку, как в примерах на следующей странице.

Примеры ввода и вывода

Ввод	Вывод
3	*\/* @ */*

Ввод	Вывод
5	***\/*** ---\ /--- ***\/*** @ ***/*** ---/ \--- ***/***

Ввод	Вывод
7	*****\ /***** -----\ /----- *****\ /***** -----\ /----- *****\ /***** @ *****/ ***** -----/ \----- *****/ ***** -----/ \----- *****/ *****

Рекомендации и советы

Из пояснения видно, что входные данные будут взяты только из одной строки, содержащей целое число в диапазоне [3 ... 1000]. Поэтому мы будем использовать, как и в предыдущих заданиях, переменную типа `int`:

```
int n;
cin >> n;
```

Мы можем разделить фигуру на три части — верхнее крыло, тело и нижнее крыло. Для того чтобы нарисовать верхнее крыло бабочки, необходимо разделить его на части — начало *, середина

\ / и конец *. Рассмотрев примеры, можно сказать, что начало координат имеет размер `n - 2`:

```
int halfRowSize = n - 2;
```

Мы также видим, что верхнее крыло бабочки имеет размер `n - 2`, поэтому мы можем сделать цикл, который повторяется половину размера строки (`halfRowSize`):

```
for (int i = 1; i <= halfRowSize; i++) {
}
```

Из примеров также видно, что в четной строке мы имеем начало *, середину \ / и конец *, а в нечетной — начало -, середину \ / и конец -. Поэтому на каждой итерации цикла необходимо выполнять проверку `if-else`, чтобы определить, является ли выводимая строка четной или нечетной. Из примеров, приведенных в условии, видно, что количество звездочек и тире в каждой строке также равно `n - 2`, то есть мы можем использовать переменную `halfRowSize` для их повторного вывода:

```
for (int i = 1; i <= halfRowSize; i++) {
    if (i % 2 != 0) {
        for (int j = 1; j <= halfRowSize; j++) {
            cout << '*';
        }
        cout << "\\ /";

        for (int j = 1; j <= halfRowSize; j++) {
            cout << '*';
        }
        cout << endl;
    }
    else {
        for (int j = 0; j <= halfRowSize; j++) {
            cout << '-';
        }
        cout << "\\ /";
        for (int j = 0; j <= halfRowSize; j++) {
            cout << '-';
        }
        cout << endl;
    }
}
```

Для создания тела бабочки мы снова можем использовать переменную `halfRowSize` и вывести на консоль ровно одну строку. Структура тела: начало (пустое место), середина `@` и конец (пробел). Из примеров видно, что количество пробелов равно `n - 1`:

```
for (int i = 1; i <= [illegible]; i++) {
    cout << ' ';
}
cout << " @" << endl;
```

Осталось вывести на консоль только нижнее крыло, которое аналогично верхнему: нужно лишь поменять местами наклонные черты:

```
for (int i = 1; i <= [illegible]; i++) {
    if ([illegible]) {
        for (int j = 1; j <= [illegible]; j++) {
            [illegible]
        }
        [illegible]

        for (int j = 1; j <= [illegible]; j++) {
            [illegible]
        }
        cout << endl;
    }
    else {
        for (int j = 1; j <= [illegible]; j++) {
            cout << '-';
        }
        [illegible]

        for (int j = 1; j <= [illegible]; j++) {
            [illegible]
        }
        cout << endl;
    }
}
```

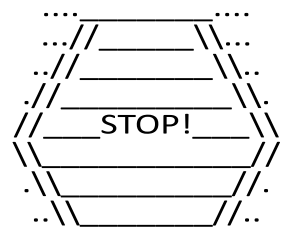
Проверка в судейской системе

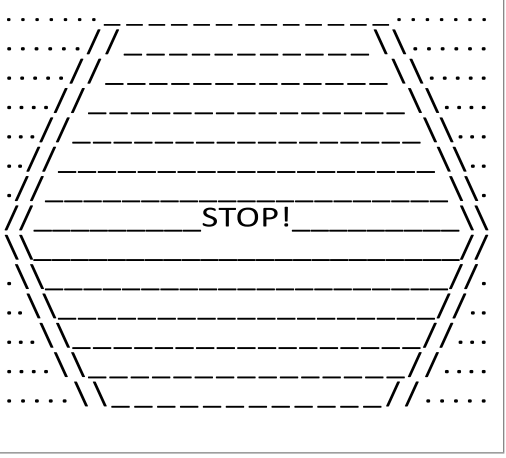
Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1367#1.

Задача: знак «Стоп»

Напишите программу, которая принимает целое число n с консоли и отрисовывает дорожный знак STOP размером, как в примерах ниже.

Примеры ввода и вывода

Ввод	Вывод
3	

Ввод	Вывод
6	

Входные данные

На вход подается целое число n в диапазоне [3 ... 1000].

Выходные данные

Выведите на консоль текстовые строки, в которых изображен знак STOP.

Рекомендации и советы

Как и в предыдущих задачах, входные данные будут считываться из одной строки, которая будет содержать одно целое число в диапазоне [3 ... 1000]:

```
int n;
cin >> n;
```

Мы можем разделить фигуру на три части — верхнюю, среднюю и нижнюю. Верхняя часть состоит из двух подчастей — начальной строки и строк, в которых символ расширяется.

Начальная строка состоит из начальных точек `.`, символов `_` в середине и точек `.` в конце. Посмотрев примеры, можно сказать, что начало имеет значение `n + 1`, и лучше выделить это значение в отдельную переменную:

```
int dots = [illegible];
```

Также необходимо создать вторую переменную, в которой мы будем хранить значение середины начальной строки размером `2 * n + 1`:

```
int underscopes = 2 * n + 1;
```

После того как мы объявили и инициализировали две переменные, мы можем вывести на консоль начальную строку:

```
for (int i = 0; i < [illegible]; i++) {
    [illegible]
}

for (int i = 0; i < [illegible]; i++) {
    [illegible]
}

for (int i = 0; i < [illegible]; i++) {
    [illegible]
}
cout << endl;
```

Для того чтобы рисовать строки, в которых знак становится «шире», нужно создать цикл, который выполняется `n` раз. Структура строки состоит из начала (`.`), `//` + середины `_` + `\ \ \` и конца (`.`). Для того чтобы можно было повторно использовать созданные пе-

ременные, необходимо уменьшить точки `dots` на 1 и подчеркивания `underscores` на 2, так как первую строку мы уже вывели, а точки и подчеркивания в верхней части рисунка на каждой строке уменьшаются:

```
underscores -= 2;
dots--;
```

На каждой последующей итерации начало и конец уменьшаются на 1, а середина увеличивается на 2:

```
for (int i = 0; i < ; i++) {
    for (int j = 0; j < ; j++) {
        cout << '.';
    }
    cout << "//";

    for (int j = 0; j < ; j++) {
        cout << '_';
    }
    cout << "\\\\";

    for (int j = 0; j < ; j++) {
        cout << '.';
    }
    cout << endl;
    underscopes += 2;
    dots--;
}
```

Средняя часть рисунка имеет начало `//` + `_`, середину `STOP!` и конец `_` + `\\\`. Количество подчеркиваний `_` равно `(underscores - 5) / 2`:

```
cout << "//";

for (int i = 0; i < ; i++) {
    cout << '_';
}
cout << "STOP!";

for (int i = 0; i < ; i++) {
    cout << '_';
}
cout << "\\\\" << endl;
```

Нижняя часть рисунка, где знак становится меньше, может быть выполнена путем повторного создания цикла, который осуществляется n раз. Структура строки — начало `. + \`, середина _ и конец `// + .`. Количество точек на первой итерации цикла должно быть равно 0 и увеличиваться на единицу на каждой последующей итерации. Поэтому можно сказать, что количество точек в нижней части рисунка равно `i`.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        cout << '.';
    }
    cout << "\\\\";

    for (int j = 0; j < underscopes; j++) {
        cout << '_';
    }
    cout << "//";

    for (int j = 0; j < i; j++) {
        cout << '.';
    }
    cout << endl;
}
```

Чтобы наша программа работала корректно, необходимо на каждой итерации цикла уменьшать количество _ на 2:

```
for (int i = 0; i < n; i++) {
    [illegible]
    underscopes -= 2;
}
```

Проверка в судейской системе

Провертьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1367#2.

Задача: стрелка

Напишите программу, которая считывает с консоли нечетное целое число `n` и рисует вертикальную стрелку с теми же размерами, что и в приведенных ниже примерах.

Примеры ввода и вывода

Ввод	Вывод
3	.###. .#.#. ##.## .#.#. ..#..

Ввод	Вывод
5	..#####.. ..#...#.. ..#...#.. ..#...#.. ###...### .#.....#. ..#...#.. ...#.#... ....#....

Ввод	Вывод
9	....#########.... ....#.......#.... ....#.......#.... ....#.......#.... ....#.......#.... ....#.......#.... ....#.......#.... ....#.......#.... #####.......##### .#.............#. ..#...........#.. ...#.........#... ....#.......#.... .....#.....#..... ......#...#...... ....#.#....... ....#........

Входные данные

На вход подается нечетное целое число n в диапазоне [3 ... 79].

Выходные данные

Выведите на консоль вертикальную стрелку, в которой символ # используется для контура стрелки, а «.» — для всего остального.

Рекомендации и советы

Из объяснения видно, что входные данные будут считываться только из одной входной строки, которая будет содержать целое число в диапазоне [3 ... 1000]. Поэтому мы будем использовать для n переменную типа int, как в предыдущих примерах:

```
int n;
cin >> n;
```

Мы можем разделить фигуру на три части — верхнюю, среднюю и нижнюю. Верхняя часть содержит две подчасти — первый ряд и тело стрелки. Из примеров видно, что количество внешних точек в первом ряду и в теле стрелки равно (n - 1) / 2. Это значение мы можем хранить в переменной outerDots:

```
int outerDots = (n - 1) /2;
```

Количество внутренних точек в теле стрелки равно (n - 2). Необходимо создать переменную innerDots, которая будет хранить это значение:

```
int innerDots = n - 2;
```

Из примеров видна структура начальной строки. Для вывода начальной строки нам необходимо использовать объявленные и инициализированные нами переменные outerDots и n:

```
for (int i = 0; i < [illegible]; i++) {
    cout << '.';
}

for (int i = 0; i < [illegible]; i++) {
    cout << '#';
}
```

```
for (int i = 0; i < [illegible]; i++) {
    cout << '.';
}
cout << endl;
```

Чтобы вывести тело стрелки на консоль, необходимо создать цикл, повторяющийся `n - 2` раза:

```
for (int i = 0; i < [illegible]; i++) {
    for (int j = 0; j < [illegible]; j++) {
        cout << '.';
    }
    cout << '#';

    for (int j = 0; j < [illegible]; j++) {
        cout << '.';
    }

    cout << '#';

    for (int j = 0; j < [illegible]; j++) {
        cout << '.';
    }
    cout << endl;
}
```

Середина фигуры состоит из начального символа #, средних символов `.` и конечного символа #. Из примеров видно, что количество # равно `outerDots`, увеличенному на 1, и для этого мы можем снова использовать ту же переменную:

```
for (int i = 0; i < [illegible]; i++) {
    [illegible]
}

for (int i = 0; i < [illegible]; i++) {
    [illegible]
}

for (int i = 0; i < [illegible]; i++) {
    [illegible]
}

cout << endl;
```

Чтобы нарисовать нижнюю часть стрелки, нужно задать новые значения для двух переменных `outerDots` и `innerDots`:

```
outerDots = 1;
innerDots = 2 * n - 5;
```

Цикл, который мы собираемся реализовать, должен осуществиться `n - 2` раза, и отдельно мы выведем последнюю строку рисунка.

На каждой итерации `outerDots` увеличивается на 1, а `innerDots` уменьшается на 2. Обратите внимание, что поскольку в предпоследней строке значение `innerDots` будет равно 1, то на следующей итерации цикла оно станет отрицательным числом. Поскольку конструктор строк не может конкатенировать в строку символ ноль или отрицательное число раз, на консоль ничего выведено не будет. Один из вариантов избежать этого — вывести последнюю строку рисунка отдельно. Высота нижней части стрелки равна `n - 1`, поэтому цикл, который выведет все строки, кроме последней, должен выполнить цикл `n - 2` раза:

```
for (int i = 0; i <  ; i++) {
    for (int j = 0; j <  ; j++) {
        
    }
    cout <<  ;

    for (int j = 0; j <  ; j++) {
        
    }
    cout <<  ;

    for (int j = 0; j <  ; j++) {
        
    }
    cout << endl;

    outerDots++;
    innerDots -= 2;
}
```

Последняя строка нашего рисунка состоит из начальной `.`, средней `#` и конечной `.`. Количество `.` равно `outerDots`:

```
for (int i = 0; i < [illegible]; i++) {
    [illegible]
}
cout << [illegible];

for (int i = 0; i < [illegible]; i++) {
    [illegible]
}
cout << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1367#3.

Задача: топор

Напишите программу, которая принимает целое число `n` и рисует топор с размерами, как показано ниже. Ширина топора равна `5 * n` столбцов.

Входные данные

Вход состоит из одного элемента (аргумента) — целого числа `n` в диапазоне [2..42].

Выходные данные

Выведите на консоль топор, как в примерах ниже.

Примеры ввода и вывода

Ввод
2
Вывод
------**-- ------*-*- *******-*- ------***-

Ввод
5
Вывод

```
---------------**--------
---------------*-*-------
---------------*--*------
---------------*---*-----
---------------*----*----
****************----*---
****************----*---
---------------*----*----
-------------********---
```

Ввод
8
Вывод

```
------------------------**----------
------------------------*-*---------
------------------------*--*--------
------------------------*---*-------
------------------------*----*------
------------------------*-----*-----
------------------------*------*----
------------------------*-------*---
**********************--------*---
**********************--------*---
**********************--------*---
**********************--------*---
------------------------*-------*---
-----------------------*---------*--
----------------------*-----------*-
----------------------**********----
```

Рекомендации и советы

Из условия задачи видно, что входные данные будут считываться только из одной строки, которая будет содержать целое число в диапазоне [2...42]. По этой причине мы будем использовать тип

int. Далее для решения задачи сначала необходимо вычислить размеры черточек слева, посередине, справа и по всей длине фигуры:

```
width = 5 * n;
leftDashes = 3 * n;
middleDashes = 0;
rightDashes = width - leftDashes - middleDashes - 2;
```

После объявления и инициализации переменных можно приступать к рисованию фигуры, начиная с вершины. Из примеров мы можем понять структуру первой строки и создать цикл, который повторяется n раз. На каждой итерации цикла средние черточки увеличиваются на 1, а черточки справа уменьшаются на 1:

```
for (int i = 0;  ; i++) {
    for (int j = 0; j <  ; j++) {
        cout <<  ;
    }
    cout << '*';

    for (int j = 0; j <  ; j++) {
        cout << '-';
    }
    cout << '*';

    for (int j = 0; j <  ; j++) {
        cout <<  ;
    }
    cout << endl;
}
```

Теперь необходимо нарисовать рукоятку топора. Для того чтобы можно было повторно использовать переменные, созданные при рисовании рукоятки топора, нужно уменьшить среднюю черточку на 1 и увеличить на 1 те, что справа и слева:

```
middleDashes  ;
rightDashes  ;
```

Мы можем нарисовать рукоятку топора, обратившись к циклу, который повторяется n/2 раз. Из примеров мы можем понять, какова его структура:

```
    for (int i = 0; i < [illegible]; i++) {
        for (int j = 0; j < [illegible]; j++) {
            cout << [illegible];
        }
        cout << [illegible];

        for (int j = 0; j < [illegible]; j++) {
            cout << [illegible];
        }
        cout << [illegible];

        for (int j = 0; j < [illegible]; j++) {
            cout << [illegible];
        }
        cout << endl;
    }
```

Нижнюю часть фигуры мы должны разделить на две подчасти — лезвие топора и последнюю строку фигуры. Головку топора мы выведем на консоль, создав цикл, повторяющийся `n/2 - 1` раз. На каждой итерации черточки слева и справа уменьшаются на 1, а средняя черточка увеличивается на 2:

```
    for ([illegible]) {
        for (int j = 0; j < [illegible]; j++) {
            cout << [illegible];
        }
        cout << '*';

        for (int j = 0; j < [illegible]; j++) {
            cout << [illegible];
        }

        cout << '*';

        for (int j = 0; j < [illegible]; j++) {
            cout << [illegible];
        }
        cout << endl;
        [illegible]
        [illegible]
        [illegible]
    }
```

Для последней строки рисунка мы снова можем использовать три уже объявленные и инициализированные переменные `leftDashes`, `middleDashes`, `rightDashes`:

```
for (int j = 0; j < [illegible]; j++) {
    cout << [illegible];
}
cout << [illegible];

for (int j = 0; j < [illegible]; j++) {
    cout << [illegible];
}
cout << [illegible];

for (int j = 0; j < [illegible]; j++) {
    cout << [illegible];
}
cout << endl;
```

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1367#4.

Глава 7.1.
Более сложные циклы

Теперь, когда мы узнали, что такое цикл `for` и что он делает, мы познакомимся с другими типами циклов, а также с некоторыми более сложными конструкциями циклов. Это расширит наши знания и поможет решать более сложные и трудные задачи. В частности, мы рассмотрим, как использовать следующие конструкции программирования:

- циклы с шагом
- циклы `while`
- циклы `do-while`
- бесконечные циклы

В этой главе мы также разберемся, что такое оператор `break` и как с его помощью прервать цикл.

Циклы с шагом

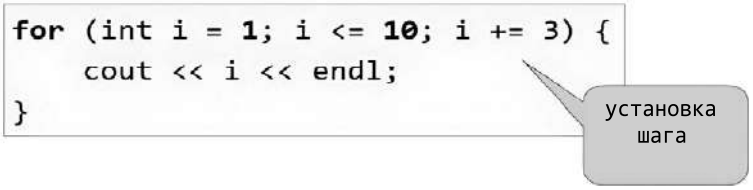
В главе 5.1 «Циклы» мы узнали, как работает цикл `for`, и теперь знаем, когда и для чего его можно использовать. В этой главе мы сосредоточимся на одной конкретной и очень важной части его конструкции, а именно на шаге.

Что такое шаг?

Шаг — это часть конструкции цикла `for`, которая указывает, на сколько нужно увеличить или уменьшить значение его ведущей переменной. Он объявляется последним в структуре цикла `for`.

Чаще всего шаг имеет размер `1`, и в этом случае вместо записи `i += 1` или `i -= 1` можно использовать операторы `i++` или `i--`. Если мы хотим, чтобы размер шага был не 1, то при увеличении ис-

пользуем оператор `i += (размер шага)`, а при уменьшении — `i -= (размер шага)`. При шаге 3 цикл будет выглядеть следующим образом:



Ниже приведена серия примеров задач, решение которых поможет нам лучше понять использование шага в цикле `for`.

Пример: числа 1...N с шагом 3

Напишите программу, которая выводит на экран числа от 1 до `n` с шагом 3. Например, если `n` = 100, то результат будет таким: 1, 4, 7, 10, ..., 94, 97, 100.

Мы можем решить эту задачу, используя следующую последовательность действий (алгоритм):

- Считываем число `n` с консоли.
- Запускаем цикл `for` от 1 до `n` с размером шага 3.
- В теле цикла выводим значение текущего шага.

```
int n;
cin >> n;

// при помощи i+= 3 увеличиваем значение i на размер шага
for (int i = 1; i <= n; i += 3) {
    cout << i << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#0.

Пример: числа от N до 1 в обратном порядке

Напишите программу, которая выводит числа от `n` до 1 в обратном порядке (шаг -1). Например, если `n = 100`, то результат будет таким: 100, 99, 98, ..., 3, 2, 1.

Решить задачу можно следующим образом:

- Считываем число `n` с консоли.
- Создаем цикл `for`, от `n` до 0.
- Определяем размер шага: -1.
- В теле цикла выводим значение текущего шага:

```
int n;
cin >> n;

for (int i = n; i >= 1; i--) {
    cout << i << endl;
}
```

Проверка в судейской системе

Проверьте пример программы: https://judge.softuni.bg/Contests/Practice/Index/1368#1.

Пример: числа от 1 до 2^n с помощью цикла for

В следующем примере мы рассмотрим использование обычного шага с размером 1 в сочетании с вычислением на каждой итерации цикла.

Напишите программу, которая выведет на экран числа от 1 до 2^n (два в степени n). Например, если n = 10, то результатом будут: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024.

```
int n;
cin >> n;
int num = 1;

for (int i = 0; i <= n; i++) {
    cout << num << endl;
    num = num * 2;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#2.

Пример: четные степени 2

Выведите четные степени 2 до 2^n: 2^0, 2^2, 2^4, 2^8, ..., 2^n. Например, если n = 10, то результатом будет 1, 4, 16, 64, 256, 1024.

Вот как можно решить эту задачу:

- Создаем переменную `num` для текущего числа, которому присваиваем начальное значение 1.
- Для шага цикла задаем значение 2.
- В теле цикла выводим значение текущего числа и увеличиваем текущее число `num` в четыре раза (в соответствии с описанием задачи):

```
int n;
cin >> n;
int num =1;

for (int i = 0; i <= n; i += 2) {
    cout << num << endl;
    num = num * 2 * 2;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#3.

Цикл while

Следующий тип циклов, который мы рассмотрим, называется циклами `while`. Их особенность заключается в том, что они повторяют блок команд до тех пор, пока не будет выполнено условие. По своей структуре они отличаются от циклов `for`, они даже имеют упрощенный синтаксис.

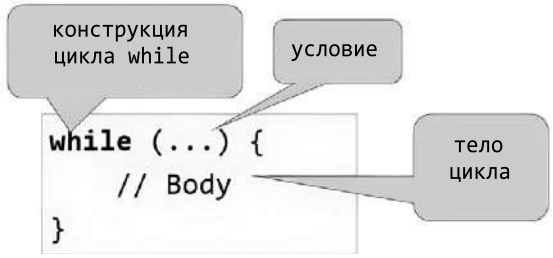
Что представляет собой цикл while?

В программировании цикл `while` используется, когда мы хотим повторять выполнение определенной логики до тех пор, пока выполняется условие.

Под условием мы понимаем любое выражение, которое возвращает `true` или `false`. Когда условие становится ложным, цикл `while`

прерывает свое выполнение, и программа продолжает выполнение кода после цикла.

Конструкция цикла `while` выглядит следующим образом:



Ниже приведена серия примеров задач, решение которых поможет нам лучше понять использование цикла `while`.

Пример: ряд чисел 2k+1

Напишите программу, которая выводит все числа ≤ n в последовательности: 1, 3, 7, 15, 31, ..., предполагая, что каждое последующее число = предыдущему числу * 2 + 1.

Вот как можно решить эту задачу:

- Для текущего числа мы создаем переменную `num`, которой присваиваем начальное значение 1.
- Для условия цикла задаем текущее число `<= n`.
- В теле цикла выведем значение текущего числа и увеличим текущее число по формуле из условия задачи.

Приведем пример реализации описанной идеи:

```
int n;
cin >> n;
int num = 1;

while (num <= n) {
    cout << num << endl;
    num = num * 2 + 1;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#4.

Пример: число в диапазоне [1 ... 100]

Введите целое число в диапазоне [1 ... 100]. Если введенное число недействительно, введите его еще раз. В этом случае недопустимым будет считаться любое число, не входящее в указанный диапазон.

Для решения этой задачи можно использовать следующий алгоритм:

- Создаем переменную `n`, которой присваиваем целочисленное значение, полученное с консольного ввода.
- В качестве условия цикла помещаем выражение, которое будет истинным, если число из входных данных не находится в диапазоне, указанном в условии.
- В теле цикла выводим на консоль сообщение "`Invalid number!`", затем присвавиваем новое значение `n` из консольного ввода.
- После проверки введенного числа выводим его значение за пределы тела цикла.

Приведем пример реализации алгоритма с использованием цикла `while`:

```
int n;
cin >> n;

while (n < 1 || n > 100) {
    cout << "Invalid number!" << endl;
    cin >> n;
}
cout << "The number is: " << n << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#5.

Наибольший общий делитель (GCD)

Наибольший общий делитель (НОД, или GCD — Greatest Common Divisor) широко используется в математике и теории чисел. В этой главе мы научимся его вычислять.

Определение НОД: наибольший общий делитель двух натуральных чисел `a` и `b` — это наибольшее число, которое без остатка делит и `a`, и `b`.

a	b	НОД
24	16	8
67	18	1

a	b	НОД
12	24	12
15	9	3

a	b	НОД
10	10	10
100	88	4

Евклидов алгоритм

В следующей задаче мы воспользуемся одним из первых опубликованных алгоритмов нахождения НОД — алгоритмом Евклида.

Пока не достигнем остатка 0:

- Делим большее число на меньшее.
- Берем остаток от деления.

Псевдокод алгоритма Евклида:

```
while b ≠ 0
    oldB = b
    b = a % b
    a = oldB
print a
```

Пример: нахождение наибольшего общего делителя (НОД, GCD)

Введите целые числа `a` и `b` и найдите НОД (`a`, `b`).

Решим задачу с помощью алгоритма Евклида:

- Создаем переменные `a` и `b`, которым присваиваем *целочисленные значения*, взятые из консольного ввода.
- В качестве условия цикла задаем выражение, которое будет *истинным*, если число `b` отлично от 0.
- В *теле цикла* следуем инструкциям из псевдокода:
  - Создаем временную переменную, которой присваиваем текущее значение `b`.
  - Присваиваем новое значение `b`, которое является остатком от деления `a` и `b`.
  - Переменной `a` присваиваем предыдущее значение переменной `b`.
- Когда цикл завершен и мы нашли НОД (GCD), мы выводим его на экран.

Пример реализации алгоритма:

```
int a, b;
cin >> a >> b;

while (b != 0) {
    int oldB = b;
    b = a % b;
    a = oldB;
}

cout << "GCD = " << a << endl;
```

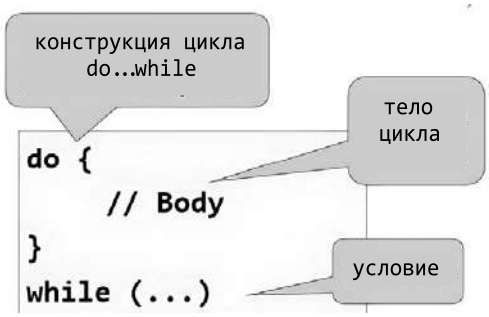
Проверка в судейской системе

Проверьте пример программы: https://judge.softuni.bg/Contests/Practice/Index/1368#6.

Цикл do-while

Следующий цикл, который мы рассмотрим — `do-while`. По структуре он похож на `while`, но между ними есть существенное различие. Оно заключается в том, что в `do-while` тело выполняется как минимум один раз. Почему так происходит? При построении этого цикла условие всегда проверяется после его тела, что, в свою очередь, гарантирует, что при первом запуске цикла код будет выполнен, а проверка конца цикла будет применяться на каждой последующей итерации `do-while`:

Мы рассмотрим примеры задач, решение которых поможет нам лучше понять цикл `do-while`. Он представляет собой удобный способ выполнения циклической проверки значений, приводящей к выполнению заданных фрагментов программы.



Пример: вычисление факториала

Для натурального числа n вычислить `n! = 1 * 2 * 3 * ... * n`. Например, если `n = 5`, то результат будет таким: `5! = 1 * 2 * 3 * 4 * 5 = 120`.

Вот как можно вычислить факториал:

- Создаем переменную `n`, которой присваиваем целое значение, взятое из консольного ввода.
- Создаем еще одну переменную — `fact`, начальное значение которой равно 1. Мы будем использовать ее для вычисления и хранения факториала.
- В качестве условия цикла используем `n > 1`, так как при каждом выполнении вычислений в теле цикла значение `n` будет уменьшаться на 1.
- В теле цикла:
  - Присваиваем `fact` новое значение, которое является результатом умножения текущего значения `fact` на текущее значение `n`.
  - Уменьшаем значение `n` на -1.
  - За пределами тела цикла выводим итоговое значение факториала.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#7.

Пример: сложение цифр числа

Суммировать цифры целого положительного числа `n`. Например, если `n` = 5634, то каким будет результат?

Для решения этой задачи мы можем использовать следующую идею:

- Создаем переменную `n`, которой присваиваем значение, равное числу, введенному пользователем.
- Создаем вторую переменную `sum`, начальное значение которой равно 0. Мы будем использовать ее для вычисления и хранения результата.

- В качестве условия цикла будем использовать `n > 0`, так как после каждого вычисления результата в теле цикла мы будем удалять последнюю цифру `n`.
- В теле цикла:
  - Присваиваем сумме `sum` новое значение, которое является результатом сложения текущего значения `sum` с последней цифрой числа `n`.
  - Присваиваем новое значение `n`, которое является результатом удаления последней цифры из `n`.
- За пределами тела цикла выводим итоговое значение суммы.

Приведем пример реализации описанной идеи:

```
int n;
cin >> n;
int sum = 0;
do {
    sum = sum + (n % 10);
    n = n / 10;
}
while (n > 0);
cout << "Сумма цифр: " << sum << endl;
```

Проверка в судейской системе

Проверьте пример программы: https://judge.softuni.bg/Contests/Practice/Index/1368#8.

Бесконечные циклы и оператор break

До сих пор мы знакомились с различными типами циклов, изучали их конструкции и области применения. Далее мы узнаем, что такое бесконечный цикл, когда он возникает и как можно прервать его выполнение с помощью оператора `break`.

Бесконечный цикл. Что это такое?

Бесконечным циклом называется цикл, который бесконечно повторяет выполнение своего тела. В циклах `while` и `do-while` провер-

ка завершения представляет собой условное выражение, которое всегда возвращает `true`. Бесконечный цикл `for` возникает при отсутствии условия завершения. Вот как выглядит бесконечный цикл `while`:

```
while (true) {
    cout << "Бесконечный цикл" << endl;
}
```

А вот так выглядит бесконечный цикл `for`:

```
for (;;) {
    cout << "Бесконечный цикл" << endl;
}
```

Оператор прерывания

Мы уже знаем, что бесконечный цикл выполняет определенный код бесконечно, но что делать, если мы хотим принудительно выйти из цикла в какой-то момент по какому-то условию? На помощь приходит оператор `break`, то есть оператор остановки, или прерывания.

Оператор `break` останавливает выполнение цикла в момент его вызова и продолжает его с первой строки после завершения цикла. Это означает, что текущая итерация цикла не будет завершена, соответственно, не будет выполнен и остальной код в теле цикла.

Пример: простые числа

В следующей задаче нам необходимо выполнить проверку на наличие простого числа. Прежде чем перейти к ней, давайте вспомним, что такое простые числа.

Определение: целое число является простым, если оно делится без остатка только на себя и на 1. По определению, простые числа положительны и больше 1. Наименьшим простым числом является 2.

Можно считать, что целое число `n` является простым, если `n > 1` и `n` не делится на число от 2 до `n-1`.

Вот первые простые числа: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, ...

В отличие от простых чисел *составные числа* — это целые числа, которые можно получить путем перемножения нескольких простых чисел.

Вот несколько примеров составных чисел:

- 10 = 2 * 5
- 42 = 2 * 3 * 7
- 143 = 13 * 11

Алгоритм проверки того, является ли целое число простым: проверить, что `n` > 1 и делится ли `n` на 2, 3, ..., `n-1` без остатка.

- Если оно делится на любое из этих чисел, то оно *составное*.
- Если оно не делится ни на одно из этих чисел, значит оно *простое*.

Мы можем оптимизировать алгоритм, вместо того чтобы проверять его до `n-1`, чтобы проверять делители до `√n`. Подумайте, почему.

Пример: проверка простого числа. Оператор break

Необходимо проверить, является ли число `n` простым. Для этого проверим, делится ли `n` на числа от 2 до √`n`.

Вот алгоритм проверки простого числа, расписанный по шагам:

- Создаем переменную `n`, которой присваиваем целое число, введенное с консольного ввода.
- Создаем булеву переменную `is_prime` с начальным значением `True`. Мы предполагаем, что число является простым, пока не доказано обратное.
- Создаем цикл `for` с начальным значением переменной цикла, равным 2, при условии, что ее текущее значение `<= √n`. Шаг цикла равен 1.
- В теле цикла проверяем, есть ли остаток от деления `n` на текущее значение. Если остатка от деления нет, то меняем `is_prime` на `False` и принудительно выходим из цикла с помощью оператора `break`.
- В зависимости от значения `is_prime` выводим, является ли число простым (`True`) или составным (`False`) соответственно.

Приведем пример реализации описанного алгоритма:

```
int n;
cin >> n;
bool isPrime = true;

for (int i = 2; i <= sqrt(n); i++) {
    if (n % i == 0) {
        isPrime = false;
        break;
    }
}

if (isPrime) {
    cout << "Простое" << endl;
}
else {
    cout << "Составное" << endl;
}
```

Нам остается проверить, больше ли вводимое число 1, поскольку по определению числа 0, 1, -1 и -2 не являются простыми.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#9.

Пример: оператор break в бесконечном цикле

Напишите программу, которая проверяет, является ли число `n` четным, и если является, то выводит его на экран. Четным считается число, которое делится на 2 без остатка. Если число неверное, вернемся к повторному вводу и выведем сообщение о том, что введенное число не является четным.

Вот идея, как можно решить эту задачу:

- Создаем переменную `n`, которой присваиваем начальное значение 0.
- Создаем бесконечный цикл `while`, устанавливая условие в `True`.
- В *теле цикла*:
  - Берем целочисленное значение из консольного ввода и присваиваем его `n`.

- Если число четное, выходим из цикла по команде `break`.
- В противном случае выводится сообщение о том, что число *не является четным*. Итерации продолжаются до тех пор, пока не будет введено четное число.

- Выводим на экран четное число.

Приведем пример реализации этой идеи:

```
int n = 0;

while (true) {
    cout << "Введите четное число: " << endl;
    cin >> n;

    if (n % 2 == 0) {
        break;
    }
    cout << "Число не является четным." << endl;
}
cout << "Введено четное число: " << n << endl;
```

Проверка в судейской системе

Проверьте пример программы в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1368#10.

Вложенные циклы и оператор break

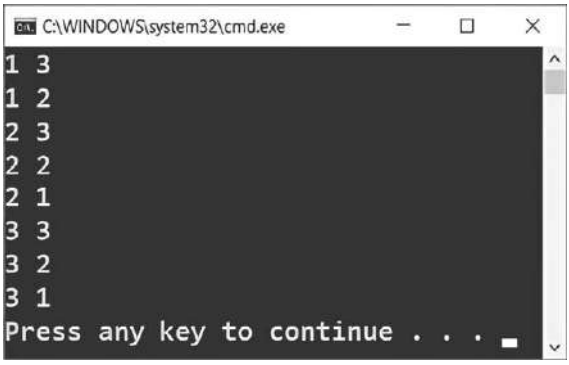
Теперь, когда мы узнали, что такое вложенные циклы и как работает оператор `break`, пришло время понять, как они работают вместе. Для лучшего понимания напишем пошаговую программу, которая должна составить все возможные комбинации из пар чисел. Первое число в комбинации увеличивается от 1 до 3, а второе — уменьшается от 3 до 1. Задача должна выполняться до тех пор, пока `i + j` не будет равно 2 (то есть `i = 1` и `j = 1`).

Желаемый результат:

Вот ошибочное решение, которое на первый взгляд кажется правильным:

```
for (int i = 1; i <= 3; i++) {
    for (int j = 3; j >= 1; j--) {
        if (i + j == 2) {
            break;
        }

        cout << i << " " « j << endl;
    }
}
```

Если мы оставим нашу программу в таком виде, то результат будет следующим:



Почему это работает именно так? Как мы видим, в результате нет никаких «1 1». Когда программа доходит до момента, когда `i = 1` и `j = 1`, она входит в проверку `if` и выполняет операцию `break`. При этом происходит выход из внутреннего цикла, но продолжается выполнение внешнего цикла. По мере роста `i` программа входит во внутренний цикл и выводит результат.

Когда мы используем оператор `break` во *вложенном цикле*, он прерывает выполнение только внутреннего цикла.

Каково же правильное решение? Одним из способов решения этой задачи является объявление булевой переменной, которая следит за тем, следует ли продолжать цикл. При необходимости выхода (из всех вложенных циклов) значение этой переменной изменяется на `true`; выходим из внутреннего цикла с `break`, а из внеш-

него — с последующей проверкой. Приведем пример реализации этой идеи:

```
bool hasToEnd = false;
for (int i = 1; i <= 3; i++) {
    if (hasToEnd == false) {
        for (int j = 3; j >= 1; j--) {
            if (i + j == 2) {
                hasToEnd = true;
                break;
            }

            cout << i << " " « j << endl;
        }
    }
}
```

Таким образом, когда `i + j = 2`, программа установит переменную `hasToEnd = true` и приведет к выходу из внутреннего цикла. При следующем возврате внешнего цикла проверка `if` не позволит программе добраться до внутреннего цикла и прервет выполнение.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1368#11.

Задачи с циклами

В этой главе мы познакомились с несколькими новыми типами циклов, которые могут быть использованы для итераций с более сложной логикой программирования. Давайте решим несколько задач, используя новые знания.

Задача: числа Фибоначчи

Числа Фибоначчи в математике образуют ряд, который выглядит следующим образом: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Формула для формирования ряда имеет вид:

```
F0 = 1
F1 = 1
Fn = Fn-1 + Fn-2
```

Следует ввести целое число n и вычислить n-е число Фибоначчи.

Примеры ввода и вывода

Ввод (n)	Вывод	Комментарий
10	89	F(11) = F(9) + F(8)
5	8	F(5) = F(4) + F(3)
20	10946	F(20) = F(19) + F(18)

Ввод (n)	Вывод
0	1
1	1
2	2

Рекомендации и советы

Идея решения задачи:

- Создаем переменную n, которой присваиваем целочисленное значение из консольного ввода.
- Создаем переменные f0 и f1, которым присваиваем значение 1, так как последовательность начинается.
- Создаем цикл for от нуля до конечного значения n - 1.
- В теле цикла:
  - Создаем *временную* переменную fNext, которой присваиваем следующее число в последовательности Фибоначчи.
  - Присваиваем текущее значение f1 значению f0.
  - Присваиваем значение временной переменной fNext значению f1.
- За пределами цикла выводим n-ое число Фибоначчи. Пример реализации:

```
int n;
cin >> n;
int f0 = 1;
int f1 = 1;

for (int i = 0; i < n - 1; i++) {
    int fNext = f0 + f1;
    f0 = f1;
    f1 = fNext;
}
cout << f1 << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#12.

Задача: пирамида чисел

Выведите числа `1 ... n` в виде пирамиды, как показано в примерах ниже. В первой строке мы выводим одно число, во второй — два, в третьей — три и так далее, пока числа не закончатся. В последней строке мы отображаем столько чисел, сколько получится, пока не дойдем до `n`.

Ввод	Вывод
7	1 2 3 4 5 6 7

Ввод	Вывод
5	1 2 3 4 5

Ввод	Вывод
10	1 2 3 4 5 6 7 8 9 10

Рекомендации и советы

Мы можем решить задачу с помощью двух вложенных циклов (по строкам и столбцам) с выводом в них и выходом при достижении последнего числа. Теперь распишем эту идею более подробно:

- Создаем переменную `n`, которой присваиваем целочисленное значение, считанное с консоли.
- Создаем переменную `num` с начальным значением 1. В ней будет храниться количество выведенных чисел. На каждой итерации мы будем увеличивать ее на 1 и выводить на экран.
- Создаем внешний цикл `for`, который будет отвечать за строки таблицы. Назовем переменную цикла `row` и установим ее начальное значение 1. В качестве условия ставим `row <= n`. Размер шага равен 1.
- В теле цикла создаем внутренний цикл `for`, который будет отвечать за столбцы таблицы. Назовем переменную цикла `col` и установим ее начальное значение равным 1. В качестве условия ставим `col <= row` (`row` = количество цифр в строке). Размер шага равен 1.

- В теле вложенного цикла:
  - Проверяем, что `col > 1`, если да — выводим расстояние. Если не делать этой проверки, а выводить расстояние напрямую, то в начале каждой строки будет стоять лишнее.
  - Выводим число `num` в текущей ячейке таблицы и увеличиваем его на 1.
  - Проверяем, не превышает ли `num > n`. Если `num` больше `n`, то прерываем внутренний цикл.
- Выводим пустую строку для перехода к следующей.
- Снова проверяем, не превышает ли `num > n`. Если больше, то прерываем выполнение программы с помощью `break`.

```
int n;
cin >> n;
int num = 1;

for (int row = 1; row <= n; row++) {
    for (int col = 1; col <= row; col++) {
        if (col > 1) {
            cout << " ";
        }

        cout << num;
        num++;

        if (num > n) {
            break;
        }
    }
    cout << endl;

    if (num > n) {
        break;
    }
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1368#13.

Задача: таблица чисел

Выведите числа `1` … `n` в виде таблицы, как показано в примерах ниже.

Примеры ввода и вывода

Ввод	Вывод
3	1 2 3 2 3 2 3 2 1

Ввод	Вывод
4	1 2 3 4 2 3 4 3 3 4 3 2 4 3 2 1

Ввод	Вывод
2	1 2 2 1

Ввод	Вывод
5	1 2 3 4 5 2 3 4 5 4 3 4 5 4 3 4 5 4 3 2 5 4 3 2 1

Рекомендации и советы

Мы можем решить задачу с помощью двух вложенных циклов и некоторых вычислений в них:

- Читаем с консоли размер таблицы в целочисленной переменной `n`.
- Создаем цикл `for`, который будет отвечать за строки таблицы. Назовем переменную цикла `row` и зададим ее начальное значение 0. В соответствии с условием задаем `row < n`. Размер шага равен 1.
- В теле цикла создаем вложенный цикл `for`, который будет отвечать за столбцы таблицы. Переменную цикла назовем `col` и установим ее начальное значение равным 0. В качестве условия ставим `col < n`. Размер шага равен 1.
- В теле вложенного цикла:
  - Создаем переменную `num`, которой присваиваем результат текущей строки + текущего столбца + 1 (+1, так как мы начинаем считать с 0).

- Проверяем, не превышает ли `num > n`. Если `num` больше `n`, присваиваем `num` новое значение, равное удвоенному `n` за вычетом текущего значения `num`. Это делается для того, чтобы ни в одной из ячеек таблицы не было превышения `n**`.
    - Выводим число из текущей ячейки таблицы.
- Выводим пустую строку во внешнем цикле для перехода к следующей строке.

```
int n;
cin >> n;
for (int row = 0; row < n; row++) {
    for (int col = 0; col < n; col++) {
        int num = row + col + 1;

        if (num > n) {
            num = 2 * n - num;
        }
        cout << num << " ";
    }
    cout << endl;
}
```

Проверка в судейской системе

Проверьте пример программы: https://judge.softuni.bg/Contests/Practice/Index/1368#14.

Что мы узнали из этой главы?

Мы можем использовать циклы `for` с шагом:

```
for (int i = 1; i <= n; i += 3) {
    cout << i << endl;
}
```

Запустить и протестировать пример можно в Интернете: https://repl.it/@vncpetrov/ForLoopWithStep.

Циклы `while`/`do-while` повторяются до тех пор, пока выполняется условие:

```
int num = 1;
while (num <= n) {
    cout << num++ << endl;
}
```

Запустить и протестировать пример можно в Интернете: https://repl.it/@vncpetrov/WhileLoop.

Если нам необходимо прервать выполнение цикла, то мы делаем это с помощью оператора `break`:

```
int n = 0;

while (true) {
    cin >> n;

    if (n % 2 == 0) {
         break; // четное число -> выход из цикла
    }
    cout << "Число не является четным." << endl;
}
cout << "Введено четное число: " << n << endl;
```

Запустить и протестировать пример можно в Интернете: https://repl.it/@vncpetrov/OperatorBreak.

Глава 7.2.
Более сложные циклы — экзаменационные задачи

Мы уже научились выполнять блок команд более одного раза с помощью цикла `for`. В предыдущей главе мы рассмотрели еще несколько конструкций циклов, которые могут помочь нам в решении более сложных задач, а именно:

- циклы с шагом
- вложенные циклы
- циклы `while`
- циклы `do-while`
- Бесконечные циклы и выход из цикла (оператор `break`)

Экзаменационные задачи

Давайте закрепим наши знания, решив несколько более сложных задач с циклами, которые даются на вступительных экзаменах.

Задача: генератор паролей

Напишите программу, которая принимает два целых числа n и l и генерирует в алфавитном порядке все возможные «пароли», состоящие из следующих пяти символов:

- Символ 1: цифра от 1 до `n`.
- Символ 2: цифра от 1 до `n`.
- Символ 3: маленькая буква из первых `l` букв алфавита.
- Символ 4: маленькая буква из первых `l` букв алфавита.
- Символ 5: цифра от 1 до `n`, больше первых двух цифр.

Примеры ввода и вывода

Ввод	Вывод
2 4	11aa2 11ab2 11ac2 11ad2 11ba2 11bb2 11bc2 11bd2 11ca2 11cb2 11cc2 11cd2 11da2 11db2 11dc2 11dd2

Ввод	Вывод
3 1	11aa2 11aa3 12aa3 21aa3 22aa3

Ввод	Вывод
4 2	11aa2 11aa3 11aa4 11ab2 11ab3 11ab4 11ba2 11ba3 11ba4 11bb2 11bb3 11bb4 12aa3 12aa4 12ab3 12ab4 12ba3 12ba4 12bb3 12bb4 13aa4 13ab4 13ba4 13bb4 21aa3 21aa4 21ab3 21ab4 21ba3 21ba4 21bb3 21bb4 22aa3 22aa4 22ab3 22ab4 22ba3 22ba4 22bb3 22bb4 23aa4 23ab4 23ba4 23bb4 31aa4 31ab4 31ba4 31bb4 32aa4 32ab4 32ba4 32bb4 33aa4 33ab4 33ba4 33bb4

Ввод	Вывод
3 2	11aa2 11aa3 11ab2 11ab3 11ba2 11ba3 11bb2 11bb3 12aa3 12ab3 12ba3 12bb3 21aa3 21ab3 21ba3 21bb3 22aa3 22ab3 22ba3 22bb3

Входные данные

Входные данные считываются с консоли и состоят из двух целых чисел: n и l в диапазоне [1...9], по одному на строку.

Выходные данные

Все пароли должны быть выведены на консоль *в алфавитном порядке, разделенные пробелом.*

Рекомендации и советы

Мы можем мысленно разделить решение задачи на три части:

- Чтение входных данных — в данной задаче это чтение двух чисел `n` и `l`, каждое в отдельной строке.
- Обработка входных данных — использование вложенных циклов для перебора всех возможных символов для каждого из пяти символов пароля.
- Вывод результатов — отображение каждого пароля, удовлетворяющего требованиям.

Чтение входных данных

Для чтения входных данных объявим две переменные целочисленного типа `int` - `n` и `l`:

```
int n, l;
cin >> n;
cin >> l;
```

Вывод результата

Одним из способов поиска решения этой задачи является создание пяти циклов `for`, вложенных друг в друга, по одному для каждого символа пароля. Для обеспечения условия, что последняя цифра `d3` больше первых двух, будем использовать встроенную функцию `max(...)`, которая находится в библиотеке `algorithm`. Для переменных циклов, в которых будут храниться символы пароля, остановимся на следующих типах: для числовых символов (`d1`, `d2` и `d3`) — тип `int`, а для буквенных символов (`l1` и `l2`) — тип `char`:

```
for (int d1 = 1; d1 < n; d1++) {
    for (int d2 = 1; d2 < n; d2++) {
        for (char l1 = 'a'; l1 < 'a' + l; l1++) {
            for (char l2 = 'a'; l2 < ('a' + l); l2++) {
                for (int d3 = max(d1, d2) + 1; d3 <= n; d3++) {
                    cout << d1 << d2 << l1 << l2 << d3 << " ";
                }
            }
        }
    }
}
```

Знаете ли вы?

- Мы можем определить цикл `for` с переменной типа `char`:

  ```
  for (char ch = 'a'; ch <= 'z'; ch++)
  ```

- Мы можем прочитать переменную типа `char` из консоли с помощью следующего оператора:

  ```
  char ch;
  cin >> ch;
  ```

- Мы можем преобразовать заглавную букву в строчную с помощью встроенной функции в языке C++:

  ```
  ch = tolower(ch);
  ```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1369#0.

Задача: магические числа

Напишите программу, которая принимает одно целое магическое число и выводит все возможные шестизначные числа, для которых произведение их цифр равно магическому числу.

Пример: “Магическое число” → 2

- `111112 → 1 * 1 * 1 * 1 * 1 * 2 = 2`
- `111121 → 1 * 1 * 1 * 1 * 2 * 1 = 2`
- `111211 → 1 * 1 * 1 * 2 * 1 * 1 = 2`
- `112111 → 1 * 1 * 2 * 1 * 1 * 1 = 2`
- `121111 → 1 * 2 * 1 * 1 * 1 * 1 = 2`
- `211111 → 2 * 1 * 1 * 1 * 1 * 1 = 2`

Входные данные

Входные данные считываются с консоли и состоят из *одного целого числа в* диапазоне [1 ... 600 000].

Выходные данные

Выведите на консоль все *магические числа*, разделенные *пробелом*.

Примеры ввода и вывода

Ввод	2
Вывод	111112 111121 111211 112111 121111 211111

Ввод	8
Вывод	111118 111124 111142 111181 111214 111222 111241 111412 111421 111811 112114 112122 112141 112212 112221 112411 114112 114121 114211 118111 121114 121122 121141 121212 121221 121411 122112 122121 122211 124111 141112 141121 141211 142111 181111 211114 211122 211141 211212 211221 211411 212112 212121 212211 214111 221112 221121 221211 222111 241111 411112 411121 411211 412111 421111 811111

Ввод	531441
Вывод	999999

Рекомендации и советы

Решение следует той же концепции, что и предыдущая задача (снова нужно сгенерировать все комбинации для n элементов). Следуя этим шагам, попробуйте решить задачу самостоятельно.

- Объявите и инициализируйте переменную типа `int` и считайте данные с консоли.
- Вложите шесть циклов `for` один в другой, для каждой цифры искомого шестизначного числа.
- В последнем цикле с помощью `if` проверьте, равно ли произведение шести цифр магическому числу.

Приведем пример реализации этой идеи:

```
int magicNumber;
[illegible]

for (int d1 = 0; d1 <= 9; d1++) {
    for (int d2 = 0; [illegible]) {
        for ([illegible]) {
            for ([illegible]) {
                for ([illegible]) {
                    for ([illegible]) {
                        if (d1 * d2 * d3 * d4 * d5 * d6 == magicNumber) {
                            [illegible]
                        }
                    }
                }
            }
        }
    }
}
```

В предыдущей главе мы рассмотрели другие циклические конструкции. Рассмотрим пример решения той же задачи с использованием цикла `while`. Сначала нам необходимо записать входное магическое число в соответствующую переменную и инициализировать шесть переменных, по одной для каждой из шести цифр числа, которое мы ищем в качестве результата:

```
int magicNumber;
cin >> magicNumber;

int d1, d2, d3, d4, d5, d6;
```

Затем мы приступим к описанию циклов `while`:

- Инициализируем первый разряд: `d1 = 1`.

- Для каждого цикла зададим условие: число должно быть меньше или равно 9.
- В начале каждого цикла задаем значение следующего разряда, в данном случае: `d2 = 1`. Во вложенных циклах `for` мы инициализируем переменные во внутренних циклах при каждом инкременте внешних циклов. Мы хотим добиться такого же поведения и здесь.
- В конце каждого цикла мы будем увеличивать соответствующий разряд на единицу: `d1 += 1`, `d2 += 1`, `d3 += 1` и т. д.
- В самом внутреннем цикле мы выполним проверку и при необходимости выведем данные на консоль.

```
d1 = 1;
while (d1 <= 9) {
    d2 = 1;
    while (d2 <= 9) {
        d3 = 1;
        while (d3 <= 9) {
            [illegible]
                [illegible]
                    [illegible]
                        [illegible]
                            if (d1 * d2 * d3 * d4 * d5 * d6 == magicNumber) {
                                cout << d1 << d2 << d3 << d4 << d5 << d6 << " ";
                            }
                            [illegible]
                        }
                        [illegible]
                    }
                    [illegible]
                }
            d3++;
        }
        d2++;
    }
    d1++;
}
```

Как видим, задача может быть решена с помощью различных типов циклов. Разумеется, для каждой задачи существует наиболее подходящий вариант. Для того чтобы отработать каждый цикл, попробуйте решить каждую из следующих задач с использованием всех изученных циклов.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1369#1.

Задача: стоп-число

Напишите программу, которая выводит на консоль все числа от `N` до `M`, кратные 2 и 3 без остатка, в обратном порядке.

Мы будем считывать с консоли еще одно стоп-число — `S`. Если какое-либо из чисел, кратных 2 и 3, равно стоп-числу, его не следует выводить, и программа должна завершиться. В противном случае выведите все числа до `N`, удовлетворяющие условию.

Примеры ввода и вывода

Ввод	Вывод	Комментарии
1 30 15	30 24 18 12 6	Числа от 30 до 1, которые одновременно делятся на 2 и 3 без остатка, это: 30, 24, 18, 12 и 6. Число 15 не равно ни одному, поэтому последовательность продолжается.

Ввод	Вывод	Комментарии
1 36 12	36 30 24 18	Числа от 36 до 1, которые одновременно делятся на 2 и 3 без остатка, это: 36, 30, 24, 18, 12 и 6. Число 12 равно стоп-числу, поэтому мы остановимся на 18.

Ввод	Вывод
20 1000 36	996 990 984 978 972 966 960 954 948 942 936 930 924 918 912 906 900 894 888 882 876 870 864 858 852 846 840 834 828 822 816 810 804 798 792 786 780 774 768 762 756 750 744 738 732 726 720 714 708 702 696 690 684 678 672 666 660 654 648 642 636 630 624 618 612 606 600 594 588 582 576 570 564 558 552 546 540 534 528 522 516 510 504 498 492 486 480 474 468 462 456 450 444 438 432 426 420 414 408 402 396 390 384 378 372 366 360 354 348 342 336 330 324 318 312 306 300 294 288 282 276 270 264 258 252 246 240 234 228 222 216 210 204 198 192 186 180 174 168 162 156 150 144 138 132 126 120 114 108 102 96 90 84 78 72 66 60 54 48 42

Входные данные

Считайте с консоли три числа, каждое в отдельной строке:

- `N` — целое число: `0 ≤ N < M`.
- `M` — целое число: `N < M ≤ 10000`.
- `S` — целое число: `N ≤ S ≤ M`.

Выходные данные

Выведите на консоль в одну строку все числа, удовлетворяющие условию, разделенные пробелом.

Рекомендации и советы

Задачу можно разделить на *четыре* логические части:

- Чтение входных данных.
- Проверка всех чисел в заданном интервале (соответственно, с помощью цикла).
- Проверка условий задачи на соответствие каждому числу в рассматриваемом интервале.
- Вывод чисел.

Первая часть тривиальна — мы читаем из консоли три целых числа, соответственно, будем использовать тип переменной `int`.

Во второй части инициализируем цикл `for`. Здесь есть небольшая загвоздка — в условии указано, что числа должны быть выведены в обратном порядке. Это означает, что начальным значением переменной `i` будет большее число, которое, как видно из примера, равно `M`. Соответственно, конечное значение `i` должно быть `N`. То, что мы будем выводить результаты в обратном порядке, и значения `i` говорят нам о том, что шаг будет уменьшаться на 1:

```
for (int i = m; i >= n; i--)
```

После того как мы инициализировали цикл `for`, наступает очередь третьей части задачи — проверить условие, делится ли заданное число на 2 и на 3 без остатка. Мы сделаем это с помощью простой проверки `if`, которую оставим читателю для самостоятельной работы.

Другая загвоздка в этой задаче заключается в том, что, помимо вышеуказанной проверки, мы должны выполнить еще одну — равно ли число стоп-числу, выданному нам консолью в третьей строке.

Чтобы дойти до этой проверки, проверяемое число должно пройти вышеуказанную проверку. Для этого мы построим еще один оператор `if`, который вложим в предыдущий. Если условие истинно, то задача состоит в том, чтобы остановить программу, что в данном случае мы можем сделать с помощью оператора `break`, который выведет нас из цикла `for`.

Соответственно, если условие проверки совпадения числа со стоп-числом возвращает `false`, то по умолчанию наша программа должна продолжить вывод. На этом, собственно, и заканчивается четвертая, заключительная часть нашей программы.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1369#2.

Задача: особые числа

Напишите программу, которая *считывает одно целое число* `N` и генерирует все возможные *особые числа* от 1111 до 9999. Чтобы считаться особым, число `N` должно соответствовать *условию*:

- `N` должно делиться на каждую из своих цифр без остатка.

Пример: если `N` = 16, то 2418 — особое число:

- 16 / 2 = 8 без остатка
- 16 / 4 = 4 без остатка
- 16 / 1 = 16 без остатка
- 16 / 8 = 2 без остатка

Входные данные

Входные данные считываются с консоли и состоят из *одного целого числа* в диапазоне [1 ... 600 000].

Выходные данные

Выведите на консоль все особые числа, разделенные пробелом.

Примеры ввода и вывода

Ввод	Вывод
11	1111

Ввод	Вывод	Комментарии
3	1111 1113 1131 1133 1311 1313 1331 1333 3111 3113 3131 3133 3311 3313 3331 3333	3 / 1 = 3 без остатка 3 / 3 = 1 без остатка 3 / 3 = 1 без остатка 3 / 3 = 1 без остатка

Ввод	Вывод
16	1111 1112 1114 1118 1121 1122 1124 1128 1141 1142 1144 1148 1181 1182 1184 1188 1211 1212 1214 1218 1221 1222 1224 1228 1241 1242 1244 1248 1281 1282 1284 1288 1411 1412 1414 1418 1421 1422 1424 1428 1441 1442 1444 1448 1481 1482 1484 1488 1811 1812 1814 1818 1821 1822 1824 1828 1841 1842 1844 1848 1881 1882 1884 1888 2111 2112 2114 2118 2121 2122 2124 2128 2141 2142 2144 2148 2181 2182 2184 2188 2211 2212 2214 2218 2221 2222 2224 2228 2241 2242 2244 2248 2281 2282 2284 2288 2411 2412 2414 2418 2421 2422 2424 2428 2441 2442 2444 2448 2481 2482 2484 2488 2811 2812 2814 2818 2821 2822 2824 2828 2841 2842 2844 2848 2881 2882 2884 2888 4111 4112 4114 4118 4121 4122 4124 4128 4141 4142 4144 4148 4181 4182 4184 4188 4211 4212 4214 4218 4221 4222 4224 4228 4241 4242 4244 4248 4281 4282 4284 4288 4411 4412 4414 4418 4421 4422 4424 4428 4441 4442 4444 4448 4481 4482 4484 4488 4811 4812 4814 4818 4821 4822 4824 4828 4841 4842 4844 4848 4881 4882 4884 4888 8111 8112 8114 8118 8121 8122 8124 8128 8141 8142 8144 8148 8181 8182 8184 8188 8211 8212 8214 8218 8221 8222 8224 8228 8241 8242 8244 8248 8281 8282 8284 8288 8411 8412 8414 8418 8421 8422 8424 8428 8441 8442 8444 8448 8481 8482 8484 8488 8811 8812 8814 8818 8821 8822 8824 8828 8841 8842 8844 8848 8881 8882 8884 8888

Рекомендации и советы

Решите задачу самостоятельно, используя то, что вы узнали из двух предыдущих задач.

Вспомните разницу между операторами целочисленного деления (/) и деления с остатком (%) в языке C++.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1369#3.

Задача: разряды числа

Напишите программу, которая считывает с консоли целое число в диапазоне [100 ... 999], а затем выводит его заданное количество раз, изменяя перед каждым выводом следующим образом:

- Если число делится на 5 без остатка, *вычтите* из него *его первую цифру*.
- Если число делится на 3 без остатка, *вычтите* из него *вторую цифру*.
- Если ни одно из вышеперечисленных условий не действует, *добавьте к нему третью цифру*.

Выведите на консоль `N` строк, каждая из которых содержит `M` чисел, являющихся результатом вышеописанных действий.

- `N` = сумма первой и второй цифры числа.
- `M` = сумма первой и третьей цифры числа.

Входные данные

Входные данные считываются с консоли и представляют собой целое число в диапазоне [100 ... 999].

Выходные данные

Выведите на консоль все целые числа, являющиеся результатом вышеупомянутых вычислений, в соответствующих строках и столбцах, как в примерах.

Примеры ввода и вывода

Ввод	Вывод	Комментарии
132	129 126 123 120 119 121 123 120 119 121 123 120	(1 + 3) = 4 и (1 + 2) = 3 → 4 строки с 3 числами в каждой Ввод числа 132 → деление на 3 → 132 - 3 = = 129 → деление на 3 → 129 - 3 = = 126 → деление на 3 → 126 - 3 = = 123 → деление на 3 → 123 - 3 = = 120 → деление на 5 → 120 - 1 = ...121 → ни на 5, ни на 3 → 121 + 2 = 123

Ввод	Вывод	Комментарии
376	382 388 394 400 397 403 409 415 412 418 424 430 427 433 439 445 442 448 454 460 457 463 469 475 472 478 484 490 487 493 499 505 502 508 514 520 517 523 529 535 532 538 544 550 547 553 559 565 562 568 574 580 577 583 589 595 592 598 604 610 607 613 619 625 622 628 634 640 637 643 649 655 652 658 664 670 667 673 679 685 682 688 694 700 697 703 709 715 712 718	10 строк с 9 числами в каждой Введено число 376 → ни 5, ни 3 → 376 + 6 → = 382 → ни 5, ни 3 → 382 + 6 = 388 + 6 = 394 + 6 = 400 → делится на 5 → 400 - 3 = 397

Рекомендации и советы

Решите эту задачу самостоятельно, используя то, что вы узнали из предыдущих задач. Помните, что для каждой цифры вводимого числа необходимо определить отдельную переменную.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1058#4.

Глава 8.1. Подготовка к практическому экзамену — часть I

В этой главе мы рассмотрим несколько задач, уровень сложности которых соответствует уровню задач на практическом экзамене по основам программирования. Мы проверим и отработаем на практике все знания, полученные из этой книги и курса «Основы программирования».

Практический экзамен по «Основам программирования»

Курс «Основы программирования» завершается практическим экзаменом. В экзамен включено шесть задач, на решение которых отводится 4 часа. Каждая из экзаменационных задач будет посвящена одной из тем, изученных на курсе. Темы заданий следующие:

- задача с простыми вычислениями (без проверок);
- задача с одной проверкой;
- задача с более сложными проверками;
- задача с одним циклом;
- задача с вложенными циклами (рисование фигуры на консоли);
- задача с вложенными циклами и более сложной логикой.

Система онлайн-оценки (судейская)

Все экзамены и домашние задания автоматизированы с помощью судейской системы: https://judge.softuni.bg. Для каждого из заданий предусмотрены открытые (нулевые) тесты, кото-

рые помогут понять, что ожидается от задания, и исправить свои ошибки, а также конкурсные тесты, которые являются скрытыми и проверяют правильность работы над заданием. Доступ к судейской системе осуществляется с помощью учетной записи softuni.bg.

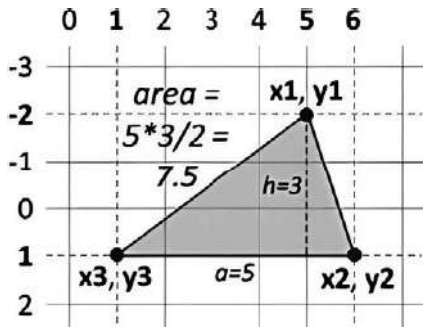
Как происходит тестирование в судейской системе? Вы загружаете исходный код и в меню под ним выбираете «запустить как программу на языке C++». Программа проверяется с помощью серии тестов, и за каждый успешный тест начисляются баллы.

Задачи с простыми вычислениями

В первой задаче практического экзамена по «Основам программирования» рассматриваются простые вычисления без проверок и циклов. Приведем несколько примеров.

Задача: площадь треугольника на плоскости

Треугольник на плоскости задается координатами трех его вершин. Сначала задается вершина (x1, y1). Затем задаются две другие вершины (x2, y2) и (x3, y3), которые лежат на общей горизонтальной прямой (то есть имеют одинаковые координаты Y). Напишите программу, которая вычисляет площадь треугольника по координатам трех его вершин.



Входные данные

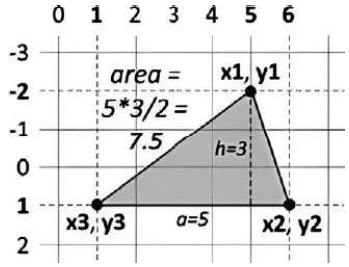
Консоль выдает шесть целых чисел (по одному на строку): x1, y1, x2, y2, x3, y3.

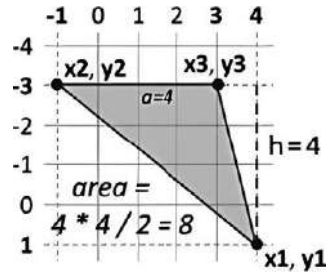
- Все вводимые числа находятся в диапазоне [-1000 ... 1000].
- Гарантируется, что y2 = y3.

Выходные данные

Выведите на консоль площадь треугольника.

Примеры ввода и вывода

Ввод	Вывод	Визуализация	Комментарии
5 -2 6 1 1 1	7.5		Сторона треугольника: a = 6 - 1 = 5 Высота треугольника: h = 1 - (-2) = 3 Площадь треугольника: S = a * h / 2 = = 5 * 3 / 2 = 7,5

Ввод	Вывод	Визуализация	Комментарии
4 1 -1 -3 3 -3	8		Сторона треугольника: a = 3 - (- 1) = 4 Высота треугольника: h = 1 - (-3) = 4 Площадь треугольника: S = a * h / 2 = 4 * 4 / 2 = 8

Рекомендации и советы

В задачах такого типа, где на вход передаются некоторые координаты, очень важно обращать внимание на порядок их передачи и правильно представлять, какие координаты и в каком порядке мы будем использовать. В данном случае на вход передаются координаты x1, y1, x2, y2, x3, y3, именно в таком порядке. Если мы не будем соблюдать эту последовательность, то решение станет неверным. Сначала пишем код, который считывает входные данные:

```
int x1, y1, x2, y2, x3, y3;
cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
```

Нам необходимо вычислить сторону и высоту треугольника. Из рисунков и из условия `y2 = y3` следует, что одна из сторон всегда параллельна горизонтальной оси. Это означает, что ее длина равна длине отрезка между координатами x2 и x3, который равен разности между большей и меньшей координатами. Аналогично можно вычислить и высоту. Она всегда будет равна разности y1 и y2

(или y3, так как они равны). Поскольку мы не знаем, будет ли x2 *всегда* больше x3, будет ли y1 ниже или выше стороны треугольника, мы будем использовать абсолютные значения разности, чтобы всегда получать положительные числа, поскольку отрезок не может иметь отрицательную длину. Также в начале программы необходимо включить библиотеку cmath для доступа к функции fabs(...):

```
int a = fabs(x2 - x3);
int h = fabs(y2 - y1);
```

Используем известную нам из школьного курса формулу для нахождения площади треугольника. Важно помнить, что, несмотря на то, что на входе мы получаем только целые числа, площадь не всегда будет целым числом. Поэтому для нее мы используем переменную типа double. Нам также необходимо преобразовать правую часть уравнения, поскольку если в качестве параметров в уравнение передаются целые числа, то и результат будет целым числом:

```
double s = (double)h*a / 2;
```

Осталось только вывести площадь на консоль:

```
cout << s << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#0.

Задача: перемещение кирпичей

Следующий пример экзаменационной задачи — рассчитать, сколько тележек необходимо для перемещения заданного количества кирпичей (при условии, что тележка имеет ограниченную грузоподъемность).

Описание задачи

Строительные рабочие должны переместить в общей сложности x кирпичей. Число рабочих — w, работают они одновременно. Они перевозят кирпичи на тележках, каждая из которых вмещает m кирпичей. Напишите программу, которая считывает целые числа x, w и m и вычисляет, *какое минимальное количество рейсов* нужно сделать рабочим, чтобы перевезти кирпичи.

Входные данные

Вводятся *три целых числа*, по одному на строку:

- Количество кирпичей `x` считывается из первой строки.
- Количество рабочих `w` считывается из второй строки.
- Вместимость тележки `m` считывается из третьей строки.

Все вводимые числа — целые в диапазоне [1 ... 1000].

Выходные данные

Выведите на консоль минимальное количество рейсов, необходимых для транспортировки кирпичей.

Ввод	Вывод	Комментарии
120 2 30	2	У нас есть 2 рабочих, каждый из которых перевозит по 30 кирпичей за рейс. Всего рабочие перевозят 60 кирпичей за рейс. Чтобы перевезти 120 кирпичей, необходимо ровно 2 рейса.

Ввод	Вывод	Комментарии
355 3 10	12	У нас есть 3 рабочих, каждый из которых перевозит по 10 кирпичей за рейс. Всего рабочие перевозят 30 кирпичей за рейс. Чтобы перевезти 355 кирпичей, необходимо ровно 12 рейсов: за 11 полных рейсов перевозят 330 кирпичей, а в последний 12-й рейс перевозят последние 25 кирпичей.

Ввод	Вывод	Комментарии
5 12 30	1	У нас есть 5 рабочих, каждый из которых перевозит по 30 кирпичей за рейс. В общей сложности рабочие перевозят 150 кирпичей за рейс. Для того чтобы перевезти 5 кирпичей, достаточно 1 рейса (хотя и неполного, всего с 5 кирпичами).

Чтение входных данных

Производится стандартно, нужно лишь внимательно следить за последовательностью считывания данных.

```
int x, w, m;
cin >> x >> w >> m;
```

Расчет количества кирпичей на один рейс

Подсчитываем, сколько кирпичей перевозят рабочие за рейс:

```
int bricksInOneCourse = w * m;
```

Деление общего количества кирпичей на количество кирпичей, перевозимых за один рейс, дает количество рейсов, необходимых для их перевозки. Обратите внимание, что при делении на целые числа остаток игнорируется и всегда округляется в меньшую сторону. Чтобы избежать этого, преобразуем правую часть уравнения в `double` и воспользуемся функцией `ceil(...)` из библиотеки `cmath`, чтобы всегда округлять результат в большую сторону. Если кирпичи можно перевезти *целым количеством рейсов*, деление вернет целое число, и округлять будет нечего. Соответственно, если нет, то результатом деления будет точное число рейсов, но с десятичной дробью. Десятичная часть округляется в большую сторону, чтобы получить требуемый рейс для оставшихся кирпичей:

```
double totalCourses = ceil((double)x / bricksInOneCourse);
```

Наконец, мы выводим результат на консоль:

```
cout << totalCourses << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#1.

Задачи с однократной проверкой

Вторая задача практического экзамена по «Основам программирования» охватывает *условные конструкции* и *простые вычисления*. Приведем несколько примеров.

Задача: точка на отрезке

На горизонтальной прямой находится горизонтальный отрезок, заданный координатами x двух его концов: первого и второго. На этой же горизонтальной прямой расположена точка, заданная координатой x. Напишите программу, которая проверяет, находится ли точка внутри или вне отрезка, и вычисляет расстояние до ближайшего конца отрезка.

Входные данные

Консоль считывает три целых числа (по одному на строку):

- На первой строке расположено число `first` — один конец отрезка.

- На второй строке вводится `second` — другой конец отрезка.
- В третьей строке указываются координаты point — местоположение точки.

Все вводимые числа являются целыми и находятся в диапазоне [-1000 ... 1000].

Выходные данные

Выведите результат на консоль:

- В первой строке выведите «`in`» или «`out`» — находится ли точка на отрезке или вне его.
- Во второй строке выведите расстояние от точки до ближайшего конца отрезка.

Примеры ввода и вывода

Ввод	Вывод	Визуализация
10 5 7	in 2	

Ввод	Вывод	Визуализация
8 10 5	out 3	

Ввод	Вывод	Визуализация
1 -2 3	out 2	

Чтение входных данных

Считываем входные данные:

```
int first, second, point;
cin >> first >> second >> point;
```

Вычисление минимального расстояния до ближайшего конца

Так как мы не знаем, какая точка находится слева, а какая справа, мы создадим две переменные, которые будут обозначать это. Так как левая точка всегда имеет меньшую координату `x`, то для ее нахождения мы будем использовать функцию `min(...)`. Для того

чтобы использовать эту функцию, нам необходимо включить в начало программы библиотеку `algorithm`. Соответственно, правым всегда является тот, у которого больше координата `x`, и для его нахождения мы будем использовать функцию `max(...)`. Также найдем расстояние от точки `x` до двух точек. Так как мы не знаем их положения относительно друг друга, то для получения положительного результата воспользуемся функцией `fabs(...)` из библиотеки `cmath`:

```
int left = min(first, second);
int right = max(first, second);

int distanceLeft = fabs(left - point);
int distanceRight = fabs(right - point);
```

Найдем меньшее из двух расстояний с помощью функции `min(...)`:

```
int minDistance = min(distanceLeft, distanceRight);
```

Определение того, находится ли точка на отрезке или вне его

Остается выяснить, находится ли точка на отрезке. Точка будет лежать на отрезке, если она совпадает с двумя другими точками или ее координата `x` лежит между ними. В противном случае точка лежит вне отрезка. После проверки мы выводим одно из двух сообщений («`in`» или «`out`») в зависимости от того, какое условие выполнено:

```
if (point >= left && point <= right) {
    cout << "in" << endl;
}
else {
    cout << "out" << endl;
}
```

Наконец, мы выводим найденное ранее расстояние:

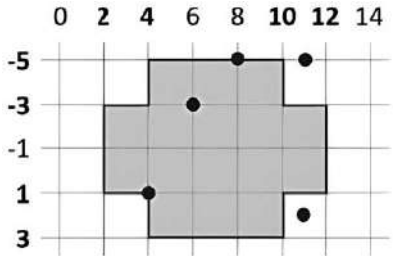
```
cout << minDistance << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#2.

Задача: точка на рисунке

Напишите программу, которая проверяет, находится ли точка (с координатами x и y) внутри или вне следующей фигуры на рисунке.



Входные данные

Консоль считывает *два целых числа* (по одному на строку): x и y.

Все вводимые числа являются целыми числами в диапазоне [-1000 ... 1000].

Выходные данные

Выведите на консоль «`in`» или «`out`» — находится ли точка внутри или вне фигуры (нахождение точки на контуре считается нахождением внутри).

Ввод	Вывод
8 -5	in

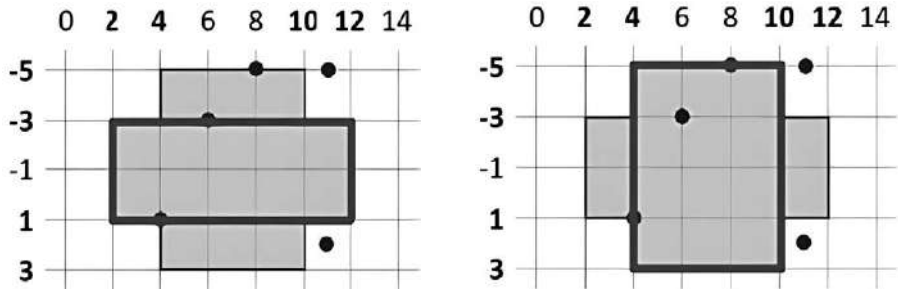
Ввод	Вывод
6 -3	in

Ввод	Вывод
11 -5	out

Ввод	Вывод
11 2	out

Рекомендации и советы

Чтобы узнать, находится ли точка на фигуре, разделим фигуру на два прямоугольника:



Для того чтобы точка находилась на фигуре, достаточно, чтобы она находилась в одном из них (в любом или обоих одновременно).

Определение местоположения точки

Считываем входные данные с консоли:

```
int x, y;
cin >> x >> y;
```

Создаем две переменные, которые будут указывать, находится ли точка в одном из прямоугольников:

```
bool pointInRect1 = x >= 2 && x <= 12 && y >= -3 && y < = 1;
bool pointInRect2 = x >= 4 && x <= 10 && y >= -5 && y <= 3;
```

При выводе сообщения на экран мы будем проверять, приняла ли какая-либо из переменных значение `true`. Для нахождения точки на рисунке достаточно, чтобы только одна из них была `true`:

```
if (pointInRect1 || pointInRect2) {
    cout << "in" << endl;
}
else {
    cout << "out" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#3.

Задачи с более сложными проверками

Третья задача практического экзамена по «Основам программирования» включает в себя несколько вложенных проверок в сочетании с простыми вычислениями. Приведем несколько примеров:

Задача: дата через пять дней

Следующий пример экзаменационного задания — вычислить дату через пять дней после заданного числа (день + месяц), учитывая, что оно может быть в следующем месяце.

Описание задачи

Есть два числа `d` (день) и `m` (месяц), которые образуют дату. Напишите программу, выводящую дату, которая будет через *пять дней после определенной даты*. Например, через пять дней после 28.03 будет дата 2.04.

Мы предполагаем, что месяцы апрель, июнь, сентябрь и ноябрь имеют 30 дней, февраль — 28 дней, а остальные — 31 день. Месяцы должны выводиться с ведущим нулем, если они содержат одну цифру (например 01, 08).

Входные данные

Входные данные считываются с консоли и состоят из двух строк:

- В 1-й строке мы читаем целое число `d` в диапазоне [1 ... 31] — день. Номер дня не должен превышать количество дней в этом месяце (например, 28 для февраля).
- Во 2-й строке мы читаем целое число `m` в диапазоне [1 ... 12] — месяц. Месяц 1 — это январь, месяц 2 — это февраль, ..., месяц 12 — это декабрь. Месяц может содержать ведущий ноль (например, апрель может быть записан как 4 или 04).

Выходные данные

Выведите одну строку с датой через пять дней в формате **день.месяц**. Месяц должен быть двузначным числом с ведущим нулем (если необходимо). День должен быть выведен без ведущего нуля.

Примеры ввода и вывода

Ввод	Вывод
28 03	2.04

Ввод	Вывод
27 12	1.01

Ввод	Вывод
25 1	30.01

Ввод	Вывод
26 02	3.03

Чтение и обработка входных данных

Считываем входные данные с консоли:

```
int d, m;
cin >> d >> m;
```

Чтобы было проще выполнять проверки, создадим переменную, которая будет содержать количество дней, входящих в заданный нами месяц:

```
int daysInMonth = 31;
if (m == 2) {
    daysInMonth = 28;
}
if (m == 4 || m == 6 || m == 9 || m == 11) {
    daysInMonth = 30;
}
```

Увеличиваем день на 5:

```
d += 5;
```

Мы проверяем, не стал ли день больше, чем количество дней в месяце. Если да, то из полученного дня нужно вычесть дни меся-

ца, чтобы получить наш день, которому соответствует день следующего месяца:

```
if (d > daysInMonth) {
    d -= daysInMonth;
}
```

Как только мы перешли в следующий месяц, это должно быть отмечено увеличением первоначально установленного месяца на 1. Необходимо проверить, не стало ли оно больше 12, и если стало, то скорректировать. Поскольку при увеличении на 5 дней мы не можем пропустить более одного месяца, достаточно следующей проверки:

```
if (d > daysInMonth) {
    d -= daysInMonth;
    m++;

    if (m > 12) {
        m = 1;
    }
}
```

Осталось только вывести результат на консоль. Важно правильно отформатировать вывод, чтобы отобразить ведущий ноль в первых девяти месяцах:

```
if (m <= 9) {
    cout << d << ".0" << m << endl;
}
else {
    cout << d << "." << m << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#4.

Задача: сумма трех чисел

Даны три целых числа. Напишите программу, которая проверяет, *равна ли сумма двух чисел третьему*. Например, если числа 3, 5 и 2, то сумма двух чисел равна третьему: 2 + 3 = 5.

Входные данные

С консоли считываются три целых числа, по одному в строке. Числа находятся в диапазоне [1 ... 1000].

Выходные данные

- Выведите на консоль текстовую строку с решением задачи в формате «`a + b = c`», где `a`, `b` и `c` — три входных числа, `a` `a ≤ b`.
- Если задача не имеет решения, выведите «`No`» на консоль.

Примеры ввода и вывода

Ввод	Вывод
3 5 2	`2 + 3 = 5`

Ввод	Вывод
2 2 4	`2 + 2 = 4`

Ввод	Вывод
1 1 5	No

Ввод	Вывод
2 6 3	No

Чтение входных данных

Получаем данные из консоли:

```
int a, b, c;
cin >> a >> b >> c;
```

Составление шаблона для решения

Мы можем проверить, равна ли сумма пары чисел третьему числу. У нас есть три возможных случая:

- `a + b = c`
- `a + c = b`
- `b + c = a`

Мы напишем шаблон, который впоследствии будет дополнен необходимым кодом. Если ни одно из трех вышеперечисленных условий не выполняется, мы заставим нашу программу вывести на экран «No»:

```
if (a + b == c) {
    // TODO
}
else if (a + c == b) {
    // TODO
} else if (b + c == a) {
    // TODO
}
else {
    cout << "No" << endl;
}
```

Написание кода в шаблоне

Теперь необходимо определить, в каком порядке оба сумматора будут отображаться на выходе программы. Для этого мы сделаем вложенное условие, которое будет проверять, какое из двух чисел больше. В первом случае это будет выглядеть следующим образом:

```
if (a + b == c) {
    if (a > b) {
        cout << b << " + " << a << " = " << c << endl;
    }
}
```

Аналогичным образом добавим два других случая. Полный код проверок и вывод программы будут выглядеть следующим образом:

```
if (a + b == c) {
    if (b < a) {
        cout << b << " + " << a << " = " << c << endl;
    }
    else {
        cout << a << " + " << b << " = " << c << endl;
    }
}
else if (a + c == b) {
```

```
    if (a < c) {
        cout << a << " + " << c << " = " << b << endl;
    }
    else {
        cout << c << " + " << a << " = " << b << endl;
    }
} else if (b + c == a) {
    if (b < c) {
        cout << b << " + " << c << " = " << a << endl;
    }
    else {
        cout << c << " + " << b << " = " « a << endl;
    }
}
else {
    cout << "No" << endl
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#5.

Задачи с одним циклом

Четвертая задача практического экзамена по основам программирования включает в себя один цикл с простой логикой. Экзаменующиеся должны знать, как работать с простыми циклами и как реализовать простую логику программы с помощью циклов, как считывать и обрабатывать последовательность чисел с консоли и как выполнять вычисления и проверки.

Давайте решим несколько примеров задач с помощью простых циклов.

Задача: суммы с шагом 3

В следующем примере экзаменационной задачи нужно вычислить три суммы, удерживая числа из заданной последовательности, оставаясь на определенных позициях с шагом 3.

Описание задачи

Дано `n` целых чисел: `a1`, `a2`, …, `an`. Вычислите суммы:

- `sum1 = a1 + a4 + a7 + ...` (числа складываются, начиная с первого, с шагом 3).
- `sum2 = a2 + a5 + a8 + ...` (числа складываются, начиная со второго, с шагом 3).
- `sum3 = a3 + a6 + a9 + ...` (числа складываются, начиная с третьего, с шагом 3).

Входные данные

Входные данные считываются с консоли. В первой строке находится целое число `n` (`0 ≤ n ≤ 1000`). В следующих `n` строках нам дается `n` целых чисел в диапазоне [`-1000 ... 1000`]: `a1`, `a2`, ..., `an`.

Выходные данные

На консоль мы должны вывести три строки, содержащие три суммы, в формате, как в примере.

Примеры ввода и вывода

Ввод	Вывод
4 7 -2 6 12	sum1 = 19 sum2 = -2 sum3 = 6

Ввод	Вывод
5 3 5 2 7 8	sum1 = 10 sum2 = 13 sum3 = 2

Ввод	Вывод
2 3 5	sum1 = 3 sum2 = 5 sum3 = 0

Ввод	Вывод
1 5	sum1 = 5 sum2 = 0 sum3 = 0

Чтение входных данных

Мы возьмем количество чисел из консоли и объявим начальные значения трех сумм:

```
int n;
cin >> n;

int sum1 = 0;
int sum2 = 0;
int sum3 = 0;
```

Поскольку мы заранее не знаем, сколько чисел будем обрабатывать, то будем считывать их по одному в цикл, который будет повторяться n раз, и обрабатывать их в теле цикла:

```
for (int i = 0; i < n; i++) {
    int a;
    cin >> a;
    // TODO
}
```

Чтобы определить, к какой из трех сумм нужно прибавить число, разделим его порядковый номер на три и используем остаток. Для того чтобы определить, на каком порядковом номере цикла мы находимся, будем использовать переменную `i`, которая отслеживает количество оборотов цикла. Если результат `i % 3` равен нулю, то это означает, что мы прибавляем это число к первой сумме, когда оно равно 1 — ко второй и, соответственно, когда оно равно 2 — к третьей:

```
for (int i = 0; i < n; i++) {
    int a;
    cin >> a;

    if (i % 3 == 0) {
```

```
        sum1 += a;
    }
    else if (i % 3 == 1) {
        sum2 += a;
    }
    else if (i % 3 == 2) {
        sum3 += a;
    }
}
```

Наконец, мы выведем результат на консоль в формате, требуемом условием:

```
cout << "sum1 = " << sum1 << endl;
cout << "sum2 = " << sum2 << endl;
cout << "sum3 = " << sum3 << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#6.

Задача: последовательность возрастающих элементов

Следующая примерная экзаменационная задача посвящена нахождению самой длинной возрастающей подпоследовательности заданной последовательности целых чисел.

Описание задачи

Дана серия из `n` чисел: `a1`, `a2`, ..., `an`. Вычислите длину самой длинной возрастающей последовательности последовательных элементов в этом ряду чисел.

Входные данные

Входные данные считываются с консоли.

В первой строке записано целое число `n` (`0 ≤ n ≤1000`).

В следующих `n` строках нам дается `n` целых чисел в диапазоне [`-1000 ... 1000`]: `a1`, `a2`, ..., `an`.

Выходные данные

На консоль мы должны вывести одно число — длину самой длинной возрастающей последовательности.

Примеры ввода и вывода

Ввод	Вывод
3 5 2 4	2

Ввод	Вывод
4 2 8 7 6	2

Ввод	Вывод
4 5 6 7 8	2

Ввод	Вывод
4 1 2 4 4	3

Чтение входных данных и создание рабочих переменных

Чтобы решить эту задачу, нам нужно мыслить немного более алгоритмично. Нам дана *последовательность чисел*, и мы должны проверить, будет ли *каждое последующее больше предыдущего*, и если да, то посчитать, насколько длинной будет последова-

тельность, в которой это условие выполняется. Затем нужно найти, какая из этих последовательностей *самая длинная*. Для этого создадим несколько переменных, которые будем использовать при решении задачи:

```
int n;
cin >> n;
int countCurrentLongest = 0;
int countLongest = 0;
int aPrev = 0;
int a = 0;
```

Примечание: Поскольку переменные имеют одинаковый тип (`int`), мы могли бы объявить и инициализировать их в одной строке, разделив запятой:

Переменная `n` — это количество чисел, которые мы получим из консоли. В переменной `countCurrentLongest` мы будем хранить количество элементов в увеличивающейся последовательности, которую мы в данный момент считаем. Например, для последовательности: 5, 6, 1, 2, 3 `countCurrentLongest` будет равным 2, когда мы достигнем второго элемента счета (5, 6, 1, 2, 3), и станет равен 3, когда мы достигнем последнего элемента (5, 6, 1, 2, 3), так как возрастающий ряд 1, 2, 3 имеет 3 элемента. Мы будем использовать `countLongest`, чтобы сохранить самый длинный возрастающий ряд.

Другие переменные, которые мы будем использовать, это `a`, число, на котором мы находимся в данный момент, и `aPrev`, предыдущее число, которое мы будем сравнивать с `a`, чтобы узнать, растет ли ряд.

Определение возрастающей последовательности

Мы начинаем итерацию по числам и проверяем больше ли текущее число `a` предыдущего `aPrev`. Если это условие выполняется, то последовательность возрастает, и нам необходимо увеличить ее счет на 1. Мы сохраняем это в переменной, отслеживающей длину ряда, в котором мы находимся в данный момент, а именно `countCurrentLongest`. Если число `a` не больше предыдущего, то это означает, что начинается новая подстрока, и нужно начинать отсчет с 1.

Наконец, после всех проверок aPrev становится тем числом, которое мы используем в данный момент, и мы начинаем цикл с начала, со следующего введенного a.

Приведем пример реализации описанного алгоритма:

```
for (int i = 0; i < n; i++) {
    cin >> a;

    if (a > aPrev) {
        countCurrentLongest++;
    }
    else {
        countCurrentLongest = 1;
    }
    aPrev = a;
}
```

Поиск и вывод самой длинной последовательности

Осталось выяснить, какой из всех рядов является самым длинным. Для этого в цикле мы будем проверять, стал ли ряд, в котором мы находимся в данный момент, длиннее, чем длина самого длинного из найденных на данный момент рядов. Весь цикл будет выглядеть следующим образом:

```
for (int i = 0; i < n; i++) {
    cin >> a;

    if (a > aPrev) {
        countCurrentLongest++;
    }
    else {
        countCurrentLongest = 1;
    }

    if (countCurrentLongest > countLongest) {
        countLongest = countCurrentLongest;
    }
    aPrev = a;
}
```

Наконец, мы выводим длину самого длинного найденного ряда:

```
cout << countLongest << endl;
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#7.

Задачи по рисованию фигур на консоли

Пятая задача практического экзамена по основам программирования требует использования одного или нескольких *вложенных циклов* для *рисования фигуры* на консоли. Экзаменующиеся должны уметь логически мыслить и строить простые алгоритмы с использованием вычислений, проверок и вложенных циклов для решения задач, то есть мыслить алгоритмически. Эта задача больше посвящена алгоритмическому мышлению, чем кодированию и циклам. Давайте решим несколько примеров задач с вложенными циклами (и рисованием фигур).

Задача: идеальный бриллиант

Следующая примерная экзаменационная задача посвящена использованию вложенных циклов и вычислений для вывода на консоль ромба заданного размера, как показано ниже в примерах.

Описание задачи

Напишите программу, которая считывает целое число `n` с консоли и рисует *бриллиант* размером `n` (см. примеры ниже).

Входные данные

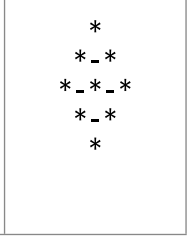
На вход подается целое число `n` в диапазоне [1 ... 1000].

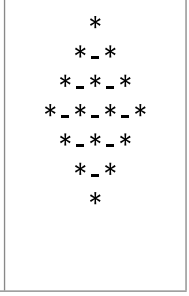
Выходные данные

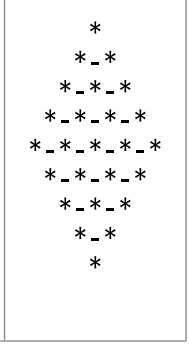
На консоль должен быть выведен бриллиант, как показано в примерах ниже.

Примеры ввода и вывода

Ввод	Вывод
2	* *-* *

Ввод	Вывод
3	

Ввод	Вывод
4	

Ввод	Вывод
5	

Рекомендации и советы

В заданиях на рисование фигур самое важное, что нужно учитывать, это *последовательность*, в которой мы будем рисовать. Какие элементы *повторяются* и *с каким шагом*? Мы хорошо видим, что верхняя и нижняя части ромба *одинаковы*. Самый простой спо-

соб решить задачу — создать *цикл*, рисующий *верхнюю часть*, а затем еще один *цикл*, рисующий *нижнюю часть* (противоположную верхней).

Чтение входных данных

Считаем число `n` с консоли:

```
int n;
cin >> n;
```

Вывод на экран верхней части бриллианта

Начинаем выводить верхнюю половину нашего ромба-бриллианта. Хорошо видно, что каждая строка начинается с нескольких пробелов и *. Если присмотреться, то можно заметить, что пробелы всегда равны `n - количеству строк` (первая строка — `n-1`, вторая — `n-2` и так далее). Начнем с того, что нарисуем количество пробелов, а также первую звездочку. В конце строки набираем `cout << endl;` для перехода на новую строку. Обратите внимание, что мы начинаем считать с 1, а не с 0. После этого нам остается только несколько раз добавить `-*`, чтобы закончить строку.

Приведем фрагмент кода для рисования вершины ромба:

```
for (int i = 1; i < n; i++) {
    for (int j = 0; j < n - i; j++) {
        cout << ' ';
    }
    cout << "*";
    //TODO: Нарисуйте остальную часть линии
    cout << endl;
}
```

Осталось дописать в каждую строку необходимое количество элементов `-*`. В каждой строке необходимо добавить `i - 1` таких элементов (в первой 1-1 -> 0, во второй -> 1 и т. д.).

Приведем полный код для построения вершины ромба:

```
for (int i = 1; i < n; i++) {
    for (int j = 0; j < n - i; j++) {
        cout << ' ';
    }
```

```
    cout << "*";

    for (int j = 0; j < i - 1; j++) {
       cout << "-*";
    }
    cout << endl;
}
```

Вывод на экран нижней части бриллианта

Чтобы нарисовать нижнюю часть, нам нужно перевернуть верхнюю часть в обратную сторону. Считать будем от `n - 1`, так как если начать с `n`, то мы дважды нарисуем средний ряд. Не забывайте менять шаг с `++` на `--`.

Здесь приведен код для рисования нижней части:

```
for (int i = n - 1; i >= 1; i--) {
    for (int j = 0; j < n - i; j++) {
        cout << ' ';
    }
    cout << "*";

    for (int j = 0; j < i - 1; j++) {
        cout << "-*";
    }
    cout << endl;
}
```

Осталось только собрать всю программу: сначала прочитать входные данные, вывести верхнюю часть ромба, а затем нижнюю.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#8.

Задача: прямоугольник со звездочками в центре

Следующая примерная экзаменационная задача посвящена использованию вложенных циклов и вычислений для вывода на консоль прямоугольника заданного размера со звездочками в середине.

Описание задачи

Напишите программу, которая считывает с консоли целое число n и рисует *прямоугольник* размером n *с двумя звездочками в центре*, как показано в примерах ниже.

Входные и выходные данные

На вход подается целое число n в диапазоне [2 ... 1000].

Прямоугольник должен быть выведен на консоль, как показано в примерах ниже.

Примеры ввода и вывода

Ввод	Вывод
2	%%%% %**% %%%%

Ввод	Вывод
3	%%%%%% % % % ** % % % %%%%%%

Ввод	Вывод
4	%%%%%%%% % % % ** % %%%%%%%%%%

Ввод	Вывод
5	%%%%%%%%%% % % % % % ** % % % % % %%%%%%%%%%

Чтение входных данных

Считываем входные данные:

```
int n;
cin >> n;
```

Вывод на экран первой и последней строк

Первое, что легко заметить, это то, что первая и последняя строки содержат `2 * n` символов %. Начнем с этого, а затем нарисуем середину прямоугольника:

```
for (int i = 0; i < n * 2; i++) {
    cout << "%";
}

// TODO: Отрисовать среднюю часть прямоугольника

for (int i = 0; i < n * 2; i++) {
    cout << "%";
}
```

Вывод средних строк

Из приведенных примеров видно, что середина фигуры всегда имеет нечетное количество рядов. Обратите внимание, что при задании четного числа количество рядов равно предыдущему нечетному числу (2 -> 1, 4 -> 3 и т. д.).

Мы создадим переменную, которая будет представлять количество рядов нашего прямоугольника, и изменим ее, если число `n` будет четным:

```
int numRows;
if (n % 2 == 0) {
    numRows = n - 1;
}
else {
    numRows = n;
}
```

Далее мы нарисуем прямоугольник без звездочек. Каждая строка имеет символ % для начала и конца и `2 * n - 2` пустых места между ними (ширина равна `2 * n` и мы отнимаем 2 для двух символов

процента в конце). Не забудьте перенести код последней строки после цикла:

```
cout << endl;
for (int i = 0; i < numRows; i++) {
    cout << "%";
    //T000: Разместить звездочки
cout << "%" << endl;
}
```

Мы можем запустить и протестировать код до этого места. Все, кроме двух звездочек в середине, должно работать корректно.

Добавление звездочек в центр прямоугольника

Теперь нам нужно добавить звездочки в тело цикла. Мы проверим, находимся ли мы в среднем ряду. Если находимся, то нарисуем ряд со звездочками, если нет, то обычный ряд. Ряд со звездочками имеет `n - 2` пробела (`n` — половина длины, и мы вычитаем звездочку и знак процента), две звездочки и снова `n - 2` пробела. Два знака процента в начале и конце строки мы оставляем без проверки:

```
cout << endl;
for (int i = 0; i < numRows; i++) {
    cout << "%";

    if (i == numRows / 2) {
        for (int j = 0; j < n - 2; j++) {
            cout << ' ';
        }
        cout << "**";

        for (int j = 0; j < n - 2; j++) {
            cout << ' ';
        }
    }
    else {
        for (int j = 0; j < n * 2 - 2; j++) {
            cout << ' ';
        }
    }
    cout << "%" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1370#9.

Задачи на использование вложенных циклов с более сложной логикой

Последняя (шестая) задача практического экзамена по основам программирования требует использования *нескольких вложенных циклов и более сложной логики внутри них*. Задачи проверяют способность экзаменующихся мыслить алгоритмически и решать нетривиальные задачи кодирования, требующие вложенных циклов с более сложной логикой и вычислениями, а также чтения с консоли и вывода данных на консоль.

Давайте решим несколько примеров задач с вложенными циклами и более сложной логикой.

Задача: сочетания из четырех нарастающих чисел

Следующая примерная экзаменационная задача посвящена использованию вложенных циклов и программной логики для генерации всех возможных комбинаций из четырех возрастающих чисел в заданном диапазоне.

Описание задачи

Для заданной пары чисел `a` и `b` найдите все четыре числа `n1`, `n2`, `n3`, `n4`, для которых `n1 ≤ n2 < n3 < n4 ≤ b`.

В комбинаторике такой выбор подмножества из заданного множества (или диапазона) называется сочетанием (https://en.wikipedia.org/wiki/Combination), поэтому суть задачи состоит в том, чтобы *сгенерировать все сочетания из четырех элементов из заданного диапазона целых чисел*.

Ввод	Вывод
3 7	3 4 5 6 3 4 5 7 3 4 6 7 3 5 6 7 4 5 6 7

Ввод	Вывод
5 7	No

Ввод	Вывод
10 13	10 11 12 13

Входные и выходные данные

Входные данные содержат два целых числа `a` и `b` в диапазоне [0 ... 1000], по одному в строке.

Вывод содержит все *числа в партиях по четыре*, в порядке возрастания, по одному в строке.

Чтение входных данных

Будем считывать входные данные из консоли и создадим дополнительную переменную `count`, которая будет отслеживать *существующие диапазоны чисел*:

```
int a, b;
cin >> a >> b;
int count = 0;
```

Реализация с двумя числами

Нам будет проще всего решить задачу, если мы логически разделим ее на части. Если нам нужно вывести все ряды от числа между `a` и `b`, мы сделаем это с помощью одного *цикла*, который берет все числа от `a` до `b`. Давайте подумаем, как это сделать с сериями из двух чисел. Ответ прост — будем использовать *вложенные циклы*:

```
for (int i = a; i <= b; i++)
    for (int j = i + 1; j <= b; j++) {
        cout << i << " " << j << endl;
    }
}
```

Мы можем протестировать недописанную программу, чтобы проверить, правильно ли она работает до сих пор. Она должна вывести все пары чисел `i`, `j`, для которых `i ≤ j`.

Поскольку каждое последующее число в ряду должно быть больше предыдущего, второй цикл будет выполняться, начиная с `i + 1` (следующего большего числа). Соответственно, если нет ряда из двух возрастающих чисел (`a` и `b` равны), то второй цикл не выполнится, и на консоль ничего не будет выведено.

Реализация с четырьмя числами

Аналогичным образом остается реализовать вложенные циклы для четырех чисел.

Чтобы узнать, существует ли такая последовательность, добавим инкремент счетчика (`count`), который мы инициализировали в начале:

```
for (int i = a; i <= b; i++) {
    for (int j = i + 1; j <= b; j++) {
        for (int k = j + 1; k <= b; k++) {
            for (int l = k + 1; l <= b; l++) {
               cout << i << " " << j << " " « k « " " << l << endl;
               count++;
            }
        }
    }
}
```

Наконец, мы проверим, равен ли счетчик 0, и в случае положительного ответа выведем на консоль сообщение «`No`»:

```
if (count == 0) {
    cout << "No" << endl;
}
```

Проверка в судейской системе

Проверьте пример программы: https://judge.softuni.bg/Contests/Practice/Index/1370#10.

Задача: создание прямоугольников

Следующая примерная экзаменационная задача заключается в использовании вложенных циклов и программной логики для генерации всех возможных прямоугольников, имеющих цело-

численные координаты в заданном диапазоне и заданную минимальную площадь.

Описание задачи

По заданному числу `n` и минимальной площади `m` сгенерируйте все возможные прямоугольники с целочисленными координатами в диапазоне `[-n...n]` с площадью не менее `m`. Сгенерированные прямоугольники должны быть выведены в следующем формате:

- `(слева (left), сверху (top)) (справа (right), снизу (bottom)) -> площадь (area)`

Прямоугольники задаются с помощью левого верхнего и правого нижнего угла. Действуют следующие неравенства:

- `-n ≤ left < right ≤ n`
- `-n ≤ top < bottom ≤ n`

Примеры ввода и вывода

Ввод	Вывод
1 2	(-1, -1) (0, 1) -> 2 (-1, -1) (1, 0) -> 2 (-1, -1) (1, 1) -> 4 (-1, 0) (1, 1) -> 2 (0, -1) (1, 1) -> 2

Ввод	Вывод
2 17	No

Ввод	Вывод
3 36	(-3, -3) (3, 3) -> 36

Входные данные

С консоли вводятся два числа, по одному на строку:

- Целое число `n` в диапазоне [1 ... 100] — задает минимальную и максимальную координаты вершины.
- Целое число `m` в диапазоне [0 ... 50 000] — задает минимальную площадь генерируемых прямоугольников.

Выходные данные

- Описанные прямоугольники должны быть выведены на консоль в таком формате, как показано в примерах ниже.
- Если для указанных `n` и `m` нет прямоугольников, выведите «`No`».
- Порядок прямоугольников в выводе неважен, реализуйте его по своему усмотрению.

Чтение входных данных

Считайте входные данные из консоли. Также создайте *счетчик*, который будет хранить количество найденных прямоугольников:

```
int n, m;
cin >> n >> m;
int count = 0;
```

Примерная идея для решения

Очень важно уметь представить себе задачу, прежде чем приступать к ее решению. В нашем случае требуется найти прямоугольники в системе координат. Известно, что левая точка всегда будет иметь координату `x` меньшую, чем правая. Соответственно, верхняя точка всегда будет иметь координату `y` меньшую, чем нижняя.

Чтобы найти все прямоугольники, нужно выполнить цикл, аналогичный циклу в предыдущей задаче, но на этот раз не каждый последующий цикл будет начинаться со следующего числа, так как некоторые координаты могут быть равны (например, `left` и `top`):

```
for (int left = -n; left < n; left++) {
    for (int top = -n; top < n; top++) {
        for (int right = left + 1; right <= n; right++) {
            for (int bottom = top + 1; bottom <= n; bottom++) {
                //TODO
            }
        }
    }
}
```

Вычисление площади прямоугольника и вывод результатов на экран

С помощью переменных `left` и `right` мы будем отслеживать горизонтальные координаты, а с помощью `top` и `bottom` — вертикальные.

Важно знать, какие это координаты, чтобы правильно вычислить стороны прямоугольника. Теперь нам необходимо найти сторону прямоугольника и проверить, больше или равна ли она `m`. Одна из сторон будет представлять собой разность между `left` и `right`, а другая — разность между `top` и `bottom`.

Поскольку координаты могут поменяться местами, мы будем использовать абсолютные значения. Снова добавим счетчик в цикл, считая только выписанные прямоугольники. Важно заметить, что порядок записи — `left`, `top`, `right`, `bottom`, поскольку именно такой порядок указан в условии:

```
for (int left = -n; left < n; left++) {
    for (int top = -n; top < n; top++) {
        for (int right = left + 1; right <= n; right**) {
           for (int bottom = top + 1; bottom <= n; bottom++) {
             int area = fabs(right - left) * fabs(bottom - top);

             if (area >= m) {
                cout << "(" << left << ", " << top << ")" << "
                  (" << right << ", " << bottom << ") -> " <<
                  area << endl;
                count++;
}
```

Наконец, мы выводим «No», если таких прямоугольников не существует:

```
if (count == 0) {
    cout << "No" << endl;
}
```

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1370#11.

Глава 8.2. Подготовка к практическому экзамену — часть II

В этой главе мы рассмотрим задания практического экзамена по основам программирования, проведенного в SoftUni ранее. Задачи дадут вам представление о том, что вас может ожидать на вступительном экзамене по программированию. Экзамен охватывает материал, изученный в данной книге и на курсе «Основы программирования» в SoftUni (https://softuni.org/curriculum).

Типы экзаменационных задач

Традиционно вступительный экзамен в SoftUni состоит из шести практических задач по программированию, перечисленных ниже:

- задача с *простыми вычислениями* (без условий);
- задача с *простым условием* (простые проверки);
- задача с *более сложными условиями* (вложенные проверки и множественные проверки);
- задача с *простым циклом* (например, итерация от `1` до `N`);
- задача с *вложенными циклами* (например, рисование 2D-фигуры на консоли);
- задача с *вложенными циклами и более сложной логикой* (циклы и проверки вместе).

Давайте рассмотрим *реальную экзаменационную тему*: шесть экзаменационных задач и их решения.

Задача: расстояние

Напишите программу, которая вычисляет, какое *расстояние проехал автомобиль* (в километрах), если известны *начальная скорость* (км/ч), *начальный отрезок времени* в минутах, затем *скорость увеличивается на 10 %, второй отрезок времени*, затем *скорость уменьшается на 5 %*, и *время до конца поездки*. Чтобы рассчитать расстояние, нужно *перевести минуты в часы* (например, 70 минут = 1,1666 часа).

Входные данные

Входные данные поступают с консоли и состоят из четырех строк:

- Начальная скорость в км/ч — целое число в диапазоне [1 ... 300].
- Первый временной интервал (таймфрейм) в минутах — целое число в диапазоне [1 ... 1000].
- Второй временной интервал в минутах — целое число в диапазоне [1 ... 1000].
- Третий временной интервал в минутах — целое число в диапазоне [1 ... 1000].

Выходные данные

Выведите на консоль число *пройденных километров, отформатированное до второй цифры после точки.*

Примеры ввода и вывода

Ввод	Вывод	Комментарии
90 60 70 80	330.90	Расстояние с начальной скоростью: 90 км/ч * 1 час (60 мин) = 90 км После увеличения скорости: 90 + 10 % = 99.00 км/ч * 1,166 часа (70 мин) = 115.50 км После снижения скорости: 99 - 5 % = 94.05 км/ч * 1.33 часа (80 мин) = 125.40 км Общее количество пройденных километров: 330.9 км

Ввод	Вывод	Комментарии
140 112 75 190	917.12	Расстояние с начальной скоростью: 140 км/ч * 1.86 часа (112 мин) = 261.33 км После увеличения скорости: 140 + 10 % = 154.00 км/ч * 1.25 часа (75 мин) = 192.5 км После снижения скорости: 154,00 - 5 % = 146.29 км/ч * 3,16 часа (190 мин) = 463.28 км Общее количество пройденных километров: 917.1166 км

Рекомендации и советы

Возможно, на первый взгляд такое описание может показаться обманчивым и неполным, что еще больше усложняет относительно простую задачу. Давайте разделим ее на несколько *подзадач* и попробуем решить каждую из них по очереди, что приведет нас к конечному результату:

- Наша начальная подзадача будет заключаться в *считывании входных данных*, введенных пользователем, и *сохранении их в соответствующих переменных*.
- *Выполнение основной логики* программы, которая в нашем случае представляет собой пакет простых вычислений числовых значений, которые у нас уже есть.
- *Расчет и формирование конечного результата.*

Основная часть логики программирования заключается в вычислении того, каким будет пройденное расстояние после всех изменений скорости. Поскольку во время выполнения программы часть имеющихся у нас данных изменяется, мы можем разделить программный код на несколько логически разделенных частей:

- Вычисление расстояния, пройденного с начальной скоростью.
- Изменение скорости и подсчет пройденного расстояния.
- Последнее изменение скорости и расчет.
- Сложение.

Чтение входных данных

Для чтения данных с консоли мы используем функцию `cin`:

```
int initialSpeed;
cin >> initialSpeed
```

По условию входные данные вводятся в четыре отдельные строки, поэтому предыдущий код мы должны выполнить в общей сложности четыре раза:

```
int initialSpeed; firstInterval, secondInterval, thirdInterval;
cin >> initialSpeed;
cin >> firstInterval;
// ...
```

Таким образом, мы смогли успешно справиться с первой подзадачей — приемом входных данных.

Преобразование входных данных и централизация логики

Для выполнения вычислений мы решили использовать тип `double`. Первоначально мы сохраняем вспомогательную переменную, которую будем использовать неоднократно. Такой подход к централизации дает нам гибкость и возможность изменять общий результат работы программы с минимальными усилиями.

В случае необходимости изменить значение достаточно сделать это в одном месте кода, что экономит время и силы:

```
const double minutesPerHour = 60.0;
```

Зарезервированное слово `const`, помещенное перед типом `double`, указывает на то, что переменная `minutesPerHour` является константой. После сохранения в константе значение 60.0 не может быть изменено. На первый взгляд использование констант в задачах, которые мы рассматриваем в этой книге, может показаться излишним, однако они находят свое применение и в крупномасштабных проектах. Например, они позволяют предотвратить будущие ошибки при разработке программного обеспечения, поскольку если компилятор заметит попытку изменить константу, то сразу же выдаст ошибку времени компиляции. Кроме того, в зависимости от компилятора использование констант может несколько ускорить выполнение программы и/или несколько уменьшить объем памяти, используемой программой.

Избегание повторяющегося кода (централизация логики программы) в задачах, которые мы рассматриваем в данной книге, на первый взгляд может показаться излишним, но этот подход очень важен при создании больших приложений в реальной рабочей среде, а его применение на начальном этапе обучения поможет вам сформировать качественный стиль программирования.

Расчет расстояния поездки

Мы рассчитываем время в пути (в часах), разделив его на 60 (минут в часе). Расстояние в пути рассчитывается умножением начальной скорости на пройденное время (в часах). После этого мы меняем скорость, увеличивая ее на 10 %, как указано в описании задачи. Вычисление процентов, а также последующих пройденных расстояний, происходит следующим образом:

- Временной интервал (в часах) рассчитывается путем деления предоставленного временного интервала в минутах на количество минут, содержащихся в часе (60).
- Пройденное расстояние рассчитывается путем умножения временного интервала (в часах) на скорость, полученную после увеличения.
- Следующим шагом будет снижение скорости на 5 %, как указано в условии задачи.

Оставшееся расстояние вычисляем способом, описанным в первых двух пунктах. Вот пример кода, который реализует описанные выше шаги:

```
double firstIntervalHours = firstInterval / minutesPerHour;
double firstDistance = initialSpeed * firstIntervalHours;

double speedAfterIncrease = initialSpeed + ((initialSpeed *
   10.0) / 100.0);
double secondIntervalHours = secondInterval / minutesPerHour;
double secondDistance = speedAfterIncrease * secondIntervalHours;
```

Вычисление и вывод результатов

К настоящему времени нам удалось выполнить две наиболее важные подзадачи — сбор и обработку данных. Осталось только

вычислить конечный результат. Поскольку по условию он должен быть отформатирован до двух десятичных знаков, мы можем сделать это следующим образом:

```
double finalDistance = firstDistance + secondDistance +
   thirdDistance;
cout << fixed;
cout << setprecision(2) << finalDistance << endl;
```

В случае корректной работы программы с входными данными из условия задачи вы убедитесь, что она работает правильно.

Проверка в судейской системе

Проверьте свое решение в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1371#0.

Задача: замена плитки

У Гарри есть сбережения, которые он хочет потратить на замену плитки на полу в ванной. Пол прямоугольный, а плитка треугольная. Напишите программу, которая вычислит, хватит ли его сбережений. Считайте из консоли ширину и длину пола, а также одну из сторон треугольника с высотой, направленной к ней. Необходимо рассчитать, *сколько плиток нужно*, чтобы покрыть пол. Количество плиток нужно округлить до большего целого числа и добавить еще пять плиток в качестве запасных. Также считайте с консоли *цену за плитку* и *сумму, заплаченную за работу*. Все цены и денежные расчеты производятся в `lv` (болгарских левах, BGN).

Входные данные

Следующие семь строк должны быть прочитаны из консоли:

- Сбережения Гарри.
- Ширина пола.
- Длина пола.
- Сторона треугольника.
- Высота треугольника.

- Цена плитки в `lv` (болгарских левах, BGN).
- Вознаграждение, выплачиваемое мастеру.

Все числа — вещественные в диапазоне [0.00 ... 5000.00].

Выходные данные

На консоль должно быть выведено в виде одной строки:

- Если денег достаточно: `"Осталось {Остаток средств} lv"`.
- Если денег не хватает: `"Вам понадобится на {Недостаточно средств} lv больше"`.

Результат должен быть отформатирован до второй цифры после десятичной точки.

Примеры ввода и вывода

Ввод	Вывод	Комментарии
500 3 2.5 0.5 0.7 7.80 100	Осталось 25.60 lv.	Площадь пола → 3 * 2.5 = 7.5 Площадь плитки → 0.5 * 0.7 / 2 = 0.175 Необходимо плитки → 7.5 / 0.175 = 42.857... = 43 + 5 запасных плиток = 48 Общая сумма → 48 * 7.8 + 100 (мастеру) = 474.4 474.4 < 500 → 25.60 lv останется

Ввод	Вывод	Комментарии
1000 5.55 8.95 0.90 0.85 13.99 321	Вам понадобится на 1209.65 lv больше.	Площадь пола → 5.55 * 8.95 = 49.67249 Площадь плитки → 0.9 * 0.85 / 2 = 0.3825 Необходимо плитки → 49.67249 / 0.3825 = 129.86... = 130 + 5 запасных плиток = 135 Общая сумма → 135 * 13.99 + 321 (мастеру) = 2209.65 2209.65 > 1000 → 1209.65 lv недостаточно

Рекомендации и советы

В данной задаче наша программа должна принять больше входных данных и выполнить большее количество вычислений, хотя решение идентично.

Прием входных данных мы выполняем известным способом. Обратите внимание, что в разделе условия о входных данных указано, что все входные данные будут вещественными числами, и поэтому мы будем использовать тип `double`.

Теперь, когда у нас уже есть все для выполнения логики программирования, мы можем пойти дальше. Как рассчитать необходимое количество плиток, чтобы покрыть весь пол? Требование, чтобы плитка имела треугольную форму, может привести к путанице, но практически задача требует простых вычислений. Площадь пола мы можем вычислить по формуле для нахождения площади прямоугольника, а площадь одной плитки — по соответствующей формуле для площади треугольника.

Для того чтобы рассчитать количество необходимых плиток, делим площадь пола на площадь одной плитки (не забываем добавить пять дополнительных плиток, которые были указаны в требованиях).

Обратите внимание: в требованиях сказано, что мы должны округлить число плиток, полученное при делении, до большего числа, а затем прибавить 5. Поищите подробную информацию о системной функции `ceil(...)`, которая это делает.

К итоговому результату мы можем прийти, рассчитав общую сумму, необходимую для покрытия всего пола, сложив стоимость плитки с затратами на мастера, которые мы имеем из исходных данных. Общие затраты на плитку можно рассчитать, умножив количество плиток на цену за плитку. Достаточно ли будет имеющейся у нас суммы, можно узнать, сравнив собранные на данный момент деньги (из исходных данных) и общую стоимость:

```
if (budget >= totalCost) {
    // TODO: вывести сообщение
}
else {
    // TODO: вывести сообщение
}
```

Проверка в судейской системе

Проверьте свое решение в судейской системе SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1371#1.

Задача: цветочный магазин

Цветочный магазин предлагает три вида цветов: *хризантемы*, *розы* и *тюльпаны*. Цены зависят от сезона.

Сезон	Хризантемы	Розы	Тюльпаны
весна / лето	2.00 BGN/шт	4.10 BGN/шт	2.50 BGN/шт
осень / зима	3.75 BGN/шт	4.50 BGN/шт	4.15 BGN/шт

В праздничные дни цены на все цветы увеличиваются на 15 %. Предоставляются следующие скидки:

- При покупке более 7 тюльпанов весной — 5 % от стоимости всего букета.
- При покупке 10 и более роз в зимний период — 10 % от стоимости всего букета.
- При покупке более 20 цветов в общей сложности в любой сезон — 20 % от стоимости всего букета.

Скидки предоставляются в вышеописанном порядке и могут комбинироваться. Все скидки действуют после повышения цены в праздничный день. Цена за составление букета всегда равна 2 USD. Напишите программу, которая вычисляет *стоимость букета*.

Входные данные

Входные данные считываются с консоли и содержат пять строк:

- В 1-й строке указывается *количество купленных хризантем* — целое число в диапазоне [0... 200].
- Во 2-й строке указывается *количество купленных роз* — целое число в диапазоне [0 ... 200].
- 3-я строка содержит *количество купленных тюльпанов* — целое число в диапазоне [0 ... 200].
- 4-я строка — *время года* — [`Весна`, `Лето`, `Осень`, `Зима`].
- В 5-й строке указывается, *является ли этот день праздничным* — [`Y` = да / `N` = нет].

Выходные данные

Выведите на консоль одно число — *цену цветов*, отформатированную до второй цифры после точки.

Примеры ввода и вывода

Ввод	Вывод	Комментарии
2 4 8 Весна Y	46.14	Цена: 2*2.00 + 4*4.10 + 8*2.50 = 40.40 BGN Праздник: 40.40 + 15 % = 46.46 BGN Скидка 5 % при покупке > 7 тюльпанов весной: 44.14 Цветы в количестве 20 или менее: без скидки 44.14 + 2 за составление букета = 46.14 BGN

Ввод	Вывод	Комментарии
3 10 9 Зима N	69.39	Цена: 3*3.75 + 10*4.50 + 9*4.15 = 93.60 BGN Не праздник: без повышения цены Скидка 10 % при покупке 10 и более роз в зимний период: 84.24 Всего цветов более 20: скидка 20 % = 67.392 67.392 + 2 за составление букета = 69.392 BGN

Рекомендации и советы

Разделим задачу на несколько более мелких подзадач, как описано ниже.

Разделение постоянных значений в переменных

Внимательно прочитав условия задачи, мы понимаем, что нам снова нужно произвести простые вычисления, однако на этот раз нам понадобятся дополнительные логические условия. Нам нужно уделить больше внимания моменту внесения изменений в конечную цену, чтобы суметь правильно выстроить логику нашей программы.

Для начала *разделим уже определенные значения в переменные*, как мы делали это в предыдущих задачах:

```
// Начальный прейскурант
const double roseAutumnWinterPrice = 4.5;
const double roseSpringSummerPrice = 4.1;
const double tulipAutumnWinterPrice = 4.15;
const double tulipSpringSummerPrice = 2.5;
const double chrysantemumAutumnWinterPrice = 3.75;
const double chrysantemumSpringSummerPrice = 2;
const double arrangePrice = 2;
```

Аналогично поступаем и с другими уже определенными значениями:

```
// Возрастание цен
const double pricelncreasePercentage = 15.0;

// Снижение цен
const double tulipPriceDecreasePencentage = 5.0;
const double rosePriceDecreasePercentage = 10.0;
const double totalPriceDecreasePercentage = 20.0;

// Пороги снижения цен
const double tulipPriceDecreaseThreshold = 7.0;
const double rosePriceDecreaseThreshold = 10.0;
const double totalPriceDecreaseThreshold = 20.0;
```

Чтение входных данных

Наша следующая подзадача — корректное чтение входных данных из консоли. Для этого мы используем уже известный способ:

```
int chrysantemumsPurchased, rosesPurchased, tulipsPurchased;
string season, isSpecialDay;
cin >> chrysantemumsPurchased;
// ...
```

Структурирование логики программы

Давайте подумаем, как лучше структурировать логику нашей программы. Из условия видно, что путь программы в основном делится на две части: весна/лето и осень/зима. Разделение произведем с помощью условного оператора `if-else`, а перед этим зададим переменные для цен на отдельные цветы и для конечного результата:

```
if (season == "Зима || season == "Осень") {
    rosesPrice = rosesPurchases * roseAutumnWinterPrice;
    chrysantemumsPrice = chrysantemumsPurchases *
  chrysantemumAutumnWinterPrice;
    tulipsPrice = tulipsPurchased * tulipAutumnWinterPrice;
    totalCost = rosesPrice + chrysantemumsPrice + tulipsPrice;
}
```

```
else {
    // TODO
}
```

Нам остается провести несколько проверок скидок на различные виды цветов в зависимости от сезона и изменить конечный результат.

Проверка в судейской системе

Проверьте пример программы в судейской системе университета SoftUni: https://judge.softuni.bg/Contests/Practice/Index/1371#2.

Задача: оценки

Напишите программу, которая вычисляет *статистику оценок на экзамене*. Вначале программа считывает *количество студентов*, присутствовавших на экзамене, и для каждого студента — его *оценку*. В конце программа должна вывести процент студентов, имеющих оценки от 2.00 до 2.99, от 3.00 до 3.99, от 4.00 до 4.99, 5.00 и выше, а также средний балл за экзамен.

Примечание: мы используем *болгарскую систему оценки знаний*, где шкала оценок начинается с 2.00 (неудовлетворительно) и заканчивается 6.00 (превосходно): https://en.wikipedia.org/wiki/Grading_systems_by_country#Bulgaria.

Входные данные

Считайте с консоли последовательность чисел, каждое в отдельной строке:

- В первой строке — количество студентов, присутствовавших на экзамене — целое число в диапазоне [1 ... 1000].
- Для каждого отдельного студента в отдельной строке — оценка на экзамене — вещественное число в диапазоне [2.00...6.00].

Выходные данные

Выведите на консоль пять строк, содержащих следующую информацию:

- «Лучшие студенты: {процент студентов с оценкой 5.00 и выше}%».

- «Между 4.00 и 4.99: {от 4,00 до 4,99 включительно}%».
- «Между 3,00 и 3,99: {от 3,00 до 3,99 включительно}%».
- «Неудовлетворительно: {менее 3,00} %».
- «Среднее: {средний балл}».

Результаты должны быть отформатированы до *второй цифры после десятичной точки.*

Ввод	Вывод	Комментарии
6 2 3 4 5 6 2.2	Лучшие студенты: 33.33% Между 4.00 и 4.99: 16.67% Между 3.00 и 3.99: 16.67% Неудовлетворительно: 33,33% Среднее: 3.70	5 и более: 2 ученика = 33,33% от 6 Между 4.00 и 4.99: 1 студент Между 3.00 и 3.99: 1 студент Ниже 3: 2 студента Средний балл: 2 + 3 + 4 + 5 + 6 + 2,2 = 22. 2 / 6 = 3.70

Ввод	Вывод	Комментарии
10 3.00 2.99 5.68 3.01 4 4 6.00 4.50 2.44 5	Лучшие студенты: 30.00% Между 4.00 и 4.99: 30.00% Между 3.00 и 3.99: 20.00% Неудовлетворительно: 20.00% Среднее: 4.06	5 и более: 3 студента = 30% от 10 Между 4.00 и 4.99: 3 студента = 30% от 10 От 3,00 до 3,99: 2 студента = 20% от 10 Ниже 3: 2 студента = 20% от 10 Средний балл составляет: 3 + 2.99 + 5.68 + 3.01 + 4 + 4 + 6 + 4.50 + 2.44 + 5 = 40.62 / 10 = 4.062

Советы и рекомендации

Разделим задачу на более мелкие подзадачи, как описано ниже.

Чтение входных данных и создание вспомогательных переменных

По требованиям мы видим, что сначала мы будем считывать количество студентов, а затем их оценки. Сперва в переменной `int` мы будем считывать количество студентов. Для того чтобы считать и обработать сами оценки, мы будем использовать цикл `for`. На каждой итерации цикла будет считываться и обрабатываться одна оценка:

```
int numberOfStudents;
cin >> numberOfStudents;

for (int i = 0; i < numberOfStudents; i++) {
    // TODO
}
```

Перед выполнением кода цикла `for` мы выделяем переменные, в которых будем хранить количество студентов для каждой группы: низкие баллы (до 2,99), баллы от 3 до 3,99, баллы от 4 до 4,99 и баллы выше 5. Также нам понадобится еще одна переменная для хранения суммы всех оценок, которую мы будем использовать для вычисления среднего балла всех студентов:

```
int numberOfFailedStudents = 0;
int numberOfAverageStudents = 0;
int numberOfGoodStudents = 0;
int numberOfExcellentStudents = 0;
double totalResult = 0;
```

Распределение учащихся по группам

Запустим цикл и объявим в нем еще одну переменную, в которой будем хранить текущий введенный счет. Переменная будет иметь тип `double`, и на каждой итерации мы будем проверять, каково ее значение. В соответствии с этим значением мы увеличиваем количество студентов в соответствующей группе на 1, не забывая при этом увеличивать и *общее количество оценок*, которые мы также отслеживаем:

```
int numberOfStudents;
cin >> numberOfStudents;

for (int i = 0; i < numberOfStudents; i++) {
    double grade;
    cin >> grade;

    totalResult += grade;
    if (grade < 3) {
        numberOfFailedStudents++;
    }
    // else if ()
    // TODO: проверить остальные группы
}
```

Мы можем рассчитать, какой процент от общего числа студентов занимает та или иная группа, умножив число студентов в этой группе на 100, а затем разделив на общее число студентов.

При выполнении этих вычислений обратите внимание на тип числовых данных, с которыми вы работаете.

Конечный результат форматируется привычным способом до второй цифры после точки.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1371#3.

Задача: рождественская шапка

Напишите программу, которая считывает с консоли целое число `n` и рисует рождественскую шапку шириной `4 n + 1` столбцов и высотой `2 * n + 5` строк, как в примерах ниже.

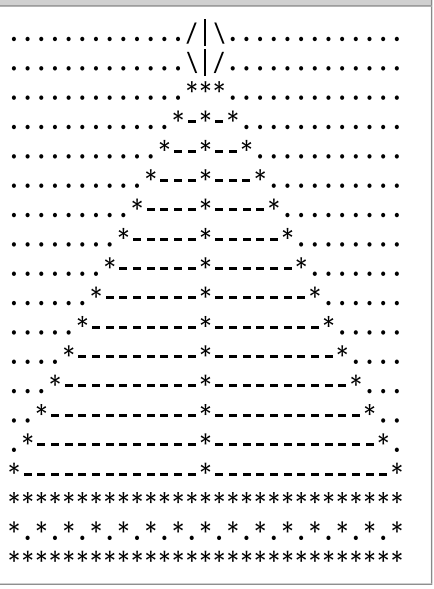
Входные данные

На вход с консоли подается *целое число* `n` в диапазоне [3 ... 100].

Выходные данные

Выведите на консоль рождественскую шапку, точно такую же, как в примерах.

Ввод	Вывод
4	.......|/|\....... ....|/|/....... ....***....... ......*-*-*...... .....*--*--*..... ....*---*---*.... ...*----*----*... ..*-----*-----*.. .*------*------*. *-------*-------* ***************** *.*.*.*.*.*.*.*.* *****************

Ввод	Вывод
7	

.....
.....
.....

Рисование динамической части фигуры

Чтобы нарисовать динамическую, или переменную, часть фигуры, мы будем использовать *цикл*. Размер цикла будет от 0 до той ширины, которую мы имеем по условию, а именно `4 * n + 1`. Поскольку мы будем использовать эту формулу в нескольких местах в коде, хорошей практикой будет объявить ее в *отдельной переменной*. Перед запуском цикла следует *объявить переменные для количества отдельных символов*, участвующих в динамической части: точек и тире. Проанализировав примеры, можно также составить формулы для начальных значений этих переменных. Изначально тире равно 0, но понятно, что количество точек можно вычислить, вычтя 3 из *общей ширины* (количество символов, составляющих верхнюю часть рождественской шапки), а затем разделив на 2, так как количество точек с обеих сторон шапки одинаково.

```
.......***.......
......*-*-*......
.....*--*--*.....
....*---*---*....
...*----*----*...
..*-----*-----*..
.*------*------*.
*-------*-------*
```

Осталось выполнить тело цикла, так как после каждого рисунка мы уменьшаем количество точек на 1 и увеличиваем количество черточек на 1. Не забудем нарисовать между ними по одной звездочке. Последовательность рисования в теле цикла следующая:

- Символьная строка из точек
- Звездочка
- Символьная строка из тире
- Звездочка
- Символьная строка из тире
- Звездочка
- Символьная строка из точек

В случае правильной работы программы мы получим результаты, идентичные приведенным в примерах.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1371#4.

Задача: сочетания букв

Напишите программу, которая выводит на консоль все *сочетания из трех букв* в заданном диапазоне, пропуская комбинации, *содержащие определенную букву*. Выведите количество сочетаний.

Входные данные

Входные данные считываются из консоли и содержат три строки:

- Строчная (малая) буква из английского алфавита, обозначающая начало диапазона — между «a» и «z».
- Строчная английская буква для конца диапазона — между первой буквой и «z».
- Строчная английская буква — от «a» до «z» — поскольку сочетания, содержащие эту букву, пропускаются.

Выходные данные

Выведите на одной строке все сочетания, соответствующие требованиям, с указанием их номера, разделенного пробелом.

Примеры ввода и вывода

Ввод	Вывод	Комментарии
a c b	aaa aac aca acc caa cac cca ccc 8	Все возможные сочетания с буквами "a", "b" и "c": aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca bcb bcc caa cab cac cba cbb cbc cca ccb ccc Сочетания, содержащие 'b', недействительны. Осталось 8 правильных сочетаний.

Ввод	Вывод
a c z	aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca bcb bcc caa cab cac cba cbb cbc cca ccb ccc 27

Ввод	Вывод
f k h	fff ffg ffi ffj ffk fgf fgg fgi fgj fgk fif fig fii fij fik fjf fjg fji fjj fjk fkf fkg fki fkj fkk gff gfg gfi gfj gfk ggf ggg ggi ggj ggk gif gig gii gij gik gjf gjg gji gjj gjk gkf gkg gki gkj gkk iff ifg ifi ifj ifk igf igg igi igj igk iif iig iii iij iik ijf ijg iji ijj ijk ikf ikg iki ikj ikk jff jfg jfi jfj jfk jgf jgg jgi jgj jgk jif jig jii jij jik jjf jjg jji jjj jjk jkf jkg jki jkj jkk kff kfg kfi kfj kfk kgf kgg kgi kgj kgk kif kig kii kij kik kjf kjg kji kjj kjk kkf kkg kki kkj kkk 125

Рекомендации и советы

Чтение входных данных

По условиям задачи мы имеем входные данные в трех строках, каждая из которых представлена одним символом таблицы ASCII (http://www.asciitable.com).

Для преобразования входных данных к типу можно воспользоваться функцией `cin` в языке C++ следующим образом:

```
char startLetter;
cin >> startLetter;
```

Вывод всех символов от начала до конца

Давайте подумаем, как мы могли бы прийти к конечному результату. Если условием задачи является вывод всех символов от начала до конца (пропуская определенную букву), то как мы поступим?

Самый простой и лучший способ — использовать цикл, перебирающий все символы и выводящий на экран те из них, которые отличаются от буквы, которую нам нужно пропустить. Одним из преимуществ языка C++ является возможность использования другого типа данных для циклической переменной:

```
for (char i = 'a'; i <= 'z'; i ++) {
    cout << i << " ";
}
```

В результате выполнения приведенного выше кода все буквы от `a` до `z` включительно выводятся на одной строке и разделяются пробелом. Похоже ли это на конечный результат нашей задачи? Нужно придумать, как вывести не 1, а 3 символа, как указано в ус-

ловии. Выполнение программы очень похоже на игровой автомат. Там мы в основном выигрываем, если нам удается выстроить в ряд несколько одинаковых символов. Допустим, на автомате есть места для трех символов. Когда мы останавливаемся на каком-либо символе в первом месте, в двух других местах продолжают выстраиваться символы из всех возможных мест. В нашем случае все возможные буквы — от начальной до конечной, указанной пользователем, и решение нашей программы идентично тому, как работает игровой автомат.

Вывод сочетания из трех символов

Используется цикл, в котором перебираются все символы от начальной до конечной буквы включительно. На каждой итерации первого цикла запускаем второй цикл с теми же параметрами (но только в том случае, если буква первого цикла верна, то есть не совпадает с той, которую нам нужно исключить по условию). На каждой итерации второго цикла запускаем еще один с теми же параметрами и той же проверкой. Таким образом, у нас получится три вложенных цикла, в теле последнего из которых мы будем добавлять символы к конечному результату:

```
for (char i = startLetter; i <= endLetter; i++) {
    if (i != exceptLetter) {

    }
}
```

Не будем забывать, что от нас требуется также вывести общее количество найденных правильных комбинаций, причем они должны быть выведены на одной строке, разделенные пробелом. Эту подзадачу мы оставляем читателю.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1371#5.

Глава 9.1.
Задачи для чемпионов — часть I

В этой главе мы предлагаем читателю несколько более сложных задач, направленных на развитие алгоритмических навыков и освоение приемов программирования для решения задач повышенной сложности.

Более сложные задачи по изученному материалу

Мы вместе решим несколько задач по программированию, которые охватывают материал, изученный в книге, но они более сложные, чем обычные задачи вступительных экзаменов в SoftUni. Если вы хотите стать чемпионом по основам программирования, рекомендуем потренироваться в решении таких сложных задач, чтобы облегчить себе сдачу экзаменов.

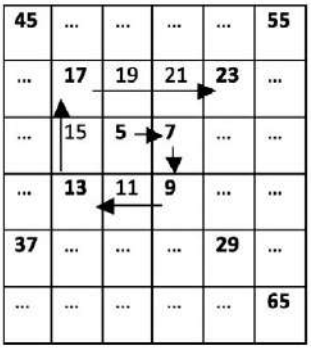
Задача: пересекающиеся последовательности

У нас есть две последовательности:

- *последовательность Трибоначчи* (по аналогии с последовательностью Фибоначчи), где *каждое число является суммой трех предыдущих* (при заданных трех числах);
- последовательность, генерируемая *числовой спиралью*, определяемая путем зацикливания по спирали (справа, снизу, слева, сверху, справа, снизу, слева, сверху и т. д.) матрицы чисел, начиная с ее центра, с заданным начальным числом и шагом приращения, сохраняя текущие числа в последовательности при каждом осуществленном повороте.

Напишите программу, которая находит *первое число, встречающееся в обеих последовательностях, заданных вышеупомянутым способом.*

Пример



Пусть последовательность Трибоначчи начинается с 1, 2 и 3. Это значит, что первая последовательность будет содержать числа 1, 2, 3, 6, 11, 20, 37, 68, 125, 230, 423, 778, 1431, 2632, 4841, 8904, 16377, 30122, 55403, 101902 и так далее.

При этом пусть числа в спирали начинаются с 5, а спираль увеличивается на 2 на каждом шаге.

Тогда вторая последовательность будет содержать числа 5, 7, 9, 13, 17, 23, 29, 37 и так далее. Мы видим, что 37 — это первое число, которое встречается в последовательности Трибоначчи и в спиральной последовательности, и это и есть искомое решение задачи.

Входные данные

Входные данные должны считываться с консоли.

- В первых трех строках входных данных мы прочитаем *три целых числа*, представляющих *первые три числа* в последовательности Трибоначчи, положительные ненулевые числа, отсортированные в порядке возрастания.
- В следующих двух строках ввода мы прочитаем *два целых числа*, представляющих первое число и *шаг* для каждой ячейки матрицы для спирали чисел. Числа, представляющие спираль, являются положительными ненулевыми числами.

Входные данные всегда будут действительны и всегда будут иметь описанный формат. Проверять не нужно.

Выходные данные

Результат должен быть выведен на консоль.

В единственной строке вывода мы должны вывести *наименьшее число, которое встречается в обеих последовательностях*.

Если в диапазоне [1 ... 1 000 000] нет числа, которое встречается в обеих последовательностях, выведите «No».

Ограничения

- Все числа на входе будут находиться в диапазоне [1 ... 1 000 000].
- Допустимое время программы: 0,3 секунды.
- Разрешенная память: 16 МБ.

Примеры ввода и вывода

Ввод	Вывод
1 2 3 5 2	37

Ввод	Вывод
13 25 99 5 2	13

Ввод	Вывод
99 99 99 2 2	No

Ввод	Вывод
4 1 7 23 3	71

Рекомендации и советы

Задача представляется достаточно сложной, поэтому мы разобьем ее на более простые подзадачи: обработка входных данных, генерация ряда Трибоначчи, генерация числовой спирали и нахождение наименьшего общего числа для двух рядов.

Обработка входных данных

Первым шагом в решении задачи является чтение и обработка входных данных.

Входные данные состоят из пяти целых чисел: трех для ряда Трибоначчи и двух для числовой спирали:

```
int tribonacciFirst, tribonacciSecond, tribonacciThird;
cin >> tribonacciFirst >> tribonacciSecond >> tribonacciThird;

int spiralCurrent, spiralStep;
cin >> spiralCurrent >> spiralStep;
```

После того как мы получили исходные данные, необходимо подумать о том, как мы будем генерировать числа в двух строках.

Генерация ряда Трибоначчи

Для ряда Трибоначчи мы будем каждый раз складывать три предыдущих значения, а затем сдвигать значения этих чисел (трех предыдущих) на одну позицию вперед в ряду, то есть значение первого числа должно принять значение второго, значение второго — значение третьего, а значение третьего станет новым найденным числом.

Когда мы закончим работу с числом, мы добавим его значение в вектор (список). Для использования векторов в языке C++ необходимо включить библиотеку `vector` с помощью директивы `#include <vector>` в начале кода. После создания вектора добавление в него элементов осуществляется с помощью функции `push_back(...)`.

Поскольку в условии задачи сказано, что числа в ряду не превышают 1 000 000, мы можем прекратить генерацию этого ряда именно при достижении 1 000 000:

```
vector<int> tribonacciNumbers;
tribonacciNumbers.push_back(tribonacciFirst);
tribonacciNumbers.push_back(tribonacciSecond);
tribonacciNumbers.push_back(tribonacciThird);

int tribonacciCurrent = tribonacciThird;
while (tribonacciCurrent < 1000000) {
    tribonacciCurrent = tribonacciFirst + tribonacciSecond +
  tribonacciThird;

tribonacciNumbers.push_back(tribonacciCurrent);
tribonacciFirst = tribonacciSecond;
tribonacciSecond = tribonacciThird;
tribonacciThird = tribonacciCurrent;
}
```

Генерация числовой спирали

Нам необходимо выяснить зависимости между числами в числовой спирали, чтобы мы могли легко генерировать каждое последующее число, не обращаясь к матрицам и их обходам. Если внимательно посмотреть на рисунок из условия, то можно заметить, что на каждые два «витка» спирали пропускаемые числа увеличиваются на 1, то есть от 5 до 7 и от 7 до 9 мы не пропускаем ни одного числа, а непосредственно прибавляем с шагом ряда. От 9 до 13 и от 13 до 17 мы пропускаем одно число, то есть прибавляем в два раза больше шага. От 17 до 23 и от 23 до 29 пропускаем два числа, то есть прибавляем три раза шаг и т. д.

Таким образом, мы видим, что для первых двух у нас есть `последнее число + 1 * шаг`, следующие два мы складываем с `2 * шаг` и так далее. Каждый раз, когда мы захотим перейти к следующему числу спирали, нам придется производить подобные вычисления:

```
spiralCurrent += spiralStep * spiralStepMul;
```

Необходимо позаботиться о том, чтобы каждые два числа наш множитель (назовем его «коэффициент») увеличивался на 1 (`spiralStepMul++`), что достигается простой проверкой на четность (`spiralCount % 2 == 0`). Весь код, начиная с генерации спирали в векторе, приведен ниже:

```
vector<int> spiralNumbers;
spiralNumbers.push_back(spiralCurrent);
int spiralCount = 0;
int spiralStepMul = 1;

while (spiralCurrent < 1000000) {
    spiralCurrent += spiralStep * spiralStepMul;
    spiralNumbers.push_back(spiralCurrent);
    spiralCount++;
    if (spiralCount % 2 == 0) {
        spiralStepMul++;
    }
}
```

Нахождение общего числа для последовательностей

После того как мы сгенерировали числа в обоих рангах, можно приступать к их объединению и построению окончательного ре-

шения. Как оно будет выглядеть? Для каждого из чисел одного ряда (начиная с меньшего) мы проверим, существует ли оно в другом ряду. Первое число, удовлетворяющее этому критерию, и будет ответом на поставленную задачу.

Поиск во втором векторе будет осуществляться линейно, а для более любопытных мы дадим возможность оптимизировать его с помощью техники, называемой *бинарным поиском (*Binary Search), поскольку второй вектор генерируется отсортированным, то есть удовлетворяет требованию применения этого вида поиска.

Для обработки каждого числа из двух векторов мы будем использовать особый цикл `for`. С помощью приведенного ниже синтаксиса каждый из двух циклов поочередно просматривает все числа в векторах и записывает их в соответствующую переменную.

Код для поиска решения будет выглядеть следующим образом:

```
bool found = false;
for (int tribonacciNumber : tribonacciNumbers) {
    for (int spiralNumber : spiralNumbers) {
        if (tribonacciNumber == spiralNumber &&
   tribonacciNumber <= 1000000) {
            cout << tribonacciNumber << endl;
            found = true;
            break;
        }
    }

    if (found) {
        break;
    }
}

    if (!found) {
        cout << "No" << endl;
}
```

В решении задачи для хранения значений используются векторы. Векторы не являются необходимым условием решения задачи. Существует *альтернативное решение*, которое генерирует числа и работает с ними напрямую, а не записывает их в вектор. Мы можем на каждом шаге проверять, совпадают ли числа в двух сериях. Если совпадают, то мы выводим это число на консоль и завершаем

выполнение программы. В противном случае мы будем видеть текущее число, в каком ряду оно меньше, и генерировать следующее, в котором мы «отстаем». Идея заключается в том, что мы будем генерировать числа из того ряда, который «дальше», пока не пропустим текущее число в другом ряду, а затем наоборот, и если за это время найдем совпадение, то завершим выполнение:

```
while (tribonacciCurrent <= 1000000 && spiralCurrent <=
1000000) {
    if (tribonacciCurrent == spiralCurrent) {
        cout << "TODO: Вывести и остановить выполнение" << endl;
    }
    else if (tribonacciCurrent < spiralCurrent) {
        cout << "TODO: Генерировать следующее число
  Трибоначчи" << endl;
    }
    else {
        cout << "TODO: Генерировать следующее число спирали"
  << endl;
    }
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1372#0.

Задача: магические даты

Дата указывается в формате «чч-мм-гггг», например 17-04-2018. Мы вычисляем *вес этой даты*, беря все ее цифры, умножая каждую цифру на следующие за ней и суммируя все полученные результаты. К примеру, у нас восемь цифр: `17032007`, поэтому вес равен `1*7 + 1*0 + 1*3 + 1*2 + 1*0 + 1*0 + 1*7 + 7*0 + 7*3 + 7*2 + 7*0 + 7*0 + 7*7 + 0*3 + 0*2 + 0*0 + 0*0 + 0*7 + 3*2 + 3*0 + 3*0 + 3*7 + 2*0 + 2*0 + 2*7 + 0*0 + 0*7 + 0*7 = 144`.

Наша задача — написать программу, которая находит все магические даты между двумя определенными годами (включительно), соответствующие заданному весу. Даты должны быть выведены в порядке возрастания (по дате) в формате «чч-мм-гггг». Мы будем ис-

пользовать только действительные даты в традиционном календаре (в високосные годы 29 дней в феврале).

Входные данные

Входные данные должны быть считаны с консоли. Они состоят из трех строк:

- Первая строка содержит целое число: начальный год.
- Вторая строка содержит целое число: конечный год.
- Третья строка содержит целое число: вес поиска по датам.

Входные данные всегда будут действительны и всегда будут иметь описанный формат. Проверять не нужно.

Выходные данные

Результат должен быть выведен на консоль в виде последовательных дат в формате «`чч-мм-гггг`», отсортированных по дате в порядке возрастания. Каждая строка должна быть в отдельной строке. Если магических дат не существует, выведите «`No`».

Ограничения

- Начальный и конечный год — это целые числа в диапазоне [1900 ... 2100].
- Магический вес — это целое число в диапазоне [1 ... 1000].
- Допустимое время программы: 0.25 секунды.
- Разрешенная память: 16 МБ.

Примеры ввода и вывода

Ввод	Вывод
2007 2007 144	17-03-2007 13-07-2007 31-07-2007

Ввод	Вывод
2003 2004 1500	No

Ввод	Вывод
2011 2012 14	01-01-2011 10-01-2011 01-10-2011 10-10-2011

Ввод	Вывод
2012 2014 80	09-01-2013 17-01-2013 23-03-2013 11-07-2013 01-09-2013 10-09-2013 09-10-2013 17-10-2013 07-11-2013 24-11-2013 14-12-2013 23-11-2014 13-12-2014 31-12-2014

Рекомендации и советы

Начнем с добавления необходимой функциональности и чтения входных данных из консоли.

В данном случае мы добавили библиотеки `sstream` и `iomanip` для преобразования строк в дату и наоборот и чтения трех целых чисел:

```
#include <iostream>
#include <iomanip>
#include <sstream>

using namespace std;
int main()
{
    int firstYear, secondYear, numberToSearchFor;
    cin >> firstYear >> secondYear >> numberToSearchFor;

    //...
}
```

Имея начальный и конечный год, для удобства вычислений необходимо определить, как мы будем перебирать все даты, чтобы не задумываться о том, сколько дней в месяце, високосный ли это год и так далее.

Цикл по датам

Для перебора дат мы воспользуемся функциональностью, предоставляемой типом `tm` в языке C++. Определим переменную для начальной даты, для чего воспользуемся `stringstream` и методом `get_time(...)`, с помощью которого можно создать объект типа `tm` из строки. Мы знаем, что год — это начальный год, который мы получили в качестве параметра, а месяц и день должны быть январем и 1-м числом соответственно:

```
tm date = {};
stringstream ss("01-01-" + to_string(firstYear));
ss >> get_time(&date, "%d-%m-%Y");
```

Получив начальную дату (в переменной `date`), мы хотим создать цикл, который будет выполняться до тех пор, пока мы не превысим конечный год (или пока мы не пройдем 31 декабря в конечном году, если мы сравниваем целые даты), увеличивая `date` на один день на каждом шаге.

Для инкремента на один день при каждом обороте будем использовать `tm_mday`, который является частью данных каждой переменной типа `tm`. В C++ для этого необходимо увеличить `tm_mday` на 1, а затем вызвать метод `mktime`. Добавление одного дня к дате поможет C++ позаботиться о переходе к следующему месяцу и году, отслеживать количество дней в месяце и принять во внимание високосные годы:

```
date.tin mday++;
mktime(&date);
```

Важной особенностью работы с датами с использованием типа `tm` в C++ является то, что переменная `tm_year`, которая может использоваться для получения года даты, имеет значение 0 для 1900 года и значение 119 для 2019 года.

Поэтому каждый раз, когда мы хотим получить фактический год, мы должны прибавлять к `tm_year` значение 1900.

Другая особенность заключается в том, что месяц январь отмечен значением 0 в `tm_mon` и значение 11, соответственно, используется для декабря. Здесь мы, чтобы получить номер месяца в диапазоне от 1 до 12, как требуется в задаче, добавляем к `tm_mon` единицу.

В конечном итоге наш цикл будет выглядеть следующим образом:

```
while ((date.tm_year + 1900 <= secondYear) {

    // TODO: проверить дату для numberToSearchFor

    date.tm_mday++;
    mktime(&date);
}
```

Расчет веса даты

Каждая дата состоит ровно из восьми символов (цифр) — двух для дня (`d1`, `d2`), двух для месяца (`d3`, `d4`) и четырех для года (`d5 – d8`). Это означает, что у нас всегда будет один и тот же расчет, и мы можем воспользоваться этим, чтобы *определить формулу статически* (то есть не использовать циклы, обращаясь к разным числам из даты, а написать всю формулу). Чтобы написать ее, нам понадобятся все цифры из даты в отдельных переменных, чтобы произвести все необходимые умножения.

Используя операции деления и взятия остатка над отдельными компонентами даты, через `tm_mday`, `tm_mon` и `tm_year`, мы можем извлечь каждую цифру:

```
int d1 = date.tm_mday / 10; // Первая цифра дня
int d2 = date.tm_mday % 10; // Вторая цифра дня

int month = date.tm_mon + 1;
int d3 = month / 10; // Первая цифра месяца
int d4 = month % 10; // Вторая цифра месяца

int year = date.tm_year + 1900;
int d5 = year / 1000; // Первая цифра года
int d6 = (year / 100) % 10; // Вторая цифра года
int d7 = (year / 10) % 10; // Третья цифра года
int d8 = year % 10; // Четвертая цифра года
```

Поясним здесь одну из наиболее интересных строк. Рассмотрим пример с извлечением второй цифры года (`d6`). Здесь мы делим год на 100 и берем остаток 10. Чего мы добиваемся таким образом? Во-первых, при делении на 100 мы убираем последние 2 цифры года (пример: `2018 / 100 = 20`). С остатка от деления на 10 берем последнюю цифру получившегося числа (`20 % 10 = 0`) и таким образом получаем 0, который является второй цифрой числа 2018.

Осталось произвести вычисления, которые дадут нам магический вес заданной даты. Чтобы не выписывать все умножения, как показано в примере, применим простую группировку. Для этого нужно умножить каждую цифру на следующую за ней. Вместо того чтобы писать `d1 * d2 + d1 * d3 + ... + d1 * d8` мы можем сократить это выражение до `d1 * (d2 + d3 + ... + d8)`, следуя математическим правилам группировки при умножении и сложении. Применив такое же упрощение к оставшимся умножениям, мы получим следующую формулу:

```
int weight = d1 * (d2 + d3 + d4 + d5 + d6 + d7 + d8)
             d2 * (d3 + d4 + d5 + d6 + d7 + d8) +
             d3 * (d4 + d5 + d6 + d7 + d8) +
             d4 * (d5 + d6 + d7 + d8) +
             d5 * (d6 + d7 + d8) +
             d6 * (d7 + d8) +
             d7 * d8;
```

Вывод результатов

После того как мы вычислили вес даты, необходимо проверить, соответствует ли он искомому магическому весу, чтобы понять, следует ли его выводить или нет. Проверка может быть выполнена с помощью стандартного оператора `if`, причем при выводе необходимо следить за тем, чтобы дата имела правильный формат. Для этого мы воспользуемся другим вспомогательным методом языка C++ — `put_time(...)`, который принимает дату типа `tm` и символьный формат (в нашем случае `%d - %m - %Y`) и выводит данные о дате в указанном формате (в нашем случае `дата-месяц-год`):

```
if (weight == numberToSearchFor) {
    cout << put_time(&date, "%d-%m-%Y") << endl;
    found = true;
}
```

Внимание: при обходе дат от начала до конца они всегда будут располагаться в порядке возрастания в соответствии с условием.

Наконец, если мы не нашли ни одной даты, удовлетворяющей условиям, то в переменной `found` будет стоять значение `false` и выведено «No»:

```
if (! found) {
    cout << "No" << endl;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1372#1.

Задача: пять особых букв

Даны два числа: *начальное* (`start`) и *конечное* (`end`). Напишите программу, которая *генерирует все сочетания из пяти букв*, каждая из которых входит в набор `{'a', 'b', 'c', 'd', 'e'}` так, что вес этих пяти букв является числом в диапазоне `[start … end]` включительно. Выведите их в алфавитном порядке в одну строку, разделив пробелом.

Вес букв вычисляется следующим образом:

```
weight('a') = 5;
weight('b') = -12;
weight('c') = 47;
weight('d') = 7;
weight('e') = -32;
```

Вес последовательности букв `c1`, `c2`, ..., `cn` вычисляется путем удаления всех повторяющихся букв (справа налево), а затем вычисляется по формуле:

```
weight(c1c2...cn) = 1 * weight(c1) + 2 * weight(c2) + … + n *
weight(cn)
```

Например, вес `bcddc` рассчитывается следующим образом.

Сначала мы удалим повторяющиеся буквы и получим `bcd`. Затем применяем формулу: `1 * weight('b') + 2 * weight('c') + 3 * weight('d') = 1 * (-12) + 2 * 47 + 3 * 7 = 103`.

Другой пример: `weight("cadae") = weight("cade") = 1 * 47 + 2 * 5 + 3 * 7 + 4 * (-32) = -50`.

Входные данные

Входные данные считываются с консоли и состоят из двух чисел:

- Начальное число.
- Конечное число.

Входные данные всегда будут действительны и всегда будут иметь описанный формат. Проверять не нужно.

Выходные данные

Результат должен быть выведен на консоль в виде последовательности строк, *расположенных в алфавитном порядке*. Каждая строка должна быть отделена от следующей одним пробелом. Если вес любой из пяти буквенных строк не существует в указанном диапазоне, выведите «Нет».

Ограничения

- Числа для начала и конца — это целые числа в диапазоне [-10000 ... 10000].
- Допустимое время программы: 0,25 секунды.
- Разрешенная память: 16 МБ.

Примеры ввода и вывода

Ввод	Вывод	Комментарии
40 42	bcead bdcea	вес("bcead") = 41 вес("bdcea") = 40

Ввод	Вывод
300 400	Нет

Ввод	Вывод
-1 1	bcdea cebda eaaad eaada eaadd eaade eaaed eadaa eadad eadae eadda eaddd eadde eadea eaded eadee eaead eaeda eaedd eaede eaeed eeaad eeada eeadd eeade eeaed eeead

Ввод	Вывод
200 300	baadc babdc badac badbc badca badcb badcc badcd baddc bbadc bbdac bdaac bdabc bdaca bdacb bdacc bdacd bdadc bdbac bddac beadc bedac eabdc ebadc ebdac edbac

Рекомендации и советы

Чтение входных данных

Как и в любой другой задаче, мы начинаем решение с чтения и обработки входных данных:

```
int firstNumber, secondNumber;
cin >> firstNumber >> secondNumber;
```

В задаче есть несколько ключевых моментов — генерация всех комбинаций длины 5 из 5 заданных букв, удаление повторяющихся букв и вычисление веса для уже упрощенного слова. Ответом будет любое слово, вес которого находится в заданном интервале `[firstNumber, secondNumber]`.

Генерация всех сочетаний

Чтобы сгенерировать все *сочетания длиной 1*, используя 5 символов, мы используем цикл от 0 до 4, поскольку хотим, чтобы каждый номер цикла соответствовал одному символу. Чтобы сгенерировать все *сочетания длиной 2*, используя 5 символов (то есть «aa», «ab», «ac», ..., «ba», ...), мы создадим *два вложенных цикла, каждый*

из которых будет проходить по цифрам от 0 до 4, так как мы снова убедимся, что каждая цифра соответствует определенному символу. Мы повторим этот шаг пять раз, так что в итоге у нас будет пять вложенных циклов с индексами `i1`, `i2`, `i3`, `i4` и `i5`:

```
for (int i1 = 0; i1 < 5; i1++) {
    for (int i2 = 0; i2 < 5; i2++) {
        for (int i3 = 0; i3 < 5; i3++) {
            for (int i4 = 0; i4 < 5; i4++) {
                for (int i5 = 0; i5 < 5; i5++) {
```

Учитывая все пятизначные комбинации, необходимо найти способ «превратить» эти пять цифр в слово с буквами от «`a`» до «`e`». Один из способов сделать это — переопределить строковую переменную, содержащую имеющиеся у нас буквы:

```
string pattern = "abcde";
```

и для каждой цифры берем букву из определенной позиции. Таким образом, число 00000 станет «`aaaaa`», а число 02423 — «`acecd`». Строку из пяти букв мы можем составить следующим образом:

```
string fullWord;
fullWord += pattern [i1];
fullWord += pattern [i2];
fullWord += pattern [i3];
fullWord += pattern [i4];
fullWord += pattern [i5];
```

Другой способ: мы можем преобразовать цифры в буквы, используя их порядок в таблице ASCII. Выражение `(char)((int)'a' + i)` даст нам результат `'a'` при `i = 0`, `'b'` при `i = 1`, `'c'` при `i = 2` и т. д.

Таким образом, теперь мы имеем все пятибуквенные комбинации и можем продолжить решение следующей части задачи.

Внимание: поскольку мы выбрали `pattern`, соответствующий алфавитному порядку букв, и циклы осуществляются соответствующим образом, алгоритм будет генерировать слова в алфавитном порядке, и дополнительная сортировка перед выводом не требуется.

Удаление повторяющихся букв

После того как строка готова, необходимо удалить все повторяющиеся символы. Эту операцию мы будем выполнять, добавляя буквы слева направо в новую строку, причем каждый раз перед до-

бавлением буквы мы будем проверять, есть ли она там уже — если есть, то мы ее пропустим, а если нет, то добавим. Для начала добавим в исходную строку первую букву:

```
string word;
word += pattern[i1];
```

Затем проделаем то же самое с остальными четырьмя, каждый раз проверяя их наличие с помощью метода `find(...)`. Если метод `find(...)` возвращает отрицательное число -1, это означает, что искомая строка не найдена. Для служебного значения «не найдено» мы используем `string::npos`. Мы также можем реализовать эту часть кода с помощью цикла по символам `fullWord`, но оставим это читателю для тренировки. Более «ленивый» способ с копированием-вставкой показан ниже:

```
if (word.find(pattern[i2]) == string::npos) {
    word += pattern[i2];
}

if (word.find(pattern[i3]) == string::npos) {
    word += pattern[i3];
}

if (word.find(pattern[i4]) == string::npos) {
    word += pattern[i4];
}

if (word.find(pattern[i5]) == string::npos) {
    word += pattern[i5];
}
```

Метод `find(...)` возвращает индекс конкретного элемента, если он найден, или `-1`, если элемент не найден. Таким образом, если мы получим значение `-1`, это будет означать, что у нас еще нет этой буквы в новой строке уникальных букв и мы можем ее добавить, а если мы получим значение, отличное от `-1`, это будет означать, что у нас уже есть эта буква и мы не будем ее добавлять.

Расчет веса

Вычисление веса сводится к обходу уникального слова (`word`), полученного на последнем шаге, для каждой буквы необходимо взять ее вес и умножить на позицию.

Для каждой буквы в обходе необходимо вычислить, на какую величину мы будем умножать ее позицию, например, с помощью серии операторов `if`:

```
int weight = 0;
for (int i = 0; i < word.length(); i++) {
    int multiplier = 0;

    if (word[i] == 'a') {
        multiplier = 5;
    }

    if (word[i] == 'b') {
        multiplier = -12;
    }

    if (word[i] == 'c') {
        multiplier = 47;
    }

    if (word[i] == 'd') {
        multiplier = 7;
    }

    if (word[i] == 'e') {
        multiplier = -32;
}

    weight += multiplier * (i + 1);
}
```

Получив значение данной буквы, необходимо умножить его на ее позицию. Поскольку индексы в строке отличаются на 1 от реальных позиций, то есть индекс 0 — это позиция 1, индекс 1 — позиция 2 и т. д., то к индексам прибавим 1. Поэтому в последней строке цикла мы имеем выражение:

```
weight += multiplier * (i + 1);
```

Все промежуточные результаты должны быть сложены в общий итог (`weight`) для каждой буквы 5-буквенной комбинации.

Подготовка выходных данных

Нужно ли выводить слово, определяется его весом. Нам необходимо условие, определяющее, находится ли текущий вес в интерва-

ле [начало ... конец], заданном нам на входе в начале программы. Если да, то мы выводим полное слово (fullWord).

Будьте внимательны, не выводите слово из уникальных букв (word). Оно нам нужно только для расчета веса!

Слова разделяются пробелом, и мы будем накапливать их в промежуточной переменной result, которая вначале определяется как пустая строка:

```
if (firstNumber <= weight && weight <= secondNumber) {
    result += fullWord + " ";
}
```

Последние штрихи

Условие выполняется только в том случае, если в представленном интервале нет ни одного слова. Чтобы узнать, найдено ли такое слово, можно просто проверить, имеет ли строка result свое начальное значение (а именно пустую строку), используя метод empty(), и если да — вывести «No», в противном случае вывести всю строку без последнего пробела. Удалить последний пробел из result можно с помощью метода substr(...), который вернет подстроку, начинающуюся с 0 и имеющую длину: длина строки минус 1.

```
if (result.empty()) {
    cout << "No" << endl;
}
else {
    cout << result.substr(0, result.size() - 1) << endl;
}
```

Для более любознательных читателей мы оставим домашнее задание — оптимизировать работу циклов, придумав, как уменьшить количество итераций внутренних циклов.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1372#2.

Глава 9.2. Задачи для чемпионов — часть II

В этой главе мы рассмотрим еще три задачи, которые мы относим к категории «для чемпионов», то есть более трудные, чем стандартные задачи, приведенные в этой книге.

Более сложные задачи по изученному материалу

Прежде чем перейти к рассмотрению конкретных задач, следует пояснить, что они могут быть решены проще при наличии дополнительных знаний по программированию и языку C++ (функции, массивы, рекурсия и т. д.), но при любом решении, которое мы дадим сейчас, будет использоваться только материал, рассмотренный в этой книге. Цель — научиться строить более сложные алгоритмы на основе имеющихся знаний.

Задача: дни страстного шопинга

У Лины настоящая страсть к шопингу. Как только у нее появляется немного денег, она сразу же отправляется в ближайший торговый центр (молл) и старается потратить как можно больше на одежду, сумки и обувь. Но больше всего она любит зимние распродажи. Наша задача — проанализировать ее странное поведение и *подсчитать покупки*, которые Лина совершает при входе в торговый центр, а также *деньги, которые у нее остаются после окончания шопинга*. Все цены и суммы денег указаны в BGN (болгарских левах, `lv`).

В *первой строке ввода* будет передана сумма, которой Лина располагает *перед началом* покупок. После этого, получив команду

«`mall.Enter`», Лина войдет в торговый център и начнет своершать покупки, пока не будет подана команда «`mall.Exit`». Когда Лина начнет делать покупки, в каждой строке ввода вам будут даны строки, которые являются действиями, выполняемыми Линой. Каждый символ в строке — это *покупка или другое действие*. Строковые команды содержат только символы таблицы ASCII. ASCII-код каждого знака связан с тем, *сколько Лина должна заплатить за каждый из товаров*. Вам нужно интерпретировать символы следующим образом:

- Если символ — *заглавная буква*, Лина получает скидку 50 %, то есть вы должны уменьшить имеющиеся у нее деньги на 50 % от числового представления символа из таблицы ASCII.
- Если символ — *строчная буква*, Лина получает скидку 70 %, то есть вы должны уменьшить имеющиеся у нее деньги на 30 % от числового представления символа из таблицы ASCII.
- Если символ «%», Лина совершает покупку, которая уменьшает ее деньги вдвое.
- Если символ «*», Лина снимает деньги со своей дебетовой карты и добавляет 10 lv к имеющимся средствам.
- Если символ *отличается от всех вышеперечисленных*, Лина совершает покупку без скидки, и в этом случае вам следует просто вычесть значение символа из таблицы ASCII из ее доступных средств.

Если определенная стоимость ее покупок *больше*, чем текущие доступные средства, Лина *не совершает покупку*. Средства Лины *не могут быть меньше 0*.

Покупки заканчиваются, когда подается команда «`mall.Exit`». Когда это произойдет, вам нужно *вывести количество сделанных покупок и сумму денег*, которые остались у Лины.

Входные данные

Входные данные должны быть считаны из консоли. В *первой строке* ввода будет указана *сумма, которой Лина располагает перед началом покупок*. В каждой из следующих строк будет опреде-

ленная команда. После того как вы считаете команду «`mall.Enter`», в каждой из следующих строк вам будут переданы строки, *содержащие информацию о покупках/действиях*, которые Лина хочет совершить. Эти строки будут передаваться до тех пор, пока не будет дана команда «`mall.Exit`».

Всегда будет подаваться только одна команда «`mall.Enter`», а также только одна команда «`mall.Exit`».

Выходные данные

Выходные данные должны быть выведены на консоль. Когда шопинг завершится, вы должны вывести на консоль определенный вывод в зависимости от того, какие покупки были сделаны.

- Если *покупок не было* — `"Покупок нет. Осталось денег: {оставшиеся средства} lv."`.
- Если *сделана хотя бы одна покупка* — `"{количество покупок} покупок". Осталось денег: {оставшиеся средства} lv"`.

Средства должны быть выведены с точностью до двух символов после точки.

Ограничения

- Деньги — это число с плавающей точкой в диапазоне: [`0 – 7.9 x` 10^{28}].
- Количество строк между «`mall.Enter`» и «`mall.Exit`» будет находиться в диапазоне: [`1-20`].
- Количество символов в каждой строке, представляющей команду, будет находиться в диапазоне: [`1-20`].
- Допустимое время выполнения: 0,1 секунды.
- Разрешенная память: 16 МБ.

Примеры ввода и вывода

Ввод	Вывод

`110`
`mall.Enter`
`%`
`mall.Exit`
	`1 покупка. Осталось денег: 55.00 lv.`

Ввод	Вывод
100 mall.Enter Ab ** mall.Exit	2 покупки. Осталось денег: 58.10 lv.

Ввод	Вывод	Комментарии
110 mall.Enter d mall.Exit	1 покупка. Осталось денег: 80.00 lv.	'd' имеет ASCII-код, равный 100. 'd' - строчная буква, поэтому Лина получает скидку 70%. Она тратит 30% от 100, что составляет 30 lv. После этой покупки у нее остается: 110 - 30 = 80 lv.

Рекомендации и советы

Разделим решение задачи на три основные части:

- Обработка входных данных.
- Алгоритм решения задачи.
- Форматирование выходных данных.

Давайте подробно рассмотрим каждую из частей.

Обработка входных данных

Входные данные состоят из нескольких компонентов:

- В первой строке указаны *все деньги*, которые Лина имеет на покупки.
- В каждой из следующих строк будет какая-либо *команда*.

Первая часть чтения входных данных тривиальна:

```
double shoppingMoney;
cin >> shoppingMoney;
```

Но во второй части есть одна деталь, на которую необходимо обратить внимание. Условие гласит следующее: «В каждой из следующих строк будет определенная команда. При получении команды «`mall.Enter`» в каждой последующей строке будут появляться стро-

ки, содержащие информацию о покупках/действиях, которые хочет совершить Лина».

Здесь нужно иметь в виду, что со второй строки мы должны начать читать команды, но лишь после того, как мы получим команду «mall.Enter», нам необходимо начать их обработку. Как мы можем это сделать? Хорошим вариантом будет использование цикла while или do-while. Вот пример решения, как можно пропустить все команды до получения команды «mall.Enter»:

```
string command;
cin >> command;

while (command != "mall.Enter") {
    cin >> command;
}

cin >> command;
```

Здесь стоит отметить, что вызов cin после окончания цикла используется для перехода к первой обрабатываемой команде.

Алгоритм решения задачи

Сам алгоритм решения задачи прост — мы продолжаем считывать команды из консоли до тех пор, пока не будет выдана команда «mall.Exit». За это время мы обрабатываем каждый ASCII-символ (char) каждой команды в соответствии с правилами, указанными в условии, и одновременно изменяем количество денег у Лины и сохраняем количество покупок.

Рассмотрим первые две проблемы, с которыми сталкивается наш алгоритм.

Первая проблема связана с тем, как мы можем читать команды до тех пор, пока не встретим «mall.Exit». Решение, как мы видели выше, заключается в использовании цикла while.

Вторая проблема — это задача доступа к каждому символу заданной команды. Учитывая, что входные данные команды имеют тип string, наиболее простым способом доступа к каждому символу команды является цикл for.

Вот как выглядит использование двух таких циклов:

```
while (command != "mall.Enter") {
    for (char action : command) {

    }

    cin >> command;
}
```

Обработка символов команд

Следующая часть алгоритма заключается в обработке символов из команд в соответствии с правилами, указанными в требованиях:

- Если символ — *заглавная буква*, Лина получает скидку 50 %, то есть вы должны уменьшить имеющиеся у нее деньги на 50 % от числового представления символа из таблицы ASCII.
- Если символ — *строчная буква*, то скидка 70 %, то есть вы должны уменьшить имеющиеся у нее деньги на 30 % от числового представления символа из таблицы ASCII.
- Если символ «%», Лина совершает покупку, которая уменьшает ее деньги вдвое.
- Если символ «*», Лина снимает деньги со своей дебетовой карты и добавляет 10 lv к имеющимся средствам.
- Если символ отличается от всех вышеперечисленных, Лина совершает покупку без скидки, и в этом случае вам следует просто вычесть значение символа из таблицы ASCII из ее доступных средств.

Давайте рассмотрим проблемы, с которыми мы столкнемся в первом условии. Первая из них — как определить, является ли тот или иной символ *заглавной буквой*. Мы можем использовать один из следующих способов.

Учитывая, что буквы в алфавите имеют определенный порядок, мы можем использовать проверку `(action >= ‘A’) && (action <= ‘Z’)`.

Другая проблема — как пропустить символ, если он обозначает операцию, требующую больше денег, чем есть у Лины. Это можно сделать с помощью конструкции `continue`.

Пример проверки первой части условия выглядит следующим образом:

```
if ((action >= 'A') && (action <= 'Z')) {
    double price = action * 0.5;
    if (shoppingMoney < price) {
        continue;
    }

    shoppingMoney -= price;
    purchases++;
}
```

Примечание: `purchases` — переменная типа `int`, в которой хранится количество всех покупок.

Мы полагаем, что у читателя не возникнет проблем с реализацией всех остальных проверок, поскольку они очень похожи на первую.

В конце нашей задачи мы должны вывести определенный результат, зависящий от следующего условия:

- Если покупок не было — `"Покупок нет. Осталось денег: {оставшиеся средства} lv."`.
- Если сделана хотя бы одна покупка — `"{количество покупок} покупок". Осталось денег: {оставшиеся средства} lv"`.

Операции вывода тривиальны, поскольку единственное, что нам нужно учесть — это то, что *сумма должна быть выведена с точностью до двух символов после точки.*

Как мы можем это сделать? Ответ на этот вопрос оставим читателю.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1373#0.

Задача: числовое выражение

Бонни — необычайно могущественная ведьма. Поскольку ее природной силы недостаточно для успешной борьбы с вампирами и оборотнями, она начала осваивать силу Выражений. Выра-

жением очень сложно овладеть, потому что заклинание опирается на способность *быстро решать математические выражения*.

Чтобы использовать заклинание «Выражение», ведьма должна заранее знать результат математического выражения. Заклинание «Выражение» состоит из нескольких простых математических выражений. Каждое математическое выражение может содержать операторы для сложения, вычитания, умножения и/или деления.

Выражение решается без учета математических правил вычисления числовых выражений. Это означает, что приоритет применяется в соответствии с последовательностью операторов, а не с типом вычислений, которые они выполняют. Выражение может содержать скобки, так как все, что находится внутри скобок, вычисляется в первую очередь. Каждое выражение может содержать несколько скобок, но не вложенные скобки:

- Выражение, содержащее (...(...)...), является недопустимым.
- Выражение, содержащее (...)...(...), является допустимым.

Пример

Выражение

```
4 + 6 / 5 + (4 * 9 - 8) / 7 * 2
```

решается следующим образом:

```
4 + 6 / 5 + (4 * 9 - 8) / 7 * 2 =
10 / 5 + (4 * 9 - 8) / 7 * 2 =
2 + (4 * 9 - 8) / 7 * 2 =
2 + (36 - 8) / 7 * 2 =
2 + 28 / 7 * 2 =
30 / 7 * 2 =
4.285714285714286 * 2 =
8.57172857142571 =
8.57
```

Бонни очень красива, но не столь мудра, поэтому ей нужна наша помощь, чтобы овладеть силой Выражений.

Входные данные

Входные данные состоят из одной текстовой строки, передаваемой из консоли. В ней содержится *математическое выражение*

для вычисления. Строка *всегда заканчивается символом* «=». Символ «=» означает *конец математического выражения*.

Входные данные всегда действительны и всегда имеют описанный формат. Нет необходимости проверять их.

Выходные данные

Выходные данные должны быть выведены на консоль. Вывод состоит из одной строки: *результат* вычисленного математического выражения, *округленный до второй цифры после точки*.

Ограничения

- Выражения будут состоять *максимум из 2500 символов*.
- Числа каждого математического выражения будут находиться в диапазоне [1 ... 9].
- Операторами в математических выражениях всегда будут + (сложение), - (вычитание), / (деление) или * (умножение).
- Результат математического выражения будет находиться в диапазоне [-100000.00 ... 100000.00].
- Допустимое время выполнения: 0,1 секунды.
- Разрешенная память: 16 МБ.

Примеры ввода и вывода

Ввод	Вывод
`4+6/5+(4*9-8)/7*2=`	`8.57`

Ввод	Вывод
`3+(6/5)+(2*3/7)*7/2*(9/4+4*1)=`	`110.63`

Рекомендации и советы

Как обычно, сначала мы считаем и обработаем входные данные, затем решим задачу и, наконец, выведем результат, отформатированный так, как требуется по условию.

Обработка входных данных

Входные данные состоят ровно из одной строки из консоли.

Здесь у нас есть два способа обработки входных данных. Первый — это чтение всей строки с помощью команды `cin` и обращение к каждому символу (`char`) в строке через цикл `for`. Второй — чтение входных данных посимвольно с помощью команды `cin` и обработка каждого символа.

Для решения задачи воспользуемся вторым вариантом.

```
int symbol;
cin >> symbol;
```

Алгоритм решения задачи

Для решения нашей задачи нам понадобятся две переменные:

- Переменная, в которой мы будем хранить текущий результат.
- Еще одна переменная, в которой мы будем хранить текущий оператор нашего выражения.

```
double result = 0;
int expressionOperator = '+';
```

В приведенном выше коде следует уточнить, что по умолчанию оператор имеет значение +, поэтому к результату можно прибавить первое попавшееся число.

После того как мы получили исходные переменные, необходимо продумать, какой будет основная структура нашей программы. Из условия мы понимаем, что каждое выражение заканчивается на =, поэтому нам придется читать и обрабатывать символы до тех пор, пока мы не встретим =. Следующим шагом будет точное написание цикла `while`:

```
while (symbol != '=') {
    cin >> symbol;
}
```

Следующий шаг — обработка переменной `symbol`. Для нее мы имеем три возможных случая:

- Если символ является началом подвыражения, заключенного в круглые скобки, то есть встретившийся символ имеет вид (.
- Если символ представляет собой *число от 0 до 9*. Но как проверить, является ли наш символ цифрой? Здесь на помощь

приходит ASCII-код символа, и мы можем воспользоваться следующей формулой: `[ASCII-код нашего символа] - [ASCII-код символа 0] = [цифра, обозначающая символ]`. Если результат этой проверки находится в диапазоне от 0 до 9, то наш символ действительно является числом.

- Если символ является оператором, то есть +, -, * или /.

```
if (symbol == '(') {
}
else if ((0 <= symbol - '0') && (symbol - '0' <= 9)) {
}
else if (symbol == '+' ||
    symbol == '-' ||
    symbol == '/' ||
    symbol == '*') {
}
```

Реализация

Рассмотрим, какие действия необходимо предпринять в определенных нами случаях:

- Если наш символ является оператором, то все, что нам нужно сделать, это установить новое значение в переменную `expressionOperator`.
- Если наш символ — число, то необходимо изменить текущий результат выражения в зависимости от текущего оператора, то есть если `expressionOperator` — это -, то необходимо уменьшить результат на числовое представление текущего символа. Мы можем взять числовое представление текущего символа, используя формулу, которую мы использовали при проверке этого случая (`[ASCII-код нашего символа] - [ASCII-код символа 0] = [цифра, обозначающая символ]`).

```
else if (0 <= symbol - '0' && symbol - '0' <= 9) {
    switch (expressionOperator) {
        case '+':
            result += symbol - '0';
            break;
        case '-':
```

```
            result -= symbol - '0';
            break;
        case '*':
            result *= symbol - '0';
            break;
        case '/':
            result /= symbol - '0';
            break;
    }
}
else if (symbol == '+' ||
    symbol == '-' ||
    symbol == '/' ||
    symbol == '*') {
    expressionOperator = symbol;
}
```

- Если наш символ — (, то это означает начало подвыражения (выражения в круглых скобках). По определению подвыражение должно быть вычислено до модификации результата всего выражения (сначала выполняются действия в круглых скобках). Это означает, что у нас будет локальный результат для подвыражения и локальный оператор:

```
if (symbol == '(') {
    double innerResult = 0;
    int innerOpenator = '+';
    cin >> symbol;
}
```

Вычисление значения подвыражения

Далее для вычисления значения подвыражения мы используем те же методы, что и для вычисления основного выражения, — в цикле `while` считываем символы (пока не встретим символ скобки)). В зависимости от того, является ли считанный символ числом или оператором, мы модифицируем результат подвыражения. Реализация этих операций аналогична описанной выше реализации вычисления выражений, поэтому, думаем, у читателя не должно возникнуть проблем.

После завершения вычисления результата нашего подвыражения мы модифицируем результат всего выражения в зависимости от значения `expressionOperator`:

```
switch (expressionOperator) {
    case '+':
        result += innerResult;
        break;
    case '-':
        result -= innerResult;
        break;
    case '*':
        result *= innerResult;
        break;
    case '/':
        result /= innerResult;
        break;
}
```

Форматирование выходных данных

Единственный вывод, который программа должна вывести на консоль, — это результат решения выражения с точностью до двух знаков после десятичной точки. Как оформить вывод таким образом? Реализацию оставляем читателю.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1373#1.

Задача: быки и коровы

Все мы знаем игру под названием «Быки и коровы»: http://en.wikipedia.org/wiki/Bulls_and_cows. При наличии определенного четырехзначного *секретного числа* и четырехзначного *предлагаемого числа* действуют следующие правила:

- Если цифра в предложенном числе совпадает с цифрой в секретном числе и находится в *той же позиции*, у нас есть *бык*.
- Если цифра в предложенном числе совпадает с цифрой в секретном числе, но находится в *другой позиции* — *корова*.

Вот несколько примеров:

Секретное число	1	4	8	1	Комментарий
Предложенное число	8	8	1	1	Быки = 1 Коровы = 2

Секретное число	1	4	8	1	Комментарий
Предложенное число	9	9	2	4	Быки = 0 Коровы = 2

Имея определенное секретное число и относящихся к нему быков и коров, наша задача — найти все возможные предложенные числа в порядке возрастания.

Если предложенных чисел, соответствующих заданным критериям, нет, нужно вывести на экран «Нет».

Входные данные

Входные данные считываются с консоли. Они состоят из трех текстовых строк:

- Первая строка содержит *секретное число*.
- Вторая строка содержит *количество быков*.
- В третьей строке указано *количество коров*.

Входные данные всегда будут действительными. Нет необходимости проверять их.

Выходные данные

Выходные данные должны быть выведены на консоль. Вывод должен состоять из одной строки, содержащей все *предложенные числа*, разделенные пробелами. Если нет ни одного предложенного числа, соответствующего критериям, заданным в консоли, мы должны вывести «Нет».

Ограничения

- Секретное число всегда состоит из четырех цифр в диапазоне [1..9].
- Количество коров и быков всегда будет находиться в диапазоне [0..9].
- Допустимое время выполнения: 0,15 секунды.
- Разрешенная память: 16 МБ.

Примеры ввода и вывода

Ввод	Вывод
1234 3 0	1134 1214 1224 1231 1232 1233 1235 1236 1237 1238 1239 1244 1254 1264 1274 1284 1294 1334 1434 1534 1634 1734 1834 1934 2234 3234 4234 5234 6234 7234 8234 9234

Ввод	Вывод
2228 2 1	1222 2122 2212 2232 2242 2252 2262 2272 2281 2283 2284 2285 2286 2287 2289 2292 2322 2422 2522 2622 2722 2821 2823 2824 2825 2826 2827 2829 2922 3222 4222 5222 6222 7222 8221 8223 8224 8225 8226 8227 8229 9222

Рекомендации и советы

Решим задачу в несколько этапов:

- Прочитаем *входные данные*.
- Сгенерируем все возможные *четырехзначные комбинации* (кандидаты на проверку).
- Для каждой сгенерированной комбинации подсчитаем, *сколько быков и сколько коров* она имеет в соответствии с секретным числом. После подбора нужных быков и коров выведем комбинацию на экран.

Чтение входных данных

У нас есть три строки во входных данных:

- Секретное число.
- Количество требуемых быков.
- Количество требуемых коров.

Чтение входных данных производится обычным способом:

```
int guessNumber;
cin >> guessNumber;

int targetBulls;
cin >> targetBull

int targetCows;
cin >> targetCows;
```

Алгоритм решения задачи

Прежде чем приступить к написанию алгоритма решения нашей задачи, необходимо объявить флаг, указывающий, найдено ли решение:

```
bool solutionFound = false;
```

Если после завершения работы нашего алгоритма этот флаг все еще будет ложным, то мы выведем на консоль «Нет», как указано в условии:

```
if(!solutionfound) {
    cout << "Нет" << endl;
}
```

Давайте приступим к анализу нашей задачи. Нам нужно проанализировать все числа от 1111 до 9999, исключая те, которые содержат нули (например, 9011, 3401 и т. д. недействительны).

Как проще всего сгенерировать все эти числа? Мы будем использовать *вложенные циклы*. Поскольку у нас есть четырехзначное число, у нас будет четыре вложенных цикла, потому что каждый из них будет генерировать *отдельную цифру в нашем числе* для проверки:

```
for (int digit1 = 1; digit1 <= 9; digit1++) {
    for (int digit2 = 1; digit2 <= 9; digit2++) {
        for (int digit3 = 1; digit3 <= 9; digit3++) {
            for (int digit4 = 1; digit4 <= 9; digit4++) {
```

Благодаря этим циклам мы имеем доступ к каждой цифре всех чисел, которые нам нужно проверить. Следующий шаг — разделить секретное число на цифры. Этого можно легко добиться с помощью комбинации целочисленного и модульного деления:

```
int guessDigit1 = (guessNumber / 1000) % 10;
int guessDigit1 = (guessNumber / 100)% 10;
int guessDigit1 = (guessNumber / 100)% 10;
int guessDigit1 = (guessNumber / 1) %10;
```

Перед тем как приступить к анализу количества коров и быков в числе, нам осталось сделать два последних шага. Соответственно, первый — это объявление переменных-счетчиков во вложенных циклах для подсчета коров и быков для текущего числа. Второй

шаг — создание копий цифр текущего числа, которые мы будем анализировать, для предотвращения возможных проблем с работой вложенных циклов (например, если мы внесем изменения в числа):

```
int digitToCheck1 = digit1;
int digitToCheck2 = digit2;
int digitToCheck3 = digit3;
int digitToCheck4 = digit4;

int currentBulls = 0;
int currentCows = 0;
```

Теперь мы готовы приступить к анализу полученных чисел. Какую логику мы можем использовать? Самый элементарный способ проверить, сколько коров и быков содержится в числе, — это *серия проверок* `if-else`. Да, это не самый оптимальный способ, но ради того, чтобы не использовать знания, выходящие за рамки этой книги, мы выберем именно такой подход.

Какие проверки нам нужны?

Проверка на наличие быков элементарна — мы проверяем, совпадает ли первая цифра сгенерированного числа с той же цифрой секретного числа. Проверенные цифры мы удаляем, чтобы избежать повторения «быков» и «коров».

```
//Находим всех быков, считаем и удаляем (присваиваем -1 и -2)
if (digitToCheckl == guessDigit1) {
    //Бык на позиции #1 найден -> считаем и удаляем
    currentBulls++;
    guessDigitl = -1;
    digitToCheckl = -2;
```

Повторите для второй, третьей и четвертой цифр.

Проверить наличие коров можно следующим образом: сначала проверяется, совпадает ли первая цифра сгенерированного числа со второй, третьей или четвертой цифрой секретного числа.

Пример реализации:

```
//Найдем всех коров для digitToCheckl, посчитаем их и удалим
(присвоим -1)
 if (digitToCheckl == guessDigit2) {
     //Корова на позиции #2 найдена -> считаем и удаляем
```

```
        currentCows++;
        guessDigit2 = -1;
    }
    else if (digitToCheck1 == guessDigit3) {
        //Корова на позиции #3 найдена -> считаем и удаляем
        currentCows++;
        guessDigit3 = -1;
    }
    else if (digitToCheck1 == guessDigit4) {
        //Корова на позиции #4 найдена -> считаем и удаляем
        currentCows++;
        guessDigit4 = -1;
    }
```

Затем мы последовательно проверяем, совпадает ли вторая цифра сгенерированного числа с первой, третьей или четвертой цифрой секретного числа, совпадает ли третья цифра сгенерированного числа с первой, второй или четвертой цифрой секретного числа и, наконец, проверяем, совпадает ли четвертая цифра сгенерированного числа с первой, второй или третьей цифрой секретного числа.

Вывод результатов

После выполнения всех условий нам остается проверить, *совпадают ли быки и коровы в текущем сгенерированном числе с требуемыми быками и коровами, считанными с консоли*. Если это так, то мы выводим текущее число на консоль:

```
    if (currentBulls == targetBulls && currentCows == targetCows) {
        if (solutionFound) {
            cout << " ";
        }

        cout << digit1 << digit2 << digit3 << digit4;
        solutionFound = true;
    }
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1373#2.

Глава 10. Функции

В этой главе мы познакомимся с функциями и узнаем, что они собой представляют, а также каковы основные понятия при работе с ними. Мы выясним, почему их использование является хорошей практикой, как их определять и как вызывать. Мы узнаем о параметрах и возвращаемом значении функции, а также о том, как использовать это возвращаемое значение. В конце главы мы рассмотрим сложившуюся практику использования функций.

Что такое функция?

К данному моменту мы убедились, что при написании программы, решающей какую-либо задачу, проще разделить ее на части. Каждая часть отвечает за определенное действие, и это не только облегчает решение задачи, но и значительно улучшает как читаемость кода, так и прослеживаемость логики программы и поиск ошибок.

В контексте программирования функция (метод) — это именованная группа инструкций, выполняющая заданную функциональность. Эта группа инструкций логически разделена и поименована таким образом, чтобы в процессе выполнения программы можно было запускать выполнение инструкций данной группы по этому имени. Запуск выполнения инструкций в функции называется вызовом функции (по-английски — function call или invoking a function).

Функция может быть вызвана столько раз, сколько нам необходимо для решения задачи. Это избавляет нас от многократного повторения одного и того же кода, что, в свою очередь, снижает вероятность упустить ошибку при возможном исправлении рассматриваемого кода.

Рассмотрим два типа функций — простые (без параметров) и сложные (с параметрами).

Простые функции

Простые функции отвечают за выполнение действия, которое помогает решить конкретную задачу. Такими действиями могут быть, например, вывод строки на консоль, выполнение некоторой проверки, выполнение цикла и т. д.

Рассмотрим следующий пример простой функции:

```
void printHeader() {
    cout << "-------------";
}
```

Однако у функции есть и другая часть, не зависящая от вышеуказанной:

```
void printHeader();
```

Мы подробно прокомментируем различия между ними — их структуру, использование и значение — в этой главе.

В задачу этой функции входит вывод заголовка, представляющего собой последовательность символов `-`. Поэтому ее название — `printHeader`. Круглые скобки `(` и `)` всегда следуют за именем, независимо от того, как мы назвали функцию. Позже в этой главе мы рассмотрим устоявшуюся практику именования функций, а пока отметим: важно, чтобы имя описывало действие, которое она выполняет. Мы также поговорим о ключевом слове `void`.

Тело функции, находящееся между фигурными скобками `{` и `}`, содержит программный код (инструкции), решающий задачу, описанную ее именем. Тело функции записывается далее внутри, обычно через четыре пробела (одну табуляцию), которые отделяют его как отдельный блок инструкций, примыкающий к функции.

Зачем использовать функции?

К настоящему времени мы убедились, что функции помогают разделить объемную задачу на более мелкие части, что приводит к более легкому решению поставленной задачи. Это делает нашу программу не только более структурированной и удобной для чтения, но и более понятной.

С помощью функций мы избегаем повторения программного кода. Повторяющийся код — плохая практика, подобные повторы *усложняют сопровождение программы* и *приводят к ошибкам*. Если определенная часть кода встречается в программе более одного раза и ее нужно изменить, то изменения должны быть внесены во все повторения данного кода. Велика вероятность пропустить место, где необходимо внести исправления, что приведет к некорректному поведению программы.

Именно поэтому хорошей практикой является использование определенного фрагмента кода более одного раза и *определение его как отдельной функции*. Использование функций экономит много времени и сил. Кстати, каждый раз, когда мы пишем программу на языке C++, мы используем функцию `main()`.

Объявление и определение функций

В языке программирования C++ существует разница между понятиями *объявления* и *определения* функции:

- Объявление функции сообщает компилятору или интерпретатору о существовании функции с соответствующим именем и параметрами, не содержащей реализации (тела функции).
- Определение функции содержит ее реализацию (тело).

Объявление

Рассмотрим следующий пример объявления функции, которая находит площадь квадрата с заданной стороной `num`:

```
double getSquare(double);
```

Следующее утверждение эквивалентно приведенному выше, и мы прокомментируем его далее в главе:

```
double getSquare(double num);
```

Обратим внимание на следующее: приведенный фрагмент программы лишь сообщает компилятору или интерпретатору, что в данной программе будет определена и использована функция `getSquare()`.

Объявление функции происходит после указания использования пространства имен `std` (`using namespace std;`) и перед функци-

ей `main()`. В приведенном примере показаны объявления функций `getSquare()`, `multiply()` и `printHeader()`:

```
#include <iostream>
using namespace std;

double getSquare(double num);
int multiply(int, int);
void printHeader();

int main() {
}
```

Обязательные элементы при объявлении функции:

- *Тип возвращаемого значения*. В данном случае тип — `double`, что означает, что функция `getSquare(...)` вернет результат типа `double`. Возвращаемое значение может быть `int`, `double`, `char` и т. д. или `void`. Если указан тип `void`, то это означает, что функция не возвращает результат, а только выполняет операцию.
- *Имя функции*. Имя функции задаем мы сами, помня о том, что оно должно описывать операцию, которую выполняет код в ее теле. В примере имя — `getSquare`, что говорит нам о том, что задача этой функции — вычислить площадь квадрата.
- *Список параметров*. Он объявляется между круглыми скобками `(` и `)`, которые следуют за именем функции. Здесь перечисляется последовательность *параметров*, которые она будет использовать, причем подразумевается, что они могут быть разных типов (`int`, `double`, `char` и т. д.). Интересно то, что здесь можно (и даже обычно нужно) просто записать тип используемых параметров (то есть не идентифицировать их). Это допускается только при объявлении функции. Может присутствовать только один параметр, может присутствовать несколько параметров или пустой список. Если параметров нет, то пишем только круглые скобки `()`. В данном примере мы объявляем параметр `double num`.
- В конце ставим точку с запятой `;`.

Точка с запятой ; обязательна в конце объявления функции.

При объявлении функции важно соблюдать последовательность расположения ее основных элементов — сначала пишется тип возвращаемого значения, затем имя функции и список параметров, окруженных круглыми скобками (), и точку с запятой ; в конце.

Определение

После объявления функции следует ее определение. Определение функции — это описание ее тела (реализации). Тело содержит код (программный блок), реализующий логику работы функции. В приведенном примере мы вычисляем площадь квадрата, а именно `num * num`. Это и есть определение функции `getSquare(...)`, объявленной выше:

```
double getSquare(double num) {
    return num * num;
}
```

Определения функций реализуются после функции `main()`. Полный пример мы рассмотрим в ближайшее время.

Необходимые элементы при определении функции:

- *Тип возвращаемого значения*. Тип должен соответствовать типу возвращаемого значения в объявлении функции, то есть быть одинаковым. Если в объявлении написано, что функция возвращает значение типа `int`, то в определении тип должен быть `int`.
- *Имя функции*. Имя должно быть таким же, как в объявлении функции.
- *Список параметров*. Он объявляется между круглыми скобками (и). Опять же, мы следуем тому, что написано в объявлении функции. Если параметров несколько, то мы записываем их в том же порядке, что и в объявлении функции. Например, если мы объявили функцию `function(double, int, double, char)`, то в ее определении мы расположим параметры в том же порядке — `double`, `int`, `double`, `char`. Кроме того, здесь необходимо идентифицировать рассма-

триваемые параметры (в отличие от объявления, где это можно не делать). Поэтому пример идентификации параметров будет выглядеть так: `(double var1, int var2, double var3, char var4)`.

- *Реализация (тело)*. Записывается в области, отделенной фигурными скобками { и }. Эта область создается после списка параметров. Тело функции описывает алгоритм (инструкции), с помощью которого она решает задачу, то есть тело содержит код, реализующий логику функции. Если функция возвращает значение, то в конце реализации добавляется оператор `return`, за которым следует то, что мы собираемся вернуть, и символ `;`. О возвращаемых и невозвращаемых значениях мы поговорим позже.
- Здесь не ставится `;` в конце, как в объявлении функции.

Типы переменных

Когда мы объявляем переменную в теле функции, мы называем ее *локальной переменной* функции. Область, в которой эта переменная существует и может быть использована, начинается со строки, в которой мы ее объявили, и продолжается до закрывающей фигурной скобки } тела функции. Эта область называется *областью видимости переменной* (variable scope). Если мы объявляем переменную извне всех функций программы (включая функцию `main()`), то называем ее *глобальной переменной* программы. К ней можно обращаться в любой части программного кода.

Параметры функций

Это те данные (переменные или даже константы), которые передаются функции в момент ее вызова для их обработки, возврата значения или выполнения над ними какого-либо действия. При объявлении и определении функции необходимо следить за тем, чтобы параметры располагались в одном и том же порядке.

Вызов функции

Вызов функции — это начало выполнения кода, находящегося в теле функции. Для этого необходимо набрать имя функции, затем круглые скобки `()` и символ `;` для завершения строки. Если нашей

функции требуются входные данные, то они передаются в круглых скобках (), а последовательность фактических параметров должна соответствовать последовательности, переданной при объявлении функции. Приведем пример:

```
//Объявление функции
void sum(int, int, int);

//Вызов функции
int main() {
    int a1, b1, c1;
    cin >> a1 >> b1 >> c1;
    sum(a1, b1, c1);
    return 0;
}

//Определение функции
void sum(int a, int b, int c) {
    cout << a + b + c;
}
```

Функция может быть вызвана из нескольких мест программы. Один из способов — вызвать ее из функции main:

```
//Использование функции в main
int main() {
    printHeader();
    return 0;
}
```

Функция также может быть вызвана из тела любой другой функции:

```
//Использование функции в другой функции
void print() {
    printHeader();
    printFooter();
}
```

Важно: рассматриваемые здесь функции не возвращают значения (результата). Мы обсудим их позже, в главе «Возврат результата функции».

Существует возможность вызова функции из собственного тела. Это называется рекурсией, и более подробную информацию о ней

можно найти в Википедии или самостоятельно поискать в Интернете.

Обобщение

До сих пор мы подробно рассматривали элементы функции в языке C++ и их распределение в программном коде. Как оказалось, рассмотренная нами методология работы с функциями является стандартной, но не единственной.

Напишем программу, которая будет использовать функцию (`printSentence()`) для вывода предложения «`Я изучаю функции на C++`». Используем уже известный нам алгоритм:

- Объявим функцию `printSentence()` перед функцией `main()`.
- Определим нашу функцию `printSentence()` после функции `main()`.
- Вызовем функцию `printSentence()` в `main ()`.

```
// Первый подход
#include <iostream>
using namespace std;
// Объявление
 void printSentence();

int main() {
    // Вызов функции
    printSentence();
    return 0;
}

// Определение
void printSentence() {
    cout << "Я изучаю функции на C++.";
}
```

Протестировать пример можно в Интернете: https://repl.it/@vncpetrov/printSentence1.

В чем заключается другой подход? Рассмотрим следующую программу, решающую ту же задачу:

```
// Второй подход
#include <iostream>
using namespace std;
```

```
// Декларация и определение одновременно
void printSentence() {
    cout << "Я изучаю функции.";
}

int main() {
    // Вызов функции
    printSentence();
    return 0;
}
```

Протестировать пример можно в Интернете: https://repl.it/@vncpetrov/printSentence2-1.

Результат работы обеих программ одинаков. Разница лишь в том, что во втором подходе объявление и определение функции объединяются в одну, и происходит это перед функцией `main()`. Второй подход требует меньшего объема кода, но на практике часто используется первый подход, так как он обеспечивает лучшую читаемость программы и по другим причинам.

В данной книге используется первый подход (то есть с отдельным объявлением и определением). Этот подход также является рекомендуемым.

Прототип функции

Уточним еще один момент — вместо того, чтобы использовать термин «объявление функции» (как мы это делали до сих пор для наглядности), мы будем говорить о прототипе функции.

Само название говорит нам о том, что мы уже выяснили выше, а именно: мы объявляем функцию (делаем прототип) до `main()`, создаем ее (пишем ее тело) после `main()` и используем (вызываем) либо в `main()`, либо в другой функции. Единственная функция, для которой мы не делаем прототип, это `main()`.

Пример: печать пустого чека

Напишем функцию, печатающую пустой чек.

Функция должна вызывать три другие функции: одну для печати заголовка, одну для средней части и одну для нижнего колонтитула.

Часть квитанции	Текст
Верхняя часть	КАССОВЫЙ ЧЕК _____
Средняя часть	Взимается с _____ Получено _____
Нижняя часть	_____ (c) SoftUni

Пример ввода и вывода

Ввод	Вывод
(без ввода)	КАССОВЫЙ ЧЕК _____ Взимается с _____ Получено _____ _____ (c) SoftUni

Рекомендации и советы

Первым шагом будет создание функции типа `void` для вывода заголовка квитанции. Под созданием функции мы понимаем написание прототипа и определения отдельно. Дадим ей осмысленное имя, коротко и ясно описывающее ее задачу, например `printReceiptHeader()`. А вот как будет выглядеть ее определение:

```
void printReceiptHeader() {
    cout << "КАССОВЫЙ ЧЕК" « endl;
    cout << "_____" << endl;
}
```

С этого момента примеры, приводимые в главе, будут содержать только определения функций. Создание их прототипов мы оставляем читателю для самостоятельной реализации.

Аналогично создаем еще две функции — для вывода средней части чека (тела) `printReceiptBody()` и для вывода нижней части чека (footer) `printReceiptFooter()`. Затем создадим еще одну функцию (`printReceipt()`), которая будет последовательно вызывать все три функции, написанные нами у этому моменту:

```
void printReceipt {
    printReceiptHeader();
    printReceiptBody();
    printReceiptFooter()
}
```

Наконец, вызовем функцию `printReceipt()` из тела функции `main()` нашей программы:

```
int main() {
    printReceipt();
    return 0;
}
```

Количество соответствующих символов в выводе должно быть таким, как в приведенном примере.

Проверка в судейской системе

Программа, состоящая из пяти функций, вызывающих друг друга, готова, и мы можем ее запустить и протестировать, а затем отправить на проверку в судейскую систему: https://judge.softuni.bg/Contests/Practice/Index/1374#0.

Функции с параметрами

Очень часто на практике для решения той или иной задачи функции, с помощью которой мы этого добиваемся, требуется дополнительная информация, зависящая от ее задачи. Именно эту информацию представляют собой параметры функции, от которых зависит ее поведение.

Использование параметров в функциях

Как мы заметили выше, параметров может быть ноль, один или несколько. При их объявлении необходимо разделять их запятой. Они могут быть разных типов данных (`int`, `string` и т. д.), и ниже приведен пример того, как именно они будут использоваться функцией.

Мы создаем прототип функции, и самое малое, что мы указываем в списке параметров, — это тип данных параметров, то есть их не нужно именовать. Однако в определении это обязательно. Здесь же пишется тело функции.

```
// Определение
void printNumbers(int start, int end) {
    for (int i = start, i <= end; i++) {
        cout << i;
    }
}
```

Затем мы вызываем функцию, передавая ей определенные значения:

```
int main() {
    int a = 5; b = 10;
    printNumbers(a, b);
}
```

При объявлении параметров мы можем использовать различные типы переменных, при этом необходимо следить за тем, чтобы каждый параметр имел тип (и имя в определении).

Важно отметить, что при последующем вызове функции мы должны передавать значения параметров в том порядке, в котором они объявлены. Если у нас есть функция с параметрами `int` и `char`, расположенными в таком порядке, то при вызове функции мы не можем передать сначала значение для `char`, а затем для `int`.

Единственный способ поменять местами местоположение передаваемых параметров — это явно записать сначала имя параметра, как мы увидим чуть позже в одном из примеров. В общем случае это не очень хорошая практика!

Рассмотрим пример объявления функции, имеющей несколько параметров различных типов:

```
//Определение
void data(char letter, int number1, double number2 {
    cout << "Буква - это  " << letter << endl;
    cout << "Целое число - это " << number1 << endl;
    cout << "Вещественное число - это " << number2;
```

Пример: знак целого числа

Создадим функцию, которая выводит информацию о том, является ли заданное целое число положительным, отрицательным или нулевым.

Примеры ввода и вывода

Ввод	Вывод
2	Число 2 – положительное.
-5	Число -5 – отрицательное.
0	Число 0 – ноль.

Рекомендации и советы

Первым делом мы создадим функцию и дадим ей описательное имя, например `printSign(...)`. Эта функция будет иметь только один параметр (типа `int`) — число, знак которого мы хотим проверить. Следующий шаг — реализовать логику, по которой наша программа будет проверять, какой именно знак у числа. Из примеров видно, что существует три случая — число больше нуля, равно нулю или меньше нуля, а значит, в теле функции мы будем делать три проверки:

```
void printSign(int a) {
    cout << "Число" << a << "–";
    if (a == 0) {
        cout << "ноль";
    }
    else {
        if (a > 0) {
            cout << "положительное";
        }
        else {
            cout << "отрицательное";
        }
    }
}
```

Следующим нашим шагом будет чтение входного числа и вызов новой функции из тела функции `main()`:

```
int main() {
    int n;
    cin >> n;
    printSign(n)
    return 0;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#1.

Вы можете заметить, что переменная в `main()`, которую мы передаем как параметр функции `printSign(...)`, имеет имя `n`, а в `printSign(...)` — имя `a`. Мы могли бы использовать в рассматриваемой функции параметр с тем же именем, что и переменная в `main()`, то есть `n`.

В языке C++ заполнение списка параметров при вызове функций является обязательным, поскольку сами функции объявляются и количество параметров, типы и расположение данных должны быть одинаковыми.

Пример: вывод треугольника

Создадим функцию, которая выводит на экран треугольник, как показано в примерах.

Ввод	Вывод
2	1 1 2 1

Ввод	Вывод
3	1 1 2 1 2 3 1 2 1

Ввод	Вывод
4	1 1 2 1 2 3 1 2 3 4 1 2 3 1 2 1

Ввод	Вывод
1	1

Рекомендации и советы

Перед тем как создать функцию для вывода одной строки с заданными началом и концом, мы считываем входное число с консоли.

Затем мы выбираем для функции осмысленное имя, описывающее ее назначение, например `printLine(...)`, и реализуем ее:

```
void printLine(int start, int end) {
    for (int i = start; i <= end; i++) {
        cout << i;
    }
    cout << endl;
}
```

Из задач консольного рисования мы помним, что хорошей практикой является деление фигуры на несколько частей. Для удобства разделим треугольник на три части — верхнюю, среднюю и нижнюю.

Следующим шагом будет использование цикла для вывода верхней части треугольника:

```
for (int i = 0; i < n; i++) {
    printLine(1, i);
}
```

Затем выводим среднюю строку:

```
printLine(1, n);
```

Наконец, выводим нижнюю часть треугольника, на этот раз уменьшив шаг цикла.

```
for (int i = n-1; i > 0; i--) {
    printLine(1, i);
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#2.

Пример: рисование заполненного квадрата

Нарисуйте на консоли заполненный квадрат со стороной `n`, как показано в примерах.

Ввод	Вывод
4	-------- -\/\/\/- -\/\/\/- --------

Ввод	Вывод
3	------ -\/\/- -\/\/- ------

Ввод	Вывод
2	---- ----

Рекомендации и советы

Первым шагом будет чтение входных данных из консоли. Далее необходимо создать функцию, которая будет выводить первую и последнюю строки, поскольку они одинаковы. Не забудем, что нужно дать ей *описательное имя* и задать в качестве *параметра* длину ее стороны:

```
// Определение
void printHeaderFooter(int n) {
    for (int i = 0; i < n; i++) {
        cout << "-";
    }
    cout << endl;
}
```

Следующим шагом будет создание функции, которая будет выводить средние строки на консоль. Снова зададим описательное имя, например `printMiddleRow(...)`:

```
// Определение
void printMiddleRow(int n) {
    cout << "-";
    for (int i = 0; i <= n - 1; i++) {
        cout << "\\/";
    }
    cout << "-" << endl;
}
```

Отметим, что для того, чтобы вывести символ «\», мы должны ввести его в программу «\\» (чтобы экранировать символ). Нако-

нец, для рисования всего квадрата мы вызываем функции, созданные в функции `main()` программы:

```
int main() {
    int input;
    cin >> input;
    printHeaderFooter(input);
    // TODO: Отрисовать остальную часть квадрата
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#3.

Возвращаемый результат функций

До сих пор мы рассматривали функции, которые выполняют действие, например, выводят на консоль текст, число или цифру. Помимо функций этого типа существуют функции, которые могут возвращать некоторый результат выполнения — например, результат умножения двух чисел. Именно эти функции мы и рассмотрим далее.

Тип значения, возвращаемого функцией

До сих пор мы рассматривали примеры, в которых ключевое слово `void` использовалось в объявлениях функций для указания на то, что функция не возвращает результат, а выполняет определенное действие:

```
void addOne(int n) {
    n += 1;
    cout << n;
}
```

Если заменить ключевое слово `void` на какой-либо тип данных, то это укажет программе, что функция должна вернуть некоторое значение указанного типа. Это возвращаемое значение может быть любого типа — `int`, `char`, `double` и т. д.

Для того чтобы функция возвращала результат, при объявлении функции вместо `void` нужно написать тип возвращаемого значения, который мы ожидаем.

```
int plusOne(int n) {
    return n + 1;
}
```

Важно отметить, что результат, возвращаемый функцией, может иметь тип, совместимый с типом возвращаемого значения функции. Например, если объявленный тип возвращаемого значения — `double`, то мы можем вернуть результат типа `int`.

Оператор return

Для возврата результата из функции используется оператор `return`. Он должен использоваться в теле функции и указывает программе на необходимость прекратить выполнение и вернуть тому, кто вызывает функцию, значение, которое определяется выражением после оператора `return`.

Оператор `return` может также использоваться в функциях, которые не возвращают результат (`void`-функциях). После оператора не должно быть выражения для возврата. Тогда функция сама прекратит выполнение, не вернув значения. В этом случае единственное использование `return` — это выход из функции. Также возможно использование оператора `return` более чем в одном месте в теле функции.

В приведенном ниже примере мы имеем функцию, которая сравнивает два числа и возвращает `-1`, `0` или `1` в зависимости от того, меньше, равен или больше первый аргумент, чем второй, переданный в функцию. Функция использует ключевое слово `return` в трех разных местах, чтобы вернуть три разных значения в соответствии с логикой сравнения чисел:

```
// Определение
int compareTo(int number1, int number2) {
    if (number1 > number2) {
        return 1;
    }
    else {
        if (number1 == number2) {
            return 0;
        }
        else {
```

```
            return -1;
        }
    }
}
```

Код после оператора return недоступен

После оператора `return` в текущем блоке не должно быть других строк кода, поскольку выполнение функции завершается, и программа продолжается с того места, где была вызвана функция. Если после оператора `return` есть другие инструкции, то они не будут выполнены. Некоторые редакторы, в том числе и Visual Studio, выводят предупреждение о том, что обнаружен код, доступ к которому невозможен:

```
int someFunction() {
    return 7;
    return 5; // Эта часть кода недоступна
}
```

В программировании *нельзя использовать оператор* `return` *дважды*, один за другим (double return), потому что выполнение первого не позволит выполнить второй. Иногда болгарские программисты шутят про двойной `return`, чтобы объяснить, что логика программы утрачена.

Использование возвращаемого значения функции

После выполнения функции и возвращения значения это значение может быть использовано несколькими способами.

Первый — присвоить результат в качестве значения переменной совместимого типа:

```
int max = GetMax(5, 10);
```

Второй — для использования результата в выражении:

```
double total = getPrice() * + addition;
```

Третий — передать результат функции другой функции:

```
cout << function(a);


function1(function2(variable));
```

Пример: вычисление площади треугольника

Напишите функцию, которая вычисляет площадь треугольника по заданным основанию и высоте и возвращает ее в качестве результата.

Пример ввода и вывода

Ввод	Вывод
3 4	6

Рекомендации и советы

Сначала создадим функцию, которая вычислит площадь по двум переменным — длине стороны `length` и высоте `height`. На этот раз мы внимательно следим за тем, чтобы передать правильный тип данных, который мы хотим вернуть функции, а именно `float` (может быть также `double`):

```
float getTriArea(float length, float height) {
    return (length * height) / 2;
}
```

Следующим шагом будет чтение входных данных и вызов новой функции с их использованием.

Полученный результат мы записываем в соответствующую переменную и выводим на экран:

```
int main() {
    float a, b;
    cin >> a;
    cin >> b;

    float area = getTriArea(a, b);
    cout << area;

    return 0;
}
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#4.

Пример: степень числа

Напишите функцию, которая вычисляет и возвращает результат возведения числа в заданную степень.

Примеры ввода и вывода

Ввод	Вывод
2 8	256

Ввод	Вывод
3 4	81

Рекомендации и советы

Первым нашим шагом снова будет чтение входных данных из консоли.

Следующий шаг — создание функции, которая будет принимать два параметра (число и степень) и возвращать в качестве результата число типа `double`:

```
// Определение
double calculatePower(double num, double pow) {
    double result = 1;
    // Используем цикл для вычисления
    return result;
}
```

Задача может быть решена с помощью функции `pow(...)` из библиотеки `cmath`. При ее использовании рекомендуется изменить идентификатор параметра `pow`, чтобы избежать путаницы в программе.

После того как мы выполнили необходимые вычисления, нужно вызвать определенную функцию и вывести результат.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1374#5.

Функции, возвращающие несколько значений

Иногда функция должна возвращать более одного значения. Конечно, очевидно, что такая функция будет сложной и может быть разбита на отдельные функции. Но все же, если нам нужно, чтобы одна функция возвращала несколько значений, необходим другой подход. Он может быть связан с использованием массивов, векторов и т. д., но они не являются предметом изучения в данной книге. Однако рассмотрим хотя бы один универсальный способ группировки данных — введем понятие структур (или структурных типов данных).

Структуры

Если мы объявляем структуру, то это означает, что мы создаем набор переменных, логически связанных, которые, конечно, также объявляются. Приведем пример структуры:

```
struct student {
    string name;
    int age;
    char sex;
    int grade;
};
```

Переменные `name`, `age`, `sex` и `grade` являются элементами структуры `student`. Объявление `student` делается перед основной функцией `main`. Тело структуры располагается между фигурными скобками { и }, с обязательной точкой с запятой ; после закрывающей скобки }. Как же она используется в программе?

```
int main() {
    struct student pupil;
    cin >> pupil.name;
    cin >> pupil.age;
    cin >> pupil.sex;
    cin >> pupil.grade;
    return 0;
}
```

В `main()` мы объявляем переменную `pupil` со структурированным типом данных `student`, которая, в свою очередь, уже объявлена перед `main()`. Вводимые после этого значения на самом деле

являются элементами переменной pupil. Доступ к ним осуществляется путем ввода имени объявленной нами переменной структурированного типа данных (в данном случае pupil), затем вставки точки . и, наконец, ввода нужного элемента (например name). Таким образом, чтобы ввести имя данного ученика, мы используем pupil.name.

Важно отметить, что действия могут производиться только с элементами структур, но не с именами самих структур.

Структуры и функции

Вот как будет выглядеть прототип функции, использующей переменную структурированного типа данных:

```
student include(struct student pupil);
```

Определение:

```
student include(struct student pupil) {
    cin >> pupil.name;
    cin >> pupil.age;
    cin >> pupil.sex;
    cin >> pupil.grade;
    return pupil;
}
```

В данном случае это функция, которая будет возвращать переменную со структурированным типом данных student. Таким образом, функция возвращает четыре значения (для фамилии, возраста, пола и класса студента).

Пусть читатель самостоятельно напишет всю программу с использованием функции, введя четыре компонента характеристики ученика, а затем завершит программу так, чтобы функция выполнялась для каждого ученика школы с n учениками.

Варианты функций

Во многих языках программирования одна и та же функция может быть объявлена в нескольких вариантах с одним и тем же именем и разными параметрами. Это называется перегрузкой функций.

Теперь подробно рассмотрим, как пишутся такие перегруженные функции.

Сигнатура функции

В программировании функция идентифицируется по паре элементов в ее объявлении — имени функции и списку ее параметров. Эти два элемента определяют ее спецификацию, так называемую сигнатуру функции:

```
void print(char symbol) {
    cout << symbol;
}
```

В данном примере сигнатурой функции является ее имя `print`, а параметром — `char symbol`.

Если в нашей программе есть функции с одинаковыми именами, но разными сигнатурами, то мы говорим, что у нас происходит перегрузка функции (function overloading).

Варианты функций

Как уже говорилось, если мы используем одно и то же имя для нескольких функций с разными сигнатурами, это означает, что у нас есть варианты функции. Приведенный ниже код показывает, как три разные функции могут иметь одно и то же имя, но иметь разные сигнатуры и выполнять разные действия:

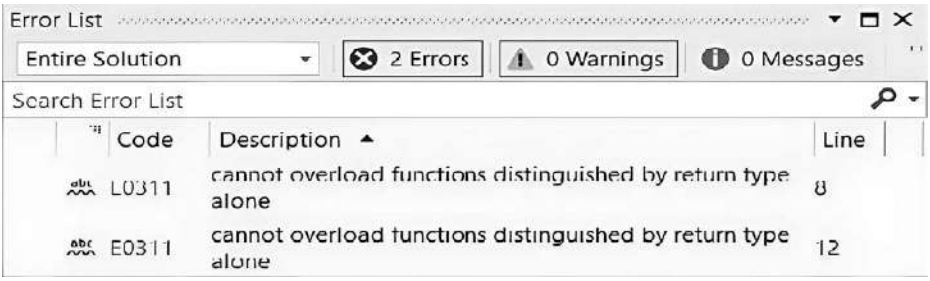
```
void print(char symbol) {
    cout << symbol;
}

void print(int number) {
    cout << number;
}

void print(char symbol, int number) {
    cout << symbol << " " << number;
}
```

Сигнатура и тип возвращаемого значения

Важно отметить, что тип, возвращаемый в результате работы функции, не является частью ее сигнатуры. Если бы возвращаемое значение было частью сигнатуры функции, то компилятор не смог бы точно определить, какую функцию следует вызвать.

Рассмотрим следующий пример — у нас есть две функции с разными типами возвращаемых значений. Однако Visual Studio показывает нам ошибку, поскольку сигнатуры обеих функций одинаковы. Соответственно, при попытке вызвать функцию с именем `print(...)` компилятор не смог решить, какую из двух функций следует выполнить:

```
8   void print(int number) {
9       cin >> number;
10  }
11
12  int print(int number) {
13      number = 7;
14  }
```



Пример: большее из двух значений

На вход подаются два значения одного типа. Значения могут быть типа `int`, `char` или `string`. Создайте функцию `getMax(...)`, которая возвращает в качестве результата большее из двух значений.

Примеры ввода и вывода

Ввод	Вывод
int 2 16	16

Ввод	Вывод
char a z	z

Ввод	Вывод
string Ivan Todor	Todor

Рекомендации и советы

Для создания этой функции сначала нужно создать три другие функции с одинаковыми именами и разными сигнатурами. Сначала создадим функцию, которая будет сравнивать целые числа:

```
// Определение
int getMax(int first, int second) {
    if (first >= second) {
        return first;
    }
    else return second;
}
```

Следуя логике предыдущей функции, мы создаем функцию с тем же именем, но которая будет сравнивать символы:

```
// Определение
char getMax(char first, char second) {
    // TODO: написать логику
}
```

Следующая функция, которую нам необходимо создать, будет сравнивать строки. Здесь логика будет несколько иной, поскольку значения типа `string` не позволяют сравнивать их с помощью операторов < и >. Мы воспользуемся функцией `compare(...)`, которая возвращает числовое значение: больше 0 (сравниваемый объект больше), меньше 0 (сравниваемый объект меньше) и 0 (для двух одинаковых объектов). Для использования функции должна быть объявлена библиотека `string`:

```
// Определение
string getMax(string first, string second) {
    if (first.compare(second) >= 0) {
        return first;
    }
    else return second;
}
```

Последний шаг — чтение входных данных, использование соответствующих переменных и вызов функции `getMax(...)` из тела функции `main()`:

```
int main() {
    string type;
    cin >> type;

    if (type == "int") {
        int first, second, max;
        cin >> first;
```

```
        cin >> second;
        max = getMax(first, second);
        cout << max;
    }
    else {
        if (type == "char") {
            char first, second, max;
            // TODO: получить максимальное значение
        }
        else {
            string first, second, max;
            // TODO: получить максимальное значение
        }
    }
    return 0;
}
```

Важно отметить, что строка может состоять из нескольких слов с пробелом между ними. Например, строка «`Software University`» является допустимой строкой. Однако для того, чтобы прочитать такую строку на языке C++, необходимо применить другой подход — использовать функцию `getline(...)`:

```
string str;
getline(cin, str);
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/ReadingStringWithGetline.

Для того чтобы воспользоваться функцией `getline(...)`, необходимо предварительно обратиться к библиотеке `string` (в зависимости от среды программирования она также может быть найдена как `cstring` или `string.h`): `#include <string>`.

В силу некоторых соображений, на которых мы сейчас не будем останавливаться, введем использование еще одной функции, являющейся частью подхода к чтению строки, содержащей пробелы: `cin.ignore()`. Эту функцию необходимо вызывать перед `getline(...)`, если до этого мы считывали значения другого типа данных (`int`, `char` и т. д.). Если строки, содержащие пробелы, читаются последовательно, то вызывать функцию между ними не нужно. Рассмотрим фрагмент программы, которая считывает и выводит различные значения, в том числе и строку, содержащую пробелы:

```
int number;
char symbol;
string word;
string sentence;

cout << "Введите число: ";
cin >> number;

cout << "Введите символ: ";
cin >> symbol;

cout << "Введите одно слово: ";
cin >> word;

cin.ignore();
cout << "Введите предложение: ";
getline(cin, sentence);

cout << number << endl;
cout << symbol << endl;
cout << word << endl;
cout << sentence <<endl;
```

Запустить и протестировать код из примера можно в Интернете: https://repl.it/@vncpetrov/ReadingAndPrintingMultipleValues.

Если строка `cin.ignore();` закомментирована, то программа не будет работать так, как ожидалось, и чтение последнего значения будет пропущено.

Строки с пробелами и строки без пробелов сравниваются функцией `compare(...)` одинаково — лексикографически.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#6.

Вложенные функции (локальные функции)

В некоторых языках программирования функция может быть объявлена и определена в теле другой функции, то есть быть *локальной*. В языке C++ это недопустимо!

В языке программирования C++ нет понятия локальной функции!

Именование функций. Лучшие практики при работе с функциями

В этой части мы познакомимся с некоторыми сложившимися практиками работы с функциями, связанными с именованием, расположением и структурой кода.

Именование функций

При именовании функции желательно использовать осмысленные имена. Поскольку каждая функция отвечает за какую-то часть нашей задачи, при ее именовании следует учитывать действие, которое она выполняет, то есть хорошей практикой является то, что имя описывает ее назначение.

Название начинается со строчной буквы и должно состоять из *глагола* или пары *глагол* + *существительное*. Форматирование имени соответствует соглашению `lowerCamelCase`, то есть первое слово начинается со строчной буквы, а каждое последующее — с прописной. После имени функции всегда следуют круглые скобки `(` и `)`.

Каждая функция должна выполнять отдельную задачу, а имя функции должно описывать, какую операцию она выполняет.

Некоторые примеры правильного именования функций:

- `FindStudent`
- `loadReport`
- `sine`

Некоторые примеры неудачного именования функций:

- `function1`
- `doSomething`
- `handleStuff`
- `simpleFunction`

Если мы не можем придумать подходящее название, то, скорее всего, функция решает несколько задач или не имеет четко определенного назначения. В этом случае стоит подумать о том, как разделить ее на несколько отдельных функций.

Именование параметров функции

При именовании параметров функций действуют практически те же правила, что и для самих функций. Разница лишь в том, что для имен параметров лучше использовать *существительное* или пару из *прилагательного и существительного*. При именовании параметров опять же соблюдается соглашение о *нижнем регистре* (`lowerCamelCase`). Следует отметить, что хорошей практикой является указание в имени параметра, какая единица измерения используется при работе с ним.

Некоторые примеры правильного именования параметров функций:

- `firstName`
- `report`
- `speedKmH`
- `usersList`
- `fontSizeInPixels`
- `font`

Некоторые примеры некорректного именования параметров функций:

- `p`
- `p1`
- `p2`
- `populate`
- `LastName`
- `last_name`

Эффективные методы работы с функциями

Напомним еще раз, что функция должна выполнять только одну конкретную задачу. Если этого достичь не удается, то необходимо подумать о том, как разбить функцию на несколько отдельных. Как мы уже говорили, имя функции должно точно и ясно описывать ее назначение. Еще одна хорошая практика программирования — избегать функций длиннее нашего экрана (приблизительно). Однако если код становится очень большим, целесообразно разбить функцию на несколько более коротких, как в следующем примере:

```
void printReceipt {
    printReceiptHeader();
    printReceiptBody();
    printReceiptFooter();
}
```

Структура и форматирование кода

При написании функций необходимо следить за соблюдением правильного отступа (отступ внутрь на одну табуляцию или четыре пробела). В языке C++ неправильный отступ не приводит к некорректной работе программы, но затрудняет чтение кода. Пример правильно оформленного кода на языке C++:

```
int main() {
    // код
    // еще код
}
```

Пример некорректно отформатированного кода на языке C++:

```
int main() {
    // код
// еще код
}
```

Если строка заголовка функции слишком длинная, рекомендуется разбить его на несколько строк, причем каждая строка после первой должна иметь отступ в две табуляции вправо (для лучшей читаемости):

```
double getTriangleArea(double length,
        double height) {
        return (length * height) / 2;
}
```

Еще одна хорошая практика при написании кода — оставлять пустую строку между функциями, после циклов и условных операторов. Также старайтесь избегать написания длинных строк и сложных выражений. Со временем вы обнаружите, что это улучшает читаемость кода и экономит время.

Мы рекомендуем всегда использовать фигурные скобки для тела проверок и циклов. Скобки не только улучшают читаемость, но и снижают вероятность того, что будет допущена ошибка и наша программа поведет себя некорректно.

Что мы узнали из этой главы?

В этой главе мы познакомились с основными принципами работы с функциями:

- Мы узнали, что *назначение функций* состоит в том, чтобы разбивать большие программы с большим количеством строк кода на более мелкие отдельные задачи.
- Мы изучили *структуру функций* — как объявлять, определять и вызывать их по имени.
- Мы рассмотрели примеры функций с *параметрами* и то, как использовать их в нашей программе.
- Мы узнали, что такое *сигнатура* и *возвращаемое значение функции*, а также роль оператора `return` в функциях.
- Мы узнали, *как лучше работать с функциями* — как называть их и их параметры, как оформлять код и т. д.

Упражнения

Чтобы закрепить навыки работы с функциями, мы решим несколько задач. В них требуется написать функцию с определенной функциональностью, а затем вызвать эту функцию, передав ей данные, считанные из консоли, как показано в примере ввода и вывода.

Задача: «Здравствуй, {имя}!»

Напишите функцию, которая принимает в качестве параметра имя и выводит на консоли «`Hello, {name}!`».

Ввод	Вывод
`Peter`	`Hello, Peter!`

Советы и рекомендации

Определите функцию `printName(string name)` и реализуйте ее, затем прочитайте имя из консоли в основной программе и вызовите функцию, передав ей считанное имя.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#7.

Задача: нахождение меньшего числа

Создайте функцию `getMin(int a, int b)`, возвращающую меньшее из двух чисел. Напишите программу, которая считывает с консоли три числа и выводит наименьшее из них. Используйте уже созданную функцию `getMin(...)`.

Примеры ввода и вывода

Ввод	Вывод
1 2 3	1

Ввод	Вывод
-100 -101 -102	-102

Рекомендации и советы

Определите функцию `getMin(int a, int b)` и реализуйте ее, а затем вызовите ее из программы, как показано ниже. Чтобы найти минимум трех чисел, сначала находим минимум первых двух чисел, затем минимум результата и третьего числа:

```
int min = getMin(getMin(num1, num2), num3);
```

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#8.

Задача: повторение строки

Напишите функцию `repeatString(str, count)`, которая принимает в качестве параметров переменную типа `string` и целое число `n` и возвращает строку, повторенную `n` раз. Результат выводится на консоль.

Примеры ввода и вывода

Ввод	Вывод
str 2	strstr

Ввод	Вывод
roki 6	rokirokirokirokirokiroki

Рекомендации и советы

Задача может быть решена двумя способами — один из них заключается в использовании `void`-функции, другой — в использовании функции, возвращающей значение типа `string`. Во втором случае это означает, что нам придется конкатенировать отдельные строки. Пример такой конкатенации, или объединения, приведен ниже:

```
string str1, str2;
cin >> str1;
cin >> str2;
str1 += str2;
cout << str1;
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/workWithTypeString.

В ходе дальнейшего обучения языку C++ мы познакомимся с другими подходами к работе со строками.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#9.

Задача: N-й разряд

Напишите функцию `findNthDigit(number, index)`, которая принимает в качестве параметров число и индекс `N` и выводит `N`-ю цифру числа (считая справа налево, начиная с `1`). Затем результат выводится на консоль.

Примеры ввода и вывода

Ввод	Вывод
83746 2	4

Ввод	Вывод
938478376	8

Ввод	Вывод
24354	2

Рекомендации и советы

Для выполнения алгоритма мы будем использовать цикл `while` до тех пор, пока число не станет равным 0. На каждой итерации цикла `while` мы будем проверять, совпадает ли текущий индекс цифры с искомым индексом. Если совпадает, то в качестве результата будет возвращена цифра индекса (`number % 10`). Если не совпадает, то мы удалим последнюю цифру числа (`number = number / 10`). Нам необходимо отслеживать, какую цифру мы проверяем, по индексу (справа налево, начиная с 1). Когда мы найдем цифру, мы вернем ее индекс.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1374#10.

Задача: система счисления

Напишите функцию `integerToBase(number, toBase)`, которая принимает в качестве параметров целое число и основание системы счисления и возвращает входное число, преобразованное в указанную систему счисления. Результат выводится на консоль.

Входное число всегда будет в системе счисления 10, а параметр основания будет находиться в диапазоне от 2 до 10.

Примеры ввода и вывода

Ввод	Вывод
3 2	11

Ввод	Вывод
4 4	10

Ввод	Вывод
9 7	12

Рекомендации и советы

Для решения задачи объявим строковую переменную, в которой будем хранить конечный результат. Затем следует выполнить вычисления, необходимые для преобразования числа:

- Вычисляем остаток от деления числа на основание.
- Вставляем остаток числа в начало строки, представляющей результат.
- Делим число на основание.
- Повторяем алгоритм до тех пор, пока входное число не станет равным 0.

При решении этой задачи мы приходим к тому, что нам необходимо преобразовать переменную типа `int` в `string`. Это можно сделать способом, описанным ниже. Мы будем использовать эту методику для преобразования не только `int`, но и других типов данных (`char`, `double` и др.) в `string`:

```
int number;
char symbol;
cin >> number;
cin >> symbol;

string numberAsString = to_string(number);
string symbolAsString = to_string(symbol);

cout << NumberAsString << endl;
cout << SymbolAsString << endl;
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/IntToStringConversion.

Саму задачу можно решить следующим образом:

```
string integerToBase(int, int);

int main() {
// TODO: Реализовать недостающую логику преобразования

    return 0;
}
string integerToBase(int number, int toBase) {
    string result = "";
    while (number != 0) {
        // TODO: Реализовать недостающую логику преобразования
    }

    return result;
}
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/IntegerToBase.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#11.

Задача: уведомления

Напишите программу, которая считывает целое число `N` и вводит в последующие строки `N` сообщений (причем для каждого сообщения считывается несколько строк). Для каждого сообщения может быть получено различное количество параметров.

Каждое сообщение начинается с `messageType: success`, `warning` или `error`:

- Если `messageType` — это `success`, то считывается `operation` + `message` (каждое в отдельной строке).
- Если `messageType` — это `warning`, то считывается только само сообщение (`message`).
- Если `messageType` — это `error`, то считывается `operation` + `message` + `errorCode` (каждое на отдельной строке).

Выведите на консоль каждое прочитанное сообщение, отформатированное в соответствии с его типом `messageType`. Следуя за строкой заголовка каждого сообщения, выведите столько символов `=`, сколько составляет длина соответствующей строки заголовка, и по-

ставьте одну пустую строку после каждого сообщения (для более детального понимания см. примеры).

Решим задачу, определив четыре функции:

- `showSuccessMessage()`,
- `showWarningMessage()`,
- `showErrorMessage()`,
- `readAndProcessMessage()`,

причем из функции `main()` будет вызываться только последняя функция:

```
// Прототипы
void showSuccessMessage(string operations, string message);
void showWarningMessage(string message);
void showErrorMessage(string operation, string message,
     int errorCode);
void readAndProcessMessage();
```

Примеры ввода и вывода

Ввод	Вывод
4 error покупка по кредитной карте Неверный адрес клиента 500	Ошибка: Не удалось выполнить покупку по кредитной карте. ========== Причина: Неверный адрес клиента. Код ошибки: 500.

Ввод	Вывод
warning Электронная почта не подтверждена	Предупреждение: Электронная почта не подтверждена. ==========

Ввод	Вывод
success регистрация пользователя Пользователь успешно зарегистрирован	Успешно выполнена регистрация пользователя. ========== Пользователь успешно зарегистрирован.

Ввод	Вывод
warning У клиента не определена электронная почта	Предупреждение: Не определена электронная почта. ==

Рекомендации и советы

Определим и реализуем четыре вышеуказанные функции в условии.

В функции `readAndProcessMessage()` мы считываем тип сообщения из консоли и в соответствии с прочитанным типом считываем остальные данные (это может быть еще одна, две или три строки). Затем мы вызываем соответствующую функцию для вывода сообщения соответствующего типа.

Для того чтобы вывести строку символов длиной в заданную строку, необходимо сначала найти длину соответствующей строки. Это можно сделать следующим образом:

```
string str = "Software University";
int length = str.size();
cout << string(length, '=') << endl;
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/GetStringLength.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1374#12.

Задача: перевод чисел в слова

Напишите функцию `letterize(number)`, которая считывает целое число и выводит его в виде английских слов в соответствии с приведенными ниже условиями:

- Выведите сотни, десятки и единицы (а также минус, если необходимо) словами в соответствии с правилами английского языка.
- Если число больше 999, то должно быть выведено сообщение `"too large"`.

- Если число меньше -999, то должно быть выведено сообщение `"too small"`.
- Если число отрицательное, то перед ним должен быть выведен `"minus"`.
- Если число не состоит из трех цифр, его не следует выводить на экран.

Примеры ввода и вывода

Ввод	Вывод
3 999 -420 1020	nine-hundred and ninety nine minus four-hundred and twenty too large

Ввод	Вывод
2 15 350	fifteen three-hundred and fifty

Ввод	Вывод
4 311 418 509 -9945	three-hundred and eleven four-hundred and eighteen five-hundred and nine too small

Ввод	Вывод
3 500 123 9	five-hundred one-hundred and twenty three nine

Рекомендации и советы

Сначала мы можем вывести в виде текста *сотни* — `(число / 100) % 10`, затем *десятки* — `(число / 10) % 10` и, наконец, *единицы* — `(число % 10)`.

Первый частный случай — это когда число *округляется точно* до 100 (например, 100, 200, 300 и т. д.). В этом случае мы выводим «one-hundred», «two-hundred» и т. д.

Второй частный случай — когда число, образованное двумя последними цифрами входного числа, *меньше 10* (например, 101, 305, 609 и т. д.). В этом случае мы выводим «one-hundred and one», «three-hundred and five», «six-hundred and nine» и т. д.

Третий частный случай — когда число, образованное двумя последними цифрами входного числа, *больше 10 и меньше 20* (например, 111, 814, 919 и т. д.).

В этом случае мы выводим «one-hundred and eleven», «eight-hundred and fourteen», «nine-hundred and nineteen» и т. д.

Проверка в судейской системе

Проверьте свое решение: https://judge.softuni.bg/Contests/Practice/Index/1374#13.

Задача: шифрование строк

Напишите функцию `encrypt(letter)`, которая шифрует букву следующим образом:

- Берутся первая и последняя цифры ASCII-кода буквы и конкатенируются в строку, которая будет представлять результат.
- В начале значения строки, представляющей результат, добавляется символ, удовлетворяющий следующему условию:
  - ASCII-код буквы + последняя цифра ASCII-кода буквы.
- Затем к концу строкового значения, представляющего результат, добавляется символ, удовлетворяющий следующему условию:
  - ASCII-код буквы - первая цифра ASCII-кода буквы.
- Функция должна возвращать в качестве результата зашифрованную строку.

Пример:

- j → p16i
  - ASCII-код j — `106` → первая цифра — `1`, последняя — `6`.
  - Соединяем первую и последнюю цифру → `16`.

- В начало строки, представляющей результат, вставьте символ, который получается из суммы ASCII-кода + последней цифры → `106 + 6` → `112` → `p`.
- В конце строки, представляющей собой результат, конкатенируйте символ, который получается в результате вычитания ASCII-кода — первой цифры → `106 - 1` → `105` → `i`.

Используя описанную выше функцию, напишите программу, которая *принимает последовательность символов, шифрует их и выводит результат в одной строке*.

Мы предполагаем, что входные данные всегда будут подходящими. Консоль должна считывать вводимые пользователем данные — целое число `N`, за которым следует по одному символу в каждой из следующих `N` строк.

Зашифруйте символы, а затем добавьте их в зашифрованную строку.

В результате на экран должна быть выведена зашифрованная строка символов, как показано в следующем примере:

- `S, o, f, t, U, n, i → V83Kp11nh12ez16sZ85Mn10mn15h`

Примеры ввода и вывода

Ввод	Вывод
7 S o f t U n i	V83Kp11nh12ez16sZ85Mn10mn15h

Ввод	Вывод
4 s l a p	x15rt18kh97Xr12o

Ввод	Вывод
7 B i r a H a x	H66<n15hv14qh97XJ72Ah97xx10w

Рекомендации и советы

Мы будем выполнять все операции, каждый раз сохраняя результаты в переменных типа `string`. Затем мы «склеим» все эти переменные, как в задаче о повторении строки. Таким образом, мы создаем итоговую переменную `result` типа `string` и выводим ее. Нам необходимо выполнить цикл `n` раз, и на каждой итерации мы будем добавлять зашифрованный символ в переменную `result`.

Для нахождения первой и последней цифр ASCII-кода воспользуемся алгоритмом, который мы использовали при решении задачи о n-й цифре, а для создания строки поступим так же, как в задаче о системах счисления.

Проверка в судейской системе

Проверьте пример программы в судейской системе: https://judge.softuni.bg/Contests/Practice/Index/1374#14.

Глава 11.
Трюки и хаки

В этой главе мы рассмотрим некоторые трюки, хаки и приемы, облегчающие работу с языком C++ в среде разработки Visual Studio. В частности, мы рассмотрим:

- Как правильно форматировать код.
- Соглашения об именовании элементов кода.
- Удобные сочетания клавиш.
- Шаблоны кода (snippets).
- Методы отладки кода.

Форматирование кода

Правильное форматирование нашего кода сделает его более читаемым и понятным в случае, если с ним придется работать кому-то еще. Это важно, поскольку на практике нам придется работать в команде с другими людьми, и очень важно писать код так, чтобы наши коллеги могли быстро в нем сориентироваться.

Для правильного оформления кода существуют определенные правила, которые в совокупности называются соглашениями, или конвенциями. Соглашения — это группа правил, общепринятых для программистов, работающих с тем или иным языком, и широко используемых. Эти конвенции помогают выработать нормы в конкретных языках — как лучше писать и что является лучшей практикой. Предполагается, что если программист следует им, то его код легко читается и понимается.

Язык C++ был создан Бьёрном Строуструпом (Страуструпом) (https://en.wikipedia.org/wiki/Bjarne_Stroustrup). Следует знать, что даже при несоблюдении конвенций код будет работать (при условии, что он написан правильно), но его будет нелегко понять. На ба-

зовом уровне это, конечно, не смертельно, но чем раньше вы привыкнете писать качественный код, тем лучше.

Существует множество кодовых конвенций для языка C++, например, одна из них, часто используемая Строуструпом, опубликована в статье «Bjarne Stroustrup's C++ Style and Technique FAQ», которую можно найти по адресу https://www.stroustrup.com/bs_faq2.html.

Соглашения, используемые в настоящей работе, описаны в следующих строках. Для форматирования кода рекомендуется использовать фигурные скобки {}:

- Открывающая скобка { должна находиться в той же строке и непосредственно после структуры, к которой она относится.
- Закрывающая скобка } должна быть вынесена на отдельную строку.

```
if (некотороеУсловие) {
    cout << "Внутри оператора if" << endl;
}
```

Видно, что команда `cout` в примере занимает четыре пустых поля (одна табуляция), что также рекомендуется Microsoft в качестве хорошей практики. Независимо от того, сколько табуляций внутри конкретного оператора с фигурными скобками начинается (то есть сколько бы вложенностей различных операторов мы ни имели), фигурные скобки {} должны находиться в начале оператора, как в приведенном ниже примере:

```
if (некотороеУсловие) {
    if (другоеУсловие) {
        cout << "Внутри оператора if" << endl;
    }
}
```

Приведем пример плохого форматирования кода относительно принятых в языке C++ правил написания кода:

```
if(некотороеУсловие)
{
cout << "Внутри оператора if" << endl;}
```

Первое, на что следует обратить внимание, это фигурные скобки {}. Первая (открывающая) скобка должна находиться сразу по-

сле условия `if`, а вторая (закрывающая) — под командой `cout << (...)`, на отдельной пустой строке. Кроме того, команда внутри оператора `if` должна иметь четыре пустых поля (одну табуляцию). Сразу после ключевого слова `if` и перед условием проверки оставляется пробел.

Это же правило распространяется на циклы `for` и любые другие конструкции с фигурными скобками `{}`. Приведем еще несколько примеров.

Верно:

```
for (int i = 0; i < 5; i++) {
    cout << i << endl;
}
```

Неправильно:

```
for(int i=0;i<5;i++){
cout << i << endl;
}
```

Для удобства работы в Visual Studio существуют быстрые клавиши, о которых мы расскажем далее в этой главе, а пока нас интересуют две конкретные комбинации. Одна комбинация предназначена для форматирования кода во всем документе, а другая — для форматирования части кода. Если мы хотим отформатировать весь код, необходимо нажать [CTRL + K + D]. В случае если требуется отформатировать только часть кода, необходимо выделить мышью ту часть, которую нужно отформатировать, и нажать [CTRL + K + F].

Давайте воспользуемся некорректным примером, который мы приводили недавно:

```
for(int i=0;i<5;i++){
cout << i << endl;
}
```

Если мы нажмем [CTRL + K + D], что является нашей комбинацией для форматирования всего документа, то получим код, отформатированный в соответствии с конвенциями языка C++, который будет выглядеть следующим образом:

```
for (int i = 0; i < 5; i++) {
    cout << i << endl;
}
```

Эта комбинация может помочь нам, если мы столкнемся с плохо отформатированным кодом.

Именование элементов кода

В этом разделе мы рассмотрим общепринятые в мире C++ соглашения об именовании проектов, файлов и переменных.

Именование проектов и файлов

Для именования проектов и файлов рекомендуется использовать описательное имя, позволяющее определить роль данного файла/проекта, и в то же время рекомендуется использовать соглашение `PascalCase`. Это соглашение об именовании элементов, в котором каждое слово, включая первое, начинается с заглавной буквы, например `ExpressionCalculator`.

Пример: в этом курсе мы начинаем с лекции «Первые шаги в программировании», поэтому примерное название проекта (Solution) для этой лекции может быть `FirstStepsInCoding`. Аналогичное соглашение применяется и к файлам проекта. Если мы возьмем в качестве примера первую задачу из этой лекции, то она называется «Hello World», и поэтому наш файл проекта будет называться `HelloWorld`.

Именование переменных

В программировании переменные хранят некоторые данные, и чтобы сделать код более понятным, имя переменной должно подсказывать ее назначение. Вот еще несколько рекомендаций по именованию переменных:

- Имя должно быть коротким, описательным и пояснять, для чего предназначена переменная.
- Имя должно состоять только из букв a–z, A–Z, цифр 0–9 и символа ‘_’.
- В языке C++ принято, чтобы переменные всегда начинались со строчной буквы и содержали строчную букву, а каждое последующее слово в них начиналось с прописной буквы (это соглашение об именовании также известно как camelCase convention).

- Следует быть осторожным с прописными и строчными буквами, так как C++ различает их. Например, `age` и `Age` — это разные переменные.
- Имена переменных не могут совпадать с ключевыми словами языка C++, например, `int` — недопустимое имя переменной.

Хотя использование *заглавных букв в именах переменных* допускается, в C++ оно не рекомендуется и считается плохим стилем именования. Хорошей практикой является использование прописных (заглавных) букв только для названия макросов (https://docs.microsoft.com/en-us/cpp/preprocessor/macros-c- cpp?view=vs-2017).

Приведем несколько примеров удачно названных переменных:

- `firstName`
- `age`
- `startIndex`
- `lastNegativeNumberIndex`

Приведем несколько примеров неудачно названных переменных, хотя с точки зрения компилятора C++ их имена корректны:

- `_firstName` (начинается с '_')
- `last_name` (содержит '_')
- `AGE` (используются заглавные буквы)
- `Start_Index` (начинается с заглавной буквы и содержит '_')
- `lastNegativeNumber_Index` (содержит '_')

Поначалу все эти правила могут показаться бессмысленными и ненужными, но со временем, набравшись опыта, вы убедитесь в необходимости норм написания качественного кода для облегчения и ускорения работы в команде. Вы поймете, что работать с кодом, написанным без соблюдения каких-либо правил написания качественного кода, крайне утомительно.

Клавиатурные сокращения в Visual Studio

В предыдущем разделе мы упомянули две комбинации, применяемые для форматирования кода. Одна комбинация [CTRL + K + D] предназначена для форматирования всего кода в файле, а вторая,

[CTRL + K + F] — для случаев, когда мы хотим форматировать только определенную часть кода. Эти сочетания называются горячими клавишами, и сейчас мы расскажем о них более подробно.

Горячие клавиши — это сочетания клавиш, позволяющие выполнять определенные действия проще и быстрее, и каждая среда разработки программного обеспечения имеет свои горячие клавиши, хотя большинство из них повторяются. Сейчас мы рассмотрим некоторые горячие клавиши в Visual Studio.

Сочетание	Действие
[CTRL + F]	Открывает окно поиска, с помощью которого мы можем искать в коде.
[CTRL + K + C]	Делает часть кода закомментированным.
[CTRL + K + U]	Раскомментирует уже закомментированный код.
[CTRL + Z]	Отмена изменения (Undo).
[CTRL + SHIFT + Z]	Это сочетание противоположно [CTRL + Z] (Redo).
[CTRL + K + D]	Форматирует код в соответствии с соглашениями по умолчанию.
[CTRL + Backspace]	Удаляет слово, расположенное слева от курсора.
[CTRL + Del]	Удаляет слово, расположенное справа от курсора.
[CTRL + Shift + S]	Сохраняет все файлы в проекте.
[CTRL + S]	Сохраняет текущий файл.

Подробнее о клавиатурных сочетаниях в Visual Studio можно узнать здесь: https://visualstudioshortcuts.com/2017/.

Шаблоны кода (code snippets)

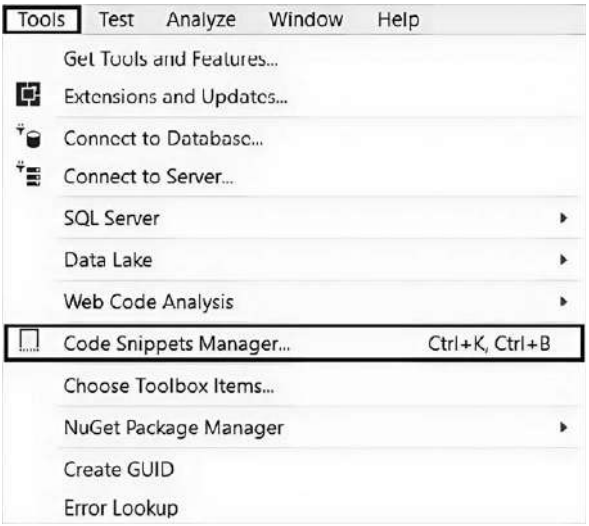
В Visual Studio существуют так называемые шаблоны кода, позволяющие создать шаблонные блоки кода. Например, если набрать `for` и нажать [Tab], то в теле нашей программы будет сгенерирован следующий код:

```
for (size_t i = 0; i < length; i++) {

}
```

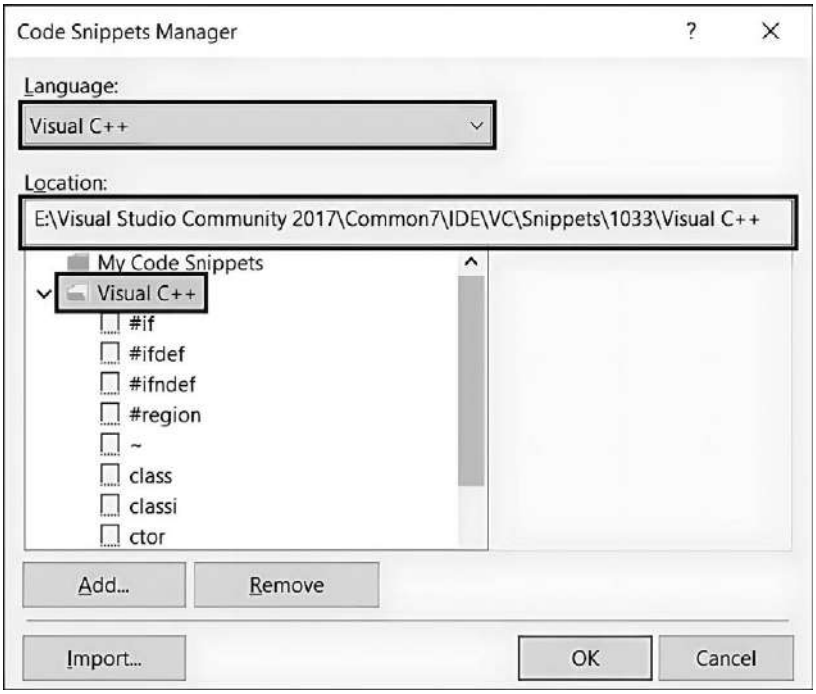
Это называется «разворачивание шаблона шорткода». На скриншоте ниже показано действие шаблона `for`:

Использование шаблонов кода

Мы можем создать свой собственный шаблон или загрузить его из Интернета и добавить (Add) или вставить (Import). Это делается из меню [Tools] -> [Code Snippets Manager], как показано на скриншоте:



В открывшемся окне необходимо выбрать [Language] -> [Visual C++], а в разделе [Locations] -> [Visual C++]. Там находятся все существующие шаблоны для языка C++:



С помощью кнопок [Add] и [Import] можно добавлять новые шаблоны. Кнопка [Remove] служит для удаления.

Приемы отладки кода

Отладка играет важную роль в процессе разработки программного обеспечения, позволяя пошагово отслеживать выполнение программы. С ее помощью можно отслеживать изменения значений локальных переменных в процессе выполнения программы и отлаживать возникающие ошибки (баги). Процесс отладки включает в себя:

- Выявление проблем (ошибок, багов).
- Нахождение кода, вызывающего проблемы.
- Исправление кода, вызывающего проблемы, с тем, чтобы программа работала корректно.

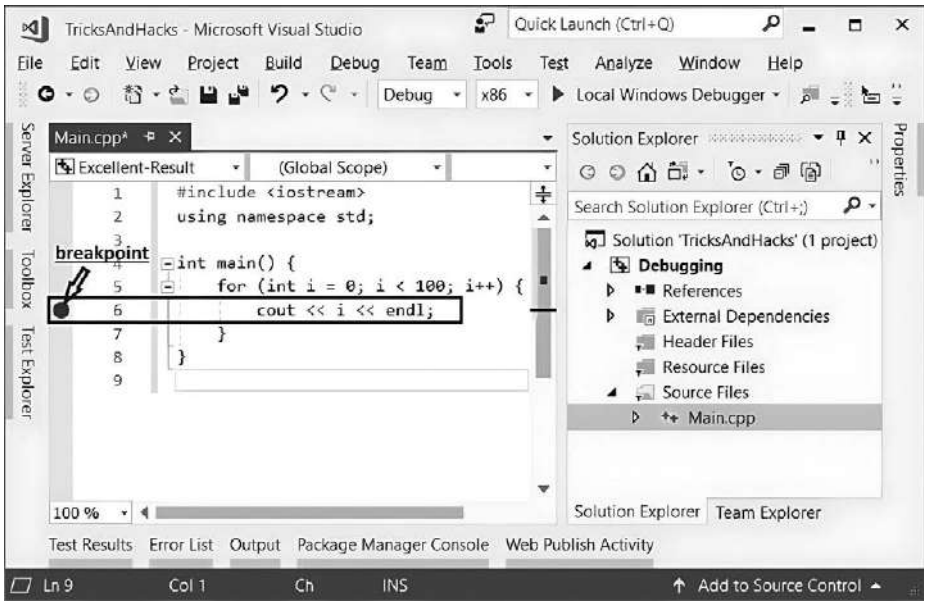
- Тестирование, позволяющее убедиться в правильности работы программы после внесения корректировок.

Visual Studio предоставляет нам встроенный отладчик, с помощью которого мы можем устанавливать точки останова в выбранных местах. При возникновении точки останова программа останавливает выполнение и позволяет последовательно выполнять оставшиеся строки. Отладка позволяет вникнуть в детали программы и увидеть, где именно возникают ошибки и что их вызывает.

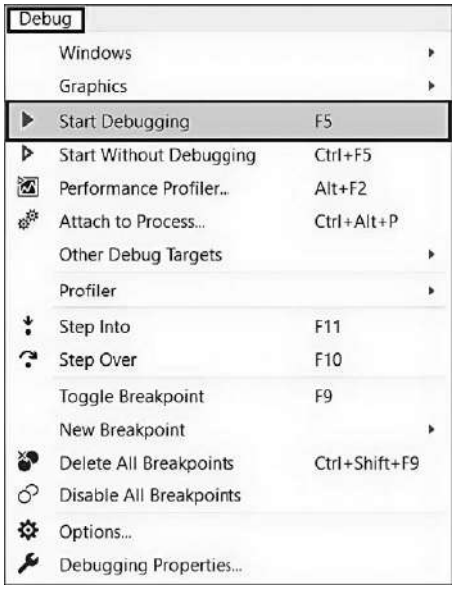
Для демонстрации работы отладчика мы воспользуемся следующей программой:

```
int main() {
    for (int i = 0; i < 100; i++) {
        cout << i << endl;
    }
}
```

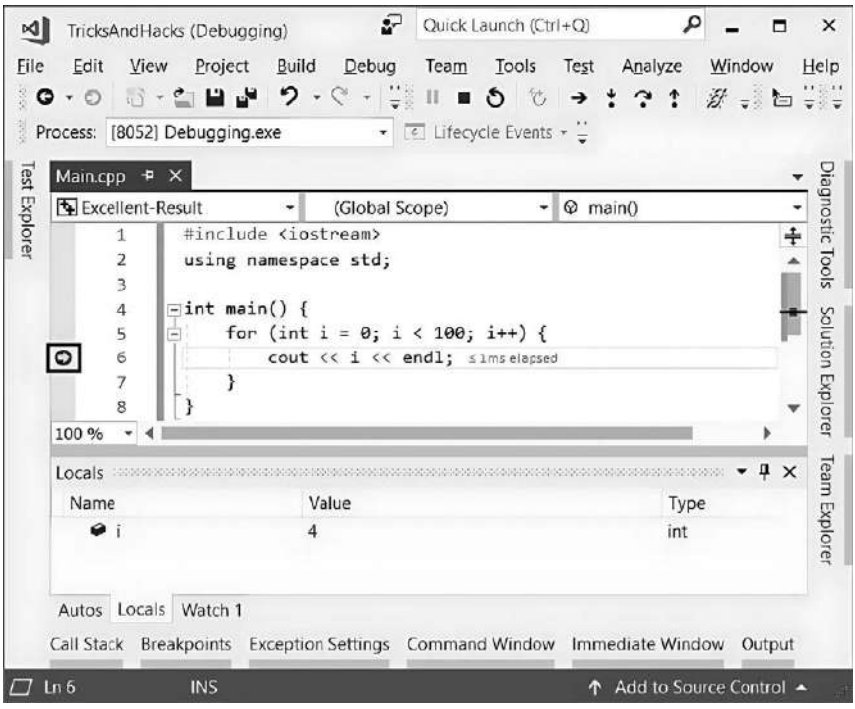
Поставим точку останова на функции `cout`. Для этого необходимо подвести курсор к строке, выводящейся на консоль, и нажать [F9]. Появится точка останова, на которой программа остановит выполнение:



Чтобы запустить программу в режиме отладки, выберите [Debug] -> [Start Debugging] или нажмите [F5]:



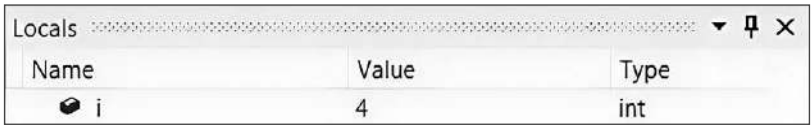
Запустив программу, мы видим, что она останавливает выполнение на строке 6, где мы поставили точку останова.



Стрелка слева указывает на текущую строку. Для перехода на следующую строку мы используем клавишу [F10]. Мы видим, что код

в текущей строке еще не выполняется. Он будет выполнен, когда мы перейдем на следующую строку.

Из окна [Locals] можно следить за изменением локальных переменных. Чтобы открыть окно, выберите [Debug] -> [Windows] -> [Locals]:



Полезные приемы

В этом разделе мы кратко напомним уже рассмотренные в этой книге приемы и методы программирования на языке C++, которые будут очень полезны для начинающих.

Вывод текста на консоль

```
string text = "некоторый текст";
cout << text << endl;
// Это выведет на консоль "некоторый текст"
```

Протестировать пример можно в Интернете: https://repl.it/@vncpetrov/PrintText.

Вставка переменных в строку

Заполнитель (placeholder) — это выражение, которое при выводе будет заменено определенным значением. Функция `printf(...)` поддерживает вывод текста по шаблону, где первым передаваемым аргументом является строка формата, за которой следует количество аргументов, равное количеству заполнителей. Функция находится в библиотеке `stdio.h`, и на нее необходимо ссылаться в начале программы:

```
string text = "некоторый текст";
int number = 5;

printf("%s %d %s\n",
    text.c_str(), number, text.c_str());
// Будет выведено "некоторый текст 5 некоторый текст"
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/VarInString.

В данном примере обратите внимание, что мы можем передавать не только текстовые переменные. Знак % используется для ука-

зания типа следующей переменной: %d используется для числового типа int, %s — для строки и т. д.

Форматирование с двумя цифрами после десятичного знака

```
double number = 5.432432;
cout << fixed << setprecision(2) << number;
// При этом на консоль будет выведено "5.43"
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/RoundingSetprecision.

Округлите результат до двух цифр после десятичной дроби с помощью fixed << setprecision(2), не забыв сослаться на библиотеку <iomanip>.

Если мы хотим округлить до двух цифр после десятичной точки, а третья цифра меньше 5, как в примере выше, то округляем «вниз» (в меньшую сторону), если же третья цифра 5 или больше — округляем «вверх» (в большую сторону), как в примере ниже:

```
double number = 5,439;
cout << fixed << setprecision(2) << number << endl;
// В результате на консоль будет выведено "5.44"
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/RoudinginSetprecision2.

Округление чисел

Если нам необходимо округление, мы можем воспользоваться функциями, которые находятся в библиотеке math.h:

- round(...) — округление производится по основному математическому правилу: если десятичная часть меньше 5, то число округляется в меньшую сторону, и наоборот, если больше 5 — в большую сторону. Важно отметить, что данная функция округляет до ближайшего целого числа:

```
double firstNumber = 5.431;
cout << round(firstNumber) << endl;
// В результате на консоль будет выведена цифра "5"
double secondNumber = 5.539;
cout << round(secondNumber) << endl;
// В результате на консоль будет выведено "6"
```

Запустить и протестировать пример можно в Интернете: https://repl.it/@vncpetrov/RoundingWithRound.

- `floor(...)` — в случае, если мы хотим, чтобы округление всегда происходило в меньшую сторону. Эта функция также округляет до целого числа. Например, если у нас есть число 5.99 и мы используем `floor(5.99)`, то получим число 5:

```
double numberToFloor = 5.99;
cout << floor(numberToFloor) << endl;
// Это выведет на консоль 5
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/RoundFloor.

- `ceil(...)` — в случае, если мы хотим, чтобы округление всегда шло в бошльшую сторону. Эта функция также округляет до целого числа. Например, если у нас есть число 5.11 и мы используем `ceil(5.11)`, то получим число 6:

```
double numberToCeiling = 5.11;
cout << ceil(numberToFloor) << endl;
// Это выведет на консоль 6
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/RoundCeil.

- `trunc(...)` — в случае, если необходимо удалить дробную часть. Например, если у нас есть число 2,63 и мы используем `trunc(2,63)`, то получим 2:

```
double numberToTrunc = 2.63;
cout << trunc(numberToTrunc) << endl;
// Это выведет на консоль 2
```

Пример можно протестировать в Интернете: https://repl.it/@vncpetrov/RoundTrunc.

Округление с заполнителем

```cpp
double num = 5.439524;
printf("%.3f\n", num);
// При этом на консоль будет выведено "5.440"
```

Запустить и протестировать код из приведенного примера можно в Интернете: https://repl.it/@vncpetrov/RoundingWithPlaceholder.

В данном случае мы добавим после числа `.3f`, что ограничит число тремя цифрами после десятичного знака. Поведение будет аналогично функции `setprecision(...)`. Следует иметь в виду, что число перед буквой `f` означает, сколько цифр будет после десятичного знака (то есть может быть, например, `2f` или `5f`). Не забудьте поставить перед числом точку — она обязательна.

Как написать условную конструкцию?

Условная конструкция `if` состоит из следующих элементов:

- Ключевое слово `if`.
- Булево выражение (условие).
- Тело условной конструкции.
- Необязательно: условие `else`.

```
if (условие) {
    // тело
}
else (условие) {
    // тело
}
```

Для простоты мы можем использовать фрагмент кода для оператора `if`:

- `if + [Tab]`

Как написать цикл for?

Для цикла `for` нам необходимы:

- Блок инициализации — в нем объявляется переменная-перечислитель (`int i`) и устанавливается ее начальное значение.
- Условие повторения (`i <= 10`).
- Обновление счетчика (`i++`).
- Тело цикла.

```
for (int i = 0; i < 5; i++) {
    // тело
}
```

Для простоты мы можем использовать шаблон кода для цикла `for`, для чего пишем `for` и нажимаем `[Tab]`.

Заключение

Если вы прочитали всю книгу, решили все задачи и пришли к данному заключению, то вас можно поздравить! Вы уже сделали первый шаг в освоении профессии программиста, но вам предстоит пройти еще долгий путь, прежде чем вы не станете действительно хорошим специалистом и не сделаете написание программ своей профессией.

Вспомните четыре основных навыка, которыми должен обладать каждый программист для выполнения своей работы:

- Навык № 1 — написание программного кода (20 % навыков программиста) — в значительной степени рассмотрен в этой книге, но вы также должны изучить основные структуры данных, классы, объекты, функции, строки и другие элементы написания кода.
- Навык № 2 — алгоритмическое мышление (30 % навыков программиста) — частично рассмотрен в данной книге и развивается в основном путем решения большого количества разнообразных алгоритмических задач.
- Навык № 3 — фундаментальные знания для профессии (25 % навыков программиста) — осваивается в течение нескольких лет путем сочетания обучения и практики (чтение книг, просмотр видеоуроков, прохождение курсов и, в особенности, написание разнообразных проектов в различных технологических областях).
- Навык № 4 — языки программирования и программные технологии (25 % навыков программиста) — осваивается в течение длительного времени, с большим количеством практики, солидным объемом чтения и написанием проектов. Эти знания и навыки быстро устаревают и требуют постоянного обновления. Хорошие программисты осваивают новые технологии каждый день.

Эта книга — лишь первый шаг!

Эта книга по основам программирования — лишь первый шаг в формировании навыков программиста. Если вы справились со всеми задачами, значит, вы получили ценные знания о принципах программирования на языке C++ на базовом уровне. Вам еще предстоит более глубоко изучить программирование и развить свое алгоритмическое мышление, а затем добавить технологические знания о языке C++ и ряде других концепций, технологий и инструментов разработки программного обеспечения.

Если вы не смогли решить все или большинство задач, вернитесь к ним и решите их! Помните: для того, чтобы стать программистом, нужно много работать и прилагать значительные усилия. Эта профессия не для ленивых. Не занимаясь программированием серьезно годами, вы не сможете этому научиться!

Как мы уже объясняли, первый и самый основной навык программиста — научиться писать код с легкостью и удовольствием. В этом и заключается задача данной книги: научить вас писать код. В дополнение к книге мы рекомендуем вам записаться на практический курс «Основы программирования» в SoftUni, который предлагается бесплатно в очном или онлайн-формате.

Куда двигаться дальше?

С помощью этой книги вы заложили прочный фундамент, который позволит вам продолжать развиваться как программисту. Если вы задаетесь вопросом, как продолжить свое развитие, рассмотрите следующие несколько вариантов:

- Получить профессию инженера-программиста при помощи SoftUni и сделать программирование своей профессией.
- Продолжить развиваться как программист своим собственным путем, например, с помощью самообучения или онлайн-учебников.
- Оставаться на уровне кодера, не занимаясь серьезным программированием.

Профессия «инженер-программист» в университете SoftUni

Первая, и поэтому рекомендуемая, возможность освоить профессию «инженер-программист» — это начать обучение по комплексной программе подготовки инженеров-программистов в университете SoftUni: https://softuni.bg/curriculum. Учебная программа SoftUni была тщательно разработана доктором Светлином Наковым и его командой, чтобы последовательно и с постепенным усложнением дать вам все навыки, необходимые для того, чтобы начать карьеру разработчика программного обеспечения в IT-компании.

Продолжительность обучения в SoftUni

Обучение в SoftUni длится 2–3 года (в зависимости от специальности и выбранных специализаций), и за это время вполне реально выйти на хороший начальный уровень (junior developer), но только при условии, что вы серьезно занимаетесь и пишете код каждый день. При хорошей успеваемости типичный студент начинает работать уже в середине обучения (примерно через 1,5 года). Благодаря развитой партнерской сети Центр карьеры SoftUni предлагает работу по специальности всем студентам SoftUni, имеющим отличные или хорошие оценки. Поступление на работу по специальности при наличии высоких оценок в SoftUni, желания работать и разумных ожиданий по отношению к работодателю практически гарантировано.

Программист становится программистом как минимум за год упорного написания кода

Имейте в виду, что для того, чтобы стать программистом, нужно приложить много усилий, написать десятки тысяч строк кода и решить сотни, а то и тысячи практических задач, и на это уходят годы! Если кто-то предлагает вам «более легкую программу» и обещает, что вы станете программистом и начнете работать по специальности через 3–4 месяца, то вас либо обманут, либо дадут настолько низкий уровень, что компании не возьмут вас даже на стажировку. Конечно, есть исключения, например, если вы не начинаете с нуля, или у вас очень развито инженерное мышление, или вы претендуете на очень низкую должность (например, техническая поддержка), но в целом программистом меньше чем за год упорного обучения и написания кода не станешь!

Вступительный экзамен в SoftUni

Для поступления в SoftUni необходимо сдать вступительный экзамен по «Основам программирования», основанный на материале данной книги. Если вы легко решаете приведенные в книге упражнения, значит, вы готовы к экзамену. Также обратите внимание на несколько глав, посвященных подготовке к практическому экзамену по программированию. Они дадут вам представление о сложности экзамена и типах задач, которые необходимо научиться решать.

Если задачи из книги и примеры экзаменов кажутся вам сложными, значит, вам необходима дополнительная подготовка. Запишитесь на бесплатный курс «Основы программирования» или внимательно прочтите книгу с самого начала, не пропуская задач из каждой учебной темы! Вы должны научиться решать их с легкостью, не помогая себе методическими указаниями и примерами решений.

Учебная программа для инженеров-программистов

После экзамена вас ждет серьезная учебная программа по подготовке инженеров-программистов в университете SoftUni. Она представляет собой серию модулей из нескольких курсов по программированию и программным технологиям, полностью ориентированных на изучение основ разработки программного обеспечения и приобретение практических навыков работы программиста с современными программными технологиями. Студентам предоставляется выбор из нескольких профессий и специализаций с акцентом на C#, Java, JavaScript, PHP и других языках и технологиях.

Каждая специальность изучается в нескольких модулях продолжительностью четыре месяца, каждый модуль содержит 2–3 курса. Занятия делятся на теоретическую подготовку (30 %) и практические упражнения, проекты и мероприятия (70 %), каждый курс заканчивается практическим экзаменом или практическим курсовым проектом.

Сколько часов в день занимает обучение?

Обучение инженера-программиста в SoftUni — очень серьезное занятие, и ему необходимо уделять не менее 4–5 часов ежедневно, а вообще рекомендуется уделять ему все свое время. Совмещать работу с учебой не всегда удается, но если вы занимаетесь чем-то лег-

ким и имеете много свободного времени, то это хороший вариант. SoftUni — хороший вариант для старшеклассников, студентов вузов и тех, кто работает на другой работе, но лучше все свое время посвятить учебе и освоению профессии. Этого не сделаешь, уделяя программированию 2–4 часа в неделю!

Формы обучения в SoftUni — очная и онлайн. При обеих формах обучения для того, чтобы освоить программу по учебному плану (который требуют софтверные компании для начала работы), необходимо много учиться. Нужно только найти на это время! Причина № 1, по которой обучающимся тяжело дается путь к профессии, — недостаточное количество времени на обучение: как минимум нужно уделять ему 20—30 часов в неделю.

SoftUni для тех, кто работает и учится

Всем, кто получил отличный результат на вступительном экзамене в SoftUni, действительно увлечен программированием и мечтает сделать его своей профессией, мы рекомендуем освободиться от других обязательств и посвятить все свое время освоению профессии «инженер-программист».

- Для работающих людей это означает увольнение с работы (и получение кредита или сокращение расходов, чтобы провести с меньшим доходом период в 1—2 года, пока они не начнут работать по новой специальности).
- Для тех, кто учится в традиционном университете, это означает значительное смещение акцента в сторону программирования и практических курсов в SoftUni, что заставит сократить время, уделяемое обучению в традиционном университете.
- Для безработных это отличный шанс направить все свое время, силы и энергию на приобретение новой, перспективной, хорошо оплачиваемой и востребованной профессии, которая обеспечит им хорошее качество жизни и долгосрочное процветание.
- Для учеников средних и старших классов это выбор того, что важнее для их развития: или изучение практического программирования в SoftUni, которое даст им профессию и работу, или полное внимание к традиционной системе образования, или разумное совмещение обоих начинаний.

Всем тем, кто не может получить отличный результат на вступительном экзамене в SoftUni, рекомендуется сосредоточиться на лучшем изучении, понимании и особенно отработке материала, изложенного в этой книге. Если вы не справитесь с задачами из этой книги, то не сможете в будущем справиться с задачами при изучении программирования и разработки программного обеспечения.

Не пропускайте занятия по основам программирования! Ни в коем случае не принимайте смелых решений и не бросайте работу или традиционный университет, строя грандиозные планы на будущую профессию инженера-программиста, если у вас нет отличной оценки на вступительном экзамене в SoftUni! Он определяет, подходит ли вам программирование, насколько оно вам нравится и есть ли у вас мотивация изучать его всерьез и заниматься этим долгие годы каждый день с радостью и удовольствием.

Изучение программной инженерии по-своему

Другой вариант развития событий после прочтения этой книги — продолжить изучение программирования вне SoftUni. Вы можете записаться на видеокурсы или пройти их, чтобы более глубоко изучить программирование на C++ или других языках и платформах разработки. Можно читать книги по программированию и программным технологиям, изучать онлайновые учебники и другие онлайн-ресурсы — в Интернете бесконечное количество бесплатного материала. Однако помните, что самое главное на пути к становлению программиста — это практические проекты!

Программистом нельзя стать, не написав много-много кода и не потренировавшись. Потратьте достаточно времени. Программистом нельзя стать за месяц или два.

В Интернете вы найдете большое количество бесплатных ресурсов: книг, учебников, видеоуроков, онлайновых и очных курсов по программированию и разработке программного обеспечения. Однако, чтобы получить работу начального уровня, придется потратить не менее года или двух.

Когда вы продвинетесь вперед, найдите возможность либо пройти стажировку в какой-либо компании (что будет практически невозможно без хотя бы годового опыта написания кода), либо при-

думать свой собственный практический проект, над которым вы будете работать несколько месяцев, а то и год, чтобы учиться методом проб и ошибок.

Не забывайте, что существует множество способов стать программистом, но все они имеют нечто общее: интенсивное написание кода и годы практики!

Онлайн-сообщества для начинающих программистов

Независимо от того, какой путь вы выберете, если вы собираетесь серьезно заниматься программированием, желательно посещать специализированные интернет-форумы, группы новостей и сообщества, где можно получить помощь от коллег и быть в курсе новинок в области программного обеспечения.

Если вы будете серьезно изучать программирование, окружите себя людьми, которые им серьезно занимаются. Вступайте в сообщества разработчиков, посещайте конференции по программированию, ходите на мероприятия для программистов, найдите друзей, с которыми вы можете говорить о программировании и обсуждать проблемы и ошибки, найдите окружение, которое сможет вам помочь. В небольших населенных пунктах у вас есть Интернет и доступ ко всему онлайн-сообществу.

Ниже приведены некоторые рекомендуемые ресурсы, которые будут полезны для вашего развития как программиста:

- https://softuni.bg — официальный сайт SoftUni. Здесь вы найдете бесплатные (и не только) курсы, семинары, видеоуроки и тренинги по программированию, программным технологиям и цифровым компетенциям.
- https://softuni.bg/forum — официальный форум университета SoftUni. Дискуссионный форум SoftUni чрезвычайно позитивен и полон коллег, готовых помочь. Если вы зададите содержательный вопрос, связанный с программированием и технологиями, изучаемыми в SoftUni, то почти наверняка получите содержательный ответ в течение нескольких минут. Попробуйте, вы ничего не теряете.

- http://www.introprogramming.info — официальный сайт книг «Введение в программирование» на C# и Java доктора Светлина Накова и команды. В этих книгах подробно рассматриваются основы программирования, базовые структуры данных и алгоритмы, ООП и другие базовые навыки, и они являются отличным продолжением для чтения после этой книги. Однако помимо чтения вам необходимо интенсивно писать код, не забывайте об этом!
- http://stackoverflow.com — Stack Overflow — один из крупнейших в мире дискуссионных форумов для программистов, где можно получить помощь по всем возможным вопросам программирования. Если вы свободно владеете английским языком, найдите Stack Overflow и задайте там свои вопросы.
- https://www.meetup.com/find/tech — ищите технологические встречи в своем городе и участвуйте в тех сообществах, которые вам нравятся. Большинство технологических встреч бесплатны, и новичкам на них рады.
- Если вас интересуют мероприятия в области IT, технологические конференции, обучение и стажировки, обратите внимание на такие крупные сайты, посвященные IT-событиям, как http://iteventz.bg и http://dev.bg.

Всем удачи!

От лица всего авторского коллектива мы желаем вам дальнейших успехов в профессии и в жизни! Мы будем очень рады, если помогли вам увлечься программированием и вдохновили смело идти к профессии «инженер-программист», которая принесет вам хорошую работу, где вы будете трудиться с удовольствием, обеспечит качественную жизнь и достаток, а также потрясающие перспективы для развития и возможности создавать впечатляющие проекты с вдохновением и увлеченностью.

16+

Справочное издание
Анықтамалық басылым

Серия «Полный курс для начинающих программистов»

Светлин Наков

C++. ОСНОВЫ ПРОГРАММИРОВАНИЯ

Ответственный за выпуск *И. В. Резько*
Оформление обложки *С. А. Арутюнян*

Подписано в печать 23.08.2024.
Формат 70x100 1/16. Бумага офсетная. Печать офсетная.
Гарнитура Noto Serif SemiCondenced.
Усл. печ. л. 39. Тираж 2000 экз. Заказ №

Книга «C++. Основы программирования» адресована в первую очередь тем, кто всерьез интересуется разработкой программного обеспечения и стремится получить необходимые практические навыки в этом непростом деле, но пока имеет мало опыта в программировании. Данный своеобразный обучающий практикум базируется на курсе «Основы программирования», который преподается новичкам в известном болгарском университете SoftUni, специализирующемся на практической подготовке программистов и ИТ-специалистов, а основным разработчиком учебного контента выступает доктор Светлин Наков, который за последние 20 лет помог более чем 100 000 новичков освоить востребованную во всем мире профессию инженера-программиста. В качестве базового языка для знакомства с основами программирования выбран широко распространенный высокопроизводительный C++ с открытым исходным кодом, сочетающий свойства как высокоуровневых, так и низкоуровневых языков и являющийся отраслевым стандартом для создания игр, операционных систем и браузеров. Освоение указанного языка осуществляется посредством хорошо себя зарекомендовавшей среды разработки Visual Studio от компании Microsoft. Изучая эту книгу, помните, что навыки программирования можно обрести только путем написания тысяч и тысяч строк кода и решения множества практических задач, поэтому обязательно решайте упражнения в каждом разделе. Удачи!

9 785171 621957